
Algoritmos y Estructuras de Datos

2025-02-17

Índice

1 Programación Orientada a Objetos	7
1.1 Conceptos esenciales	7
1.1.1 Tipos de Datos	7
1.1.2 Tipo de dato simple y compuesto	8
1.1.3 Tipo referencia y tipo valor	8
1.1.4 Tipos de Datos Abstractos	9
1.2 Conceptos esenciales de programación orientada a objetos (POO)	10
1.2.1 Definición de Objeto	10
1.2.2 Definición de Clase	10
1.3 Principios de programación orientada a objetos	11
1.3.1 Abstracción	11
1.3.2 Encapsulamiento	12
1.3.3 Herencia	14
1.3.4 Polimorfismo	15
2 Estructuras de Datos Lineales	19
2.1 Arreglos y Matrices	19
2.1.1 Arreglos	19
2.1.2 Matrices	21
2.2 Listas	24
2.2.1 Tipo de Dato Abstracto Lista	24
2.2.2 Implementación mediante arreglos	24
2.2.3 Implementación mediante listas simples enlazadas	26
2.2.4 Implementación mediante lista doblemente enlazada	28
2.2.5 Implementación mediante lista enlazada circular	31
2.2.6 Tabla comparativa de implementaciones	33
2.3 Pilas	34
2.3.1 Tipo de dato abstracto Pila	34
2.3.2 Implementación de pilas con arreglos	34
2.3.3 Implementación de pilas con listas enlazadas	35
2.4 Colas	36
2.4.1 Tipo de dato abstracto Cola	36
2.4.2 Implementación de colas con arreglos	37
2.4.3 Implementación de colas con listas enlazadas	39
2.5 Colas de prioridad	40
2.5.1 Tipo de dato abstracto Cola de prioridad	41

2.5.2	Tipos de colas de prioridad	41
2.5.3	Implementación de colas de prioridad con listas enlazadas	41
2.6	Generics	42
2.7	Referencias adicionales	44
3	Estructuras de datos jerárquicas	45
3.1	Terminología básica	46
3.2	Tipo de dato abstracto Árbol	46
3.3	Árboles Binarios de Búsqueda (BST)	47
3.3.1	Implementación	47
3.3.2	Búsqueda de elementos	47
3.3.3	Inserción de elementos	48
3.3.4	Eliminación de elementos	49
3.3.5	Recorridos	50
3.4	Árboles Heap (montículo)	52
3.4.1	Tipo de dato abstracto	53
3.4.2	Implementación de árboles heap	54
3.5	Árboles AVL	56
3.5.1	Características	56
3.5.2	Ventajas	57
3.5.3	Desventajas	57
3.5.4	Inserción	57
3.5.5	Eliminación	62
3.5.6	Búsquedas y recorridos	64
3.6	Árboles de expresión	64
3.6.1	Conversión de arboles de expresión a notación infija	65
3.6.2	Conversión de expresión a árbol de expresión	65
3.6.3	Convertir expresión infija a postfija	65
3.7	Árboles B	66
3.7.1	Características	66
3.7.2	Estructura básica en Java	67
3.7.3	Búsqueda	67
3.7.4	Inserción	68
3.7.5	Eliminación	70
3.8	Tries	75
3.8.1	El TDA Trie	76
3.8.2	Estructura básica	76
3.8.3	Inserción	77

3.8.4	Búsqueda	79
4	Algoritmos de búsqueda y ordenamiento	80
4.1	Algoritmos de búsqueda	80
4.1.1	Búsqueda lineal	80
4.1.2	Búsqueda binaria	81
4.1.3	Búsqueda Hash	82
4.2	Algoritmos de ordenamiento	85
4.2.1	Ordenamiento por selección	85
4.2.2	Ordenamiento de burbuja	87
4.2.3	Ordenamiento por inserción (Insertion sort)	90
4.2.4	Quick sort	92
4.2.5	Shellsort	93
4.2.6	Ordenamiento por mezcla (Merge sort)	95
4.2.7	Ordenamiento por Raíz (Radix sort)	97
5	Grafos	101
5.1	Conceptos importantes	102
5.2	Representación	103
5.2.1	Matriz de adyacencia	103
5.2.2	Lista de adyacencia	105
5.3	Recorridos de un grafo	106
5.3.1	Breadth-First	106
5.3.2	Depth-First	107
5.4	Camino más corto: Dijkstra	107
5.4.1	Estructura general	109
5.5	Camino más corto: Floyd	109
5.6	Warshall	111
5.7	Minimal Spanning tree	113
5.7.1	Prim	114
5.7.2	Kruskal	116
6	Administración de Memoria	118
6.1	Breve introducción a sistemas operativos	118
6.1.1	El sistema operativo como API para los developers	118
6.1.2	El sistema operativo como administrador de los recursos de la máquina.	118
6.2	Definición de Administración de Memoria	119
6.2.1	Las direcciones de memoria	119

6.3	Evolución de la Administración de memoria	120
6.3.1	Ninguna abstracción de memoria	120
6.3.2	Espacios de direcciones	123
6.3.3	Memoria virtual	123
6.4	Administración de memoria a nivel de proceso	125
6.4.1	Secciones del memory layout	126
6.4.2	Punteros	136
6.4.3	Punteros en lenguajes manejados	146
6.5	Referencias adicionales	147
7	Análisis de algoritmos	148
7.1	Definición de Algoritmo	148
7.2	Análisis de Algoritmos	149
7.3	Análisis empírico de algoritmos	150
7.4	Análisis teórico de algoritmos	151
7.4.1	Mejor, peor y caso promedio	152
7.4.2	Análisis asintótico y notaciones comunes	153
7.4.3	Notación Big O, Big Omega y Big Theta	155
7.5	Determinar la complejidad de un algoritmo	158
7.5.1	Secuencia de instrucciones	158
7.5.2	If-Then-Else	159
7.5.3	Loops	159
7.5.4	Anidamiento	159
7.5.5	Complejidad logarítmica	160
7.5.6	Recursión	160
7.5.7	Más ejemplos	161
7.6	Revisitando los algoritmos y estructuras de datos	164
7.7	Referencias	165
8	Diseño de Algoritmos	167
8.1	Divide y conquista	167
8.1.1	Ejemplos de algoritmos de divide y conquista	167
8.2	Programación dinámica (DP)	170
8.2.1	Ejemplo de programación dinámica: Longest Common Subsequence (LCS)	172
8.3	Backtracking	173
8.3.1	Ejemplo: el problema de las N-reinas	174
8.4	Algoritmos ávidos	174
8.4.1	Ejemplo: Número mínimo de monedas	175

8.5	Algoritmos Probabilísticos	175
8.5.1	Clasificación	175
8.5.2	Aleatoriedad	176
8.5.3	Algoritmos de Monte Carlo	176
8.5.4	Algoritmos de Las Vegas	177
8.5.5	Estructuras de datos probabilísticas	177
8.6	Algoritmos Genéticos	179
8.6.1	Definiciones esenciales	180
8.6.2	Representación de un individuo	180
8.6.3	Funcionamiento general	180
8.6.4	¿Cómo seleccionar los padres?	181
8.6.5	Introducir variabilidad	181
8.6.6	Recombinación	181
8.6.7	Mutación	181
8.7	Referencias	181
9	Algoritmos de búsqueda	183
9.1	Búsqueda en arrays	183
9.1.1	Búsqueda secuencial	183
9.1.2	Búsqueda binaria	184
9.1.3	Búsqueda por interpolación	185
9.1.4	Búsqueda por salto (Jump Search)	186
9.2	Pathfinding	186
9.2.1	Pathfinding básico	186
9.2.2	Movimiento aleatorio ante obstáculos	188
9.2.3	Rodear obstáculos	189
9.2.4	Pathfinding basado en grafos	191
9.2.5	Referencias	196
10	Estructuras de almacenamiento externo	197
10.1	Conceptos esenciales	197
10.2	Discos Duros (Hard-disk drives)	197
10.2.1	Componentes	198
10.2.2	Funcionamiento	198
10.2.3	Tiempos de acceso	199
10.2.4	Tiempo de transferencia	200
10.2.5	Interfaces	200

10.3	Discos de Estado Sólido (SSD)	201
10.3.1	Componentes	201
10.3.2	Desempeño	201
10.4	Particionamiento de discos	202
10.4.1	Volumen	202
10.5	Sistemas de archivos	202
10.5.1	Propósito	202
10.6	Archivos	203
10.6.1	Estructura de un archivo	203
10.6.2	Tipos de archivos	204
10.7	Cache	204
10.7.1	Funcionamiento general	205
10.7.2	Operaciones en el cache	205
10.7.3	Tipos de cache a nivel general	206
10.7.4	Tipos de cache a nivel específicos	206
10.8	Algoritmos de ordenamiento externo	206
10.8.1	External merge	207
10.8.2	External merge-sort	210
10.9	Referencias	216
11	Bases de Datos	217
11.1	Definiciones fundamentales	217
11.2	Enfoques en el manejo de la información	218
11.2.1	Enfoque orientado a archivos	218
11.2.2	Enfoque orientado a bases de datos	219
11.3	Tipos de bases de datos	220
11.3.1	Bases de datos relacionales	221
11.3.2	Bases de Datos NoSQL	226
11.3.3	Características de NoSQL	227
11.3.4	Tipos de Bases de Datos NoSQL	227
11.3.5	Casos de Uso de NoSQL	227
11.3.6	Consideraciones y Limitaciones	228
11.3.7	Ventajas de las bases de datos NoSQL	228
11.4	Referencias	228

1 Programación Orientada a Objetos

En este capítulo, se introduce la programación orientada a objetos junto a conceptos esenciales para comprender su funcionamiento e implementación en lenguajes de programación comunes.

1.1 Conceptos esenciales

Antes de entrar en detalle en POO, es importante entender algunos conceptos básicos de programación y tipos de datos. Estos conceptos son fundamentales para comprender cómo funciona la programación orientada a objetos y cómo se pueden utilizar en la práctica.

1.1.1 Tipos de Datos

Un *tipo de dato* es una clasificación que especifica qué tipo de valores puede tomar una variable, así como las operaciones que se pueden realizar sobre estos valores. Los tipos de datos se utilizan en la declaración de variables y en la definición de funciones, entre otros usos.

Por ejemplo, en [Python](#), los tipos de datos más comunes son:

- **Enteros (int)**: Números enteros como 1, 20, -5.
- **Flotantes (float)**: Números con decimales como 3.14
- **Cadenas (str)**: Secuencias de caracteres como “Hola, mundo!”
- **Booleanos (bool)**: Valores de verdad como True o False

Con respecto a las operaciones que se pueden realizar con estos tipos de datos, por ejemplo, se pueden realizar operaciones aritméticas con enteros y flotantes, concatenar cadenas, y realizar operaciones lógicas con booleanos.

Los IDEs suelen proporcionar herramientas para trabajar con tipos de datos, como autocompletado y resaltado de sintaxis, lo que facilita la escritura y comprensión del código tal y como se muestra en la siguiente imagen:

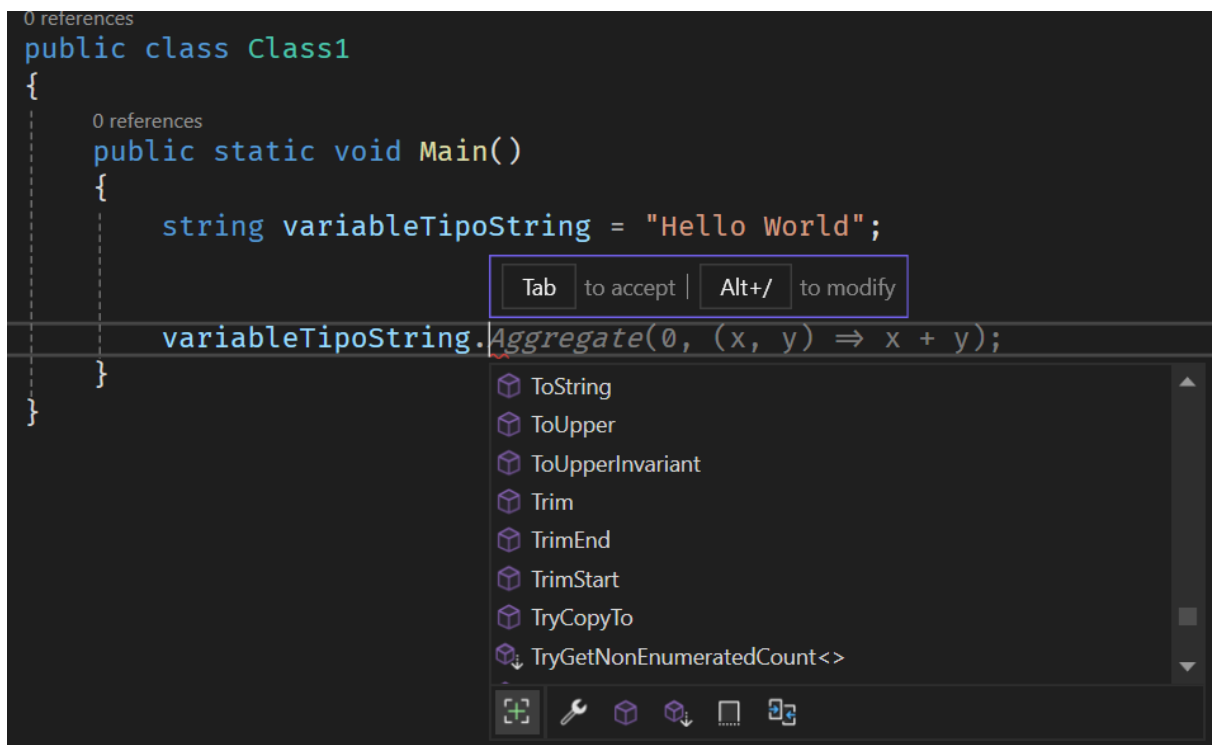


Figura 1: Ejemplo de autocompletado en el IDE según el tipo de dato

1.1.2 Tipo de dato simple y compuesto

Un *tipo de dato simple* es un tipo de dato que representa un único valor. Los tipos de datos simples son los tipos de datos básicos que se utilizan para representar valores individuales. No tiene sentido práctico separarlos en partes más pequeñas.

Un *tipo de dato compuesto* es un tipo de dato que representa una colección de valores. Los tipos de datos compuestos se utilizan para representar estructuras de datos más complejas que contienen múltiples valores de otros tipos de datos. Por ejemplo, un tipo de dato Cliente, puede contener los datos de nombre, edad, dirección, etc. **Los objetos se consideran tipos de datos compuestos.**

1.1.3 Tipo referencia y tipo valor

Un *tipo de referencia* es un tipo de dato que almacena una referencia a una ubicación en memoria. Los tipos de referencia se utilizan para acceder a memoria que puede ser compartida y modificada por múltiples partes de un programa. Por ejemplo, en [Python](#), las listas y los diccionarios son tipos de referencia. **Los objetos en la mayoría de lenguajes orientados a objetos también son tipos de referencia.**

Un *tipo valor* es un tipo de dato que almacena un valor directamente en la memoria. Los tipos de valor se utilizan para representar valores que no pueden ser compartidos ni modificados por múltiples partes de un programa. Por ejemplo, en [Python](#), los enteros y los flotantes son tipos de valor.

Usualmente los tipos de datos simples son tipos de valor, mientras que los tipos de datos compuestos son tipos de referencia. Puede leer más para el caso de C# [aquí](#).

1.1.4 Tipos de Datos Abstractos

Un *tipo de dato abstracto (TDA)* es un modelo conceptual que define un conjunto de valores y un conjunto de operaciones que se pueden realizar con esos valores. Los TDAs son una forma de abstracción que permite a los programadores trabajar con datos de manera más abstracta y genérica.

Un TDA especifica una interfaz que define las operaciones que se pueden realizar con los datos, **pero no especifica cómo se implementan esas operaciones**. Esto permite a los programadores utilizar los TDAs sin tener que preocuparse por los detalles de implementación subyacentes. Por tanto, se puede decir que un TDA tiene una vista lógica y una vista física o de implementación.

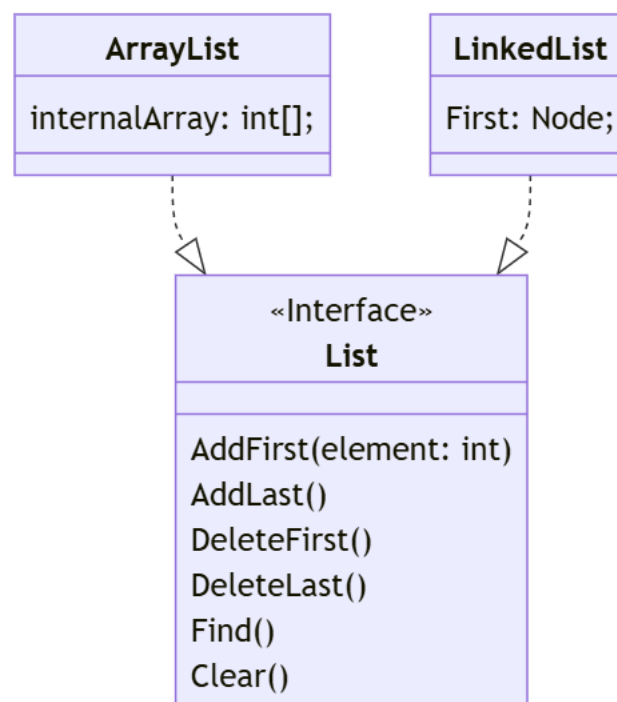


Figura 2: Diagrama de implementaciones del TDA Lista

En el diagrama anterior, se ilustra como un TDA Lista, puede ser implementado mediante Nodos con

memoria dinámica o mediante un arreglo con memoria estática. El API expuesto por el tipo Lista, no debe dar detalles de cómo se implementa internamente.

Una estructura de datos se puede entender como la implementación de TDA. En Programación Orientada a Objetos, **un TDA + implementación forman una clase**. Algunos lenguajes permiten definir *interfaces* que son un TDA puro.

1.2 Conceptos esenciales de programación orientada a objetos (POO)

POO es un paradigma de programación que modela conceptos del mundo real como *objetos* que tienen atributos y comportamientos. Los objetos son *ciudadanos de primera clase* en la programación orientada a objetos (POO), lo que significa que pueden ser manipulados y pasados como argumentos a funciones. Construir software orientado a objetos implica definir clases que representan tipos de objetos y crear instancias de esas clases (objetos) para interactuar entre sí.

Puede leer más sobre otros paradigmas de programación aquí.

1.2.1 Definición de Objeto

Estructura de datos que agrupa atributos (datos) y métodos (funciones) que operan sobre esos datos. Los objetos son instancias de clases, que definen la estructura (definida mediante los *atributos*) y comportamiento de los objetos (definido mediante los *métodos*).

La **interfaz** de un objeto se define por los métodos y atributos **públicos** que son accesibles desde fuera del objeto. Es decir, por otros objetos que usen el objeto en cuestión.

La mayoría de lenguajes de programación orientados a objetos, requieren la definición de clases para crear objetos.

1.2.2 Definición de Clase

Las clases son *plantillas que definen la estructura y comportamiento de los objetos*. Por ejemplo, la clase Televisor en C# se puede definir de la siguiente manera:

```
1 class Televisor
2 {
3     // Atributos
4     string marca;
5     int pulgadas;
6     bool encendido;
7
8     // Métodos
```

```
9     void Encender()  
10    {  
11        encendido = true;  
12    }  
13  
14    void Apagar()  
15    {  
16        encendido = false;  
17    }  
18 }
```

Para generar objetos a partir de la clase Televisor, se utilizan las siguientes instrucciones:

```
1 Televisor samsung = new Televisor();  
2 Televisor lg = new Televisor();  
3 lg.Encender();  
4 Console.WriteLine(samsung.encendido); // Imprime false
```

Un objeto opera sobre sus propios datos y no afecta la memoria de otros objetos diferentes. **Cada objeto es un bloque de memoria independiente que contiene sus propios datos y métodos.**

Clase vs Objeto

La clase define una plantilla, plano o molde para crear objetos basados en estos. Es decir, a partir de una clase se crean o instancian objetos

Definir clases es una forma de extender el lenguaje de programación, definiendo nuevos tipos de datos compuestos.

1.3 Principios de programación orientada a objetos

Son los conceptos fundamentales que rigen la programación orientada a objetos, la base para entender cómo se estructura y cómo se trabaja con este paradigma. Son los elementos distintivos que diferencian la programación orientada a objetos de otros paradigmas.

Los principios de POO son los siguientes: - Abstracción - Encapsulamiento - Herencia - Polimorfismo

1.3.1 Abstracción

Es la principal característica de POO. Permite modelar el problema por resolver en términos de objetos de alto nivel que ocultan los detalles de su implementación. La abstracción se logra a través de la creación de clases y objetos que representan entidades del mundo real. Por ejemplo, una clase *Persona* puede representar a una persona en el mundo real, con atributos como nombre, edad, etc., y métodos que permiten interactuar con la persona.

Al interactuar con objetos, pensamos en términos de su comportamiento y atributos en vez de variables y procedimientos.

Piezas de LEGO vs Plasticina

La abstracción permite que los objetos se comporten como piezas de LEGO que tienen una interfaz específica para interactuar con ellos, pero no necesitamos saber cómo están contruidos internamente. En cambio, al trabajar con plasticina, no hay una clara separación, sino que todo forma parte de un todo.

Aunque algunos lenguajes tienen la palabra reservada *Abstract*, esto no tiene relación con la abstracción como principio de POO.

1.3.2 Encapsulamiento

Los objetos son módulos autocontenidos que asocian código con sus datos. Los datos dentro de los objetos pueden o no exponerse según el programador lo decida. El **encapsulamiento** permite ocultar los detalles de implementación de un objeto y exponer solo la interfaz necesaria para interactuar con él.

Por ejemplo, en el código siguiente, los objetos de tipo *BankAccount*, ocultan los detalles de implementación de los métodos *Deposit* y *Withdraw*, y exponen solo la interfaz necesaria para interactuar con ellos. Ningún objeto externo a *BankAccount* puede acceder directamente a los atributos *accountNumber* y *balance*.

```
3 references
public class BankAccount
{
    private string accountNumber;
    private decimal balance;

    1 reference
    public BankAccount(string accountNumber)
    {
        this.accountNumber = accountNumber;
        this.balance = 0;
    }

    1 reference
    public decimal GetBalance()
    {
        // Retorna una copia del valor de balance
        return balance;
    }

    1 reference
    public void Deposit(decimal amount)
    {
        balance += amount;
    }

    1 reference
    public void Withdraw(decimal amount)
    {
        if (amount ≤ balance)
        {
            balance -= amount;
        }
        else
        {
            Console.WriteLine("Fondos insuficientes");
        }
    }
}

0 references
public class Program
{
    0 references
    public static void Main(string[] args)
    {
        BankAccount account = new BankAccount("1234567890");
        account.Deposit(1000);
        account.Withdraw(500);
        decimal balance = account.GetBalance();
        Console.WriteLine("Account balance: " + balance);

        account.balance = 1000000;
    }
}
```

readonly struct System.Decimal
Represents a decimal floating-point number.

Describe with Copilot

CS0122: 'BankAccount.balance' is inaccessible due to its protection level

Show potential fixes (Alt+Enter or Ctrl+.)

Figura 3: Ejemplo de encapsulamiento

Los lenguajes orientados a objetos permiten establecer el nivel de visibilidad que tiene un atributo o método con respecto a otros objetos o clases. Por ejemplo, C# soporta muchos modificadores de acceso, entre los que se incluyen:

- **public**: Accesible desde cualquier parte del código.
- **private**: Accesible solo desde la misma clase.
- **protected**: Accesible desde la misma clase y sus clases derivadas.

Por ejemplo, considere la siguiente clase Persona:

```
1 public class Persona
2 {
3     private string nombre;
4
5     public string GetNombre()
6     {
7         return nombre;
8     }
9
10    public void SetNombre(string nombre)
11    {
12        this.nombre = nombre;
13    }
14 }
```

En este ejemplo, el atributo nombre está declarado como private, lo que significa que solo es accesible desde la misma clase. Sin embargo, se proporcionan métodos públicos `GetNombre()` y `SetNombre()` para acceder y modificar el valor de nombre de manera controlada.

Puede leer sobre los modificadores de acceso en C# aquí. Igualmente puede leer más sobre get y set en C# aquí.

1.3.3 Herencia

La herencia permite definir jerarquías de objetos con el objetivo de reutilizar código. Cada clase puede tener máximo una clase padre de la que hereda atributos y métodos.

La clase padre se llama *superclase* y la clase hija se llama *subclase*. La superclase provee comportamiento general, mientras que las subclases proveen comportamiento especializado.

La herencia define una relación de tipo “es un/a” entre la superclase y la subclase. Por ejemplo, si tenemos una clase `Vehículo` y una clase `Automóvil`, podemos decir que *un automóvil es un vehículo*.

```
1 // Definición de la clase base
```

```
2 public class Animal
3 {
4     public string Nombre { get; set; }
5     public int Edad { get; set; }
6
7     public void Comer()
8     {
9         Console.WriteLine("El animal está comiendo");
10    }
11 }
12
13 // Definición de una subclase que hereda de Animal
14 public class Perro : Animal
15 {
16     public void Ladrar()
17     {
18         Console.WriteLine("El perro está ladrando");
19     }
20 }
21
22 // Uso de la herencia en el programa principal
23 public class Program
24 {
25     public static void Main(string[] args)
26     {
27         // Creación de una instancia de la clase base
28         Animal animal = new Animal();
29         animal.Nombre = "Animal";
30         animal.Edad = 5;
31         animal.Comer();
32
33         // Creación de una instancia de la subclase
34         // Nótese que Perro hereda de Animal
35         // las propiedades como Nombre y Edad
36         Perro perro = new Perro();
37         perro.Nombre = "Firulais";
38         perro.Edad = 3;
39         perro.Comer();
40         perro.Ladrar();
41     }
42 }
```

1.3.4 Polimorfismo

Etimológicamente significa “*muchas formas*” y se refiere a la capacidad de un objeto de comportarse de diferentes maneras. Hay dos tipos: *run-time* y *compile time*.

1.3.4.1 Run-time (tiempo de ejecución) Cuando una clase hija anula (override) un método public/protected de la clase padre. Por ejemplo:

```
1 public class Shape
2 {
3     public virtual void Draw()
4     {
5         Console.WriteLine("Drawing a shape");
6     }
7 }
8
9 public class Circle : Shape
10 {
11     public override void Draw()
12     {
13         Console.WriteLine("Drawing a circle");
14     }
15 }
16
17 public class Square : Shape
18 {
19     public override void Draw()
20     {
21         Console.WriteLine("Drawing a square");
22     }
23 }
24
25 public class Program
26 {
27     public static void Main(string[] args)
28     {
29         Shape shape1 = new Circle();
30         Shape shape2 = new Square();
31
32         shape1.Draw(); // Output: Drawing a circle
33         shape2.Draw(); // Output: Drawing a square
34     }
35 }
```

Cuando el código cliente llama el método, el método se “resuelve” en ese momento invocando el método correcto. Para encontrar el método correcto, el compilador busca en la clase real del objeto en tiempo de ejecución y en caso de no estar, sigue subiendo por la jerarquía de clases.

Polimorfismo permite tratar la clase hija como si fuera la padre. Cualquier atributo o método visible de la clase Padre se puede acceder a través de la Hija. Cualquier método o atributo que sea específico de la hija, no es accesible a través de la clase padre.

Algunos lenguajes permiten crear clases abstractas que son útiles para polimorfismo.

```
1 public abstract class Animal
```

```
2 {
3     void Respirar()
4     {
5         Console.Write("Respirando")
6     }
7
8     abstract PorDefinir(); // no tiene definición y fuerza a las hijas
                             a implementarla
9 }
10
11 public class Perro : Animal
12 {
13     void Comer()
14     {
15         Console.Write("Comiendo")
16     }
17
18     // implementa el método abstracto de la clase padre.
19     // Si no lo hace, el compilador arroja un error.
20     void PorDefinir()
21     {
22         Console.Write("Foo...")
23     }
24 }
```

Algunos lenguajes también permiten crear *interfaces* las cuales proveen polimorfismo sin formar parte de una jerarquía.

```
1 public interface Alimentable
2 {
3     void alimentar();
4 }
5
6 public class Persona : Alimentable {
7
8     // Implementa el método de la interfaz. Si no
9     // lo hace, el compilador arroja un error.
10    void alimentar()
11    {
12        Console.Write("Comiendo")
13    }
14 }
```

1.3.4.2 Compile-time La habilidad para sobrecargar (overload) métodos, es decir, crear varios métodos con el mismo nombre pero diferentes parámetros.

```
1 public class TestClass
2 {
3     void foo()
```

```
4      {  
5      }  
6  
7      void foo(int x)  
8      {  
9      }  
10  
11     void foo(string s, int x)  
12     {  
13     }  
14 }
```

2 Estructuras de Datos Lineales

¿Qué es una estructura de datos?

Una estructura de datos es una forma particular de organizar datos en una computadora para que puedan ser utilizados de manera eficiente. Se utilizan para procesar, recuperar y almacenar datos. Existe una colección mínima de estructuras de datos comunes que son utilizadas en la mayoría de los lenguajes de programación.

Las estructuras de datos lineales son estructuras de datos cuyos elementos se organizan secuencialmente, uno tras de otro. Hay un solo nivel de elementos y se recorren en una sola pasada. Cada elemento de la estructura tiene un predecesor y un sucesor, excepto el primer y el último elemento.

2.1 Arreglos y Matrices

2.1.1 Arreglos

Los *arreglos* son colecciones de elementos del mismo tipo. Los elementos se almacenan en posiciones contiguas de memoria. Es la estructura de datos más básica y se utiliza en la mayoría de los lenguajes de programación.

Terminología básica de arreglos incluye:

- *Índice*: los elementos en un array se identifican por su índice (empieza en cero usualmente). Por ejemplo, el primer elemento de un array tiene índice 0, el segundo tiene índice 1, y así sucesivamente.
- *Longitud*: el número de elementos en un array. Por ejemplo, un array con 5 elementos tiene longitud 5 y sus índices van de 0 a 4.
- *Elemento*: un valor almacenado en una posición específica del array. Por ejemplo, el elemento en la posición 0 del array es el primer elemento del array.

Visualmente, un arreglo se puede representar con la imagen siguiente. Nótese que cada elemento tiene una posición de memoria contigua (en la imagen se muestran direcciones de memoria en decimal). Cada “espacio” del arreglo es del mismo tamaño según el tipo de dato que se almacene (por ejemplo, un `int` ocupa 4 bytes, por eso observe como las direcciones aumentan de cuatro en cuatro).

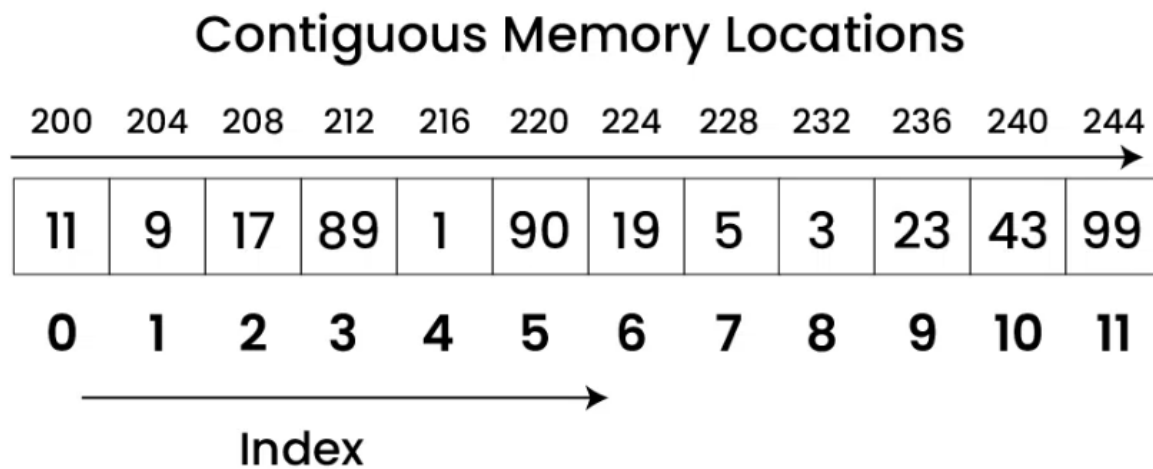


Figura 4: Representación de un arreglo en memoria

En C# una arreglo se declara de la siguiente manera:

```

1 // Arreglo de enteros. Cada posición es de 4 bytes
2 int[] arr;
3
4 // Arreglo de enteros. Cada posición es de 1 byte
5 char[] arr2;
6
7 // Arreglo de flotantes. Cada posición es de 4 bytes
8 float[] arr3;
9
10 // Ejemplo de inicialización en línea
11 int[] arr = { 1, 2, 3, 4, 5 };
12 char[] arr = { 'a', 'b', 'c', 'd', 'e' };
13 float[] arr = { 1.4f, 2.0f, 24f, 5.0f, 0.0f };
14
15 // Ejemplo de inicialización con tamaño
16 int[] arr = new int[5];
17
18 // Ejemplo de acceso a elementos
19 arr[0] = 1;
20 int x = arr[0];

```

¿ArrayList?

Algunos lenguajes como C# proveen la clase ArrayList o similar. Esto corresponde a una implementación de una lista que se verá más adelante en este documento. No se debe confundir con

un arreglo estático de posiciones contiguas

2.1.1.1 Ventajas y desventajas de los arreglos

Algunas de las ventajas de los arreglos son:

1. Tiene acceso aleatorio a los elementos. Es decir, se puede acceder a cualquier elemento en tiempo constante.
2. Es fácil de implementar y usar.
3. Cero overhead de memoria. No se necesita almacenar información adicional para mantener la estructura.

Por su parte, algunas de las desventajas de los arreglos son:

1. El tamaño del arreglo es fijo. No se puede cambiar el tamaño del arreglo una vez que se ha creado. Si se ocupa espacio adicional, se tiene que crear un nuevo arreglo y copiar los elementos del arreglo original al nuevo arreglo.
2. Es difícil de insertar y eliminar elementos. Se necesita mover todos los elementos que están después del elemento que se quiere insertar o eliminar.

¿Acceso rápido?

Acceder un elemento de un arreglo, se reduce a realizar una operación de suma y una operación de acceso a memoria. Por ejemplo, si queremos acceder al elemento en la posición 3 de un arreglo de enteros, se realiza la siguiente operación: `arr + 3 * sizeof(int)`. Donde `arr` es la dirección de memoria del primer elemento del arreglo y `sizeof(int)` es el tamaño en bytes de un entero. Esta operación se realiza en tiempo constante, es decir, no importa el tamaño del arreglo, la operación siempre se realiza en el mismo tiempo.

2.1.2 Matrices

Son arreglos de dos dimensiones. Es decir, son arreglos de arreglos. Se utilizan para representar datos en forma de tabla. Por ejemplo, una matriz de 3x3 se puede representar gráficamente como:

Columns 

Rows 

	0	1	2
0	10	8	3
1	5	20	6
2	7	9	30

Figura 5: Representación de una matriz

Para declarar e inicializar una matriz en Java, se utiliza la siguiente sintaxis:

```
1 int[][] matriz = {  
2     {1, 2, 3},  
3     {4, 5, 6},  
4     {7, 8, 9}  
5 };  
6  
7 // También se puede declarar sin inicializar  
8 int[][] matriz = new int[3][3];  
9  
10 // Y luego inicializar  
11 matriz[0][0] = 1;  
12 matriz[0][1] = 2;
```

C# provee una forma más compacta de declarar e inicializar matrices:

```
1 int[,] matriz = {  
2     {1, 2, 3},  
3     {4, 5, 6},  
4     {7, 8, 9}  
5 };
```

Dado que son arreglos de arreglos, se puede acceder a los elementos de la matriz utilizando dos índices. Por ejemplo, para acceder al elemento en la fila 1 y columna 2 de la matriz anterior, se utiliza la siguiente sintaxis: `matriz[1][2]`.

¿Memoria contigua en las matrices?

Las matrices no necesariamente se almacenan en memoria contigua. Por ejemplo, en C# las matrices se almacenan en memoria contigua, pero en Java no. En Java, las matrices se almacenan como un arreglo de arreglos, es decir, un arreglo de referencias a otros arreglos. Por lo tanto, no se puede acceder a un elemento de la matriz utilizando una sola operación de suma y una operación de acceso a memoria. Se necesita realizar dos operaciones de acceso a memoria y una operación de suma. Esto se debe a que primero se necesita acceder a la referencia del arreglo que contiene el elemento y luego acceder al elemento dentro de ese arreglo.

Aunque el término matriz se utiliza comúnmente para referirse a arreglos de dos dimensiones, también se puede utilizar para referirse a arreglos de más de dos dimensiones. Por ejemplo, un arreglo de tres dimensiones se puede inicializar en Java de la siguiente manera:

```
1 int[][][] matriz3D = {  
2     {  
3         {1, 2, 3},  
4         {4, 5, 6},  
5         {7, 8, 9}  
6     },  
7     {  
8         {10, 11, 12},  
9         {13, 14, 15},  
10        {16, 17, 18}  
11    },  
12    {  
13        {19, 20, 21},  
14        {22, 23, 24},  
15        {25, 26, 27}  
16    }  
17 };  
18  
19 // También se puede declarar sin inicializar  
20 int[][][] matriz3D = new int[3][3][3];  
21  
22 // Y luego inicializar  
23 matriz3D[0][0][0] = 1;
```

```
24 matriz3D[0][0][1] = 2;
```

2.2 Listas

Son colecciones de elementos del mismo tipo. A diferencia de los arreglos, las listas no tienen un tamaño fijo. Se pueden agregar y eliminar elementos de la lista. El orden de inserción de los elementos se respeta.

2.2.1 Tipo de Dato Abstracto Lista

Las operaciones básicas que se pueden realizar en una lista son:

- **Obtener (get):** devuelve el elemento en una posición específica.
- **Agregar (add):** añade un elemento a la lista.
- **Eliminar (remove):** elimina un elemento de la lista.
- **Contiene (contains):** verifica si un elemento está en la lista.
- **Tamaño (size):** devuelve el número de elementos en la lista.

Pueden implementarse de muchas formas. Las más comunes son:

- **ArrayList:** Implementación mediante arreglos.
- **SinglyLinkedList:** Implementación mediante listas simples enlazadas.
- **DoubleLinkedList:** Implementación mediante listas doblemente enlazadas.
- **CircularLinkedList:** Implementación mediante listas enlazadas circulares.

2.2.2 Implementación mediante arreglos

```
1 public class ArrayList implements List {
2     private int maxSize;
3     private int currentSize;
4     private int[] storage;
5
6     public ArrayList(int size) {
7         this.maxSize = size;
8         this.currentSize = 0;
9         this.storage = new int[size];
10    }
11
12    public void clear() {
13        this.currentSize = 0;
14    }
15
```

```
16     public boolean isEmpty() {
17         return this.currentSize == 0;
18     }
19
20     public int size() {
21         return this.currentSize;
22     }
23
24     public boolean contains(int element) {
25         for (int i = 0; i < this.maxSize; i++) {
26             if (this.storage[i] == element) {
27                 return true;
28             }
29         }
30         return false;
31     }
32
33     public boolean add(int element) {
34         if (this.currentSize > this.maxSize) {
35             return false;
36         }
37         this.storage[currentSize++] = element;
38         return true;
39     }
40
41     public int remove(int element) {
42         int i;
43         for (i = 0; i < this.currentSize; i++) {
44             if (this.storage[i] == element) {
45                 indexToRemove = i;
46             }
47         }
48         if (i == this.maxSize) {
49             throw new NoSuchElementException();
50         }
51         for (int j = i; j < this.currentSize - 1; j++) {
52             this.storage[j] = this.storage[j + 1];
53         }
54         this.currentSize--;
55     }
56 }
```

- Algunos lenguajes explícitamente proveen soporte para ArrayList
- **La principal ventaja** que proveen en vez de usar directamente es la abstracción de más alto nivel. El programador no tiene que lidiar con *shifts*, re-sizes u otros problemas de usar arreglos directamente
- Una estrategia que se puede utilizar para evitar lanzar una excepción al llegar al tamaño máximo, es crear un nuevo array de mayor tamaño y copiar los elementos del array actual. Pero si tendrá un *hit* de performance.

2.2.3 Implementación mediante listas simples enlazadas

Todos los enfoques utilizando listas enlazadas utilizan memoria dinámica en vez de un arreglo, es decir, asigna memoria en el *heap* cada vez que un elemento se agrega

Dado que crea la memoria *justo en el momento*, cada elemento de la lista está separado en la memoria (a diferencia de un array donde todos los elementos están en memoria contigua)

- Cada elemento de la lista es un *nodo*. Cada nodo tiene dos partes:
 - El **valor**: es el elemento relevante para el programador. El valor que solicitó registrar en la lista. Por ejemplo, `add(3)`, el 3 es el valor de interés para el programador
 - Una **referencia** al nodo siguiente que actúa como el elemento que encadena la lista.
- Visualmente, una lista enlazada se puede representar como:

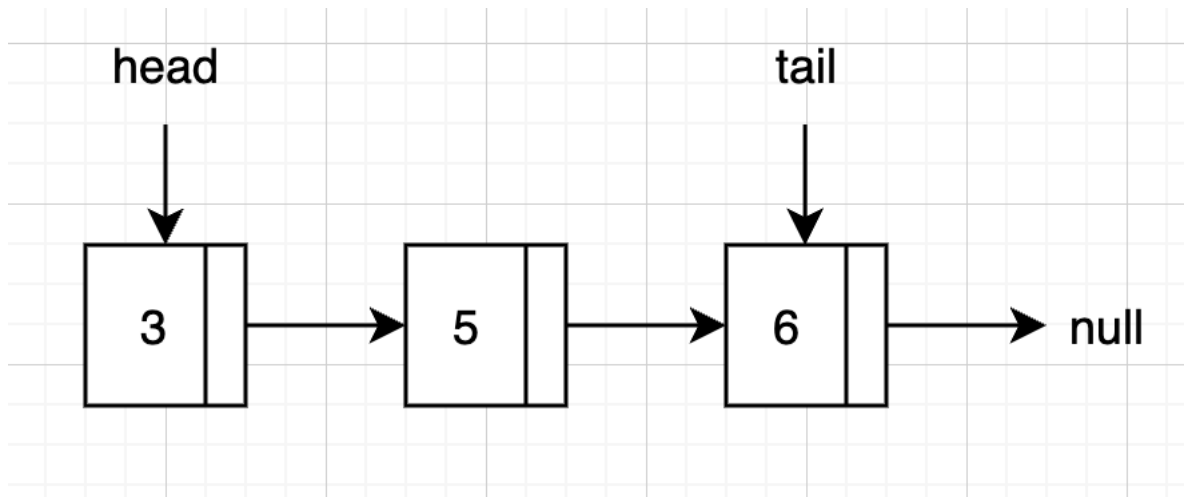


Figura 6: Visualización de una lista enlazada

- Como se puede notar, el último elemento apunta a *null* indicando el fin de la lista
- Es esencial llevar y mantener una referencia a la cabeza de la lista. Si la cabeza de la lista se pierde, se pierde toda la lista.
- Opcionalmente y para mejorar la eficiencia de la inserción al final, se puede llevar una referencia a la cola de la lista. Este tipo se conoce como *DoubleEndedLinkedList*.

2.2.3.1 Estructura general en Java

```
1 // No es public, es una clase interna para no exponer la clase nodo a
2 // otras clases fuera del paquete
3 class Node {
4     int value;
5     Node next;
```

```
6
7     public Node(int value) {
8         this.value = value;
9         this.next = null;
10    }
11 }
12
13 public class SinglyLinkedList implements List {
14     private Node head;
15     private int size;
16
17     public SinglyLinkedList() {
18         this.head = null;
19         this.size = 0;
20     }
21
22     public void clear() {
23         this.head = null;
24         this.size = 0;
25     }
26
27     public boolean isEmpty() {
28         return this.size == 0;
29     }
30
31     public int size() {
32         return this.size;
33     }
34
35     public boolean contains(int element) {
36         Node current = this.head;
37         while (current != null) {
38             if (current.value == element) {
39                 return true;
40             }
41             current = current.next;
42         }
43         return false;
44     }
45
46     public boolean add(int element) {
47         Node newNode = new Node(element);
48         if (this.head == null) {
49             this.head = newNode;
50         } else {
51             Node current = this.head;
52             while (current.next != null) {
53                 current = current.next;
54             }
55             current.next = newNode;
56         }
```



```
57     this.size++;
58     return true;
59 }
60
61 public int remove(int element) {
62     if (this.head == null) {
63         throw new NoSuchElementException();
64     }
65     if (this.head.value == element) {
66         this.head = this.head.next;
67         this.size--;
68         return element;
69     }
70     Node current = this.head;
71     while (current.next != null) {
72         if (current.next.value == element) {
73             current.next = current.next.next;
74             this.size--;
75             return element;
76         }
77         current = current.next;
78     }
79     throw new NoSuchElementException();
80 }
81 }
```

2.2.4 Implementación mediante lista doblemente enlazada

- La lista doblemente enlazada es similar a la lista simple enlazada, pero cada nodo tiene una referencia al nodo anterior y al siguiente
- Visualmente, una lista doblemente enlazada se puede representar como:

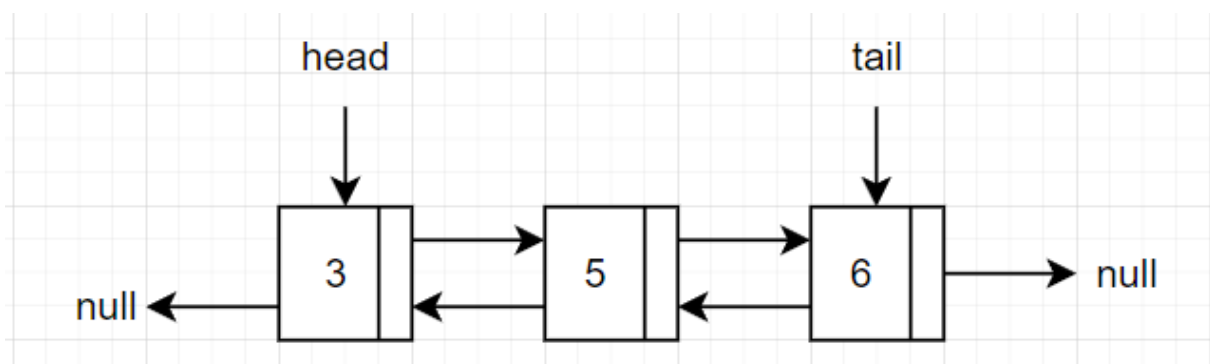


Figura 7: Visualización de lista doblemente enlazada

- La ventaja sobre la lista simple enlazada es que se puede recorrer la lista en ambas direcciones.

La desventaja es que cada nodo tiene que mantener una referencia adicional al nodo anterior, lo que consume más memoria e implica mayor complejidad en la implementación.

2.2.4.1 Estructura general en Java

```
1 class Node {
2     int value;
3     Node next;
4     Node prev;
5
6     public Node(int value) {
7         this.value = value;
8         this.next = null;
9         this.prev = null;
10    }
11 }
12
13 public class DoubleLinkedList implements List {
14     private Node head;
15     private int size;
16
17     public DoubleLinkedList() {
18         this.head = null;
19         this.size = 0;
20     }
21
22     public void clear() {
23         this.head = null;
24         this.size = 0;
25     }
26
27     public boolean isEmpty() {
28         return this.size == 0;
29     }
30
31     public int size() {
32         return this.size;
33     }
34
35     public boolean contains(int element) {
36         Node current = this.head;
37         while (current != null) {
38             if (current.value == element) {
39                 return true;
40             }
41             current = current.next;
42         }
43         return false;
44     }
45
46     public boolean add(int element) {
```

```
47     Node newNode = new Node(element);
48     if (this.head == null) {
49         this.head = newNode;
50     } else {
51         Node current = this.head;
52         while (current.next != null) {
53             current = current.next;
54         }
55         current.next = newNode;
56         newNode.prev = current;
57     }
58     this.size++;
59     return true;
60 }
61
62 public int remove(int element) {
63     if (this.head == null) {
64         throw new NoSuchElementException();
65     }
66     if (this.head.value == element) {
67         this.head = this.head.next;
68         if (this.head != null) {
69             this.head.prev = null;
70         }
71         this.size--;
72         return element;
73     }
74     Node current = this.head;
75     while (current.next != null) {
76         if (current.next.value == element) {
77             current.next = current.next.next;
78             if (current.next != null) {
79                 current.next.prev = current;
80             }
81             this.size--;
82             return element;
83         }
84         current = current.next;
85     }
86     throw new NoSuchElementException();
87 }
88 }
```

2.2.4.2 Aplicabilidad

- Cuando se necesita recorrer la lista en ambas direcciones, por ejemplo, en un editor de texto, donde se necesita recorrer el texto hacia adelante y hacia atrás o en un navegador web, donde se necesita recorrer el historial de navegación hacia adelante y hacia atrás.

2.2.5 Implementación mediante lista enlazada circular

- La lista enlazada circular es similar a la lista simple enlazada, pero el último nodo apunta al primer nodo
- Visualmente, una lista enlazada circular se puede representar como:

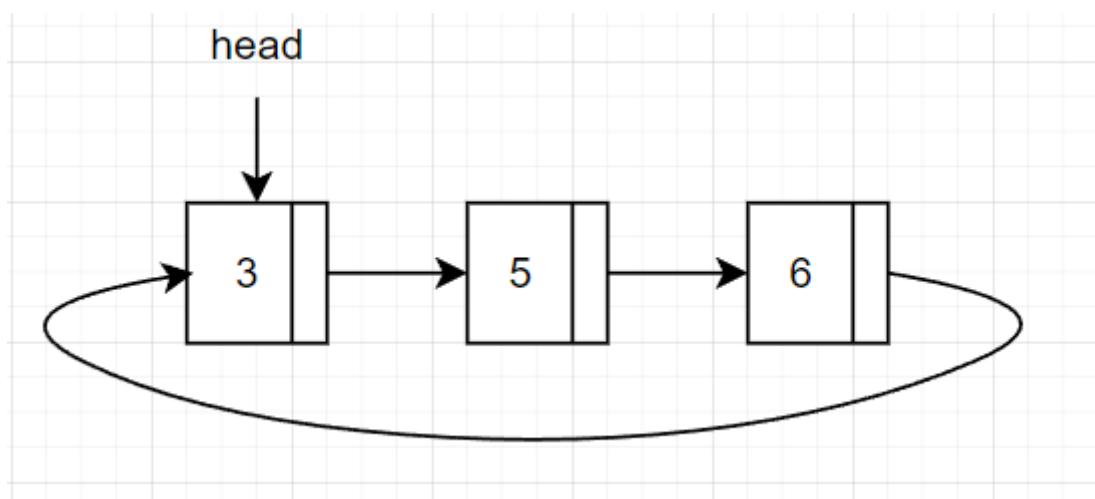


Figura 8: Visualización de lista circular

- Usualmente se implementan como circular doblemente enlazada.
- Para mejorar las inserciones, en vez de mantener la referencia a *head* se utiliza una referencia a *tail* únicamente:

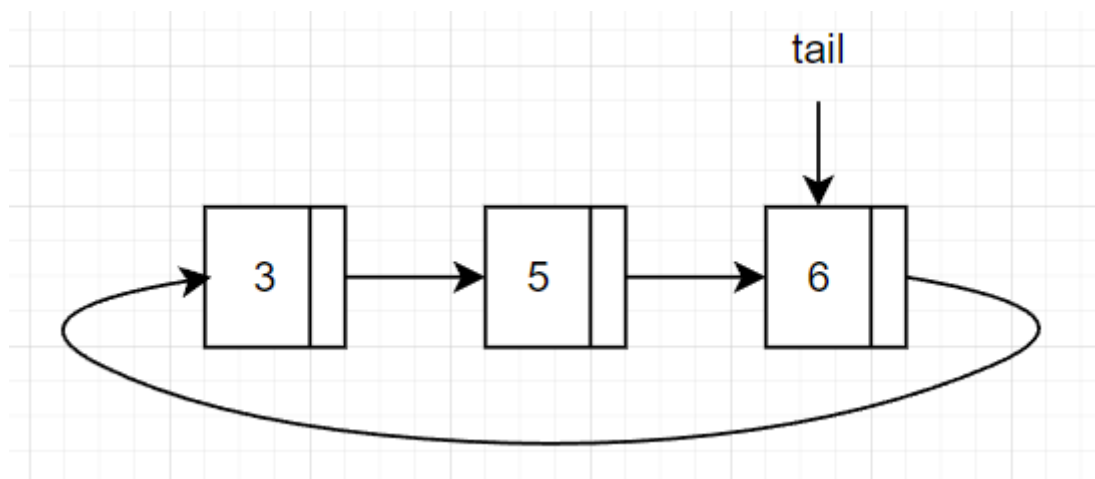


Figura 9: Optimización de listas circulares

2.2.5.1 Estructura general en Java

```
1 class Node {
2     int value;
3     Node next;
4
5     public Node(int value) {
6         this.value = value;
7         this.next = null;
8     }
9 }
10 public class CircularSinglyLinkedList {
11     private Node tail;
12     private int size;
13
14     public CircularSinglyLinkedList() {
15         this.tail = null;
16         this.size = 0;
17     }
18
19     public void clear() {
20         this.tail = null;
21         this.size = 0;
22     }
23
24     public boolean isEmpty() {
25         return this.size == 0;
26     }
27
28     public int size() {
29         return this.size;
30     }
31
32     public boolean contains(int element) {
33         if (this.tail == null) {
34             return false;
35         }
36         Node current = this.tail.next;
37         while (current != this.tail) {
38             if (current.value == element) {
39                 return true;
40             }
41             current = current.next;
42         }
43         return current.value == element;
44     }
45
46     public boolean add(int element) {
47         Node newNode = new Node(element);
48         if (this.tail == null) {
49             newNode.next = newNode;
50             this.tail = newNode;
```

```
51     } else {
52         newNode.next = this.tail.next;
53         this.tail.next = newNode;
54         this.tail = newNode;
55     }
56     this.size++;
57     return true;
58 }
59
60 public int remove(int element) {
61     if (this.tail == null) {
62         throw new NoSuchElementException();
63     }
64     Node current = this.tail.next;
65     Node prev = this.tail;
66     while (current != this.tail) {
67         if (current.value == element) {
68             prev.next = current.next;
69             this.size--;
70             return element;
71         }
72         prev = current;
73         current = current.next;
74     }
75     if (current.value == element) {
76         if (this.size == 1) {
77             this.tail = null;
78         } else {
79             prev.next = current.next;
80             if (current == this.tail) {
81                 this.tail = prev;
82             }
83         }
84         this.size--;
85         return element;
86     }
87     throw new NoSuchElementException();
88 }
89 }
```

2.2.5.2 Aplicabilidad Cuando se necesita una lista que no tenga un final o un principio, por ejemplo, una lista de reproducción de música, donde la última canción apunta a la primera o una lista de tareas pendientes, donde la última tarea apunta a la primera.

2.2.6 Tabla comparativa de implementaciones

Implementación	Acceso	Insertar/Eliminar	Uso de Memoria
ArrayList	Rápido	Costoso	Siempre ocupa todo el espacio. No tiene overhead
LinkedList	Lento	Rápido	Solo usa lo que ocupa. Tiene overhead.

2.3 Pilas

Una pila es una estructura de datos lineal que sigue el principio de LIFO (del inglés Last In, First Out), es decir, el último elemento en entrar es el primero en salir.

2.3.1 Tipo de dato abstracto Pila

La pila tiene dos operaciones básicas:

- **Apilar (push):** añade un elemento a la pila.
- **Desapilar (pop):** elimina el último elemento apilado.
- **Top (top):** permite ver el elemento que está en la cima de la pila sin desapilarlo.

	Before Push	After Push 3	After Push 5	After Pop	After Pop
1					
2					
3			+---+		
4			5		
5		+---+	+---+	+---+	
6	+---+	3	3	3	+---+
7	+---+	+---+	+---+	+---+	+---+

```

1 public interface IStack {
2     void push(int element);
3     int pop();
4     int top();
5 }

```

Al igual que en el caso de las listas, se pueden implementar pilas con arreglos o con listas enlazadas.

2.3.2 Implementación de pilas con arreglos

En la implementación de pilas con arreglos, se utiliza un arreglo unidimensional para almacenar los elementos de la pila.

```
1 public class ArrayStack implements IStack {
2     private int[] stack;
3     private int top;
4     private int size;
5
6     public ArrayStack(int size) {
7         this.size = size;
8         stack = new int[size];
9         top = -1;
10    }
11
12    public void push(int element) {
13        if (top == size - 1) {
14            System.out.println("Stack Overflow");
15        } else {
16            stack[++top] = element;
17        }
18    }
19
20    public int pop() {
21        if (top == -1) {
22            System.out.println("Stack Underflow");
23            return -1;
24        } else {
25            return stack[top--];
26        }
27    }
28
29    public int top() {
30        if (top == -1) {
31            System.out.println("Stack Underflow");
32            return -1;
33        } else {
34            return stack[top];
35        }
36    }
37 }
```

2.3.3 Implementación de pilas con listas enlazadas

```
1 public class LinkedListStack implements IStack {
2     private Node top;
3
4     public void push(int element) {
5         Node newNode = new Node(element);
6         newNode.next = top;
7         top = newNode;
8     }
9 }
```



```
9
10 public int pop() {
11     if (top == null) {
12         System.out.println("Stack Underflow");
13         return -1;
14     } else {
15         int element = top.data;
16         top = top.next;
17         return element;
18     }
19 }
20
21 public int top() {
22     if (top == null) {
23         // Preguntar a los estudiantes, que es mejor, este enfoque
24         // o
25         // lanzar excepciones como se vio en ejemplos pasados
26         System.out.println("Stack Underflow");
27         return -1;
28     } else {
29         return top.data;
30     }
31 }
32 // Otro enfoque para que la clase nodo solo pueda usarse dentro de
33 // la clase Stack
34 private class Node {
35     int data;
36     Node next;
37
38     public Node(int data) {
39         this.data = data;
40     }
41 }
42 }
```

2.4 Colas

Es una estructura de datos que sigue el principio de FIFO (del inglés First In, First Out), es decir, el primer elemento en entrar es el primero en salir. Respeta el orden de llegada de los elementos.

2.4.1 Tipo de dato abstracto Cola

```
1 public interface IQueue {
2     void enqueue(int element);
3     int dequeue();
4     int front();
5 }
```

```
5 }
```

Cola vacía:

```
1 Frente --> Cola Vacía <-- Final
```

Agregando elementos 1, 2, 3, 4 y 5:

```
1 Frente --> [ 1 ] --> [ 2 ] --> [ 3 ] --> [ 4 ] --> [ 5 ] <-- Final
```

Quitando un elemento:

```
1 Frente --> [ 2 ] --> [ 3 ] --> [ 4 ] --> [ 5 ] <-- Final
```

Agregando el elemento 6:

```
1 Frente --> [ 2 ] --> [ 3 ] --> [ 4 ] --> [ 5 ] --> [ 6 ] <-- Final
```

Quitando dos elementos:

```
1 Frente --> [ 4 ] --> [ 5 ] --> [ 6 ] <-- Final
```

2.4.2 Implementación de colas con arreglos

```
1 public class ArrayQueue implements IQueue {
2     private int[] queue;
3     private int front;
4     private int rear;
5     private int size;
6
7     public ArrayQueue(int size) {
8         this.size = size;
9         queue = new int[size];
10        front = -1;
11        rear = -1;
12    }
13
14    public void enqueue(int element) {
15        if (rear == size - 1) {
16            System.out.println("Queue Overflow");
17        } else {
18            if (front == -1) {
19                front = 0;
20            }
21            queue[++rear] = element;
22        }
23    }
24 }
```

```
25     public int dequeue() {
26         if (front == -1) {
27             System.out.println("Queue Underflow");
28             return -1;
29         } else {
30             int element = queue[front];
31             if (front == rear) {
32                 front = -1;
33                 rear = -1;
34             } else {
35                 front++;
36             }
37             return element;
38         }
39     }
40
41     public int front() {
42         if (front == -1) {
43             System.out.println("Queue Underflow");
44             return -1;
45         } else {
46             return queue[front];
47         }
48     }
49 }
```

Esta implementación tiene un problema conocido como “drifting” que ocurre cuando se hacen muchas operaciones de inserción y eliminación. La cola se desplaza hacia la izquierda y se desperdicia espacio. Para solucionar este problema se puede implementar una cola circular.

```
1  public class CircularArrayQueue implements IQueue {
2      private int[] queue;
3      private int front;
4      private int rear;
5      private int size;
6
7      public CircularArrayQueue(int size) {
8          this.size = size;
9          queue = new int[size];
10         front = -1;
11         rear = -1;
12     }
13
14     public void enqueue(int element) {
15         if ((rear + 1) % size == front) {
16             System.out.println("Queue Overflow");
17         } else {
18             if (front == -1) {
19                 front = 0;
20             }
```

```
21         rear = (rear + 1) % size;
22         queue[rear] = element;
23     }
24 }
25
26 public int dequeue() {
27     if (front == -1) {
28         System.out.println("Queue Underflow");
29         return -1;
30     } else {
31         int element = queue[front];
32         if (front == rear) {
33             front = -1;
34             rear = -1;
35         } else {
36             front = (front + 1) % size;
37         }
38         return element;
39     }
40 }
41
42 public int front() {
43     if (front == -1) {
44         System.out.println("Queue Underflow");
45         return -1;
46     } else {
47         return queue[front];
48     }
49 }
50 }
```

Con una cola circular, el frente y el final no necesariamente están al principio y al final del arreglo, respectivamente. En lugar de eso, el frente y el final se mueven a lo largo del arreglo. Cuando el final llega al final del arreglo, se mueve al principio del arreglo.

2.4.3 Implementación de colas con listas enlazadas

```
1 public class LinkedListQueue {
2     private Node front;
3     private Node rear;
4
5     public LinkedListQueue() {
6         front = null;
7         rear = null;
8     }
9
10    public void enqueue(int element) {
11        Node newNode = new Node(element);
```

```
12     if (rear == null) {
13         front = newNode;
14         rear = newNode;
15     } else {
16         rear.next = newNode;
17         rear = newNode;
18     }
19 }
20
21 public int dequeue() {
22     if (front == null) {
23         System.out.println("Queue Underflow");
24         return -1;
25     } else {
26         int element = front.data;
27         front = front.next;
28         if (front == null) {
29             rear = null;
30         }
31         return element;
32     }
33 }
34
35 public int front() {
36     if (front == null) {
37         System.out.println("Queue Underflow");
38         return -1;
39     } else {
40         return front.data;
41     }
42 }
43
44 private class Node {
45     private int data;
46     private Node next;
47
48     public Node(int data) {
49         this.data = data;
50         this.next = null;
51     }
52 }
53 }
```

2.5 Colas de prioridad

Una cola de prioridad es una estructura de datos que almacena elementos en una cola, pero en lugar de seguir un orden de llegada, los elementos se organizan de acuerdo a su prioridad. Los elementos con mayor prioridad se desencolan antes que los elementos con menor prioridad.

2.5.1 Tipo de dato abstracto Cola de prioridad

```
1 public interface IPriorityQueue {  
2     void enqueue(int element, int priority);  
3     int dequeue();  
4     int front();  
5 }
```

- **Encolar (enqueue):** Cuando un elemento se añade a la cola, se mantiene el orden de acuerdo a su prioridad, reorganizando los elementos si es necesario.
- **Desencolar (dequeue):** Se elimina el elemento con mayor prioridad primero.
- **Tope (front):** Permite ver el elemento con mayor prioridad sin desencolarlo.

2.5.2 Tipos de colas de prioridad

- Orden ascendente: el elemento con menor valor numérico tiene mayor prioridad.
- Orden descendente: el elemento con mayor valor numérico tiene mayor prioridad.

2.5.3 Implementación de colas de prioridad con listas enlazadas

Las colas de prioridad se pueden implementar de muchas maneras:

- Con arreglos
- Con listas enlazadas
- Con montículos (heaps)

Más adelante en el curso, veremos la implementación con heap. Por ahora, vamos a ver una implementación con listas enlazadas.

```
1 public class LinkedListPriorityQueue implements IPriorityQueue {  
2     private Node front;  
3     private Node rear;  
4  
5     public LinkedListPriorityQueue() {  
6         front = null;  
7         rear = null;  
8     }  
9  
10    public void enqueue(int element, int priority) {  
11        Node newNode = new Node(element, priority);  
12        if (front == null) {  
13            front = newNode;  
14            rear = newNode;  
15        } else if (priority < front.priority) {
```

```
16         newNode.next = front;
17         front = newNode;
18     } else {
19         Node current = front;
20         while (current.next != null && current.next.priority <=
21             priority) {
22             current = current.next;
23         }
24         newNode.next = current.next;
25         current.next = newNode;
26         if (newNode.next == null) {
27             rear = newNode;
28         }
29     }
30
31     public int dequeue() {
32         if (front == null) {
33             System.out.println("Queue Underflow");
34             return -1;
35         } else {
36             int element = front.element;
37             front = front.next;
38             return element;
39         }
40     }
41
42     public int front() {
43         if (front == null) {
44             System.out.println("Queue Underflow");
45             return -1;
46         } else {
47             return front.element;
48         }
49     }
50 }
```

2.6 Generics

Generics es aplicable a cualquier estructura de datos, no solo a las lineales.

Hasta ahora, los ejemplos vistos de las estructuras de datos, han sido únicamente para el tipo de dato `integer`. ¿Qué pasa si queremos implementar una estructura de datos para otro tipo de dato? Por ejemplo, si queremos implementar una pila para `String` o para `double`. En este caso, tendríamos que implementar una pila para cada tipo de dato que necesitemos. Esto no es eficiente y no es escalable.

Otra opción es utilizar el tipo de dato `Object` que es la superclase de todos los tipos de datos en Java/C#. Sin embargo, esto no es una solución óptima ya que se pierde el tipo de dato específico y se

tendría que hacer un *casting* cada vez que se quiera utilizar el dato. Esto puede resultar en errores en tiempo de ejecución.

Java y muchos otros lenguajes, proveen el concepto de **generics** para solucionar este problema. Los **generics** permiten definir clases, interfaces y métodos con un tipo de dato que se especifica en el momento de la creación de la instancia.

```
1 public class LinkedList<T> {
2     private Node<T> head;
3
4     private static class Node<T> {
5         private T data;
6         private Node<T> next;
7
8         public Node(T data) {
9             this.data = data;
10            this.next = null;
11        }
12    }
13
14    public void add(T data) {
15        Node<T> newNode = new Node<>(data);
16        if (head == null) {
17            head = newNode;
18        } else {
19            Node<T> current = head;
20            while (current.next != null) {
21                current = current.next;
22            }
23            current.next = newNode;
24        }
25    }
26
27    // Other methods for LinkedList implementation...
28 }
29
30 /// Ejemplo de instanciación
31
32 public static void main(String[] args) {
33     LinkedList<String> list = new LinkedList<>();
34     list.add("Hello");
35     list.add("World");
36     list.add("Java");
37
38     LinkedList<Integer> numbers = new LinkedList<>();
39     numbers.add(1);
40     numbers.add(2);
41
42     LinkedList<Double> decimals = new LinkedList<>();
43     decimals.add(3.14);
```



```
44     decimals.add(2.71);  
45 }
```

Una consideración importante al utilizar generics es al comparar los elementos en la estructura. Se puede aprovechar la interfaz `Comparable` de Java para forzar que los elementos de tipo `T` sean comparables entre sí.

```
1 public class LinkedList<T extends Comparable<T>> {  
2     // ...  
3  
4     private class Node<T extends Comparable<T>> {  
5         public int compareTo(T other) {  
6             return this.data.compareTo(other);  
7         }  
8     }  
9 }
```

Para cualquier tipo de datos *personalizado*, se debe implementar la interfaz `Comparable` y definir las reglas de comparación que tengan sentido dentro del contexto de la aplicación.

- Clase `Persona`: comparar por cédula
- Clase `Estudiante`: comparar por número de carné

2.7 Referencias adicionales

- <https://www.geeksforgeeks.org/introduction-to-arrays-data-structure-and-algorithm-tutorials/>

3 Estructuras de datos jerárquicas

Son estructuras de datos que permiten organizar los datos de manera jerárquica, es decir, en forma de árbol. Los datos se organizan de manera que exista un nodo raíz y cada nodo puede tener cero o más nodos hijos. Cada nodo hijo puede tener a su vez cero o más nodos hijos, y así sucesivamente.

Existen muchos tipos de árboles, cada uno con sus propias características y aplicaciones. Algunos de los árboles más comunes son:

- **Árboles binarios:** cada nodo tiene a lo sumo dos hijos.
 - **Árboles binarios de búsqueda:** Es un árbol binario con ciertas reglas que permiten realizar búsquedas eficientes.
 - **Árboles AVL:** Es un árbol binario de búsqueda balanceado.
 - **Árboles splay:** Es un árbol binario de búsqueda que reorganiza los nodos para que los nodos más utilizados estén cerca de la raíz.
 - **Árboles de expresiones:** Es un árbol que se utiliza para representar expresiones matemáticas.
 - **Árboles rojinegros:** Es un árbol binario de búsqueda balanceado.
 - **Heap:** Es un árbol binario que cumple con ciertas reglas y se utiliza para implementar colas de prioridad.
- **Arboles N-arios:** Cada nodo puede tener un número variable de hijos.
 - **Árboles B:** Es un árbol que permite almacenar grandes cantidades de datos en disco.
 - **Árboles B+:** Es una variante del árbol B que se utiliza en bases de datos y sistemas de archivos.
 - **Árboles trie:** Es un árbol que se utiliza para almacenar un conjunto de cadenas de caracteres.
 - **Árboles de sufijos:** Es un árbol que se utiliza para almacenar todas las subcadenas de una cadena de caracteres.

A excepción de algunos árboles que puede implementarse con arreglos, la mayoría de los árboles se implementan con nodos enlazados, de forma similar a las *Listas*.

3.1 Terminología básica

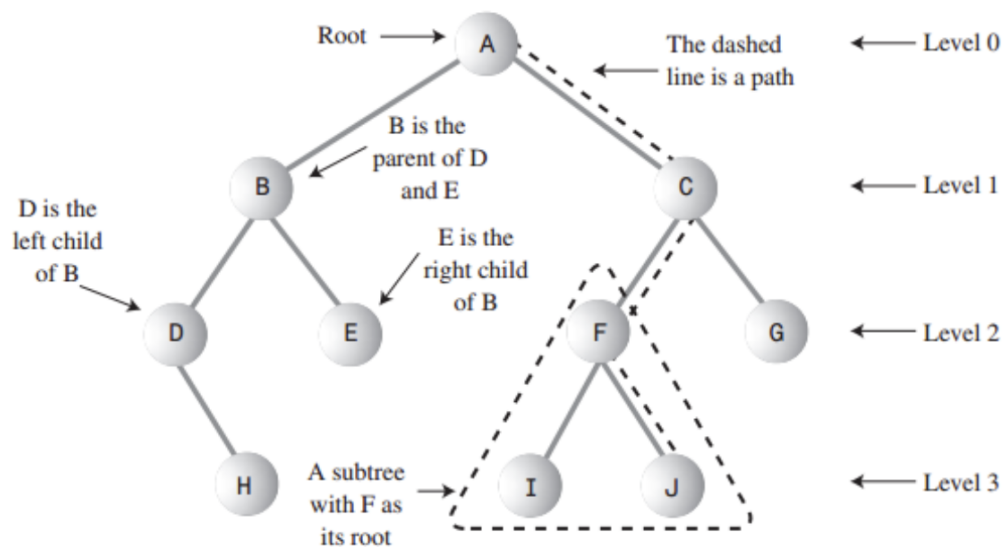


Figura 10: Árbol binario

3.2 Tipo de dato abstracto Árbol

Los árboles tienen las siguientes operaciones básicas:

- **Insertar (insert):** añade un nodo al árbol.
- **Eliminar (delete):** elimina un nodo del árbol.
- **Buscar (search):** busca un nodo en el árbol.
- **Recorrer (traverse):** recorre todos los nodos del árbol. Se puede realizar en distintos órdenes:
 - **Preorden:** primero se visita la raíz, luego el subárbol izquierdo y finalmente el subárbol derecho.
 - **Inorden:** primero se visita el subárbol izquierdo, luego la raíz y finalmente el subárbol derecho.
 - **Postorden:** primero se visita el subárbol izquierdo, luego el subárbol derecho y finalmente la raíz.
 - **Por niveles:** se recorren todos los nodos de un nivel antes de pasar al siguiente nivel.

3.3 Árboles Binarios de Búsqueda (BST)

Un BST es un árbol binario que cumple con la siguiente propiedad: para cada nodo, todos los nodos del subárbol izquierdo tienen un valor menor que el nodo y todos los nodos del subárbol derecho tienen un valor mayor que el nodo.

Los BST se pueden implementar mediante arrays o nodos enlazados. En este curso, BST se verán como nodos enlazados. Los *árboles Heap* que veremos más adelante, se implementarán con arrays.

3.3.1 Implementación

La estructura básica de un BST se puede implementar de la siguiente forma:

```
1 class TreeNode {
2     int key;
3     TreeNode left, right;
4
5     public TreeNode(int item) {
6         key = item;
7         left = right = null;
8     }
9 }
10
11 public class BinarySearchTree {
12     TreeNode root;
13
14     public BinarySearchTree() {
15         root = null;
16     }
17 }
```

3.3.2 Búsqueda de elementos

```
1     public boolean search(int key) {
2         return searchRecursive(root, key);
3     }
4
5     private boolean searchRecursive(TreeNode root, int key) {
6         if (root == null) {
7             return false;
8         }
9         if (root.key == key) {
10            return true;
11        }
12        else if (key > root.key) {
13            return searchRecursive(root.right, key);
14        }
```

```
14         } else {  
15             return searchRecursive(root.left, key);  
16         }  
17     }
```

Dado que los árboles son una estructura jerárquica, la búsqueda se puede realizar de forma recursiva (preferida por simplicidad). De tal forma, la mayoría de los métodos van en parejas: *uno público que recibe los parámetros iniciales y otro privado que realiza la operación recursiva*.

3.3.3 Inserción de elementos

La inserción usa el mismo principio de búsqueda para encontrar el lugar donde se debe insertar el nuevo nodo. **Si el nodo ya existe, no se inserta.**

```
1     public void insert(int key) {  
2         root = insertRecursive(root, key);  
3     }  
4  
5     private TreeNode insertRecursive(TreeNode root, int key) {  
6         if (root == null) {  
7             root = new TreeNode(key);  
8             return root;  
9         }  
10  
11         if (key < root.key)  
12             root.left = insertRecursive(root.left, key);  
13         else if (key > root.key)  
14             root.right = insertRecursive(root.right, key);  
15  
16         return root;  
17     }
```

Note que el método insert re-assigna el valor de root al resultado de insertRecursive. Esto es necesario para que los cambios hechos en el árbol se reflejen en la raíz. Un error de principiante es creer que el siguiente código funciona:

```
1     public void insert(int key) {  
2         insertRecursive(root, key);  
3     }  
4  
5     private void insertRecursive(TreeNode root, int key) {  
6         if (root == null) {  
7             root = new TreeNode(key);  
8             return;  
9         }  
10  
11         if (key < root.key)
```

```
12         insertRecursive(root.left, key);
13     else if (key > root.key)
14         insertRecursive(root.right, key);
15 }
```

No funciona puesto que **las referencias en Java se pasan por valor**. Esto significa que el valor de root no cambia en el método insert, por lo que el árbol no se modifica.

3.3.4 Eliminación de elementos

La eliminación en un BST considera varios casos:

- El nodo a eliminar es una hoja.
- El nodo a eliminar tiene un solo hijo.
- El nodo a eliminar tiene dos hijos.

Cuando el nodo es hoja, simplemente se elimina. **Cuando el nodo tiene un solo hijo**, se reemplaza el nodo por su hijo. **Cuando el nodo tiene dos hijos**, se reemplaza el nodo por el nodo más pequeño del subárbol derecho o por el nodo mayor del subárbol izquierdo (**solo se puede usar una de estas estrategias** en toda la implementación)

```
1     public void delete(int key) {
2         root = deleteRecursive(root, key);
3     }
4
5     private TreeNode deleteRecursive(TreeNode root, int key) {
6         if (root == null)
7             return root;
8
9         if (key < root.key)
10            root.left = deleteRecursive(root.left, key);
11        else if (key > root.key)
12            root.right = deleteRecursive(root.right, key);
13        else {
14            if (root.left == null)
15                return root.right;
16            else if (root.right == null)
17                return root.left;
18
19            root.key = minValue(root.right);
20            root.right = deleteRecursive(root.right, root.key);
21        }
22        return root;
23    }
24
25    private int minValue(TreeNode root) {
```

```
26     int minValue = root.key;
27     while (root.left != null) {
28         minValue = root.left.key;
29         root = root.left;
30     }
31     return minValue;
32 }
```

3.3.5 Recorridos

Recorrer un árbol no solo es útil para imprimirlo, sino que también es útil para realizar operaciones en todos los nodos. Los recorridos más comunes son:

- Inorden (izquierda, raíz, derecha).
- Preorden (raíz, izquierda, derecha).
- Postorden (izquierda, derecha, raíz).

```
1     public void inOrder() {
2         inOrderRecursive(root);
3     }
4
5     private void inOrderRecursive(TreeNode root) {
6         if (root != null) {
7             inOrderRecursive(root.left);
8             System.out.print(root.key + " ");
9             inOrderRecursive(root.right);
10        }
11    }
12
13    public void preOrder() {
14        preOrderRecursive(root);
15    }
16
17    private void preOrderRecursive(TreeNode root) {
18        if (root != null) {
19            System.out.print(root.key + " ");
20            preOrderRecursive(root.left);
21            preOrderRecursive(root.right);
22        }
23    }
24
25    public void postOrder() {
26        postOrderRecursive(root);
27    }
28
29    private void postOrderRecursive(TreeNode root) {
30        if (root != null) {
31            postOrderRecursive(root.left);
```

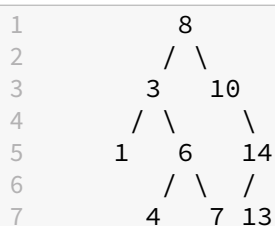
```

32         postOrderRecursive(root.right);
33         System.out.print(root.key + " ");
34     }
35 }

```

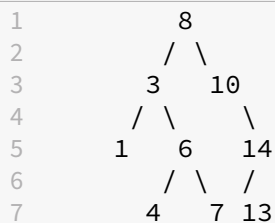
3.3.5.1 In-orden El recorrido in-orden de un árbol BST imprime los nodos en *orden ascendente*. Esto se debe a que el recorrido in-orden visita primero el subárbol izquierdo, luego la raíz y finalmente el subárbol derecho.

Por ejemplo, el siguiente árbol:



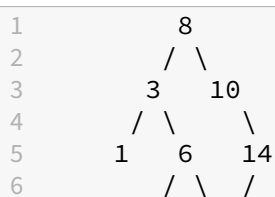
Se imprime como 1 3 4 6 7 8 10 13 14.

3.3.5.2 Pre-orden El recorrido pre-orden de un árbol BST imprime la raíz antes que los subárboles. Esto se debe a que el recorrido pre-orden visita primero la raíz, luego el subárbol izquierdo y finalmente el subárbol derecho.



Se imprime como 8 3 1 6 4 7 10 14 13.

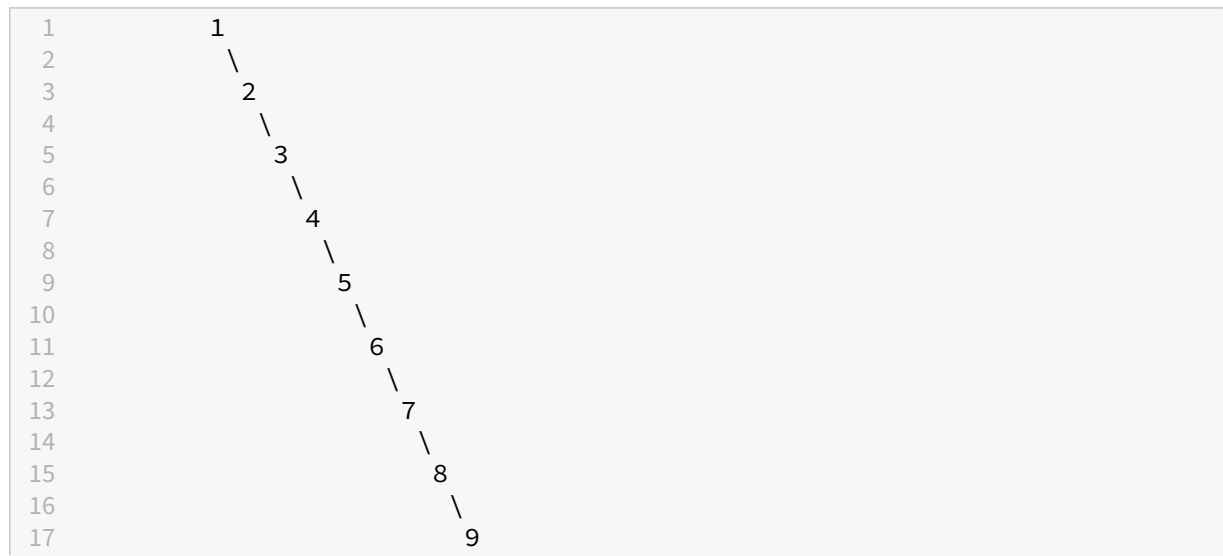
3.3.5.3 Post-orden El recorrido post-orden de un árbol BST imprime los subárboles antes que la raíz. Esto se debe a que el recorrido post-orden visita primero el subárbol izquierdo, luego el subárbol derecho y finalmente la raíz.



7	4	7	13
---	---	---	----

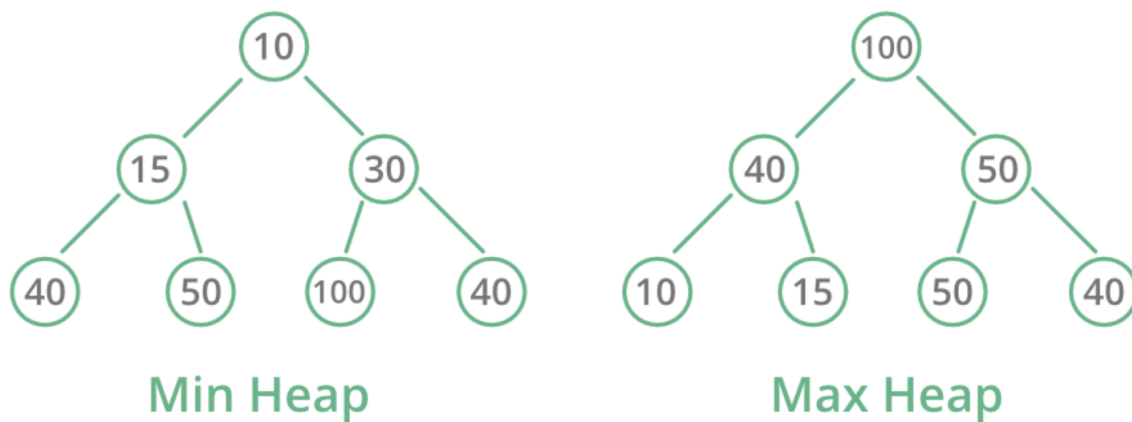
Se imprime como 1 4 7 6 3 13 14 10 8.

- El problema de los BST de no ser balanceados * Los BST pueden degenerar en listas enlazadas si los elementos se insertan en orden. Esto puede ocurrir si los elementos se insertan en orden ascendente o descendente. En este caso, la búsqueda, inserción y eliminación se convierten en operaciones de tiempo lineal.



3.4 Árboles Heap (montículo)

- Un árbol *heap* es un árbol binario completo, es decir, todos los niveles del árbol están completamente llenos, excepto posiblemente el último nivel, que se llena de izquierda a derecha.
- Cumple con la propiedad de *heap*, que establece que el valor de cada nodo es mayor o igual que el valor de sus hijos. Si el valor del nodo es mayor que el de sus hijos, se trata de un árbol **heap máximo**, si el valor del nodo es menor que el de sus hijos, se trata de un árbol **heap mínimo**.
- Visualmente, un árbol *heap* se puede representar de la siguiente forma:

**Figura 11:** Árbol Heap

- Son una opción natural para implementar colas de prioridad.
- Son la estructura esencial para posteriormente implementar *heap sort* (lo veremos más adelante en los algoritmos de ordenamiento).

3.4.1 Tipo de dato abstracto

Un heap tiene las siguientes operaciones:

- **Insertar (insert):** añade un elemento al heap y funciona de la siguiente manera:
 1. Añade el elemento al final del array.
 2. Compara el elemento con su padre y si es mayor, intercambia el elemento con su padre.
 3. Repite el paso 2 hasta que el elemento sea menor que su padre o llegue a la raíz del heap.
- **Eliminar (delete):** elimina el nodo raíz del heap,
- **Obtener (get):** permite ver el nodo raíz del heap sin eliminarlo.

```
1 public interface IHeap {  
2     void insert(int element);  
3     int delete();  
4     int get();  
5 }
```

3.4.2 Implementación de árboles heap

Normalmente se implementan sobre un array. La raíz del árbol se almacena en la posición 0 del array, y los hijos de un nodo en la posición i se almacenan en las posiciones $2 * i + 1$ y $2 * i + 2$.

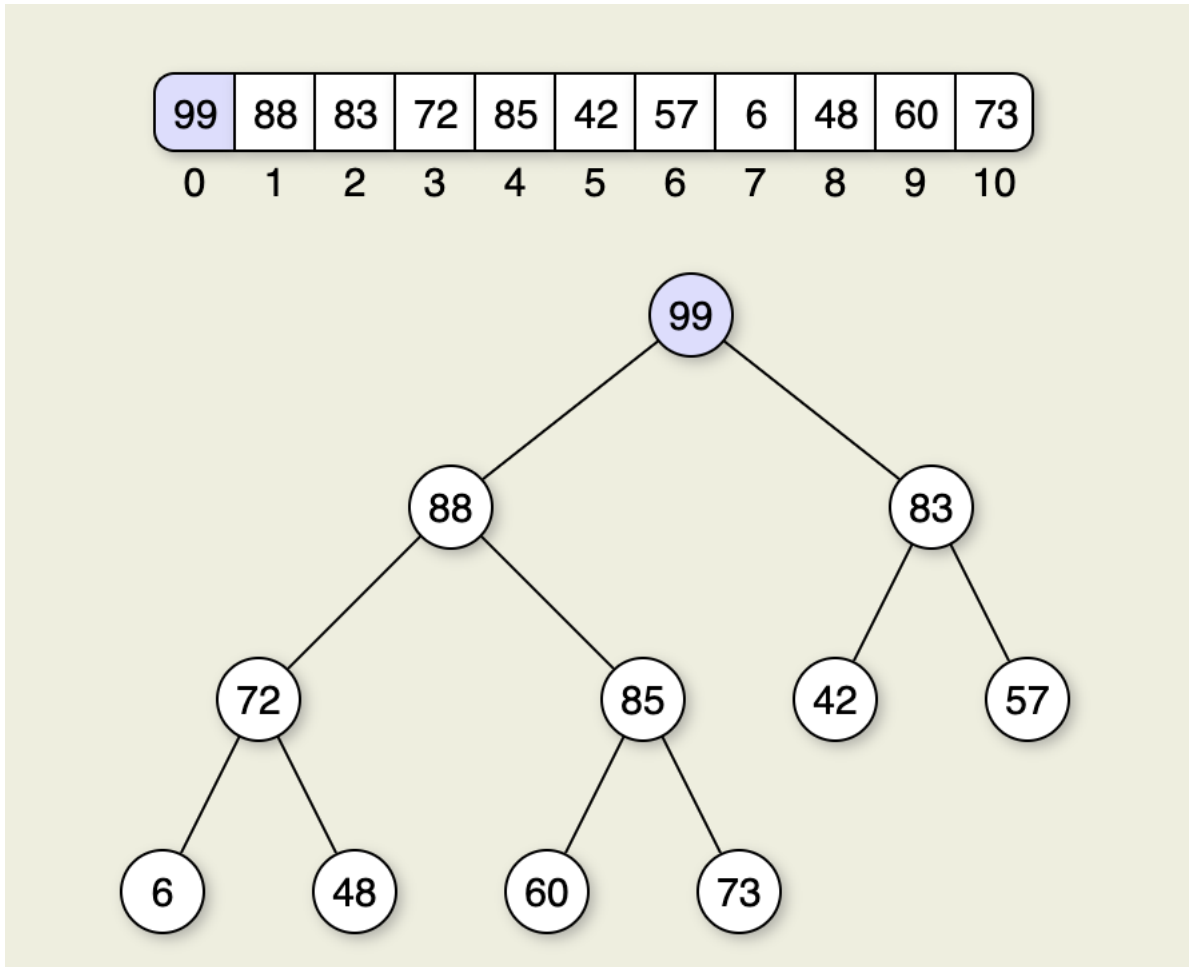


Figura 12: Árbol Heap

```
1 public class HeapTree {  
2     private int[] heap;  
3     private int size;  
4  
5     public HeapTree(int capacity) {  
6         heap = new int[capacity];  
7         size = 0;  
8     }  
9  
10    public void insert(int element) {  
11        if (size == heap.length) {
```

```
12         throw new IllegalStateException("Heap is full");
13     }
14     heap[size] = element;
15     heapifyUp(size);
16     size++;
17 }
18
19 public int delete() {
20     if (size == 0) {
21         throw new IllegalStateException("Heap is empty");
22     }
23
24     int root = heap[0];
25     heap[0] = heap[size - 1];
26     size--;
27     heapifyDown(0);
28
29     return root;
30 }
31
32 public int get() {
33     if (size == 0) {
34         throw new IllegalStateException("Heap is empty");
35     }
36
37     return heap[0];
38 }
39
40 private void heapifyUp(int index) {
41     int parentIndex = (index - 1) / 2;
42
43     while (index > 0 && heap[index] > heap[parentIndex]) {
44         swap(index, parentIndex);
45         index = parentIndex;
46         parentIndex = (index - 1) / 2;
47     }
48 }
49
50 private void heapifyDown(int index) {
51     int leftChildIndex = 2 * index + 1;
52     int rightChildIndex = 2 * index + 2;
53     int largestIndex = index;
54
55     if (leftChildIndex < size && heap[leftChildIndex] > heap[
56         largestIndex]) {
57         largestIndex = leftChildIndex;
58     }
59
60     if (rightChildIndex < size && heap[rightChildIndex] > heap[
61         largestIndex]) {
62         largestIndex = rightChildIndex;
63     }
64
65     if (index != largestIndex) {
66         swap(index, largestIndex);
67         heapifyDown(largestIndex);
68     }
69 }
```

```

61     }
62
63     if (largestIndex != index) {
64         swap(index, largestIndex);
65         heapifyDown(largestIndex);
66     }
67 }
68
69 private void swap(int index1, int index2) {
70     int temp = heap[index1];
71     heap[index1] = heap[index2];
72     heap[index2] = temp;
73 }
74 }

```

3.5 Árboles AVL

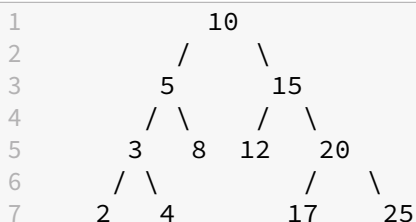
Los árboles AVL son árboles binarios de búsqueda **balanceados**. Fueron los primeros árboles auto-balanceados que se propusieron. Fueron introducidos por Adelson-Velsky y Landis en 1962.

Dado que se mantienen balanceados, las operaciones de búsqueda, inserción y eliminación tienen un tiempo de ejecución garantizado de $O(\log n)$ y no sufren de la degradación de rendimiento que sufren los árboles binarios de búsqueda no balanceados.

3.5.1 Características

- Es un árbol binario de búsqueda.
- Está balanceado en altura.
- El factor de balance de cada nodo es -1, 0 o 1.
- Las eliminaciones e inserciones pueden requerir balanceo a través de rotaciones.

El factor de balance de un nodo es la diferencia entre la altura del subárbol derecho y la altura del subárbol izquierdo.



Factores de balance de cada nodo:

Nodo	Factor de Balance
10	$4 - 4 = 0$
5	$2 - 3 = -1$
15	$3 - 2 = 1$
3	$2 - 2 = 0$
8	$1 - 1 = 0$
12	$1 - 1 = 0$
20	$2 - 2 = 0$
2	$0 - 0 = 0$
4	$0 - 0 = 0$
17	$1 - 1 = 0$
25	$1 - 1 = 0$

3.5.2 Ventajas

- Búsquedas rápidas
- Auto-balanceados

3.5.3 Desventajas

- Requieren más memoria que los árboles binarios de búsqueda no balanceados.
- Las operaciones de inserción y eliminación son más lentas que en los árboles binarios de búsqueda no balanceados.
- Las rotaciones pueden ser costosas.
- Difíciles de implementar.

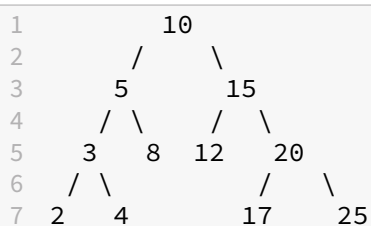
3.5.4 Inserción

Para insertar un nodo w : - Se realiza una inserción normal de un árbol binario de búsqueda para el nodo w . - Iniciando en w , se recorre el camino de búsqueda hacia la raíz. Sea z el primero nodo no balanceado, y el hijo de z que está en el camino de w a z y x el nieto de z que está en el camino de w a z . - Se rebalancea el árbol mediante rotaciones simples o dobles en el subárbol con raíz en z . Hay 4 posibles casos de rotaciones.

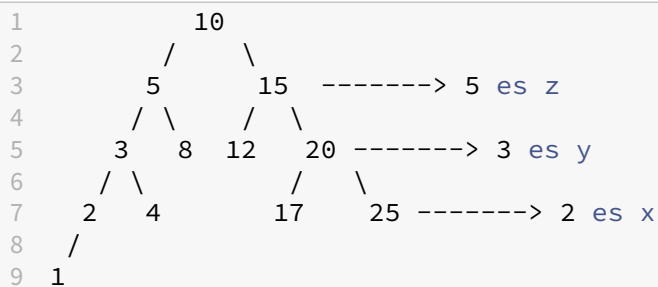
- 1 - ****Rotación derecha****: `_y_` es el hijo izquierdo de `_z_` y `_x_` es el hijo izquierdo de `_y_`.
- 2 - ****Rotación Izquierda-derecha****: `_y_` es el hijo izquierdo de `_z_` y `_x_` es el hijo derecho de `_y_`.
- 3 - ****Rotación Izquierda****: `_y_` es el hijo derecho de `_z_` y `_x_` es el hijo derecho de `_y_`.
- 4 - ****Rotación Derecha-izquierda****: `_y_` es el hijo derecho de `_z_` y `_x_` es el hijo izquierdo de `_y_`.

3.5.4.1 Ejemplo de rotación derecha

Dado el siguiente árbol AVL:



Se inserta el nodo 1:



El factor de balance del nodo 5 es 2, por lo que se debe rebalancear el árbol. El nodo 3 es el hijo de 5 que está en el camino de 1 a 5 y el nodo 2 es el nieto de 5 que está en el camino de 1 a 5. Estamos en el caso de rotación derecha.

Aplicando el siguiente código a z:

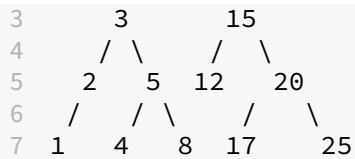
```

1 void rotateRight(Node node) {
2     Node left = node.left;
3     node.left = left.right;
4     left.right = node;
5     node = left;
6 }

```

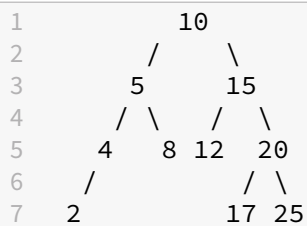
Se obtiene:



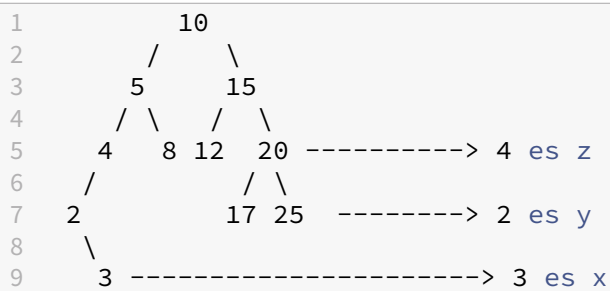


La rotación izquierda es la versión espejo de la rotación derecha.

3.5.4.2 Ejemplo de rotación izquierda-derecha Dado el siguiente árbol AVL:



Se inserta el nodo 3:



El factor de balance del nodo 5 es 2, por lo que se debe rebalancear el árbol. El nodo 4 es el hijo de 5 que está en el camino de 3 a 5 y el nodo 2 es el nieto de 5 que está en el camino de 3 a 5. Estamos en el caso de rotación izquierda-derecha.

Aplicando el siguiente código a z:

```

1 void rotateLeftRight(Node node) {
2     rotateLeft(node.left);
3     rotateRight(node);
4 }
5
6 void rotateLeft(Node node) {
7     Node right = node.right;
8     node.right = right.left;
9     right.left = node;
10    node = right;
11 }
12
13

```

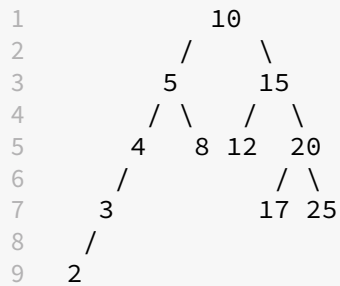


```

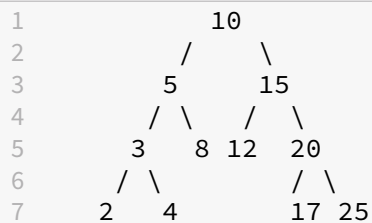
14 void rotateRight(Node node) {
15     Node left = node.left;
16     node.left = left.right;
17     left.right = node;
18     node = left;
19 }

```

Primero, se rota a la izquierda el nodo 2:



Luego, se rota a la derecha el nodo 4:



La rotación derecha-izquierda es la versión espejo de la rotación izquierda-derecha.

3.5.4.3 Implementación de inserción

```

1 class AVLNode {
2     int key, height;
3     AVLNode left, right;
4
5     AVLNode(int key) {
6         this.key = key;
7         this.height = 1;
8         this.left = this.right = null;
9     }
10 }
11
12 public class AVLTree {
13     private AVLNode root;
14
15     // Get height of node
16     private int height(AVLNode node) {
17         if (node == null)
18             return 0;

```

```
19         return node.height;
20     }
21
22     // Get balance factor of node
23     private int getBalance(AVLNode node) {
24         if (node == null)
25             return 0;
26         return height(node.right) - height(node.left);
27     }
28
29     // Right rotate subtree rooted with y
30     private AVLNode rightRotate(AVLNode y) {
31         AVLNode x = y.left;
32         AVLNode T2 = x.right;
33
34         // Perform rotation
35         x.right = y;
36         y.left = T2;
37
38         // Update heights
39         y.height = Math.max(height(y.left), height(y.right)) + 1;
40         x.height = Math.max(height(x.left), height(x.right)) + 1;
41
42         return x;
43     }
44
45     // Left rotate subtree rooted with x
46     private AVLNode leftRotate(AVLNode x) {
47         AVLNode y = x.right;
48         AVLNode T2 = y.left;
49
50         // Perform rotation
51         y.left = x;
52         x.right = T2;
53
54         // Update heights
55         x.height = Math.max(height(x.left), height(x.right)) + 1;
56         y.height = Math.max(height(y.left), height(y.right)) + 1;
57
58         return y;
59     }
60
61     // Insert a key into the tree
62     public void insert(int key) {
63         root = insertRecursive(root, key);
64     }
65
66     // Recursive function to insert a key into the tree
67     private AVLNode insertRecursive(AVLNode node, int key) {
68         // Perform normal BST insertion
69         if (node == null)
```

```
70         return new AVLNode(key);
71
72     if (key < node.key)
73         node.left = insertRecursive(node.left, key);
74     else if (key > node.key)
75         node.right = insertRecursive(node.right, key);
76     else // Duplicate keys not allowed
77         return node;
78
79     // Update height of this ancestor node
80     node.height = 1 + Math.max(height(node.left), height(node.right
81         ));
82
83     // Get the balance factor of this ancestor node
84     int balance = getBalance(node);
85
86     // If node becomes unbalanced, perform rotations
87     // Left Left Case
88     if (balance > 1 && key < node.left.key)
89         return rightRotate(node);
90
91     // Right Right Case
92     if (balance < -1 && key > node.right.key)
93         return leftRotate(node);
94
95     // Left Right Case
96     if (balance > 1 && key > node.left.key) {
97         node.left = leftRotate(node.left);
98         return rightRotate(node);
99     }
100
101     // Right Left Case
102     if (balance < -1 && key < node.right.key) {
103         node.right = rightRotate(node.right);
104         return leftRotate(node);
105     }
106
107     return node;
108 }
```

3.5.5 Eliminación

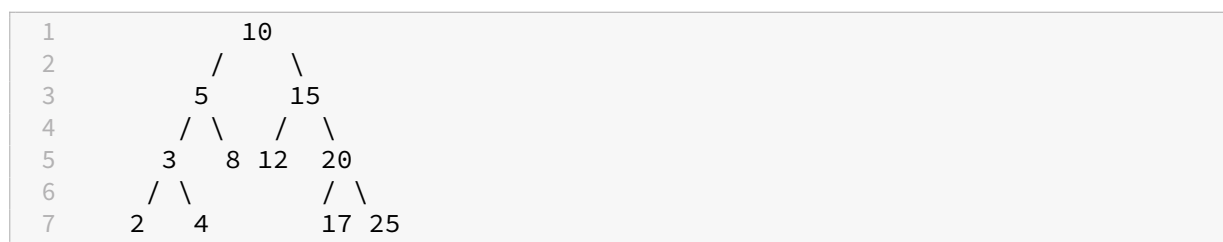
Para eliminar en un árbol AVL:

- Se realiza una eliminación normal de un árbol binario de búsqueda.
- Comenzando desde w, avanza hacia arriba y encuentra el primer nodo desequilibrado. Sea z el primer nodo desequilibrado, y el hijo de mayor altura de z, y x el hijo de mayor altura de y. Ten en cuenta que las definiciones de x e y son diferentes a las de la inserción aquí.

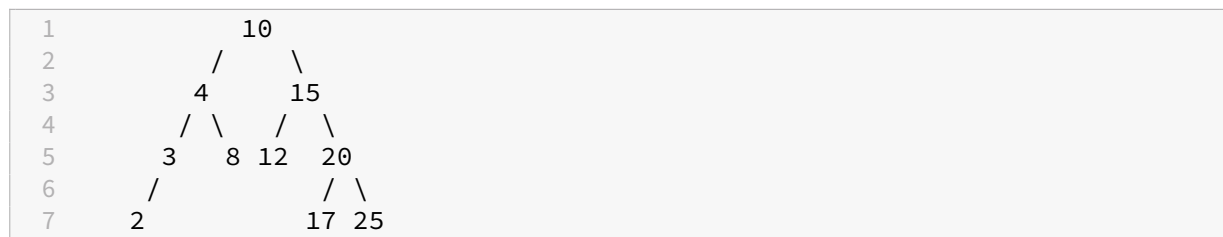
- Rebalancea el árbol realizando rotaciones apropiadas en el subárbol con raíz en z. Puede haber 4 casos posibles que deben ser manejados, ya que x, y y z pueden estar dispuestos de 4 formas diferentes. A continuación se presentan las 4 disposiciones posibles:

- **Rotación derecha:** y es el hijo izquierdo de z y x es el hijo izquierdo de y.
- **Rotación Izquierda-derecha:** y es el hijo izquierdo de z y x es el hijo derecho de y.
- **Rotación Izquierda:** y es el hijo derecho de z y x es el hijo derecho de y.
- **Rotación Derecha-izquierda:** y es el hijo derecho de z y x es el hijo izquierdo de y.

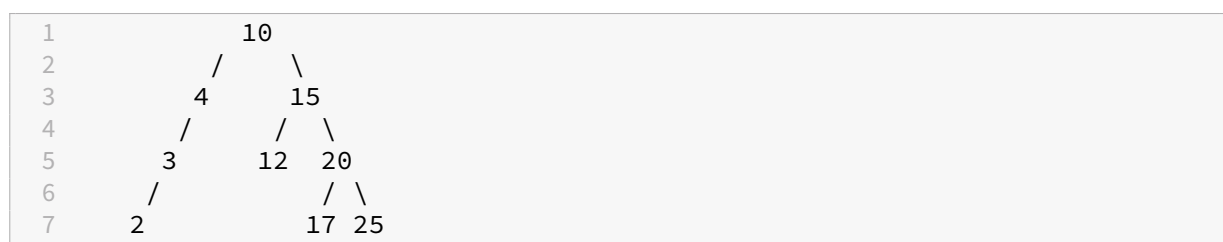
Dado el siguiente árbol, eliminemos 5:



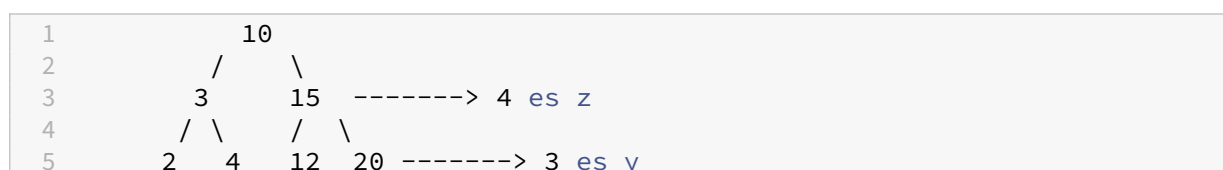
Se elimina el nodo 5 usando la lógica de eliminación de un árbol binario de búsqueda:



En este punto, no se cumple la propiedad de AVL. Se elimina 8:

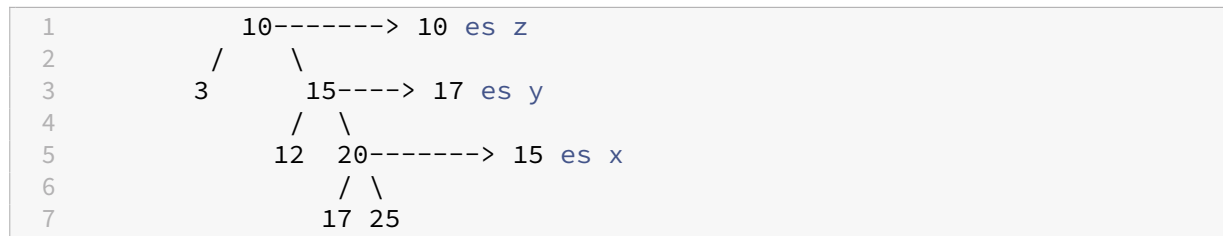


Se puede notar que 4 tiene un factor de balance -2, por lo que debe rebalancear usando rotación derecha en 4:

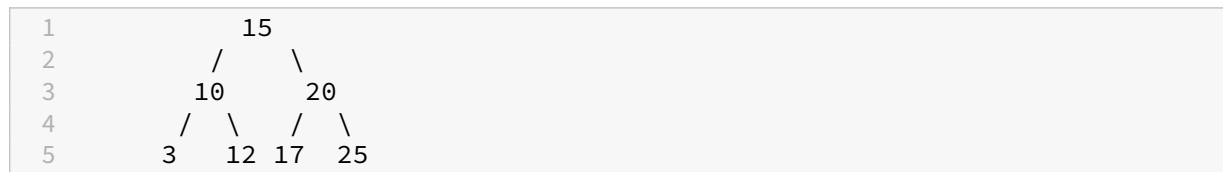




Se eliminan 2 y 4:



Estos nos dejan con un factor de balance en 10 de 2, por lo que se debe rebalancear usando rotación izquierda en 10:



3.5.6 Búsquedas y recorridos

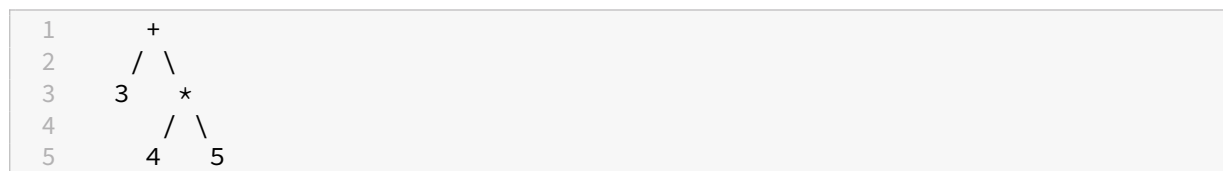
No hay cambios con respecto a BST

3.6 Árboles de expresión

Aunque no son técnicamente un TDA distinto, es relevante mencionarlos dado que su uso es muy común en la resolución de problemas de programación.

Son árboles binarios en los que las hojas son operandos y los nodos internos son operadores. Se utilizan para representar (y resolver) expresiones aritméticas o expresiones sintácticas de un lenguaje de programación en la etapa de compilación.

Por ejemplo, la expresión matemática $3 + (4 * 5)$ se puede representar con el siguiente árbol de expresión:



3.6.1 Conversión de árboles de expresión a notación infija

La notación infija es la forma tradicional de escribir expresiones matemáticas, en la que los operadores se escriben entre los operandos. Para generar la expresión, únicamente se requiere recorrer el árbol en inorden (izquierda, raíz, derecha).

```

1 private void inOrderRecursive(TreeNode root) {
2     if (root != null) {
3         inOrderRecursive(root.left);
4         System.out.print(root.key + " ");
5         inOrderRecursive(root.right);
6     }
7 }

```

Árbol	Expresión
	$(x + y) * (a - b)$
	$(x * (y - z)) * (a - f)$
	$(x * (y / -z))$
	$(A + (B * - (C + D)))$
	$((A * (X + Y)) * C)$

3.6.2 Conversión de expresión a árbol de expresión

Para convertir una expresión infija a un árbol de expresión, primero se debe convertir la expresión a notación postfija (postfix) y luego se debe recorrer la expresión postfija para construir el árbol.

3.6.3 Convertir expresión infija a postfija

Para convertir una expresión infija a postfija, se puede utilizar el algoritmo de Shunting Yard. Este algoritmo fue desarrollado por Edsger Dijkstra en 1961 y permite convertir una expresión infija a postfija en tiempo lineal.

El algoritmo utiliza dos estructuras de datos: una pila para almacenar los operadores y una cola para almacenar los operandos. El algoritmo recorre la expresión infija de izquierda a derecha y, dependiendo del tipo de token que se encuentre, realiza una de las siguientes acciones:

- Si el token es un operando, se añade a la cola.

- Si el token es un operador, se añade a la pila. Antes de añadirlo, se verifica si hay operadores en la pila con mayor precedencia. Si es así, se desapilan y se añaden a la cola.

Por ejemplo,

La cola (que contiene la expresión en postfijo) se utiliza como input para generar el árbol de expresión.

3.7 Árboles B

Los árboles B son una variante de los árboles N-arios que se utilizan para almacenar grandes cantidades de datos en disco. Los árboles B son muy utilizados en bases de datos y sistemas de archivos.

Descritos por Rudolf Bayer y Edward M. McCreight en 1972, los árboles B son árboles balanceados que se caracterizan por tener un número variable de hijos por nodo. Los árboles B son muy similares a los árboles binarios de búsqueda, pero con la diferencia de que cada nodo puede tener más de dos hijos.

Visualmente se pueden representar de la siguiente forma:

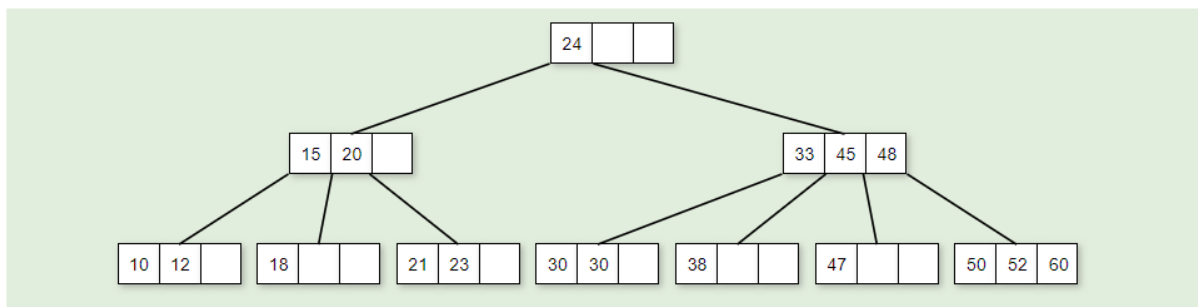


Figura 13: Árbol B

3.7.1 Características

Un árbol B de orden m , tiene las siguientes características:

- Cada nodo se compone de llaves y ramas. Las llaves están ordenadas y dividen las ramas en el orden esperado. Por ejemplo, entre las llaves 15 y 20, hay un nodo hijo, cuyas llaves serán mayores a 15 pero menores a 20.
- Todas las hojas están al mismo nivel.
- Se define con un orden m , que depende del tamaño del bloque del disco
- Cada nodo excepto la raíz **debe** contener al menos $m/2$ llaves. La raíz contiene un mínimo de una llave.

- La inserción siempre ocurre en las hojas.
- Crecen o decrecen desde la raíz.

Pueden implementarse en memoria principal o en secundaria (propósito original). En este curso, nos enfocaremos en la implementación en memoria principal.

3.7.2 Estructura básica en Java

```
1 class BTreeNode {
2     int order;
3     int[] keys;
4     int keyCount;
5     BTreeNode[] branches;
6 }
```

3.7.3 Búsqueda

Es muy similar a la búsqueda de un elemento en un BST. Los pasos se pueden resumir de la siguiente manera para buscar una llave k :

1. Iniciando en la raíz, se compara k con las llaves del nodo. Si k se encuentra, se retorna el nodo o **true**.
2. Al ir comparando a k con cada una de las llaves, si se encuentra alguna llave mayor a k (o se llega al final de las llaves, o sea k es mayor que todas), y hay una rama para dicha llave, se desciende por esa rama y se repite el proceso.
3. Si estamos en una rama, y no se encuentra k en las llaves, se return **null** o **false**

```
1 private Node Search(Node x, int key) {
2     int i = 0;
3     if (x == null)
4         return x;
5     for (i = 0; i < x.n; i++) {
6         if (key < x.key[i]) {
7             break;
8         }
9         if (key == x.key[i]) {
10            return x;
11        }
12    }
13    if (x.leaf) {
14        return null;
15    } else {
16        return Search(x.child[i], key);
17    }
```


18 }

3.7.4 Inserción

El árbol B crece hacia arriba desde la raíz. La inserción siempre ocurre en las hojas.

El proceso de inserción para una llave k sería:

- Se busca la llave en la raíz, entre las llaves del nodo.
- Dado que las llaves del nodo están ordenadas, se detiene si hay una llave mayor. Si tiene un hijo, se desciende a él y se repite el proceso.
- Si el nodo no tiene hijos:
 - Si el nodo no está lleno, se inserta en la posición correspondiente del arreglo de llaves
 - Si el nodo está lleno, se divide el nodo.

El proceso de división de un nodo n con $m-1$ llaves es el siguiente:

1. Se selecciona la llave mediana m y se sube al nodo padre.
2. Se crean dos nuevos nodos, $n1$ y $n2$.
3. Se distribuyen las llaves de n entre $n1$ y $n2$.
4. Se actualizan las ramas de $n1$ y $n2$.

El proceso de división se repite hasta llegar a la raíz.

Graficamente se puede ver de la siguiente manera (orden 5):

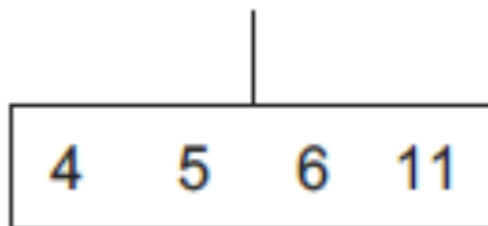
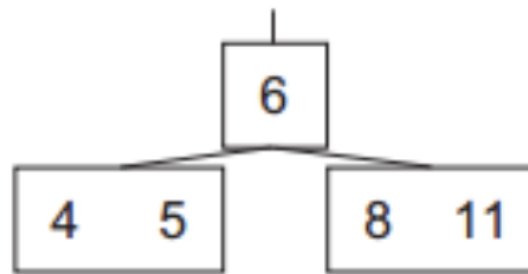
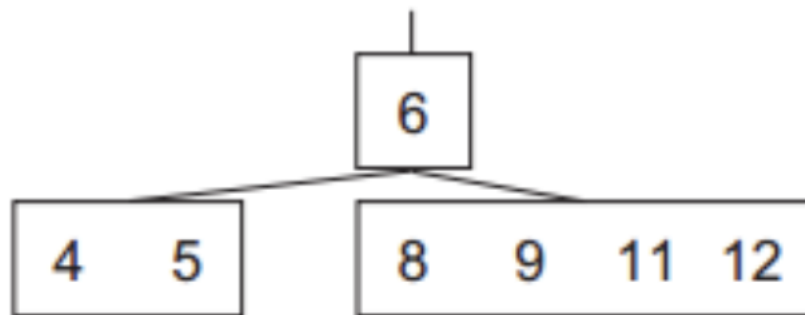


Figura 14: Árbol B

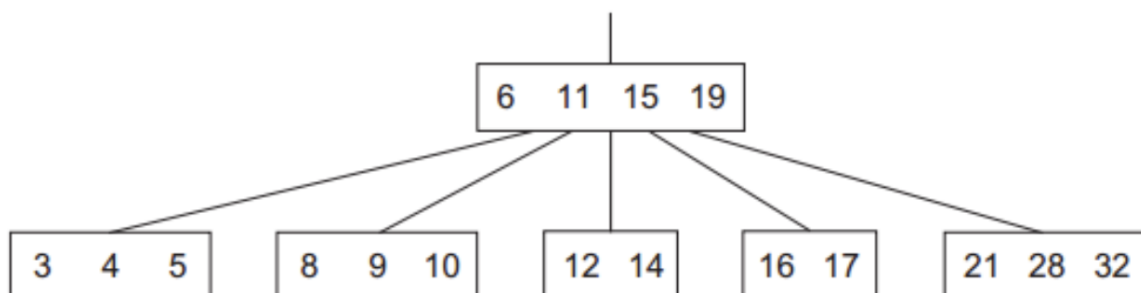
La raíz está llena. Al insertar la llave 8:

**Figura 15:** Árbol B

Varios elementos después:

**Figura 16:** Árbol B

Varios elementos después...:

**Figura 17:** Árbol B

3.7.5 Eliminación

Eliminar en un árbol B consiste de: 1. Encontrar el nodo que contiene la llave a eliminar. 2. Eliminar la llave del nodo. 3. Balancear el árbol para

3.7.5.1 Caso #1 - El nodo es hoja En este caso, la llave por eliminar está en un nodo *hoja*. Hay dos sub-casos:

1.1 El nodo tiene más de $m-1$ llaves. En este caso, simplemente se elimina la llave.

Por ejemplo en este árbol de orden 3:

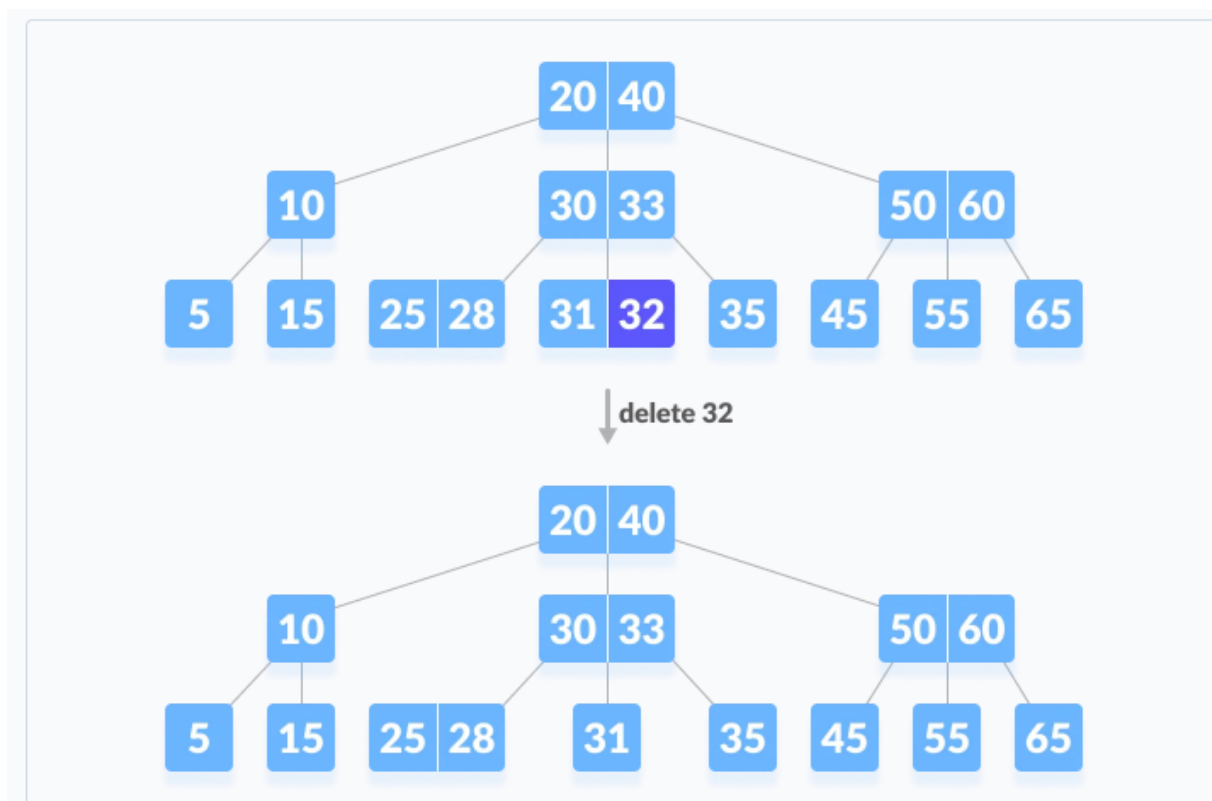
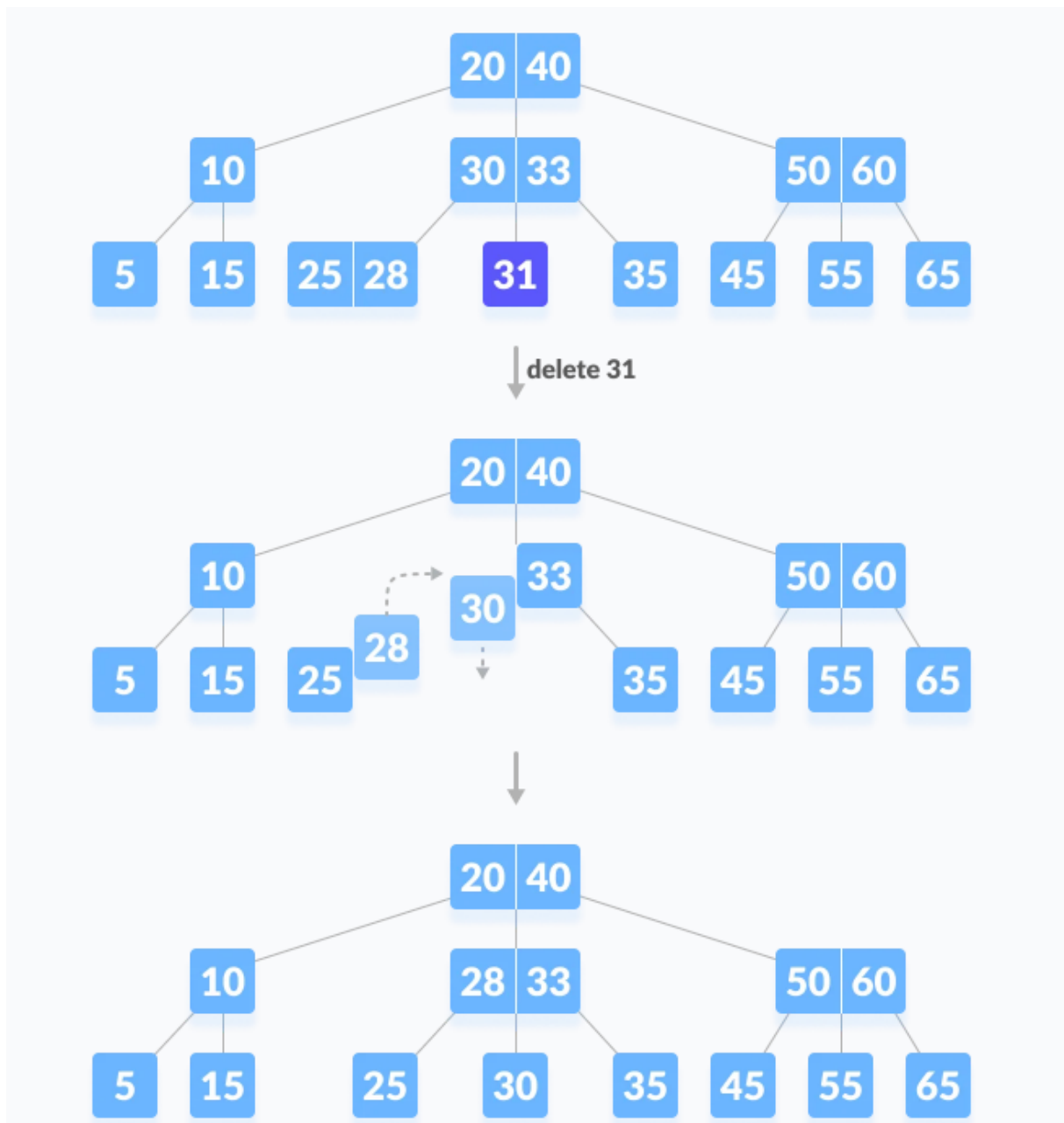


Figura 18: Árbol B

1.2 El nodo tiene $m-1$ llaves. En este caso, se *pide prestado* una llave de uno de los nodos hermanos. Primero se visita el hermano izquierdo. Si el hermano izquierdo tiene más de $m-1$ llaves, se toma la llave más grande del hermano izquierdo. Si no, se chequea el hermano derecho.

**Figura 19:** Árbol B

Si los dos hermanos tienen $m-1$ llaves, se fusionan los nodos y se elimina la llave del nodo padre. La mezcla de los nodos se hace mediante el nodo padre.

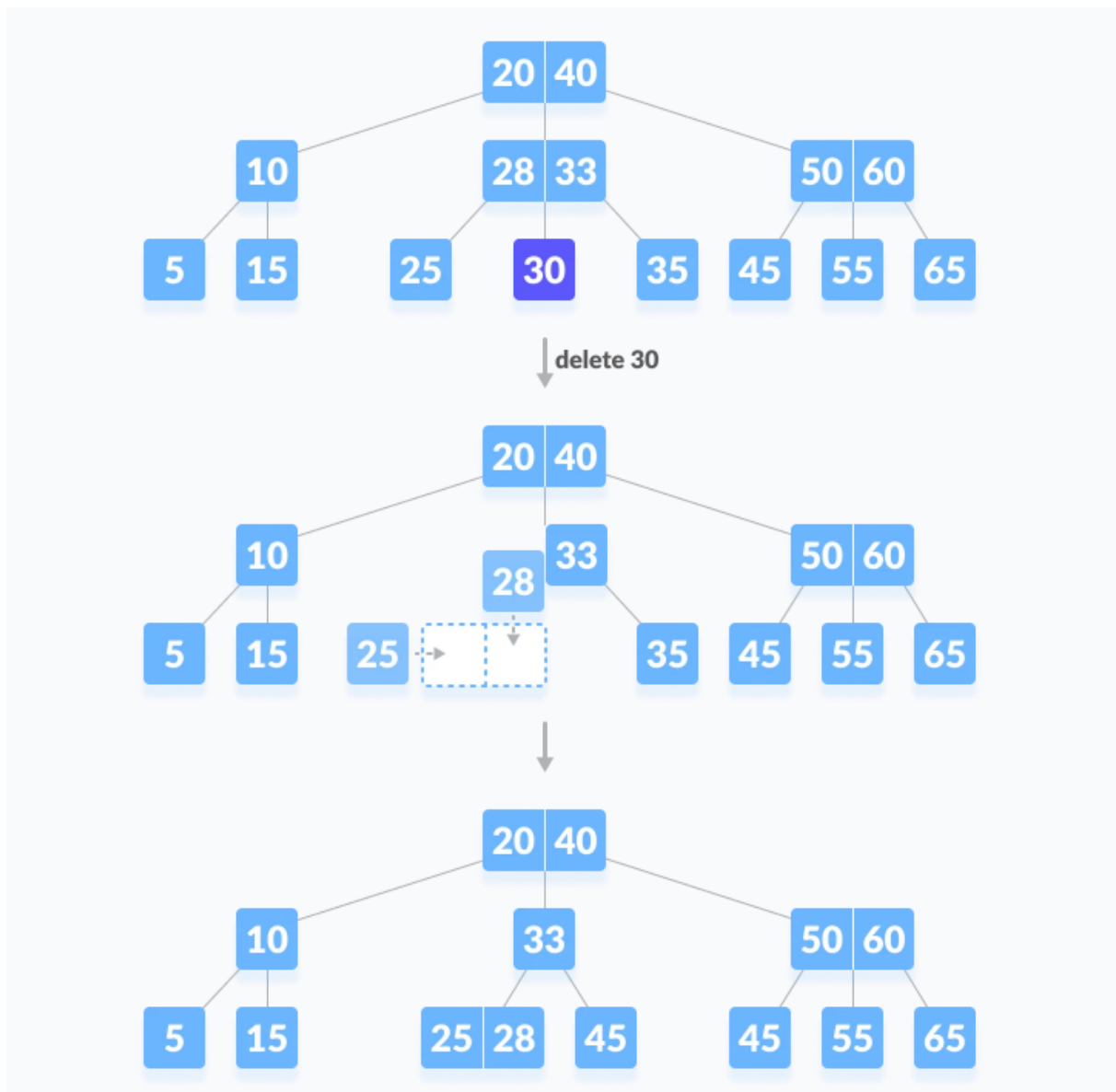
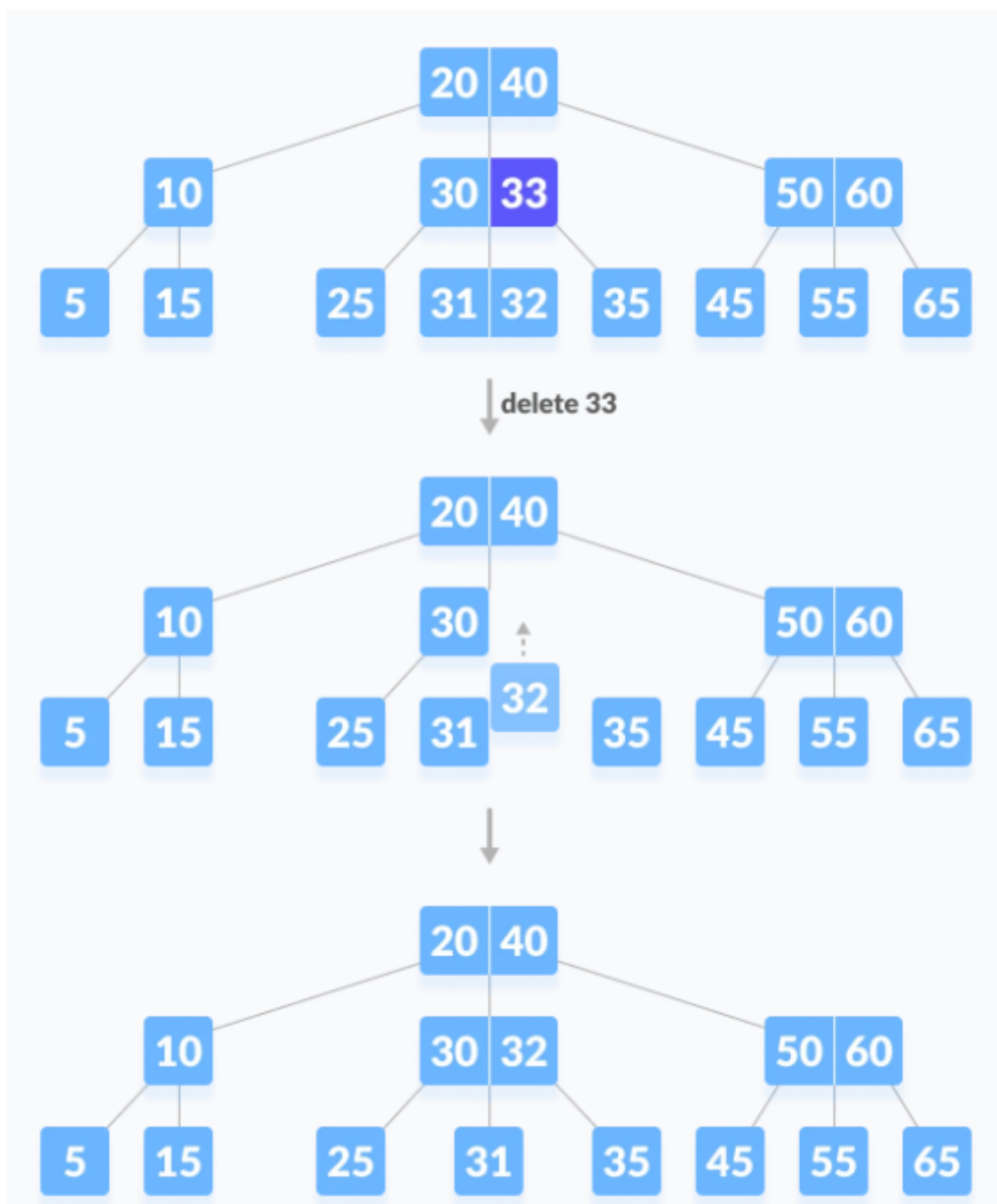


Figura 20: Árbol B

3.7.5.2 Caso #2 - El nodo es interno Si la llave por eliminar está dentro de un nodo interno, los siguientes casos pueden ocurrir:

2.1 La llave eliminada se reemplaza por la llave inmediatamente mayor (o menor) del sub-árbol derecho (o izquierdo) del nodo, si siempre y cuando el sub-árbol derecho (o izquierdo) más del mínimo de llaves.

**Figura 21:** Árbol B

2.2 Si ninguno de los hijos izquierdo o derecho tiene más de $m-1$ llaves, se fusionan los nodos y se elimina la llave del nodo padre.

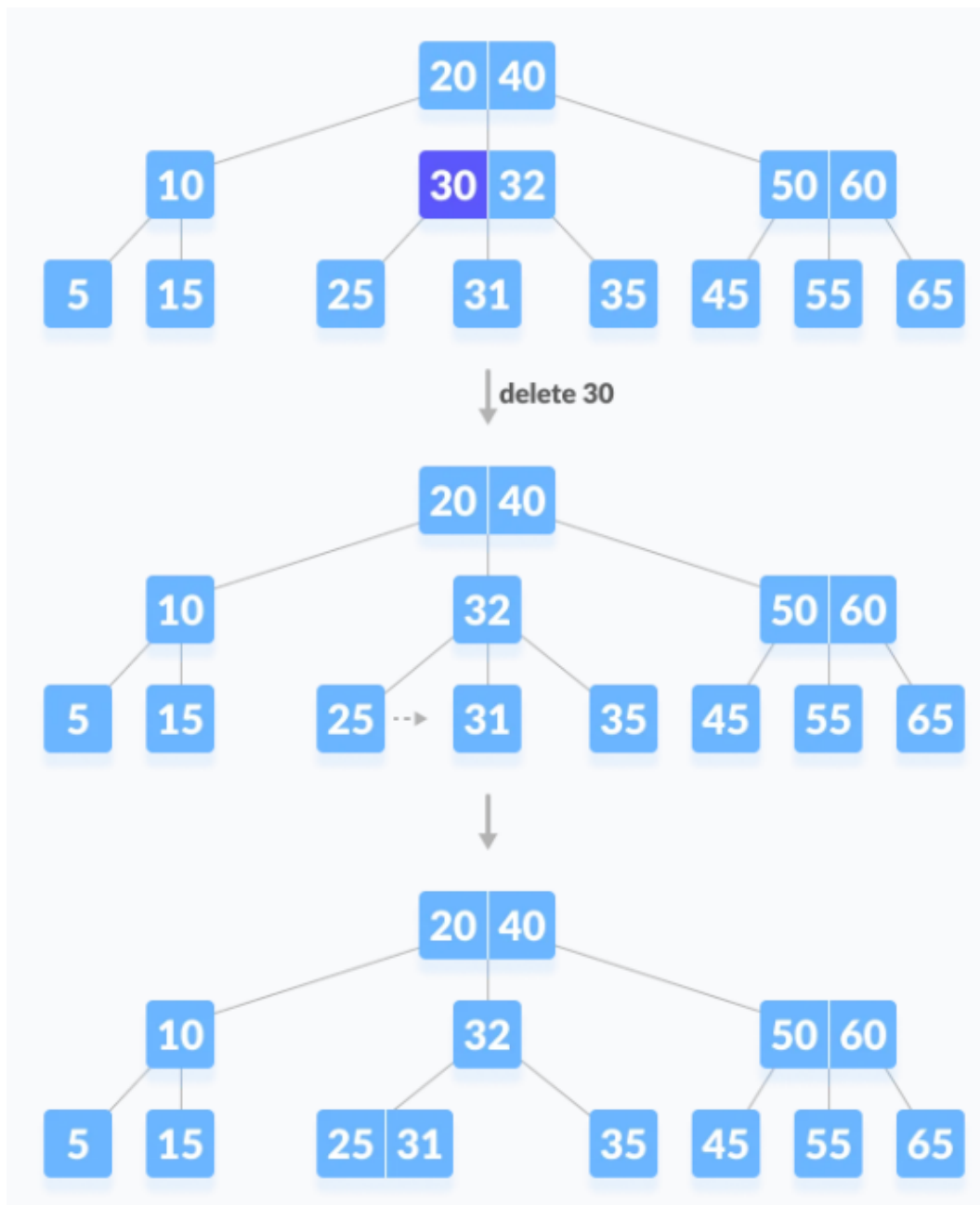
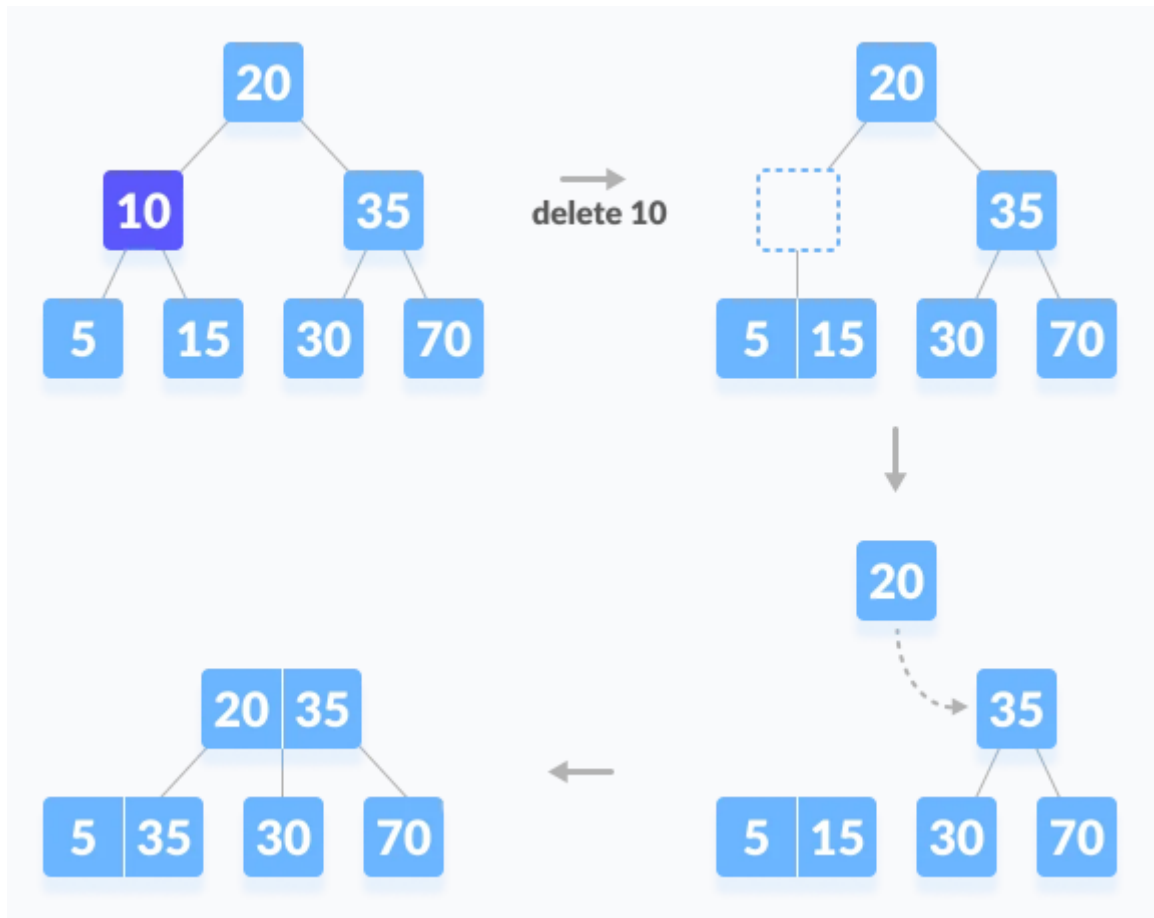


Figura 22: Árbol B

3.7.5.3 Caso #3 La eliminación ocurre en un nodo interno. Si no se puede realizar el caso #2 (anterior), se unen los hijos junto con el padre.

**Figura 23:** Árbol B

3.8 Tries

Un Trie (del inglés *reTRIEval*) es una estructura de datos que permite almacenar un conjunto de cadenas de caracteres y realizar búsquedas de palabras en ellas. Los Tries son árboles de búsqueda n-arios que almacenan cadenas de caracteres, donde cada nodo del árbol representa un carácter. Los Tries son útiles para realizar búsquedas de palabras en un conjunto de cadenas de caracteres, como en diccionarios o en motores de búsqueda.

Se pronuncia como el inglés *try*.

Visualmente, un trie se puede ilustrar como:

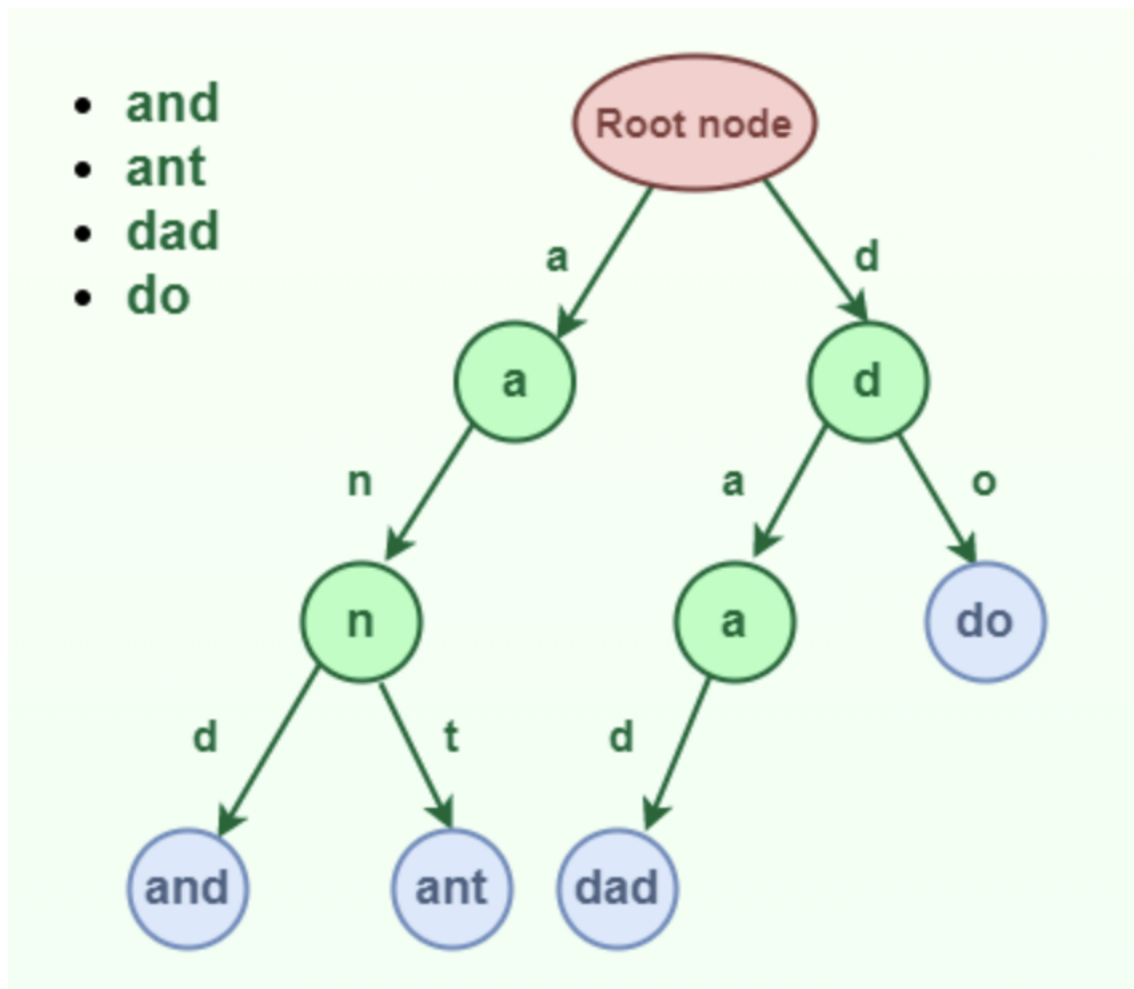


Figura 24: Trie

Cada rama de un nodo corresponde a un carácter de la llave insertada. El último nodo de cada llave se conoce como *EndOfWord*. La raíz no tiene un valor asignado.

3.8.1 El TDA Trie

El TDA Trie tiene las siguientes operaciones básicas:

- **Insertar (insert):** añade una cadena de caracteres al trie.
- **Buscar (search):** busca una cadena de caracteres en el trie.
- **Eliminar (delete):** elimina una cadena de caracteres del trie.

3.8.2 Estructura básica

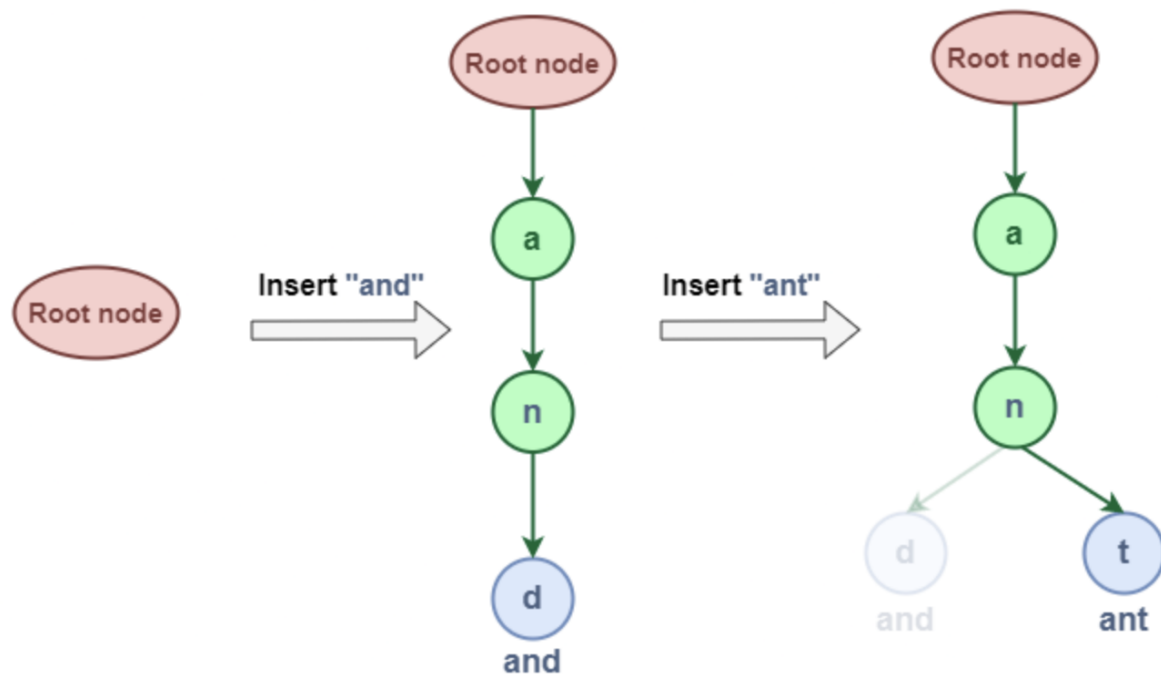
```
1 class TrieNode
2 {
3     TrieNode[] children = new TrieNode[ALPHABET_SIZE];
4     boolean isEndOfWord;
5
6     TrieNode(){
7         isEndOfWord = false;
8         for (int i = 0; i < ALPHABET_SIZE; i++)
9             children[i] = null;
10    }
11 }
12
13 class Trie
14 {
15     TrieNode root;
16 }
```

3.8.3 Inserción

Insertar una *llave* en un Trie es un proceso simple:

- Cada caracter de la llave se inserta como un nodo Trie individual.
- Los hijos de un nodo son un arreglo de punteros (o referencias) a los nodos del siguiente nivel del trie.
- El caracter de la llave actúa como un índice para el arreglo de hijos. Si la llave de entrada es nueva o una extensión de la llave existente, se construyen nodos no existentes de la llave y se marca el final de la palabra para el último nodo.
- Si la llave de entrada es un prefijo de la llave existente en el Trie, simplemente se marca el último nodo de la llave como el final de una palabra.

La longitud de la llave determina la profundidad del Trie.

**Figura 25:** Trie

```
1 void insert(String key) {
2     int level;
3     int length = key.length();
4     int index;
5
6     TrieNode pCrawl = root;
7
8     for (level = 0; level < length; level++) {
9         index = key.charAt(level) - 'a';
10        if (pCrawl.children[index] == null)
11            pCrawl.children[index] = new TrieNode();
12
13        pCrawl = pCrawl.children[index];
14    }
15
16    // mark last node as leaf
17    pCrawl.isEndOfWord = true;
18 }
```

3.8.4 Búsqueda

Buscar una llave en un Trie es similar a la operación de inserción. Sin embargo, se detiene si no hay más caracteres en la llave o si no hay más nodos en el Trie. La búsqueda puede terminar debido al final de una cadena o la falta de una llave en el Trie.

- En el primer caso, si el campo `isEndOfWord` del último nodo es verdadero, entonces la llave existe en el Trie.
- En el segundo caso, la búsqueda termina sin examinar todos los caracteres de la llave, ya que la llave no está presente en el Trie.

```
1 boolean search(String key) {
2     int level;
3     int length = key.length();
4     int index;
5     TrieNode pCrawl = root;
6
7     for (level = 0; level < length; level++) {
8         index = key.charAt(level) - 'a';
9
10        if (pCrawl.children[index] == null)
11            return false;
12
13        pCrawl = pCrawl.children[index];
14    }
15    return (pCrawl.isEndOfWord);
16 }
```

4 Algoritmos de búsqueda y ordenamiento

4.1 Algoritmos de búsqueda

Como lo indica su nombre, permiten buscar un elemento dentro de una colección de datos. La colección puede ser sobre arreglos o listas enlazadas, sin embargo, en este curso se enfocará en arreglos.

Se recomienda este video introductorio.

4.1.1 Búsqueda lineal

La búsqueda lineal es el método más sencillo para buscar un elemento en un arreglo (o una lista). Consiste en recorrer el arreglo desde el primer elemento hasta el último, comparando cada elemento con el valor que se busca.

Cuando el arreglo **no está ordenado**, la búsqueda lineal es la **única opción**. Sin embargo, cuando el arreglo está ordenado, existen algoritmos más eficientes para realizar la búsqueda, como la búsqueda binaria.

Dado el siguiente arreglo:

1	Indices	0	1	2	3	4	5	6	7	8
2	Elementos	5	3	8	6	2	7	1	4	9

Para buscar el número 7, se sigue el siguiente proceso:

1	Indices	0	1	2	3	4	5	6	7	8
2	Elementos	5	3	8	6	2	7	1	4	9
3	5 != 7	^								
4	3 != 7		^							
5	8 != 7			^						
6	6 != 7				^					
7	2 != 7					^				
8	7 == 7						^			

4.1.1.1 Implementación

```

1 public static int linearSearch(int[] arr, int target) {
2     for (int i = 0; i < arr.length; i++) {
3         if (arr[i] == target) {
4             return i;
5         }
6     }
7     return -1;
8 }

```

4.1.2 Búsqueda binaria

Es un algoritmo de búsqueda que encuentra la posición de un valor en un arreglo ordenado. A diferencia de la búsqueda lineal, que recorre el arreglo desde el primer elemento hasta el último, la búsqueda binaria divide el arreglo en dos mitades y compara el valor buscado con el elemento en el medio.

- Si el valor buscado es *menor* que el elemento en el medio, la búsqueda continúa en la mitad **izquierda** del arreglo.
- Si el valor buscado es *mayor* que el elemento en el medio, la búsqueda continúa en la mitad **derecha** del arreglo.

Este proceso se repite hasta que el valor buscado sea encontrado o hasta que el subarreglo de búsqueda sea vacío.

Dado el siguiente arreglo:

1	Indices	0	1	2	3	4	5	6	7	8
2	Elementos	1	2	3	4	5	6	7	8	9

Para buscar el número 9:

1	Indices	0	1	2	3	4	5	6	7	8	
2	Elementos	1	2	3	4	5	6	7	8	9	
3		-----									
4	5 < 9					^					
5							-----				
6	7 < 9							^			
7									-----		
8	8 < 9								^		
9										--	
10	9 == 9										^

4.1.2.1 Implementación Puede implementarse recursivamente o iterativamente. A continuación se muestra la implementación iterativa:

```

1 public static int binarySearch(int[] arr, int target) {
2     int left = 0;
3     int right = arr.length - 1;
4
5     while (left <= right) {
6         int mid = left + (right - left) / 2;
7
8         if (arr[mid] == target) {
9             return mid;
10        }
11    }

```

```
12         if (arr[mid] < target) {
13             left = mid + 1;
14         } else {
15             right = mid - 1;
16         }
17     }
18
19     return -1;
20 }
```

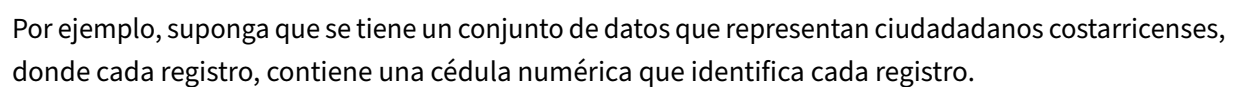
4.1.3 Búsqueda Hash

Para buscar en grandes colecciones de datos, no necesariamente ordenados, *hashing* (dispersión) provee una técnica para buscar de una forma más eficiente. La función de *hash* se utiliza para transformar una o más características de cada elemento del universo de búsqueda en un valor numérico que corresponde a un índice de un array. La búsqueda con *hash*, tiene un mejor rendimiento promedio que otros algoritmos de búsqueda.

4.1.3.1 Hash tables Es una estructura de datos que almacena datos en pares *llave-valor*:

- *Llave*: llave única para identificar un valor.
- *Valor*: dato asociado con la llave.

La llave k se utiliza como entrada para una función de hashing $h(k)$ que genera un índice i donde el valor se almacenará dentro de la tabla. Visualmente se puede representar de esta forma:



Para buscar un ciudadano en particular, se puede utilizar la cédula como llave y almacenar los datos en una tabla *hash*. Si la función de hash es $h(\text{cédula}) = \text{cédula} \% 10$, se puede almacenar los datos de la siguiente forma:

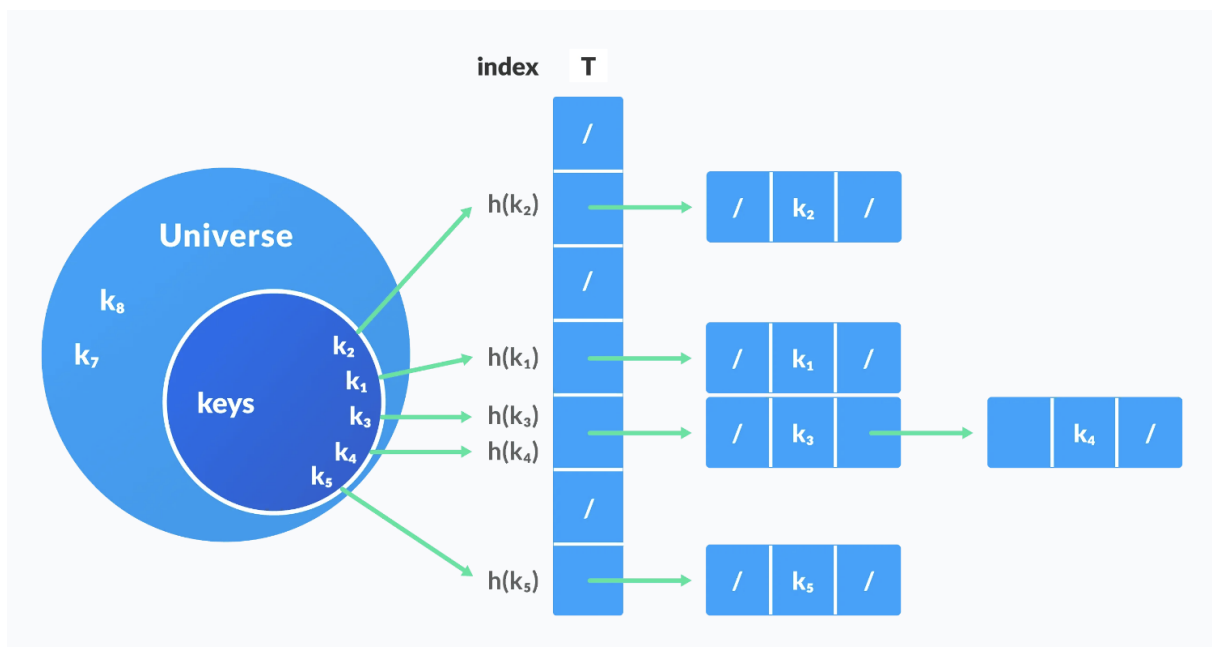
83

Índice	Cédula	Nombre	Apellido	Dirección
3	456789123	Carlos	Sánchez	Alajuela
4				
5				
6				
7				
8				
9	123456789	Juan	Pérez	San José

La función de hash seleccionada, determina la cantidad de *buckets* (espacios de almacenamiento) que se utilizarán para almacenar los datos. En este caso, se utilizó el módulo 10 para determinar el índice de almacenamiento.

El reto de las funciones hash es generar un índice único para cada llave. Si dos llaves generan el mismo índice, se produce una colisión. Las colisiones se pueden resolver de diferentes formas:

- **Separate chaining:** Cada índice de la tabla *hash* almacena una lista enlazada de elementos que colisionan. Visualmente se puede ver de la siguiente forma:



- **Open addressing:** Se busca un índice alternativo para almacenar el elemento que colisiona.

4.2 Algoritmos de ordenamiento

Son algoritmos que reciben una colección de elementos en desorden y la ordenan ascendente o descendente. Hay muchos algoritmos, y la razón de su existencia es que cada uno tiene diferentes características de rendimiento.

4.2.1 Ordenamiento por selección

Este algoritmo es el más sencillo de implementar. Funciona de la siguiente manera:

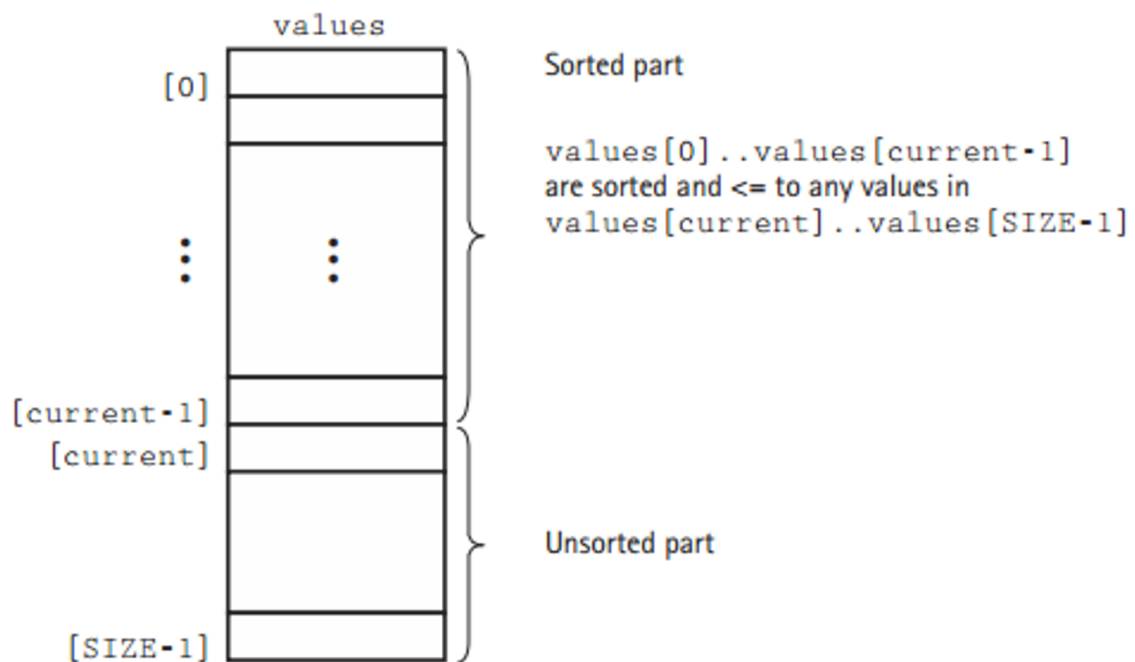
1. Busca el elemento más pequeño (o el más grande si es orden descendente) en el arreglo.
2. Intercambia el elemento más pequeño con el primer elemento del arreglo.
3. Busca el segundo elemento más pequeño en el arreglo.
4. Intercambia el segundo elemento más pequeño con el segundo elemento del arreglo.
5. Repite el proceso hasta que el arreglo esté ordenado.

Visualmente se puede ver de la siguiente forma:

values		values		values		values		values	
[0]	126	[0]	1	[0]	1	[0]	1	[0]	1
[1]	43	[1]	43	[1]	26	[1]	26	[1]	26
[2]	26	[2]	26	[2]	43	[2]	43	[2]	43
[3]	1	[3]	126	[3]	126	[3]	126	[3]	113
[4]	113	[4]	113	[4]	113	[4]	113	[4]	126
(a)		(b)		(c)		(d)		(e)	

Figura 26: Ordenamiento por selección

El arreglo se “divide” en dos arreglos “lógicos”:

**Figura 27:** División lógica del arreglo

La implementación en C# es la siguiente:

```
1 public static void selectionSort(int[] arr) {  
2     for (int i = 0; i < arr.length - 1; i++) {  
3         int minIndex = i;  
4         for (int j = i + 1; j < arr.length; j++) {  
5             if (arr[j] < arr[minIndex]) {  
6                 minIndex = j;  
7             }  
8         }  
9         int temp = arr[minIndex];  
10        arr[minIndex] = arr[i];  
11        arr[i] = temp;  
12    }  
13 }
```

4.2.1.1 Ventajas y desventajas

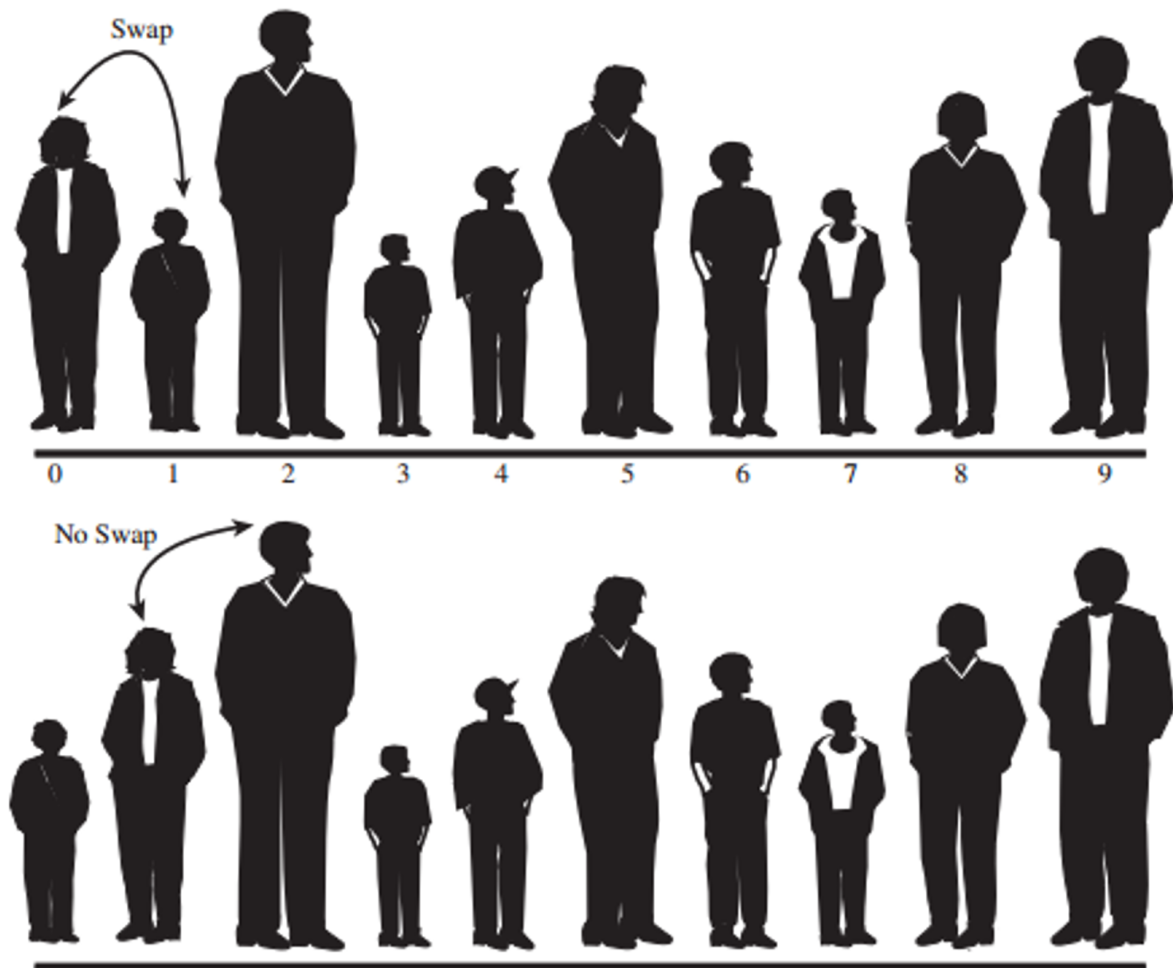
Ventajas	Desventajas
Es fácil de entender e implementar.	Tiene una complejidad de tiempo $O(n^2)$, lo que lo hace ineficiente para grandes conjuntos de datos.
No requiere memoria adicional significativa.	Siempre realiza el mismo número de comparaciones, independientemente de cómo estén ordenados los datos.
Funciona bien con listas pequeñas.	

4.2.2 Ordenamiento de burbuja

El algoritmo de burbuja es otro algoritmo de ordenamiento simple. Funciona de la siguiente manera:

1. Compara el primer elemento con el segundo. Si el primer elemento es mayor que el segundo, los intercambia.
2. Compara el segundo elemento con el tercero. Si el segundo elemento es mayor que el tercero, los intercambia.
3. Repite el proceso hasta que el arreglo esté ordenado.

Visualmente se puede ver de la siguiente forma:

**Figura 28:** Bubble sort

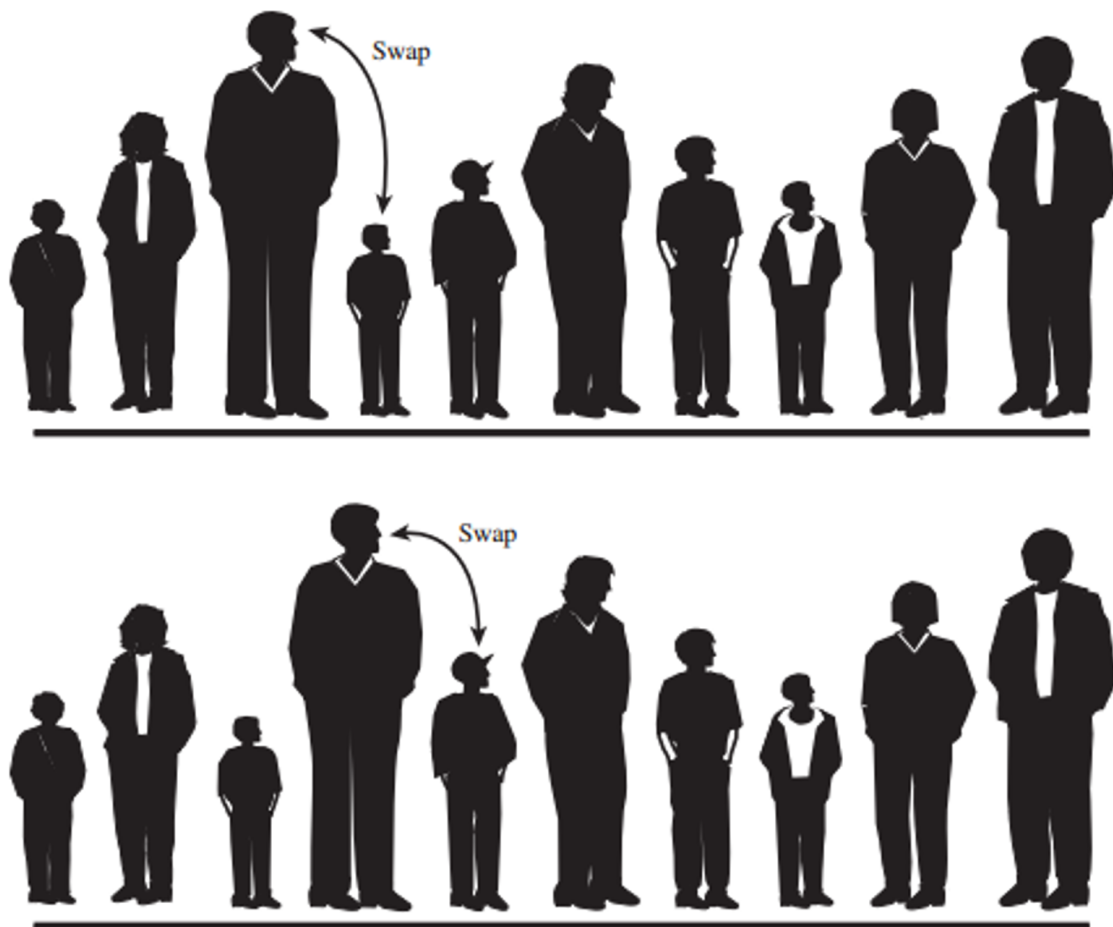


Figura 29: Bubble sort

Al final de la primera pasada:

**Figura 30:** Bubble sort

La implementación en C# es la siguiente:

```
1 public static void bubbleSort(int[] arr) {  
2     for (int i = 0; i < arr.length - 1; i++) {  
3         for (int j = 0; j < arr.length - i - 1; j++) {  
4             if (arr[j] > arr[j + 1]) {  
5                 int temp = arr[j];  
6                 arr[j] = arr[j + 1];  
7                 arr[j + 1] = temp;  
8             }  
9         }  
10    }  
11 }
```

4.2.2.1 Ventajas y desventajas Igual que el ordenamiento por selección, el ordenamiento de burbuja es fácil de entender e implementar. Sin embargo, tiene una complejidad de tiempo $O(n^2)$, lo que lo hace ineficiente para grandes conjuntos de datos.

4.2.3 Ordenamiento por inserción (Insertion sort)

El ordenamiento por inserción es un algoritmo de ordenamiento simple que funciona de la siguiente manera:

1. Comienza con el segundo elemento del arreglo. Este primer elemento se considera parte de la

lista “logica” de elementos ordenados. El resto de los elementos se consideran parte de la lista “logica” de elementos desordenados.

2. Se toma cada uno de los elementos de la lista de elementos desordenados y se inserta en la lista de elementos ordenados en la posición correcta, de atrás hacia adelante.
3. La lista de ordenados crece a medida que se insertan los elementos de la lista de desordenados. La lista de desordenados disminuye a medida que se insertan los elementos.

Visualmente se puede ver de la siguiente forma:

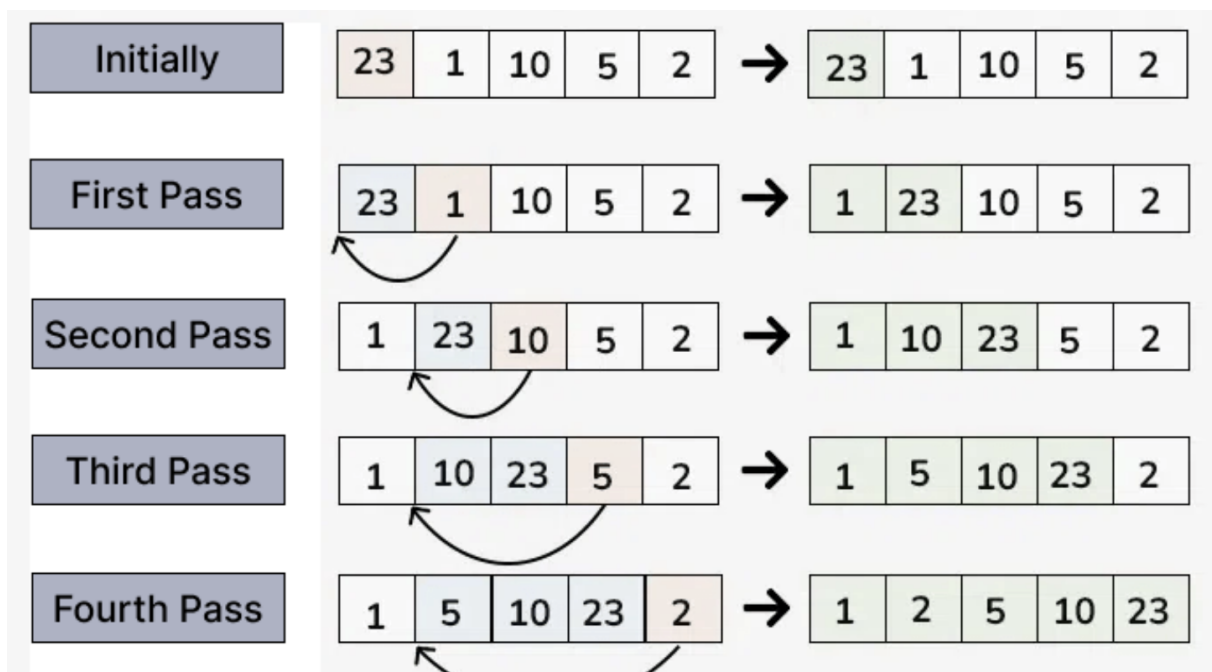


Figura 31: Insertion sort

4.2.3.1 Implementación

```

1 void sort(int arr[])
2 {
3     int n = arr.length;
4     for (int i = 1; i < n; ++i) {
5         int key = arr[i];
6         int j = i - 1;
7         while (j >= 0 && arr[j] > key) {
8             arr[j + 1] = arr[j];
9             j = j - 1;
10        }
11        arr[j + 1] = key;
12    }
13 }

```


4.2.3.2 Ventajas y desventajas El ordenamiento por inserción es un algoritmo de ordenamiento eficiente para listas pequeñas o parcialmente ordenadas. En el peor de los casos degenera a una complejidad de tiempo $O(n^2)$, pero en el mejor de los casos, tiene una complejidad de tiempo $O(n)$.

4.2.4 Quick sort

El ordenamiento rápido (Quick sort) es un algoritmo de ordenamiento eficiente y rápido. Sigue los siguientes pasos:

1. *Selección de un elemento como pivote.* Usualmente es el elemento central del arreglo.
2. *Partición del arreglo alrededor del pivote.* Se colocan los elementos menores que el pivote a la izquierda y los elementos mayores a la derecha.
3. *Recursión.* Se aplica el algoritmo de forma recursiva a los subarreglos generados (izquierda y derecha del pivote).
4. *Caso base.* Cuando el subarreglo tiene un solo elemento, se considera ordenado.

4.2.4.1 Ejecución de ejemplo Por ejemplo, dado el arreglo [8, 3, 1, 7, 0, 10, 2],

```
1 [8, 3, 1, 7, 0, 10, 2]
2   Pivote: 7
3 [3, 1, 0, 2] | 7 | [8, 10]
4
5 [3, 1, 0, 2] -> Pivote: 0 -> [ ] | 0 | [3, 1, 2]
6 [3, 1, 2] -> Pivote: 1 -> [ ] | 1 | [3, 2]
7 [3, 2] -> Pivote: 2 -> [ ] | 2 | [3]
8
9 [8, 10] -> Pivote: 10 -> [8] | 10 | [ ]
10
11 Final: [0, 1, 2, 3] | 7 | [8, 10] -> [0, 1, 2, 3, 7, 8, 10]
```

4.2.4.2 Implementación

```
1 static void QuickSort(int[] array, int left, int right)
2 {
3     if (left < right)
4     {
5         // Obtener el índice del pivote tras la partición
6         int pivotIndex = Partition(array, left, right);
7
8         // Aplicar QuickSort recursivamente a las sublistas
9         QuickSort(array, left, pivotIndex - 1);
10        QuickSort(array, pivotIndex + 1, right);
11    }
12 }
```

```
13
14 static int Partition(int[] array, int left, int right)
15 {
16     // Elegir el pivote como el elemento central
17     int pivotIndex = left + (right - left) / 2;
18     int pivot = array[pivotIndex];
19
20     // Mover el pivote al final (swap con el último elemento)
21     Swap(array, pivotIndex, right);
22
23     int partitionIndex = left;
24
25     for (int i = left; i < right; i++)
26     {
27         if (array[i] <= pivot)
28         {
29             Swap(array, i, partitionIndex);
30             partitionIndex++;
31         }
32     }
33
34     // Mover el pivote a su posición final
35     Swap(array, partitionIndex, right);
36
37     return partitionIndex;
38 }
39
40 static void Swap(int[] array, int a, int b)
41 {
42     int temp = array[a];
43     array[a] = array[b];
44     array[b] = temp;
45 }
```

4.2.4.3 Recursos adicionales Puede aprender más sobre Quick sort en los siguientes enlaces:

- <http://me.dt.in.th/page/Quicksort/>
- <https://www.geeksforgeeks.org/quick-sort/>
- <https://www.cs.usfca.edu/~galles/visualization/ComparisonSort.html>

4.2.5 Shellsort

Fue propuesto por Donald Shell en 1959.

La idea básica de Shellsort es que los elementos de un arreglo se ordenan en incrementos cada vez más pequeños. El último incremento es 1, que es el tamaño del arreglo. El algoritmo de Shellsort es

una generalización del algoritmo **InsertionSort** y busca aprovechar al máximo el mejor caso de este, es decir, cuando los elementos ya están *casi* ordenados.

Shellsort trabaja realizando sus Insertion Sorts en sublistas cuidadosamente seleccionadas, primero en sublistas pequeñas y luego en sublistas cada vez más grandes.

Shellsort rompe la lista en subconjuntos disjuntos, donde un subconjunto está definido por un “incremento”, l . Cada registro en un subconjunto dado está separado por l posiciones. Por ejemplo, si el incremento fuera 4, entonces cada registro en el subconjunto estaría separado por 4 posiciones.

Usando un enfoque sencillo, podemos definir los gaps (o incrementos) de la siguiente manera:

```
1 size / 2, size / 4, size / 8, ..., 1
```

Redondeando hacia el entero más cercano hacia arriba. Entonces, dado un array de 9 elementos, los gaps serían:

```
1 5, 3, 1
```

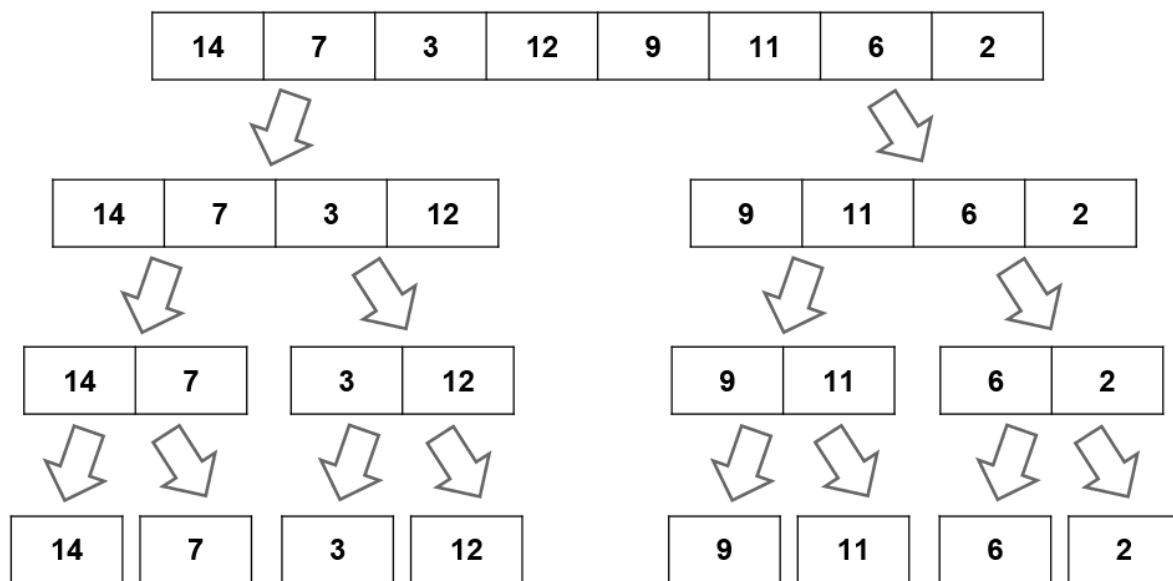
Dado el siguiente arreglo:

1	Indices	0	1	2	3	4	5	6	7	8
2	Elementos	5	3	8	6	2	7	1	4	9
3	-----									
4	gap=5	5	3	8	6	2	7	1	4	9
5		^-----^								
6		5	3	8	6	2	7	1	4	9
7		^-----^								
8		5	1	8	6	2	7	3	4	9
9		^-----^								
10		5	1	4	6	2	7	3	8	9
11		^-----^								
12		5	1	4	6	2	7	3	8	9
13	-----									
14	gap de 3	^-----^								
15		5	1	4	6	2	7	3	8	9
16		^-----^								
17		5	1	4	6	2	7	3	8	9
18		^-----^								
19		5	1	4	6	2	7	3	8	9
20		^-----^								
21		5	1	4	3	2	7	6	8	9
22		^-----^								
23		3	1	4	5	2	7	6	8	9
24		^-----^								
25		3	1	4	5	2	7	6	8	9
26		^-----^								
27		3	1	4	5	2	7	6	8	9
28	-----									
29	gap de 1	1	3	8	7	2	5	6	4	9

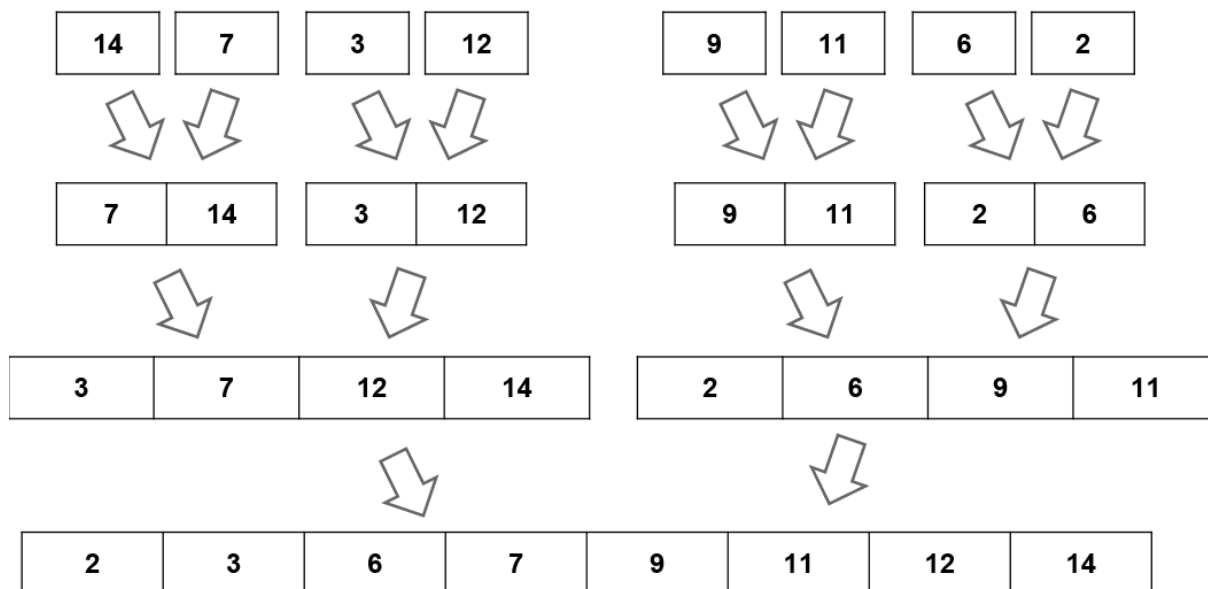
30		Λ---Λ	
31	1	3 8 7 2 5 4 6 9	
32		Λ---Λ	
33	1	3 8 7 2 5 4 6 9	
34		Λ---Λ	
35	1	3 7 8 2 5 4 6 9	
36		Λ---Λ	
37	1	3 7 2 8 5 4 6 9	
38		Λ---Λ	
39	1	3 2 7 8 5 4 6 9	
40		Λ---Λ	
41	1	2 3 7 8 5 4 6 9	
42		Λ---Λ	
43	1	2 3 7 5 8 4 6 9	
44		Λ---Λ	
45	1	2 3 5 7 8 4 6 9	
46		Λ---Λ	
47	1	2 3 5 7 4 8 6 9	
48		Λ---Λ	
49	1	2 3 5 4 7 8 6 9	
50		Λ---Λ	
51	1	2 3 4 5 7 8 6 9	
52		Λ---Λ	
53	1	2 3 4 5 7 6 8 9	
54		Λ---Λ	
55	1	2 3 4 5 6 7 8 9	
56		Λ---Λ	
57	=====		

4.2.6 Ordenamiento por mezcla (Merge sort)

Es un algoritmo de ordenamiento que sigue el paradigma de dividir y conquistar. Divide el arreglo en dos mitades, ordena las dos mitades de forma recursiva y luego combina las dos mitades ordenadas.

**Figura 32:** Merge sort, fase de división

Luego en la fase de unión,

**Figura 33:** Merge sort, fase de unión

4.2.6.1 Implementación

```
1 void mergeSort(int[] a, int low, int high) {
2     int mid;
3     if (low < high) {
4         mid = (low + high) / 2;
5         mergesort(a, low, mid);
6         mergesort(a, mid + 1, high);
7         merge(a, low, high, mid);
8     }
9     return;
10 }
11
12 void merge(int[] a, int low, int high, int mid) {
13     int i, j, k, c[50];
14     i = low;
15     k = low;
16     j = mid + 1;
17     while (i <= mid && j <= high) {
18         if (a[i] < a[j]) {
19             c[k] = a[i];
20             k++;
21             i++;
22         } else {
23             c[k] = a[j];
24             k++;
25             j++;
26         }
27     }
28     while (i <= mid) {
29         c[k] = a[i];
30         k++;
31         i++;
32     }
33     while (j <= high) {
34         c[k] = a[j];
35         k++;
36         j++;
37     }
38     for (i = low; i < k; i++) {
39         a[i] = c[i];
40     }
41 }
```

4.2.7 Ordenamiento por Raíz (Radix sort)

Es un algoritmo de ordenamiento que ordena los elementos procesando los dígitos individuales. Radix sort puede ser usado para ordenar números enteros, strings, o cualquier otro tipo de datos que puedan ser procesados en dígitos individuales.

Los algoritmos vistos previamente, consideran cada elemento como un todo. Radix sort, por otro

lado, considera cada elemento como una secuencia de dígitos. Por ejemplo, para ordenar los números [170, 45, 75, 90, 802, 24, 2, 66], se ordenan primero por el dígito de las unidades, luego por el dígito de las decenas, y así sucesivamente.

Radix significa “raíz” en latín, y se refiere a la base de un sistema numérico. Por ejemplo, en el sistema decimal, la base es 10. En el sistema binario, la base es 2.

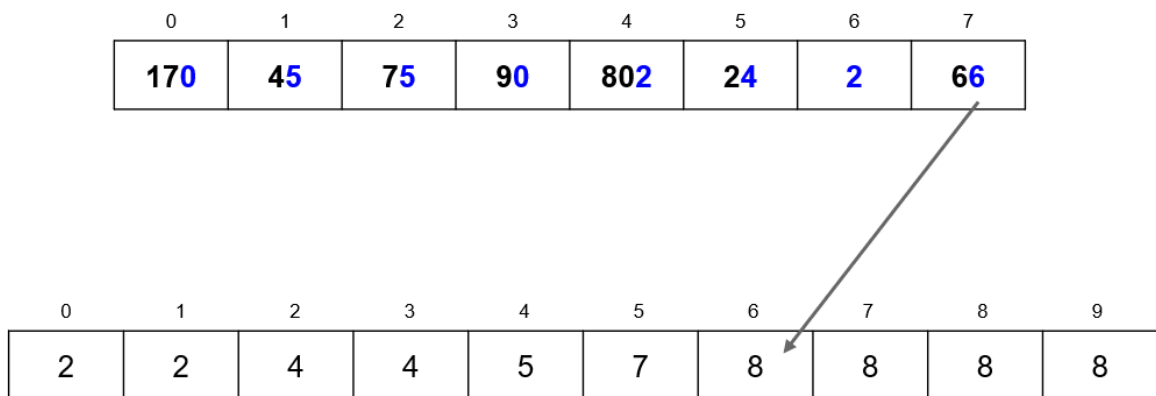
0	1	2	3	4	5	6	7
170	45	75	90	802	24	2	66

0	1	2	3	4	5	6	7	8
2	0	2	0	1	2	1	0	

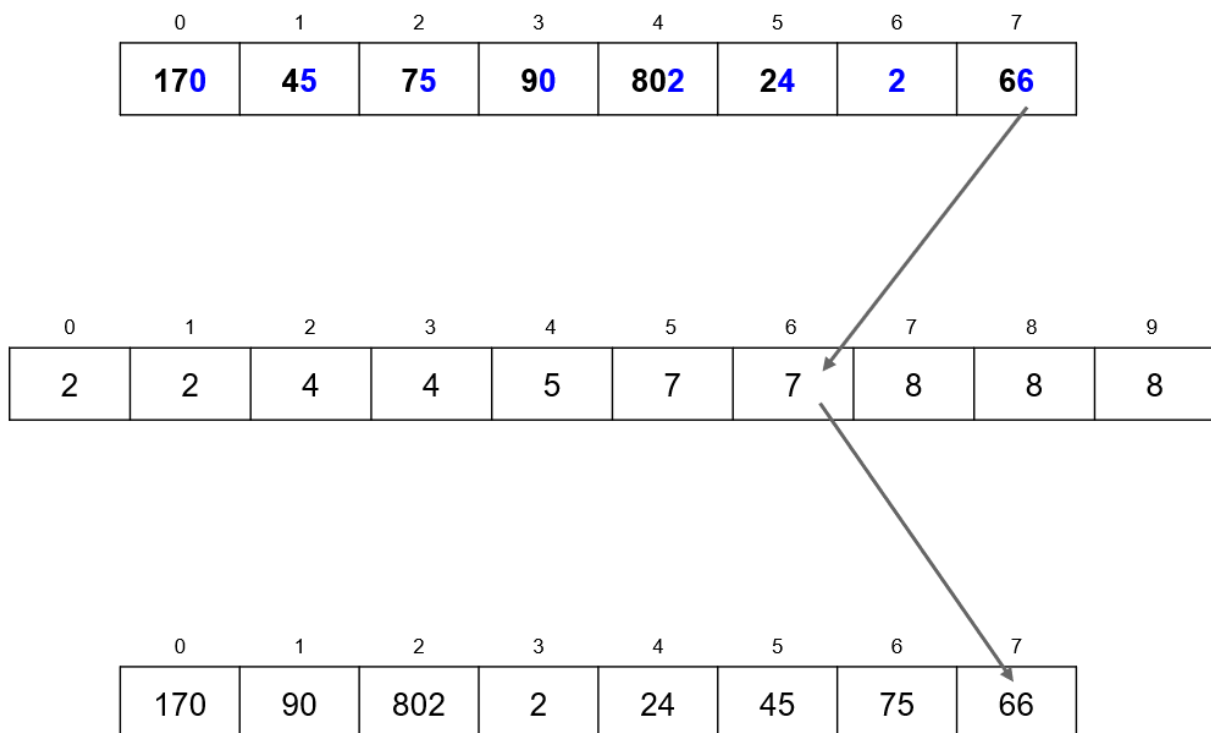
0	1	2	3	4	5	6	7

4.2.7.1 Ejecución de ejemplo

Nóte que el arreglo intermedio tiene 10 posiciones, una por cada dígito posible de la base 10. Este arreglo contiene el conteo de la aparición de cada dígito en el arreglo original. A este arreglo se le hace un ajuste (se le suma la posición anterior) para obtener la posición final de cada dígito en el arreglo ordenado.



El arreglo ajustado, mapea la posición de cada dígito en el arreglo ordenado. Al mapear, el valor correspondiente del arreglo intermedio se reduce en 1.



En resumen,

0	1	2	3	4	5	6	7
170	45	75	90	802	24	2	66

0	1	2	3	4	5	6	7	8	9
1	2	4	4	5	7	7	8	8	8

0	1	2	3	4	5	6	7
170	90	802	02	24	45	75	66

0	1	2	3	4	5	6	7	8	9
2	2	3	3	4	4	5	7	7	8

0	1	2	3	4	5	6	7
802	002	024	045	066	170	075	090

0	1	2	3	4	5	6	7	8	9
6	7	7	7	7	7	7	7	8	8

0	1	2	3	4	5	6	7
002	024	045	066	075	090	170	802

5 Grafos

Los grafos son estructuras de datos no lineales que consisten de vértices (o nodos) y aristas (o bordes). Un grafo G agrupa entidades físicas o conceptuales. Un grafo está denotado como $G = \{V, A\}$ donde V es el conjunto de vértices y A es el conjunto de aristas.

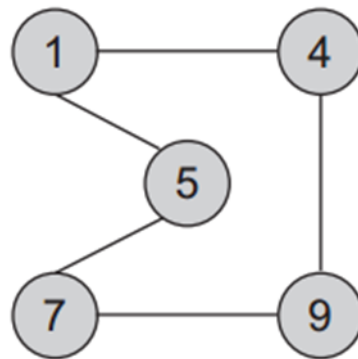


Figura 34: Grafo básico

Del grafo anterior, podemos decir que:

- $V = \{ 1, 4, 5, 7, 9 \}$
- $A = \{ (1,4), (4,1), (1,5), (5,1), (4,9), (9,4), (5,7), (7,5), (7,9), (9,7) \}$

Un grafo puede ser dirigido y no dirigido:

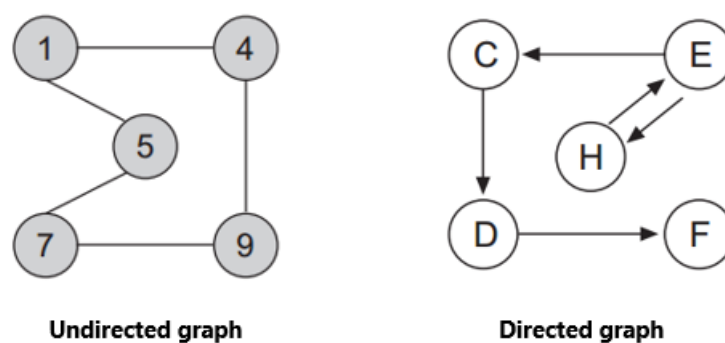


Figura 35: Tipos de grafos

- En los dirigidos se muestra la dirección de la relación entre los nodos.
- En los no dirigidos los nodos conectados son adyacentes.

5.1 Conceptos importantes

- Una arista puede tener un peso asociado denotando la magnitud asociada a la relación. Este tipo de grafos se les llama *Grafos ponderados*
- El grado de v es una cualidad de un nodo de un grafo. En un grafo no dirigido es el número de aristas que contiene v .
- En un grafo dirigido, el grado de entrada es el número de extremos de cola adyacentes a v . El grado de salida es el número de extremos de cabeza adyacentes a v .
- La ruta $P = (v_0, v_1, v_2, \dots, v_n)$ es una serie de vertices que forman la ruta desde v_0 hasta v_n . v_n y v_0 pueden ser iguales. Si los vértices entre v_0 y v_n son diferentes, la ruta se llama ruta simple.
- Un ciclo es una ruta simple que empieza y termina en el mismo nodo.

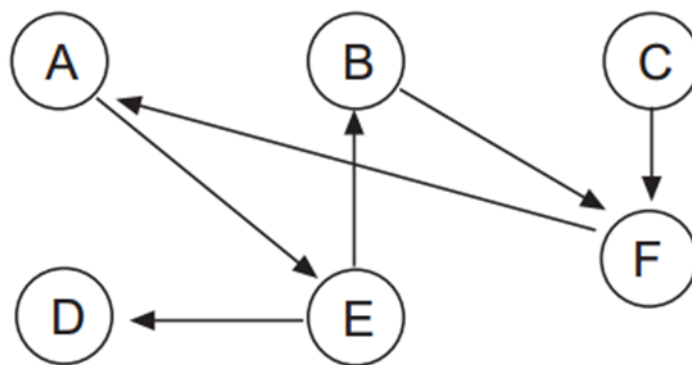


Figura 36: ciclo

- Un DAG es un grafo acíclico dirigido, o sea que no existen ciclos.
- Un grafo es conexo si existe un camino entre cualquier par de nodos que lo componen. Un grafo es fuertemente conexo si el grafo es conexo y es dirigido.



Figura 37: grafo conexo y fuertemente conexo

5.2 Representación

Los grafos se pueden representar utilizando dos enfoques diferentes:

- Usando una matriz bidimensional conocida como matriz de adyacencia
- Usando una representación dinámica conocida como lista de adyacencia

Elegir entre una representación u otra depende del tipo de array y de las operaciones que se realizarán:

- Si el grafo es denso (muchas aristas), lo mejor es escoger la matriz.
- Si el grafo es disperso, lo mejor es escoger la lista enlazada.

5.2.1 Matriz de adyacencia

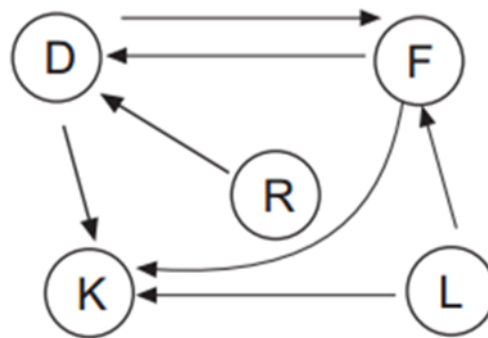
Sea $G = \{V, A\}$ donde $V = \{v_0, v_1, v_2, \dots, v_{n-1}\}$ y $A = \{(v_i, v_j)\}$. Los nodos se pueden representar mediante la matriz A de $n \times n$ conocida como matriz de adyacencia. Cada elemento de a_{ij} puede tomar uno de los siguientes valores:

$$a_{ij} \left\{ \begin{array}{l} \mathbf{0} \text{ if there is not an arc between } (v_i, v_j) \\ \mathbf{1} \text{ if there is an arc between } (v_i, v_j) \end{array} \right\}$$

Figura 38: Matriz de adyacencia

- Por ejemplo, digamos que los nodos son {D, F, K, L, R} la matriz sería:

1			0	1	1	0	0	
2			1	0	1	0	0	
3	$A =$		0	0	0	0	0	
4			0	1	1	0	0	
5			1	0	0	0	0	

**Figura 39:** alt text

- Si el grafo es ponderado:

$$P = \begin{vmatrix} 0 & 4 & 5 & 0 & 0 \\ 0 & 0 & 3 & 6 & 3 \\ 3 & 0 & 0 & 0 & 0 \\ 5 & 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{vmatrix}$$

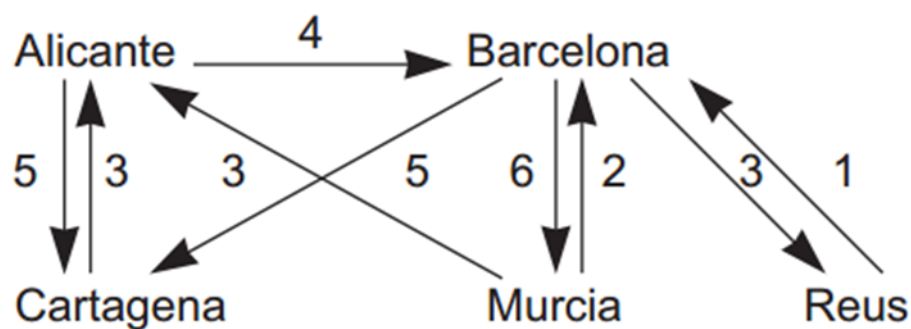


Figura 40: matriz grafo ponderado

5.2.2 Lista de adyacencia

Una lista de adyacencia es una lista vinculada donde cada elemento representa un nodo del grafo. Cada elemento contiene una lista de relaciones con otros nodos, siendo el nodo del elemento, el origen.

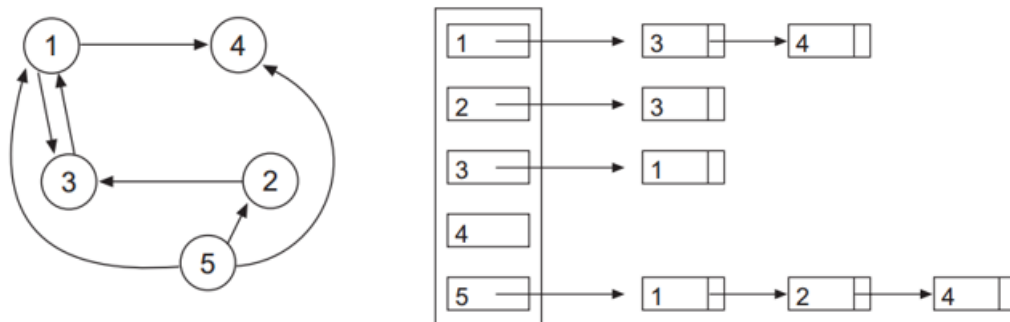


Figura 41: Lista de adyacencia

5.3 Recorridos de un grafo

Atravesar un grafo implica visitar todos los nodos accesibles comenzando desde un nodo específico

Algoritmo de recorrido básico:

- Sea V el conjunto de vértices del gráfico.
- Sea W el conjunto de nodos no visitados. Inicialmente solo contiene el nodo inicial v
- Sea Y el conjunto de nodos visitados.
- En cada paso del algoritmo, se elimina un nodo w del conjunto W , se procesa y para cada nodo adyacente de w , si no se ha visitado, se agregará a w . El nodo w se agrega a Y .
- El algoritmo termina cuando W está vacío

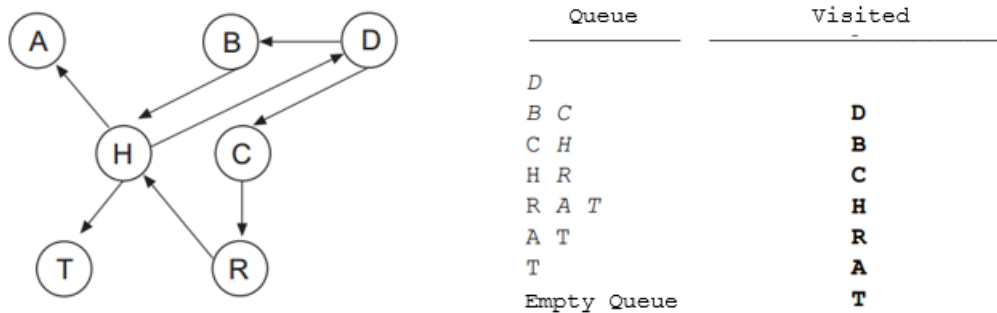
5.3.1 Breadth-First

Utiliza una cola que mantiene los vértices marcados.

FIFO logra que a partir de v , primero se procesen todos los vértices adyacentes, luego todos los adyacentes de los adyacentes de v ... y así sucesivamente

Algoritmo:

1. Marque el nodo de inicio v
2. Poner en cola el nodo de inicio v
3. Repita los pasos 4 y 5 hasta que la cola esté vacía.
4. Sacar de cola el nodo w de la cola, procesar w
5. Ponga en cola todos los nodos adyacentes a w que no estén marcados, nodos en cola marcados
6. Fin

**Figura 42:** breadth first alg

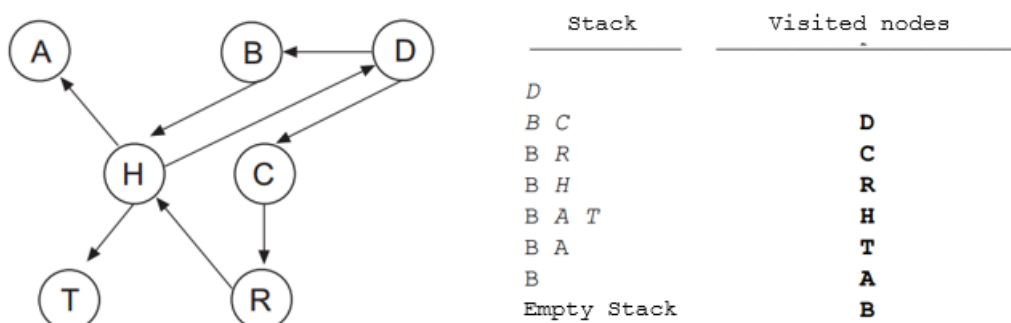
5.3.2 Depth-First

En Depth-First, el orden de procesamiento viene dado por un enfoque LIFO

Atravesar el grafo con un nodo v . v se marca como visitado y se empuja a la pila. La parte superior de la pila está reventada.

Cada nodo adyacente de v no visitado se empuja a la pila.

Esto continua hasta que no haya más elementos en la pila.

**Figura 43:** depth first

5.4 Camino más corto: Dijkstra

Uno de los problemas más comunes es determinar el camino más corto entre un par de nodos. Para este tipo de problema consideramos un grafo dirigido y ponderado.

La longitud del camino más corto es la suma del peso de cada arista. El algoritmo de Dijkstra encuentra el camino más corto desde un nodo de origen a todos los demás nodos en un gráfico con pesos positivos

Edsger Dijkstra (1930 - 2002) fue un informático holandés que dio forma a la programación informática como una ciencia reconocida.

¿Cómo funciona?

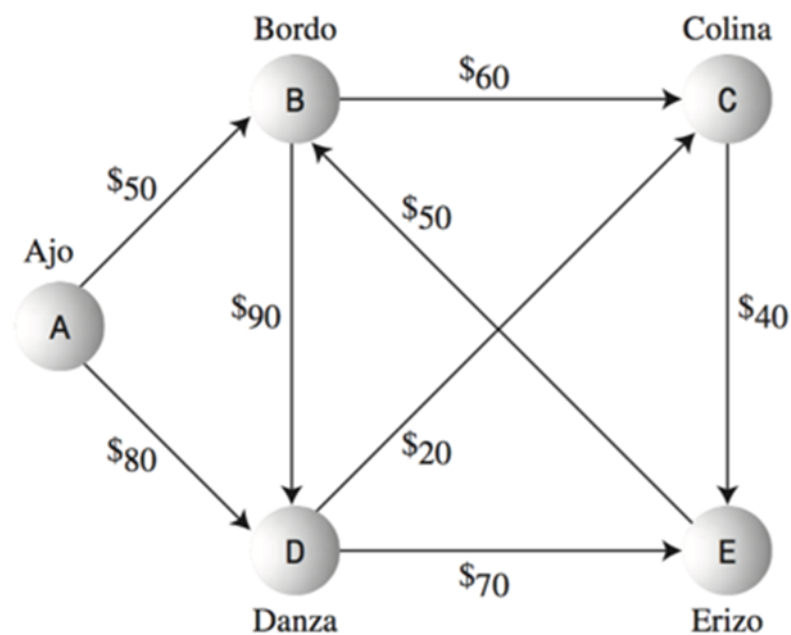


Figura 44: Dijkstra technique

1. Se utilizará una tabla donde la primera columna es el vertice, la segunda es el peso temporal que se le dará a un camino y en la tercera columna el peso final.

Vertice	Temporal	Final
A	0	0
B	50	50
C	110	110
D	80	80
E	150	150

2. Escoger un punto de salida y otro de entrada. $A \rightarrow E$.
3. Empezar por los nodos adyacentes al de salida y analizando su peso, este se coloca en la columna temporal.
4. Se debe escoger el de magnitud más corta y se coloca este número en peso final. A partir de este nodo se le analizan sus adyacentes pero la magnitud es la suma del peso final + el peso que exista en las aristas.
5. Se analiza cual es el menor y a partir de ahí se sigue analizando. *En este caso la distancia mas corta es $A \rightarrow D \rightarrow E$*

5.4.1 Estructura general

```
1  Foreach node set distance [node] = HIGH
2  SettledNodes = empty
3  UnSettledNodes = empty
4
5  Add sourceNode to UnSettledNodes
6  distance[sourceNode] = 0
7  while (UnSettledNodes is not empty) {
8      evaluationNode = getNodeWithLowestDistance(UnSettledNodes)
9      remove evaluationNode from UnSettledNodes
10     add evaluationNode to SettledNodes    evaluatedNeighbors(
        evaluationNode)
11 }
12 getNodeWithLowestDistance(UnSettledNodes){
13     find the node with the lowest distance in UnSettledNodes and return
        it
14 }
15 evaluatedNeighbors(evaluationNode){
16     Foreach destinationNode which can be reached via an edge from
        evaluationNode AND which is not in SettledNodes {
17         edgeDistance = getDistance(edge(evaluationNode, destinationNode
        ))
18         newDistance = distance[evaluationNode] + edgeDistance
19         if (distance[destinationNode] > newDistance) {
20             distance[destinationNode] = newDistance
21             evaluation.predecessor = evaluationNode
22             add destinationNode to UnSettledNodes
23         }
24     }
25 }
```

5.5 Camino más corto: Floyd

¿Cómo calcular el camino más corto de cada nodo a cada nodo? Existe una solución más directa que usar dijsktra de nodo en nodo.

El algoritmo de Floyd calcula mediante programación dinámica el camino más corto de cada nodo a cada nodo.

El algoritmo de Floyd representa el gráfico como una matriz ponderada. Cada arco (v_i, v_j) tiene un peso c_{ij} . Si el arco no existe, el valor es infinito. La diagonal de la matriz es igual a cero.

El algoritmo de Floyd determina una nueva matriz D de $n \times n$ elementos, donde cada D_{ij} es el camino mínimo de v_i a v_j

- En cada paso desde D_0 se genera una nueva matriz $D_1, D_2, \dots, D_k, D_n$. En cada paso se incluye un nuevo vértice para determinar si ese vértice mejora los caminos para que sean más cortos.

$$D_0[i, j] = \begin{cases} c_{ij} \text{ weight of the arc from vertex } i \text{ to vertex } j \\ \infty \text{ if there is no arc} \end{cases}$$

$$D_1[i, j] = \text{minimum}(D_0[i, j], D_0[i, 1] + D_0[1, j])$$

$$D_2[i, j] = \text{minimum}(D_1[i, j], D_1[i, 2] + D_1[2, j])$$

$$D_k[i, j] = \text{minimum}(D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j])$$

Figura 45: D matrix

- Otra matriz $Q_1, Q_2, \dots, Q_k, Q_n$ se genera en cada paso desde Q_0 . Q es la matriz predecesora.

$$Q_0[i, j] = \begin{cases} 0 \text{ if } i = j \text{ or } D_{ij} = \infty \\ i \text{ in all other cases} \end{cases}$$

$$Q_n[i, j] = \begin{cases} Q_{n-1}[i, j] \text{ if } D_{n-1}[i, j] \leq D_{n-1}[i, n] + D_{n-1}[n, j] \\ Q_{n-1}[n, j] \text{ if } D_{n-1}[i, j] > D_{n-1}[i, n] + D_{n-1}[n, j] \end{cases}$$

Figura 46: Q matrix

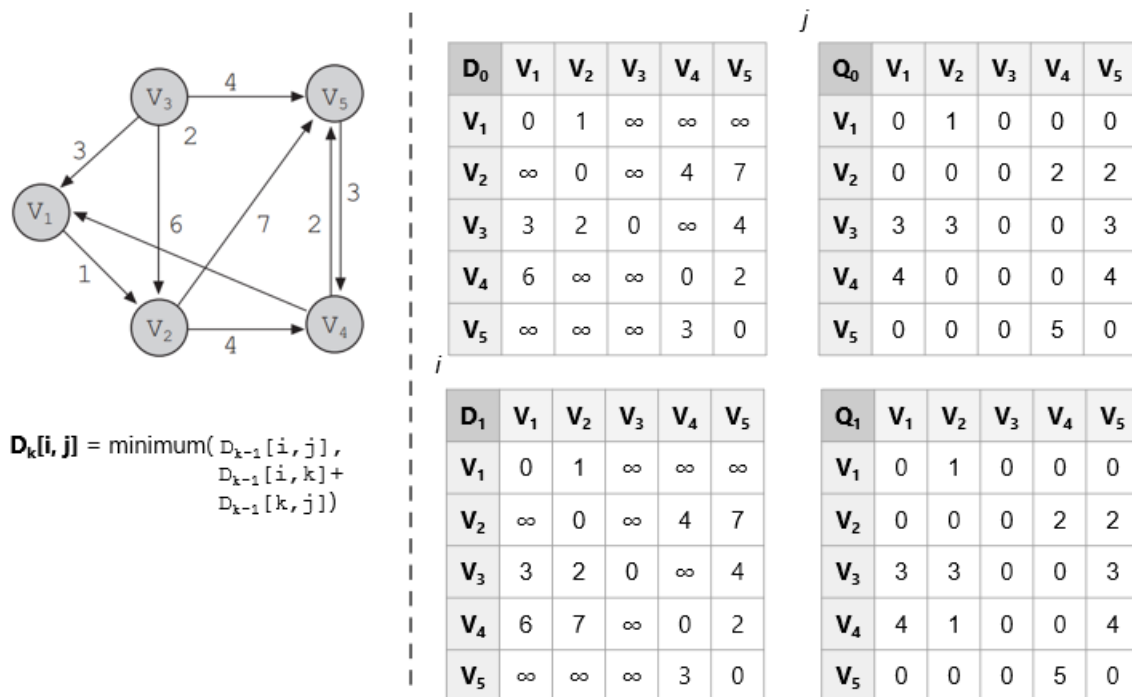


Figura 47: Floyd alg

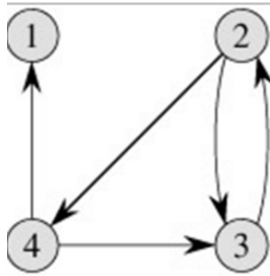
5.6 Warshall

Similar al algoritmo de Floyd. Calcula la matriz de camino P (también llamada cierre transitivo) de un grafo G de n vértices, representado por su matriz de adyacencia A.

Define una secuencia de matrices $n \times n$ $P_0, P_1, P_2, P_3, \dots, P_n$

$$\begin{aligned} \mathbf{P}_0[i, j] &= \begin{cases} 1 & \text{if there is an arc from } v_i \text{ to } v_j \\ 0 & \text{if there is no arc} \end{cases} \\ \mathbf{P}_1[i, j] &= \begin{cases} 1 & \text{if there is an arc from } v_i \text{ to } v_j \text{ that does not go through} \\ & \text{other vertex different of } v_1 \\ 0 & \text{if there is no arc} \end{cases} \\ \mathbf{P}_2[i, j] &= \begin{cases} 1 & \text{if there is an arc from } v_i \text{ to } v_j \text{ that does not go through} \\ & \text{other vertex different of } v_1 \dots v_2 \\ 0 & \text{if there is no arc} \end{cases} \\ \mathbf{P}_n[i, j] &= \begin{cases} 1 & \text{if there is an arc from } v_i \text{ to } v_j \text{ that does not go through} \\ & \text{other vertex different of } v_1 \dots v_n \\ 0 & \text{if there is no arc} \end{cases} \end{aligned}$$

Figura 48: Warshall algorithm



$$\begin{aligned}
 T^{(0)} &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{pmatrix} & T^{(1)} &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{pmatrix} & T^{(2)} &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix} \\
 T^{(3)} &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} & T^{(4)} &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}
 \end{aligned}$$

Figura 49: warshall algorithm2

5.7 Minimal Spanning tree

Se utiliza un grafo no dirigido para modelar relaciones simétricas entre vértices del gráfico. Cualquier arco (v,w) de un grafo no dirigido es igual que el arco de (w,v) .

Una tarea común es determinar si para cualquier par de vértices existe un camino que los conecte, es decir, **si es un grafo conexo**.

Un *árbol de expansión mínimo* es un subconjunto del grafo que cubre todos los vértices y cuyos bordes tienen una suma de los pesos mínimos. - Aplicado en redes

Un **árbol** es un subconjunto del grafo que está conexo y no tiene ciclos. - Si tiene n vértices, entonces tiene $n-1$ aristas. - Existe un camino único entre dos vértices cualesquiera de un árbol. - Si se agrega una ventaja, se produce un ciclo.

Si todos los vértices están en el árbol, entonces es un grafo conexo.

Dado un grafo no dirigido, encuentre el árbol de expansión mínimo

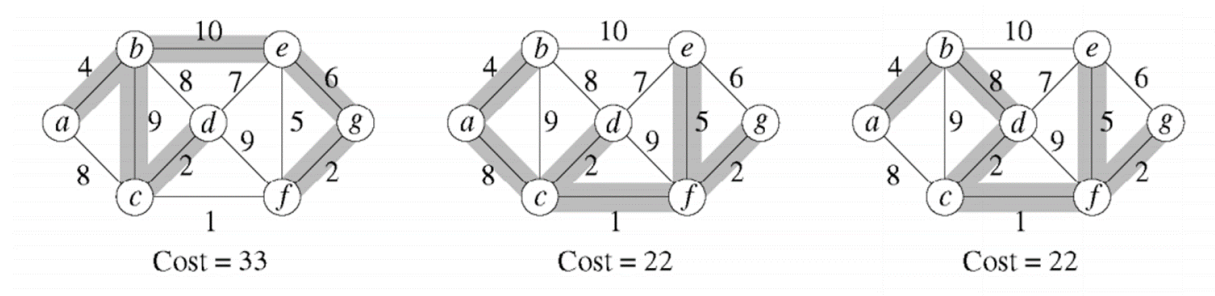


Figura 50: minimal spanning tree

- El mismo grafo puede tener varios arboles de expansión, pero no todos son el mínimo.

¿Como conseguirlo?

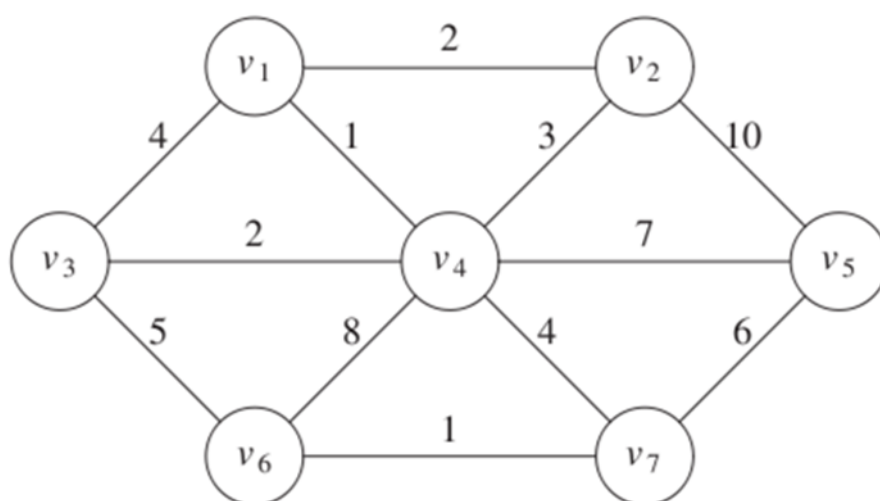
- Prim algorithm
- Kruskal algorithm

5.7.1 Prim

El árbol crece en etapas sucesivas. En cada etapa, se elige un nodo como raíz y agregamos un borde y el vértice asociado al árbol.

En cualquier punto tenemos un conjunto de vértices que ya han sido incluidos en el árbol.

El algoritmo encuentra un nuevo vértice para agregar al árbol eligiendo el borde (u,v) , tal como el costo de (u,v) es el más pequeño entre todos los bordes donde u está en el árbol y v no.

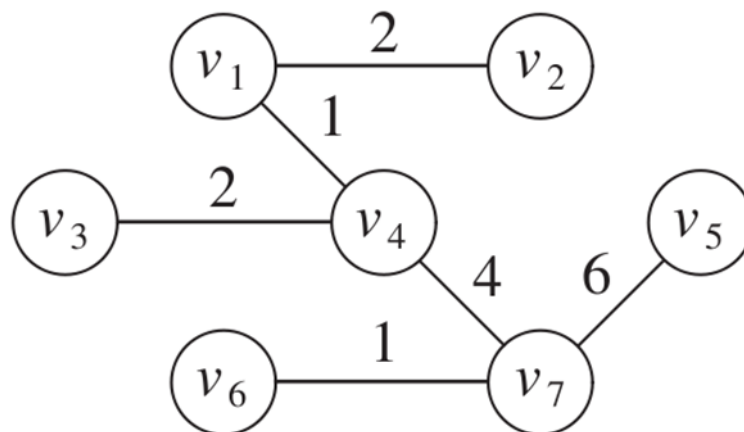
**Figura 51:** prim 1

v	$known$	d_v	p_v
v_1	F	0	0
v_2	F	∞	0
v_3	F	∞	0
v_4	F	∞	0
v_5	F	∞	0
v_6	F	∞	0
v_7	F	∞	0

Figura 52: prim table

Peso mínimo conectado a un nodo conocido

Al realizar todo el algoritmo el resultado se vería así:

**Figura 53:** prim result

v	$known$	d_v	p_v
v_1	T	0	0
v_2	T	2	v_1
v_3	T	2	v_4
v_4	T	1	v_1
v_5	T	6	v_7
v_6	T	1	v_7
v_7	T	4	v_4

Figura 54: prim table result

5.7.2 Kruskal

Selecciona bordes si el orden es de menor peso y acepta un borde si no causa un ciclo

Mantiene un bosque (colección de árboles). Inicialmente todos son árboles de un solo nodo. Agregar un borde fusiona dos árboles en uno

Si u y v están en el mismo conjunto, la arista (u,v) se rechaza, porque sumarla causaría un ciclo. u y v están en el mismo conjunto si están conectados.

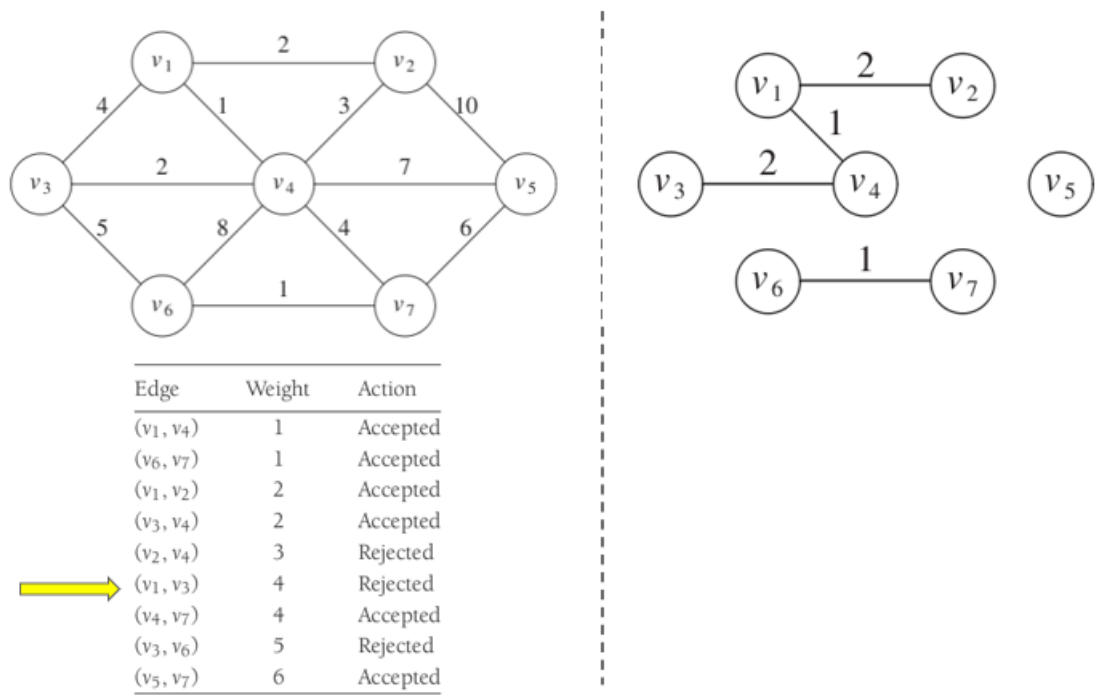


Figura 55: Kruskal

6 Administración de Memoria

La administración de memoria conlleva diferentes ideas según el contexto. En el caso de los sistemas operativos, la administración de memoria se refiere a la gestión de la memoria principal de un sistema informático. En el caso del desarrollo de software, la administración de memoria se refiere a la gestión de la memoria de un programa en ejecución. Para lograr existosamente la segunda, se requiere comprender la primera.

En este capítulo, se abordarán los conceptos básicos de la administración de memoria en sistemas operativos y se entra en detalle a la administración de memoria en programas en ejecución.

6.1 Breve introducción a sistemas operativos

Capa de software que interactúa directamente con el hardware, intermediario entre el hardware que posee una computadora y las aplicaciones de software, facilitando la gestión de recursos del sistema.

6.1.1 El sistema operativo como API para los developers

API corresponde a las siglas de la palabra *Application Programming Interface*, concepto aplicable a varios contextos: API web, API del sistema operativo, API class libraries, etc; en nuestro caso, nos interesa el API del sistema operativo. Sea cual sea el contexto, la idea es la misma: *proveer una capa de abstracción para que los desarrolladores puedan interactuar con un sistema sin necesidad de conocer los detalles de su implementación.*

Un estándar para las API de los S.O es el **posix**. El posix son las siglas de la palabra en inglés *portable operative interface for unix*. Es un **estándar del IEEE para los APIS del S.O.**

Un ejemplo de API por ejemplo para abrir un archivo, se utiliza la función **fopen** que retorna un *pointer* o *handler* al archivo.

```
fptr = fopen("archivo.txt")
```

6.1.2 El sistema operativo como administrador de los recursos de la máquina.

El hardware es complejo. Si no hubiera un ente especializado en la administración de los recursos de la máquina, los desarrolladores tendrían que lidiar con la complejidad del hardware directamente. El sistema operativo se encarga de la administración de los recursos de la máquina, permitiendo a los desarrolladores interactuar con la máquina de forma más sencilla. *El sistema operativo, hace transparentes los recursos de la máquina.*

Los recursos administrados son:

- CPU
- Memoria Principal
- Discos
- I/O

Thread vs Process

Un programa es un archivo ejecutable que contiene el código o el conjunto de instrucciones del procesador, que se almacena como un archivo en el disco. Cuando un programa se ejecuta, se carga en memoria y se convierte en un **proceso**. Un proceso activo incluye los recursos administrados por el sistema operativo que el programa necesita para ejecutarse: registros de procesador, contadores de programas, punteros de pila, páginas de memoria, etc.

Un **thread (hilo)** es un flujo de control dentro de un proceso, y cada hilo tiene su propia pila de llamadas, registros de procesador y estado de ejecución. Múltiples hilos comparten el mismo espacio de direcciones de memoria, archivos abiertos, etc.

6.2 Definición de Administración de Memoria

Es el conjunto de tareas que realiza el sistema operativo para gestionar la memoria principal de una computadora. La administración de memoria es una parte fundamental de los sistemas operativos modernos, ya que permite a los programas ejecutarse sin tener que preocuparse por la gestión de la memoria.

Algunas de las responsabilidades de la administración de memoria son:

- Asignar/Liberar memoria al inicio/fin de un proceso.
- Controlar la memoria asignada a un proceso.
- Minimizar la fragmentación.
- Mantener la integridad de los datos.

La idea de controlar la memoria asignada a un proceso, tiene como fin hacer que el mismo tenga un límite y aislamiento, para que así no afecte a otros procesos en la memoria que se estén ejecutando.

6.2.1 Las direcciones de memoria

Antes de continuar con el resto de este capítulo, es clave entender el concepto de direcciones de memoria. Para esto utilizaremos un cotidiano.

Imagine un edificio de apartamentos, en el que todos los apartamentos son exactamente del mismo tamaño y están numerados de manera consecutiva. Cada apartamento únicamente puede alojar a una sola persona. Una familia por lo tanto, ocupará varios apartamentos contiguos. El edificio puede continuar creciendo con el tiempo conforme se construyan nuevos pisos, pero la constructora tendrá un máximo de apartamentos que puede construir. La recepción del edificio puede llevar correspondencia a cualquier apartamento, incluso cuando se construyan nuevos apartamentos, pero siempre limitado por el número de apartamentos que se pueden construir y por el número de apartamentos que ya están contruídos.

Aplicando el ejemplo a la computadora, un sistema operativo (*la recepción*) tiene la capacidad de generar direcciones de memoria de n bits (usualmente 32 o 64 bits). Por ejemplo, si es son direcciones de 32 bits, el sistema operativo puede generar 2^{32} direcciones de memoria, desde la dirección 0 hasta la dirección $2^{32} - 1$. Cada dirección de memoria corresponde a una casilla (*un apartamento*) que puede alojar 1 byte (*una persona*). Las variables (*familias*) puede ser de 1 o más bytes, siempre contiguos.

Si el usuario (*la constructora*) instala más RAM, el sistema operativo podrá accederlo hasta el máximo que sus direcciones lo permitan. Por ejemplo, si el sistema operativo tiene direcciones de 32 bits, podrá acceder a 2^{32} bytes de memoria, es decir 4GB. Si el usuario instala 8GB de RAM, el sistema operativo solo podrá acceder a 4GB, ya que no tiene direcciones para más.

Suponiendo un sistema operativo con direcciones de 16 bits, el sistema operativo podrá acceder a 2^{16} bytes de memoria, es decir 64KB. El rango de direcciones sería de 0 a 65535.

6.3 Evolución de la Administración de memoria

La memoria ha sido un componente fundamental en las computadoras desde sus inicios y por ende ha necesitado del sistema operativo para administrarla. Conforme los recursos computacionales se adaptan a las necesidades de los usuarios y las prestaciones de hardware, la administración de memoria se adapta para reducir el *overhead* y mejorar la eficiencia.

A continuación se presenta la evolución de la abstracción de la memoria en los sistemas operativos.

6.3.1 Ninguna abstracción de memoria

La abstracción más simple es no tener ninguna. Las computadoras mainframe de los años 60, mini-computadoras de los 70s y computadoras personales en los 80s, no tenían abstracción de memoria. Los programas accedían directamente a la memoria **física**. Un programa con la instrucción `MOV REGISTER1, 1000` en realidad accedía a la dirección de memoria 1000.

Había cierta organización de la memoria:

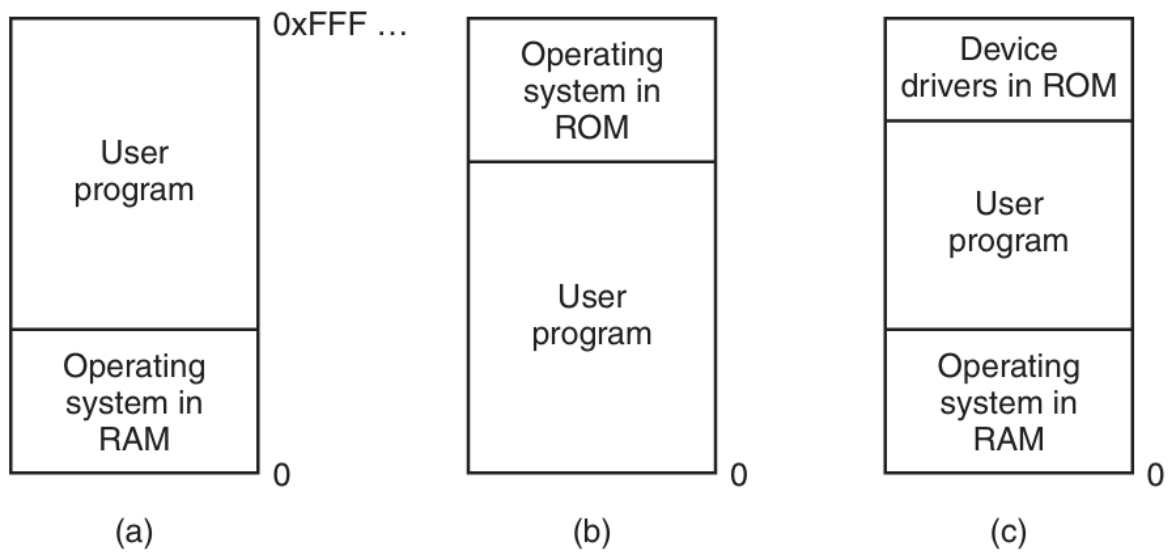
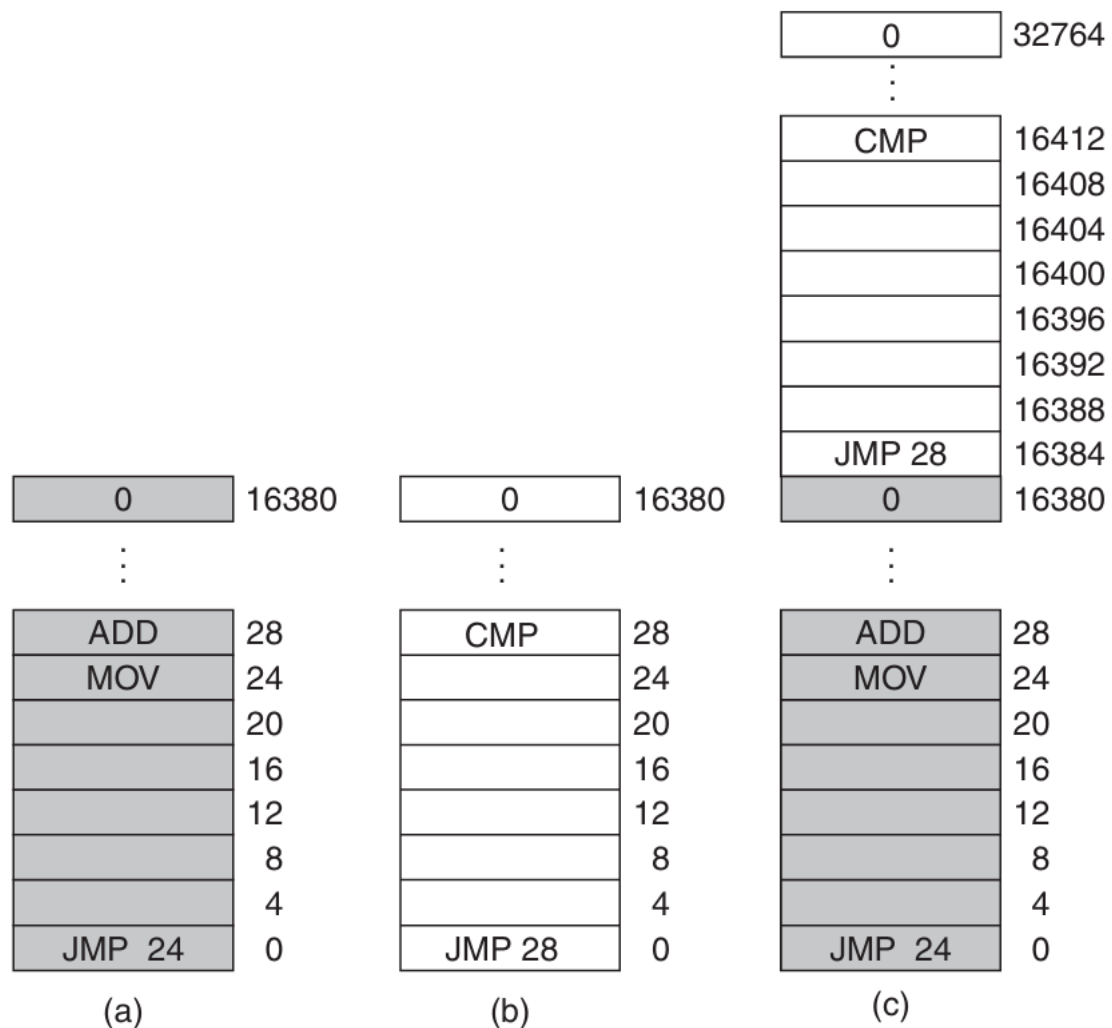


Figura 56: Organización de la memoria sin abstracción

Bajo estas condiciones, era imposible ejecutar más de un programa a la vez (*multiprogramación*), ya que cada programa accedía directamente a la memoria física. Dos o más programas cargados en memoria, podían afectarse entre sí causando errores en la ejecución. El uso de threads era posible dado que compartían la misma memoria, pero de poco valor.

**Figura 57:** El problema de la re-ubicación

El *swapping* se introduce como técnica para “pausar” la ejecución de un, guardar su estado en disco y cargar otro programa en memoria. Esta técnica permitió la multiprogramación *a cierto grado*, dado que solo un programa podía estar cargado en memoria en un momento dado. La idea era que mientras un programa estaba esperando por una operación de I/O, otro programa podía ejecutarse.

La computadora *IBM 360*, introduce una técnica para poder tener dos programas en memoria: *static relocation*. Cuando un programa se cargaba en memoria, se le asignaba una dirección base (la dirección inicial en la que se carga) y se modificaban todas las referencias a memoria para sumarle la dirección base. Esto funcionaba pero claramente no era eficiente puesto que entre más grande fuera el programa, más tarda en cargarse.

Multiprogramación

Se denomina multiprogramación a una técnica por la que dos o más procesos pueden alojarse en la memoria principal y ser ejecutados concurrentemente por el procesador o CPU. [...] la ejecución de los procesos (o hilos) se va solapando en el tiempo a tal velocidad, que causa la impresión de realizarse en paralelo (simultáneamente)

[...] En los antiguos sistemas monoprogramados, cuando un proceso en ejecución requería hacer uso de un dispositivo de E/S, el procesador quedaba ocioso mientras el proceso permaneciese en espera y no retomara su ejecución *de Wikipedia*

6.3.2 Espacios de direcciones

Constituye una abstracción para la memoria física. Cada programa tiene su propio espacio de direcciones, que va desde 0 a un valor máximo. El sistema operativo se encarga de mapear las direcciones virtuales a direcciones físicas. Dos programas puede ver la dirección 28, pero en realidad se refieren a direcciones físicas diferentes.

Bajo este enfoque, el hardware provee dos registros llamados *base* y *limite*. Cuando un programa específico se ejecuta, el sistema operativo carga el *base* y *limite* con los valores correspondientes al espacio de direcciones del programa. Cada vez que el programa accede a una dirección de memoria, el hardware verifica que la dirección esté dentro del rango permitido por el *base* y *limite* y genera la dirección física correspondiente.

Una limitante de este enfoque es que el programa completo debe caber en la memoria, lo cual es claramente poco práctico para los programas modernos con requerimientos de memoria cada vez más agresivos y que compiten con muchos otros programas ejecutándose concurrentemente.

Concurrencia vs Paralelismo

La concurrencia se refiere a la capacidad de un sistema de llevar a cabo múltiples tareas en un mismo periodo de tiempo traslapándose entre sí, pero no implica que se ejecuten a la misma vez. En paralelismo, las tareas se ejecutan al mismo tiempo. Paralelismo implica múltiples núcleos de procesamiento. La concurrencia puede lograrse con un solo núcleo bajo multiprogramación.

6.3.3 Memoria virtual

La memoria virtual es la solución para poder ejecutar programas que no caben en la memoria física. La idea básica es que cada programa tiene su propio espacio de direcciones dividido en bloques llamados *páginas*. Cada página es un rango contiguo de direcciones.

Las páginas se mapean a bloques de memoria física llamados *frames*, pero no todas las páginas necesitan estar cargadas. Cuando un programa referencia una página que está cargada en memoria (*page-hit*), el hardware mapea la dirección virtual a la dirección física correspondiente. Si la página no está cargada, el sistema operativo la carga desde el disco a un frame libre en memoria y re-ejecuta la instrucción que causó el fallo de página (*page-fault*).

Entonces, cuando un programa tiene una instrucción `MOV REGISTER1, 1000`, la dirección 1000 es una dirección virtual parte de su espacio de direcciones virtuales.

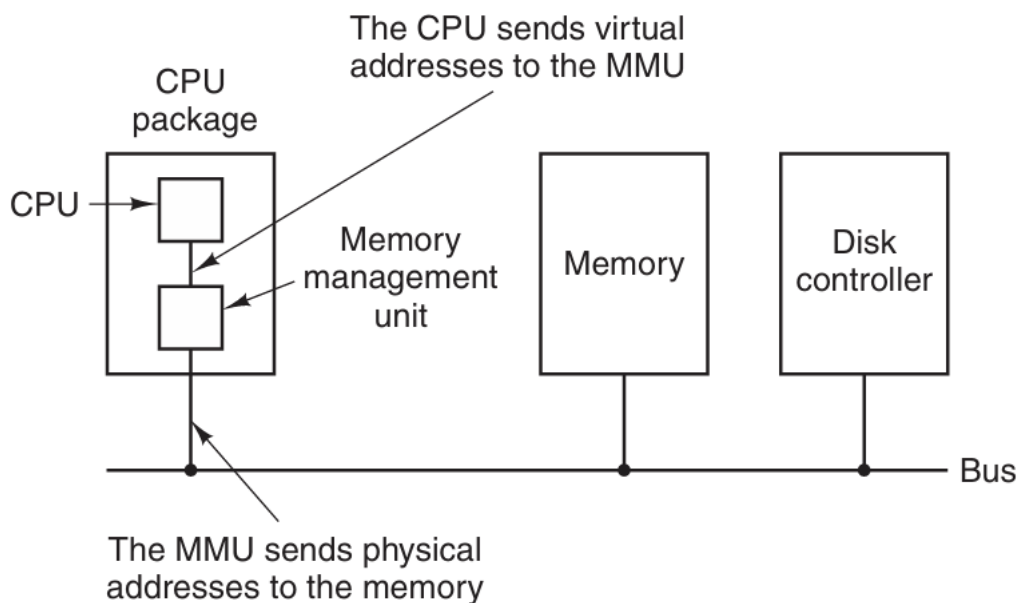


Figura 58: Funcionamiento del MMU

Como se nota en la imagen anterior, hay hardware especializado, el *Memory Management Unit (MMU)* que se encarga de mapear las direcciones virtuales a direcciones físicas. El MMU tiene una tabla de páginas que mapea las direcciones virtuales a direcciones físicas. La tabla de páginas se mantiene en memoria y el MMU la consulta cada vez que necesita mapear una dirección virtual a una dirección física.

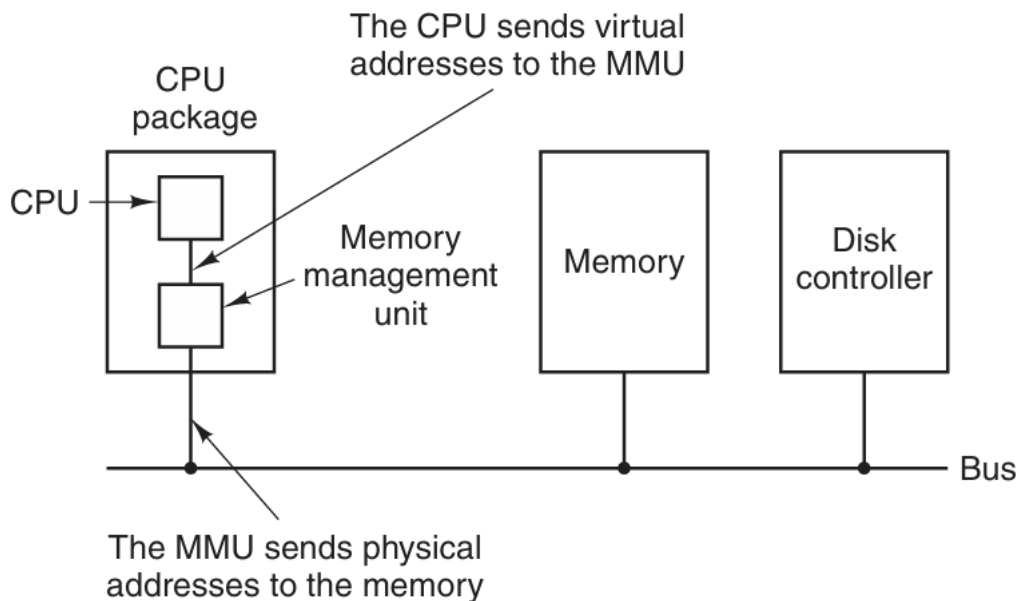


Figura 59: Relación entre direcciones virtuales y físicas

El MMU tiene una tabla que lleva el inventario de páginas cargadas y su correspondiente frame. Dicha tabla aloja información estadística sobre las páginas, como la frecuencia de uso, para poder tomar decisiones sobre qué páginas mantener en memoria y cuáles sacar. Dado que los frames son limitados, se utilizan algoritmos de reemplazo de páginas para decidir cuál página sacar de memoria cuando se necesita cargar una nueva y no hay espacio.

6.4 Administración de memoria a nivel de proceso

Para un proceso (programa en ejecución con recursos asignados), la administración de memoria se refiere a la gestión de la memoria asignada. La abstracción utilizada por el sistema operativo, es transparente para el proceso, el cuál únicamente utiliza los mecanismos disponibles para manipular la memoria.

Dependiendo del lenguaje y compilador, el layout de memoria puede variar. Un programa en C, usualmente tiene el siguiente layout de memoria:

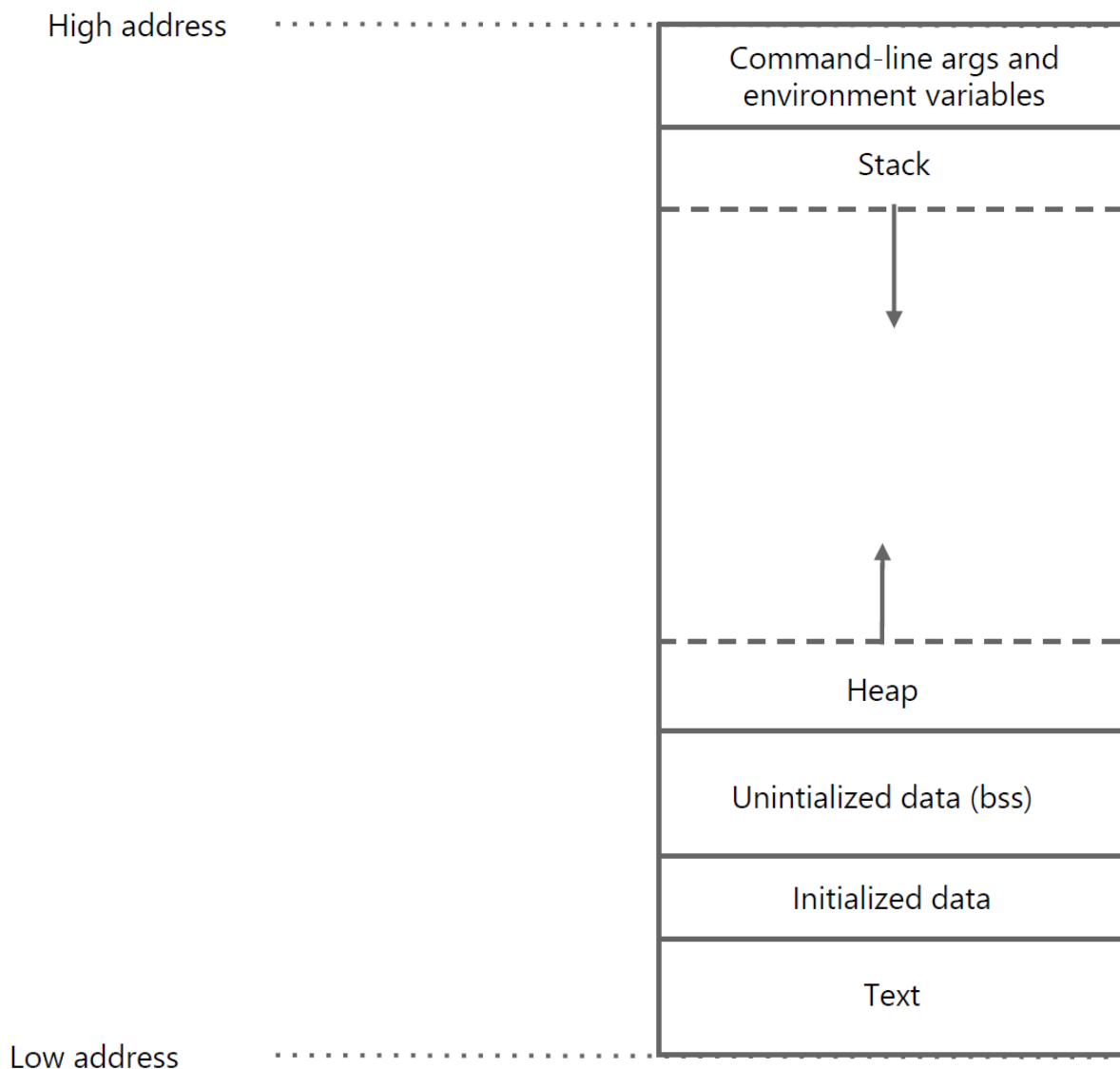


Figura 60: Layout de memoria de un programa en C

A continuación se describen a mayor detalle cada una de estas partes.

Se recomienda leer este artículo de [GeeksForGeeks](#) con detalles sumamente relevantes del memory layout

6.4.1 Secciones del memory layout

6.4.1.1 Text Contiene el código **ejecutable**. Usualmente es compartido entre procesos de un mismo programa. Es de solo lectura.

6.4.1.2 Initialized Data Llamado también el segmento de datos. Contiene las variables **globales y/o estáticas** inicializadas. Es de lectura y escritura. Tiene dos áreas: una para variables read-only y otra para variables read-write. Por ejemplo

```
1 int x = 10; //read-write
2 const int y = 20; //read-only
3 char s[] = "ho!a"; //read-write
4
5
6 int main() {
7     // Código principal
8 }
```

¿static en C? Al usarlo sobre variables que están dentro de una función, permite que el valor de las mismas persista entre llamadas. Al usarlo sobre funciones o variables de ámbito global, garantiza que dicho elemento sólo exista en la unidad de compilación en la que se encuentre declarado. *fuentes: stackoverflow*

6.4.1.3 Uninitialized Data Segment Usualmente llamado el segmento bss (block started by symbol). Contiene las variables **globales y/o estáticas** no inicializadas. Es de lectura y escritura. Los datos en este segmento, son inicializados en cero por el compilador antes de que el programa empiece a ejecutarse.

Por ejemplo,

```
1 int x; //uninitialized
2 int y; //uninitialized
3
4 int main() {
5     // Código principal
6
7     static int z; //uninitialized
8 }
```

6.4.1.4 Command-line arguments and environment variables Esta sección de la memoria se carga con los argumentos de la línea de comandos y las variables de entorno. Por *argumentos de la línea de comandos* se entiende los argumentos que se pasan al programa al momento de ejecutarlo. Por ejemplo, `./programa -a -b -c_`. Estos argumentos se pueden acceder programáticamente mediante:

```
1 int main(int argc, char* argv[]) {
2     // argc es el número de argumentos
3     // argv es un arreglo de strings con los argumentos
4     // argv[0] es el nombre del programa
```

```
5 // argv[1] es el primer argumento
6 // argv[2] es el segundo argumento
7
8 // Ejemplo
9 printf("El primer argumento es %s\n", argv[1]);
10 }
```

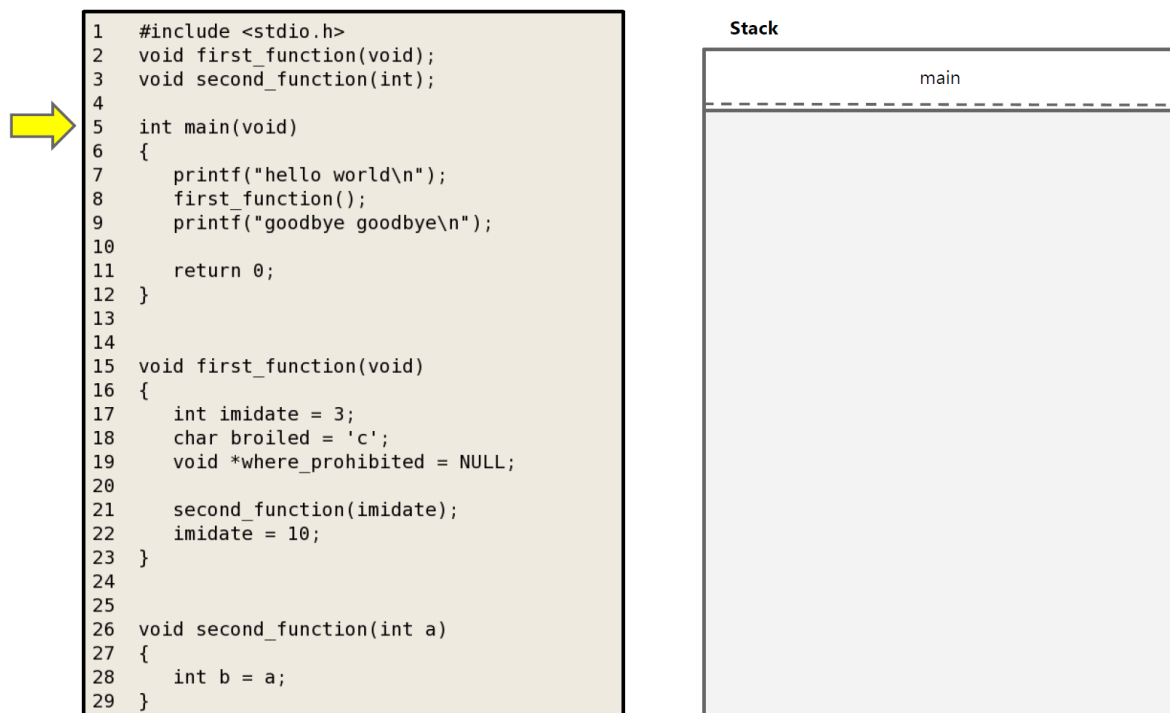
Las *variables de entorno* son variables que se definen en el sistema operativo y que pueden ser accedidas por los programas. Cuando se ejecuta un programa en la terminal, se pueden definir variables de entorno. Por ejemplo (en bash), `export MYVAR="Ejemplo de variable"`. Estas variables se pueden acceder programáticamente mediante la variable global `environ`.

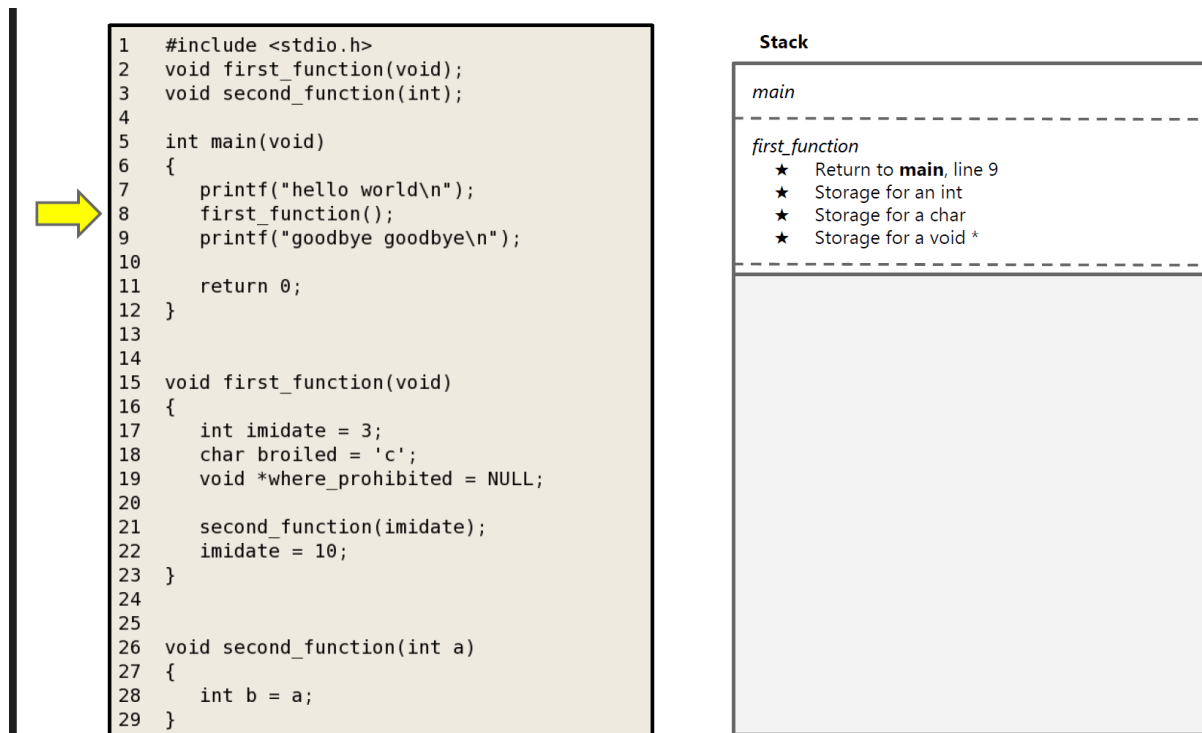
```
1 extern char** environ;
2
3 int main() {
4     // Iterar sobre las variables de entorno
5     for (int i = 0; environ[i] != NULL; i++) {
6         printf("%s\n", environ[i]);
7     }
8 }
```

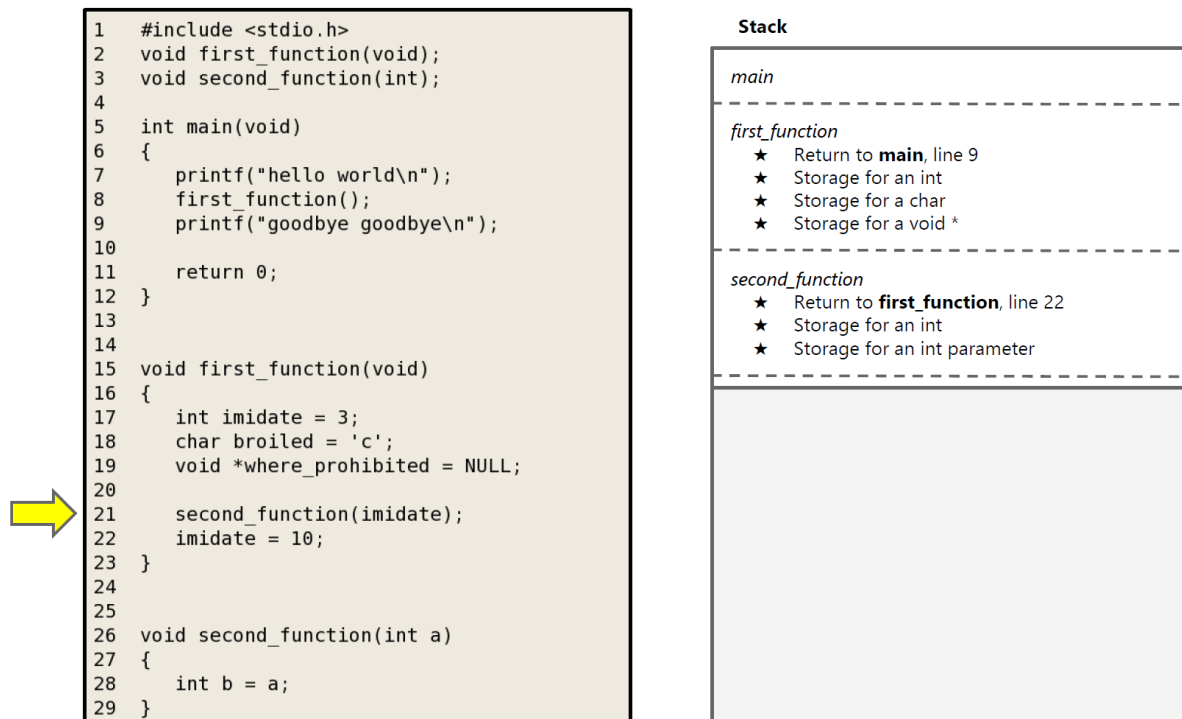
6.4.1.5 Stack Es una sección de la memoria que se utiliza para almacenar las variables locales de las funciones y los argumentos de las mismas. Es de tamaño fijo y se expande y contrae dinámicamente. Utilizar el stack es transparente (e inevitable) para el programador, por lo que el manejo de la memoria es menos propenso a errores.

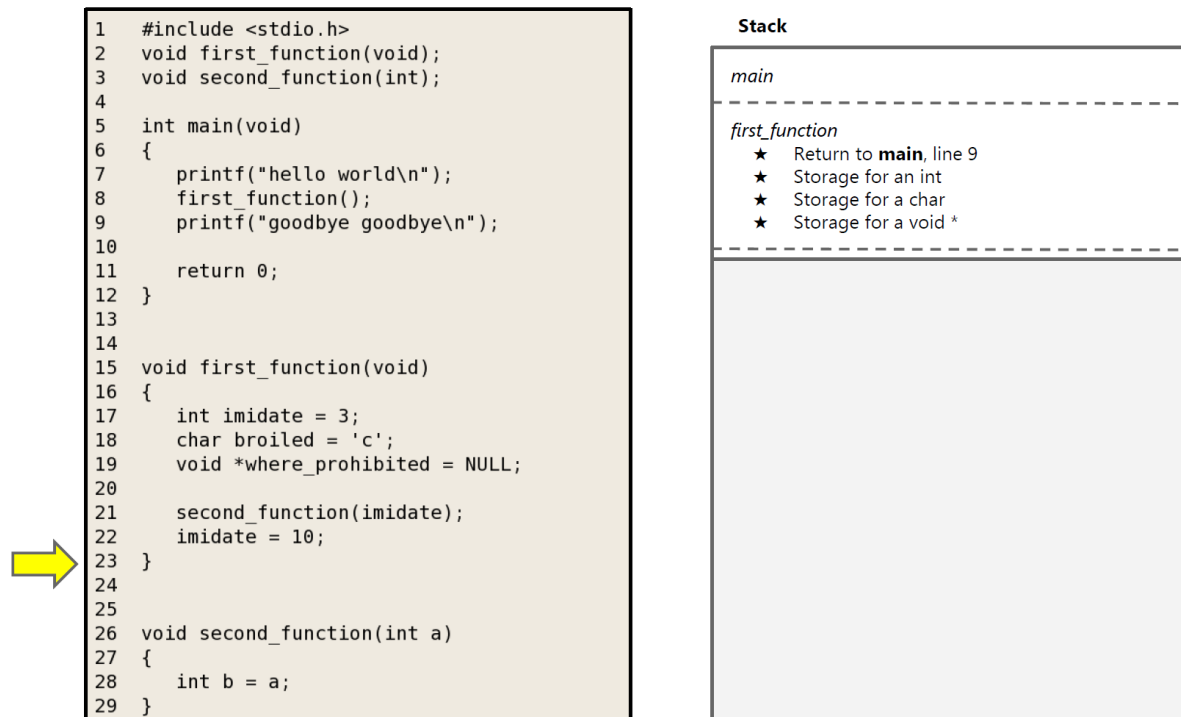
El stack es *LIFO* (Last In, First Out). Cada vez que se llama a una función, se crea un *stack frame* que contiene las variables locales de la función, los argumentos y el *return address* (la dirección a la que se debe regresar una vez que la función termina). Cuando dicha función termina, el *stack frame* se elimina (se marca la memoria como libre). Es por tal razón que las variables locales se les llama *automáticas*.

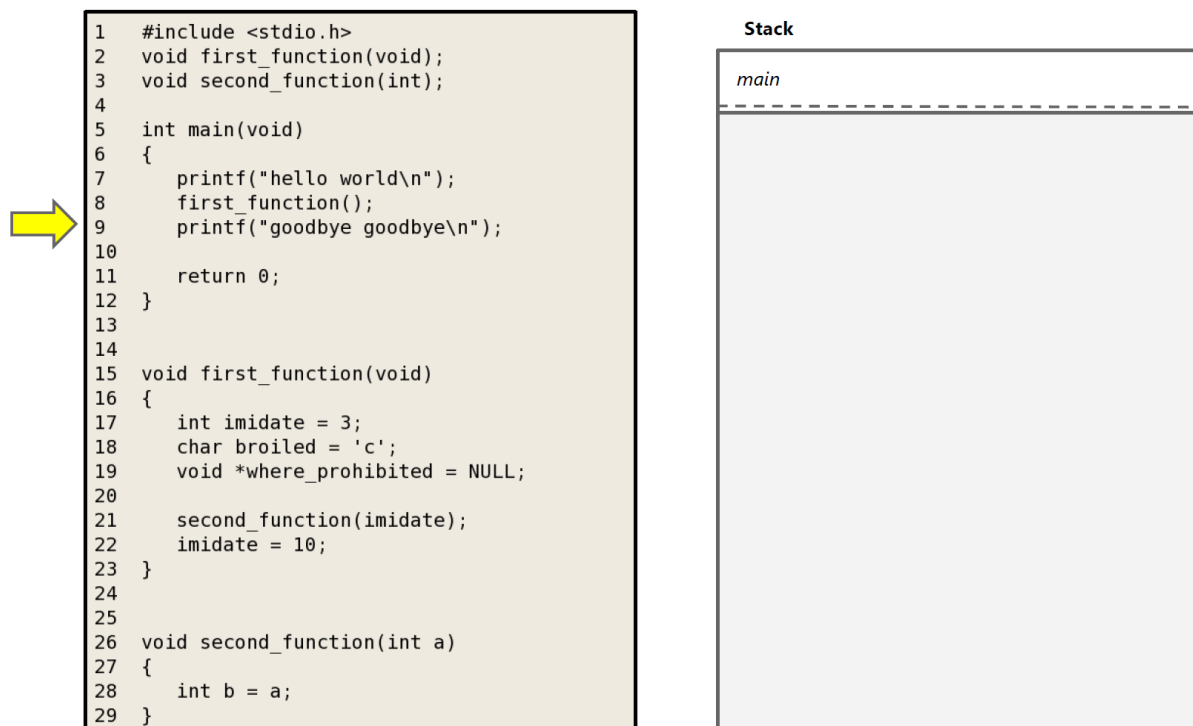
Las variables locales almacenables en el stack deben ser de tamaño conocido al momento de la compilación. Por esta razón, memoria dinámica como listas enlazadas no puede almacenarse en el stack.

**Figura 61:** Visualización del stack (1 de 5)

**Figura 62:** Visualización del stack (2 de 5)

**Figura 63:** Visualización del stack (3 de 5)

**Figura 64:** Visualización del stack (4 de 5)

**Figura 65:** Visualización del stack (5 de 5)

6.4.1.6 Heap Es una sección de la memoria que se utiliza para almacenar datos que no tienen un tamaño conocido al momento de la compilación y cuyo tiempo de vida es controlado por el programador. Por ejemplo, listas enlazadas, árboles, etc. El heap es de tamaño variable y se expande y contrae dinámicamente. El programador es responsable de manipular la memoria en el heap, es decir, asignar, des-asignar y re-dimensionarla.

Para interactuar con la memoria, el programador utiliza funciones como `malloc`, `free`, `realloc`, `calloc` y `new/delete` (en C++). Estas funciones conforman el API del heap en C/C++.

¿Es posible evitar usar el Heap?

Sí, es posible evitar usar el heap. Sin embargo, esto implica que el programador únicamente podrá utilizar variables de tamaño conocido en tiempo de compilación, limitando la flexibilidad y el alcance de lo que el programa puede realizar.

El siguiente código muestra un ejemplo de cómo se puede utilizar el heap en C:

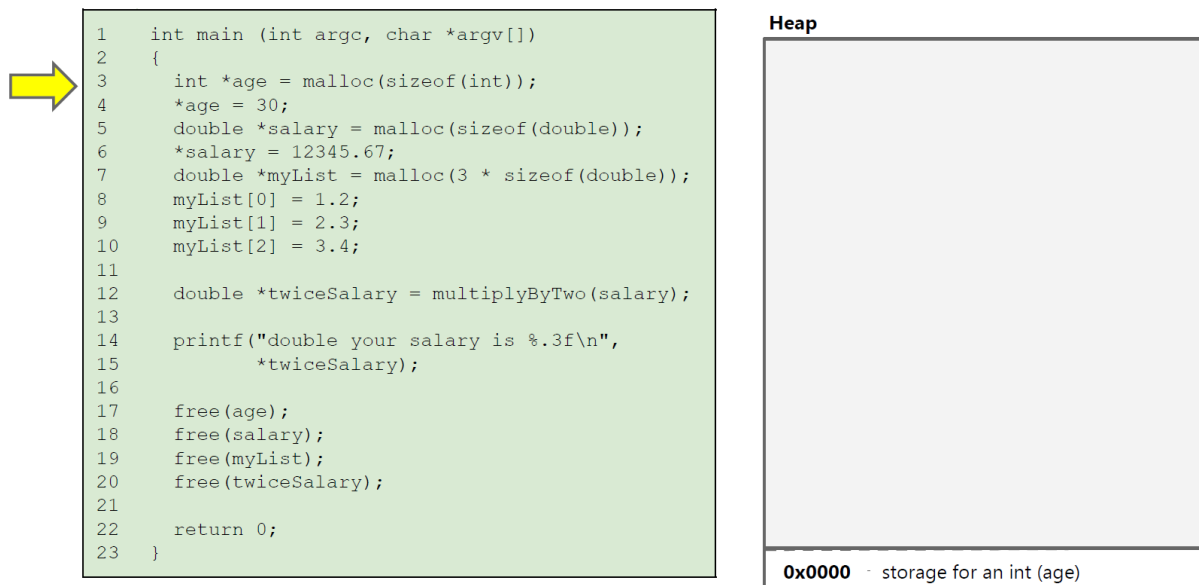


Figura 66: Visualización del heap (1 de 4)

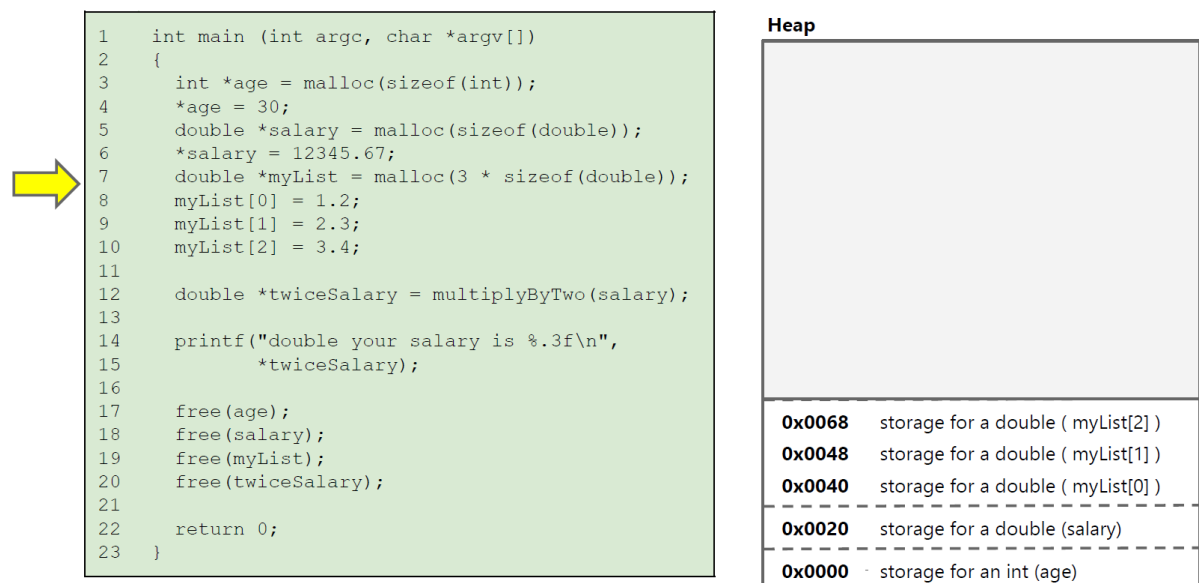


Figura 67: Visualización del heap (2 de 4)

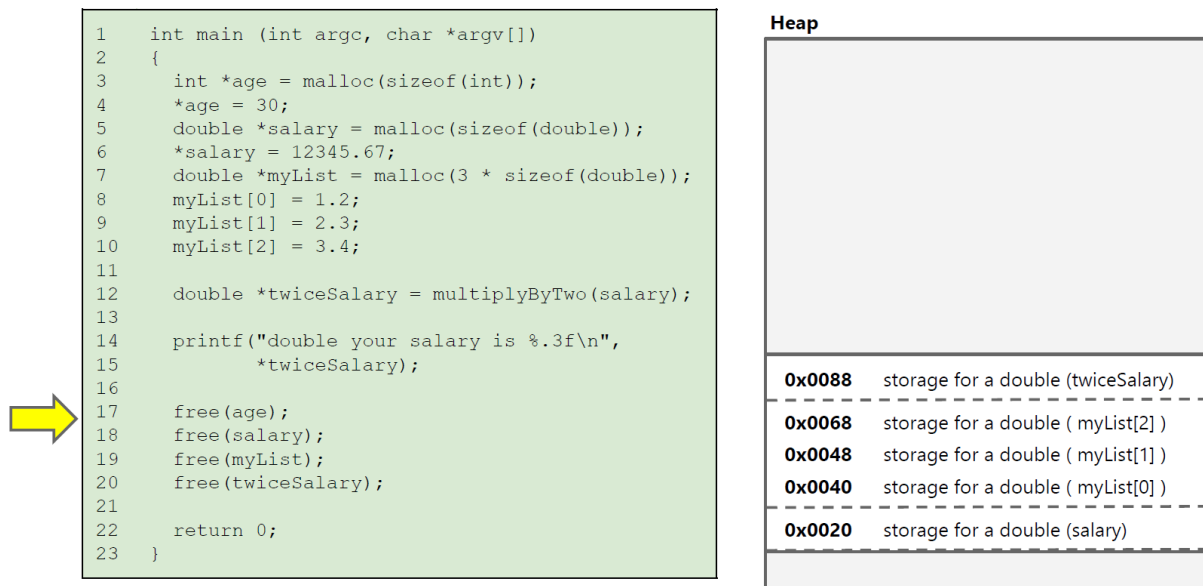


Figura 68: Visualización del heap (3 de 4)

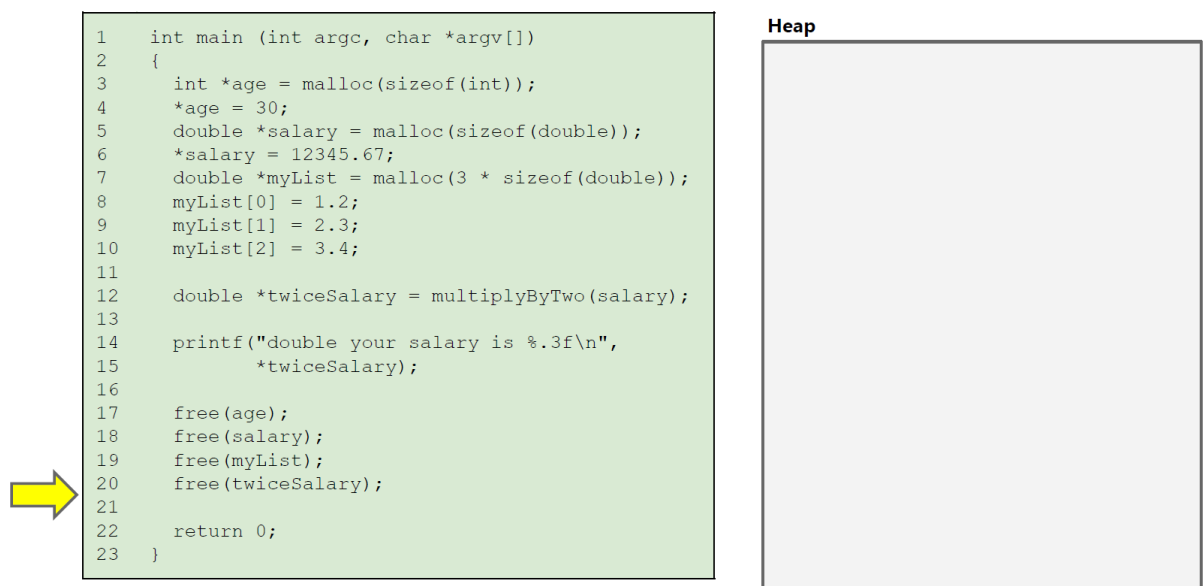


Figura 69: Visualización del heap (4 de 4)

¿Qué pasa si no se libera la memoria en el heap?

Si no se libera la memoria en el heap, se produce una fuga de memoria (*memory leak*). Esto significa

que la memoria asignada al programa no se libera y se pierde. Con el tiempo, el programa puede quedarse sin memoria y fallar.

En los ejemplos de código anteriores se utilizan punteros, por ejemplo `int* age = malloc(sizeof(int))`. En la siguiente sección se abordarán los punteros en mayor detalle.

6.4.1.7 Heap vs Stack

Heap	Stack
Memoria dinámica	Memoria estática
Tamaño variable	Tamaño fijo
Programador es responsable de la memoria	Programador no es responsable de la memoria
No es transparente	Transparente
Propenso a errores (bugs introducidos por el programador)	Menos propenso a errores (bugs del sistema operativo)

6.4.2 Punteros

Es un tipo de datos especial definido como parte del API del Heap. Al ser un tipo de datos, define un rango posible de valores que puede contener y un conjunto de operaciones que soporta.

- Rango de valores de un puntero: direcciones de memoria
- Operaciones soportadas: asignación, des-asignación, aritmética de punteros, acceso a memoria

Para declarar un puntero se utiliza código similar al siguiente:

```
1 int* ptr = malloc(sizeof(int));
2 char* cptr = malloc(sizeof(char));
3 double* dptr = malloc(sizeof(double));
4 MyClass* myptr = new MyClass(); // En C++, dado que C no soporta clases
5 void* vptr = malloc(100);
6
7 // Si NULL no estuviera definido, se podría usar 0 o definirlo
  manualmente
8 int* nPtr = NULL;
9
10 // 0 es el único valor literal que se puede asignar a un puntero en C.
11 // Equivalente a NULL
12 int* nPtr2 = 0;
13 int* nPtr3 = nullptr; // En C++ 14
```

Si visualizamos la memoria como una tabla con columnas y filas, para el código anterior tendríamos:

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	ptr	0x64	4B	Stack	Pointer
0x4	cptr	0x68	4B	Stack	Pointer
0x8	dptr	0x69	4B	Stack	Pointer
0x12	myptr	0x72	4B	Stack	Pointer
0x16	vptra	0x92	4B	Stack	Pointer
0x1A	nPtr	0x0	4B	Stack	Pointer
0x1E	nPtr2	0x0	4B	Stack	Pointer
0x22	nPtr3	0x0	4B	Stack	Pointer
0x64		0	4B	Heap	int
0x68		"	1B	Heap	char
0x69		0	8B	Heap	double
0x72		0	20B	Heap	MyClass
0x92		0	100B	Heap	?

Las direcciones de la tabla son ficticias y no corresponden a direcciones reales de memoria. Asumimos que el heap empieza en la dirección hexadecimal 64. Asumimos que cada dirección de 32 bits, es decir 4 bytes. Asumimos que la clase `MyClass` tiene un tamaño de 20 bytes.

Como se puede notar, para acceder a la memoria, se necesita dos componentes: la llamada al API (en este caso `malloc`) y un puntero para poder acceder a la memoria creada por el API. El puntero siempre estará en el stack, y es una variable automática como cualquier otra. Sin embargo, al liberarse junto con el frame, la memoria en el Heap no se libera.

Una llamada a `malloc` sin asignar un puntero, por ejemplo

```
1 malloc(sizeof(int));
```

Resulta en una tabla de memoria como:

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x64		0	4B	Heap	int

Pero al no haber ningún pointer en el stack que almacene la dirección 0x64, dicha memoria es inaccesible y se produce una fuga de memoria.

No hay nada “mágico” con respecto a los punteros. Son simplemente un tipo de dato como cualquier otro. La diferencia es que los punteros contienen direcciones de memoria en lugar de valores.

¿Cuál es el propósito de declarar pointers con tipo si todos ocupen el mismo espacio?

Type check: El compilador pueda hacer type checking. Por ejemplo, si se declara un puntero de tipo **int**, el compilador no permitirá asignarle una dirección de memoria de un **char**.

Read/Write size: El compilador sabe cuántos bytes leer o escribir al acceder a la memoria a través de un puntero.

Aritmética de pointers: El compilador sabe cuántos bytes sumar o restar al hacer aritmética de punteros.

A continuación se describen las operaciones más comunes con punteros.

6.4.2.1 Operador Address-Of (&) El operador **unario Address-Of**, designado por **&** (no confundir con el operador binario **&** para boolean) se utiliza para obtener la dirección de memoria de una variable. Por ejemplo, si se tiene una variable **int x = 10**, se puede obtener la dirección de memoria de **x** mediante **&x**. Se puede aplicar a memoria en el stack o en el heap.

```
1 int x = 10;
2 int* ptr = &x;
```

Para el código anterior, la tabla de memoria sería:

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	x	10	4B	Stack	int
0x4	ptr	0x0	4B	Stack	Pointer

Considere el siguiente código:

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6
7     int x = 20;
8
9     // Pointer pointing towards x
10    int* ptr = &x;
11
12    cout << "The address of the variable x is :- " << ptr;
13    return 0;
14 }
```

Dicho programa generará la siguiente salida (espacio de direcciones de 48b):

```
1 The address of the variable x is: 0x7ffbf7b3b7c
```

6.4.2.2 Operador de indirección/de-referencia (*) El operador **unario de-referencia**, designado por ***** (no confundir con el operador binario *****), se utiliza para acceder al valor almacenado en la dirección de memoria apuntada por un puntero. Por ejemplo, si se tiene un puntero **int* ptr** que apunta a la dirección de memoria de una variable **x**, se puede acceder al valor de **x** mediante ***ptr**.

Por ejemplo, el siguiente código:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     int x = 3899;
7     int* price;
8
9     price = &x;
10
11    cout << "The address of x is : " << &x;
12    cout << "The value of price : " << price;
13    cout << "The value stored at the variable pointed by price is: " <<
        (*price);
14    cout << "The value x is: " << x;
15    return 0;
16 }
```

Genera la siguiente salida:

```
1 The address of x is : 0x7ffbf7b3b7c
2 The value of price : 0x7ffbf7b3b7c
```



```
3 The value stored at the variable pointed by price is: 3899
4 The value x is: 3899
```

El operador de indirección también sirve para modificar el valor de la variable apuntada por el puntero. Por ejemplo, el siguiente código:

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     int x = 3899;
7     int* price;
8
9     price = &x;
10
11     cout << "The value of x is: " << x << endl;
12     cout << "The value of price is: " << *price << endl;
13
14     *price = 1000;
15
16     cout << "The value of x is: " << x << endl;
17     cout << "The value of price is: " << *price << endl;
18
19     return 0;
20 }
```

Genera la siguiente salida:

```
1 The value of x is: 3899
2 The value of price is: 3899
3 The value of x is: 1000
4 The value of price is: 1000
```

La tabla de memoria para el código anterior sería inicialmente:

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	x	3899	4B	Stack	int
0x4	price	0x0	4B	Stack	Pointer

y luego de la instrucción `*price = 1000;`,

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	x	1000	4B	Stack	int
0x4	price	0x0	4B	Stack	Pointer

6.4.2.3 Sharing Es un concepto que se refiere a la posibilidad de tener dos o más punteros que apunten a la misma dirección de memoria. Por ejemplo, si se tiene un puntero `int* ptr` que apunta a la dirección de memoria de una variable `x`, se puede tener otro puntero `int* ptr2` que apunte a la misma dirección de memoria de `x`.

```
1 int x = 20;
2 int* ptr = &x;
3 int* ptr2 = ptr;
```

La tabla de memoria para el código anterior sería:

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	x	10	4B	Stack	int
0x4	ptr	0x0	4B	Stack	Pointer
0x8	ptr2	0x0	4B	Stack	Pointer

Mediante cualquiera de los punteros `ptr` o `ptr2`, se puede leer/modificar el valor de `x`. Por ejemplo:

```
1 int x = 20;
2 int* ptr = &x;
3 int* ptr2 = ptr;
4
5 *ptr = 30;
6 cout << x; // Imprime 30
7 cout << *ptr2; // Imprime 30
8 cout << *ptr; // Imprime 30
```

6.4.2.4 Shallow-copy vs Deep-copy *Shallow-copy* implica copiar el puntero, pero no el valor al que apunta. Por ejemplo, para el siguiente código:

```
1 int x = 20;
2 int* ptr = &x;
3 int* ptr2 = ptr;
```

La tabla de memoria para el código anterior sería:

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	x	10	4B	Stack	int
0x4	ptr	0x0	4B	Stack	Pointer
0x8	ptr2	0x0	4B	Stack	Pointer

Nótese que solo hay un espacio con el valor 10. Ambos pointers “apuntan” a la misma dirección. Se pueden crear n copias de punteros, pero solo habrá un espacio de memoria al que todos apuntan.

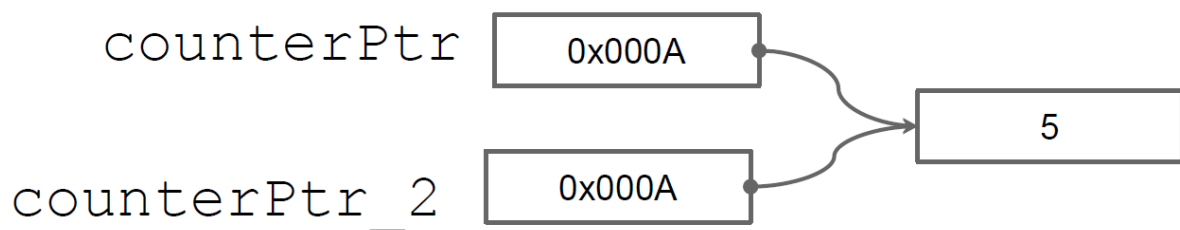


Figura 70: Shallow copy

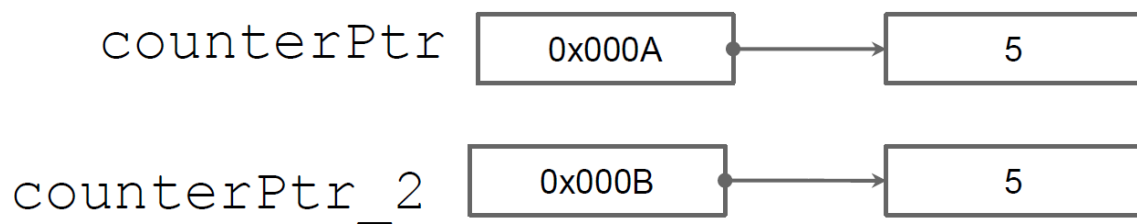
Por otro lado, *Deep-copy* implica copiar el valor al que apunta el puntero. Por ejemplo, el siguiente código:

```

1 int x = 20;
2 int* ptr = &x;
3 int* ptr2 = malloc(sizeof(int));
4 *ptr2 = *ptr;
  
```

En este caso, la tabla de memoria sería:

Dirección	Alias	Valor	Tamaño	Ubicación	Tipo
0x0	x	20	4B	Stack	int
0x4	ptr	0x0	4B	Stack	Pointer
0x8	ptr2	0x64	4B	Stack	Pointer
0x64		20	4B	Heap	int

**Figura 71:** Deep copy

Deep-copy puede implicar más trabajo que un simple `malloc`. Por ejemplo, al copiar una estructura de datos a otra, si alguno de los campos de la estructura es un puntero, se debe copiar el valor al que apunta el puntero, no el puntero en sí.

6.4.2.5 Aritmética de punteros Permite sumar o restar un número entero a un puntero. La aritmética de punteros es útil para acceder a elementos de un array o para moverse a través de una estructura de datos. Por ejemplo, considere el siguiente código:

```
1 int arr[5] = {10, 20, 30, 40, 50};
2 int* ptr = arr;
3
4 cout << *ptr; // Imprime 10
5 cout << *(ptr + 1); // Imprime 20
6 cout << *(ptr + 2); // Imprime 30
```

En este caso, `ptr` apunta al primer elemento del array `arr`. Al sumar 1 a `ptr`, se mueve al siguiente **elemento** del array. Al sumar 2 a `ptr`, se mueve dos elementos hacia adelante. Nótese que la aritmética de punteros es en términos de elementos, no de bytes. Dado que el pointer tiene un tipo, el compilador sabe cuántos bytes sumar o restar al hacer aritmética de punteros. El array `arr` ocupa 50 bytes contiguos en memoria y cada elemento ocupa 4 bytes. Por lo tanto, `ptr + 1` salta de 4 en 4 bytes.

Para efectos de arrays en C/C++, `arr[i]` es equivalente a `*(arr + i)`. Por ejemplo, el siguiente código:

```
1 int arr[5] = {10, 20, 30, 40, 50};
2
3 cout << arr[0]; // Imprime 10
4 cout << arr[1]; // Imprime 20
5 cout << arr[2]; // Imprime 30
```

Dado que acceder cualquier elemento de un array es equivalente a acceder a la dirección de memoria del primer elemento y sumarle un offset, los arrays son muy eficientes en términos de acceso a memoria. Es $O(1)$ acceder a cualquier elemento de un array.

6.4.2.6 Tipo referencia (C/C++) En C++, se puede utilizar el tipo & para definir una referencia a una variable. Una referencia es similar a un puntero, pero más seguro y más fácil de usar. Una referencia no puede ser nula y no puede ser reasignada. Una referencia es simplemente un alias para una variable.

```
1 int x = 10;
2 int& ref = x;
3
4 cout << x; // Imprime 10
5 cout << ref; // Imprime 10
```

Son muy útiles para pasar argumentos a funciones por referencia.

6.4.2.7 Paso de parámetros por valor vs por referencia En C/C++, los parámetros de una función pueden pasarse por valor o por referencia. Es decir, una función puede recibir una copia del valor de una variable o la dirección de memoria de la variable.

- **Por valor:** Se pasa una copia del valor de la variable. Los cambios a la variable dentro de la función no afectan a la variable original.

```
1 void foo(int x) {
2     x = 20;
3 }
4
5 int main() {
6     int x = 10;
7     foo(x);
8     cout << x; // Imprime 10
9 }
```

- **Por referencia:** Se pasa la dirección de memoria de la variable. Los cambios a la variable dentro de la función afectan a la variable original.

```
1 void foo(int& x) {
2     x = 20;
3 }
4
5 int main() {
6     int x = 10;
7     foo(x);
8     cout << x; // Imprime 20
9 }
```

o bien, en C:

```
1 void foo(int* x) {
2     *x = 20;
3 }
```

```
4
5 int main() {
6     int x = 10;
7     foo(&x);
8     cout << x; // Imprime 20
9 }
```

6.4.2.8 Buenas prácticas al usar punteros

- **Inicializar punteros:** Siempre inicializa los punteros. Un puntero no inicializado puede contener cualquier valor (random value) y puede causar errores difíciles de depurar o causar un segmentation fault al de-referenciarlo.

```
1 int* ptr = NULL; // Correcto
2 int* ptr; // Incorrecto
```

- **Verificar punteros antes de usarlos:** Antes de usar un puntero, verifica que no sea NULL.

```
1 if (ptr != NULL) {
2     // Usar el puntero
3 }
```

- **Liberar memoria:** Siempre libera la memoria asignada dinámicamente cuando ya no la necesites para evitar fugas de memoria.
- **Evitar punteros colgantes (dangling pointers) :** Después de liberar memoria, establece el puntero a NULL para evitar el uso accidental de punteros colgantes.

```
1 free(ptr);
2 ptr = NULL;
```

- **Usar `const` cuando sea posible:** Si un puntero no debe modificar los datos a los que apunta, decláralo como `const`.

```
1 const int* ptr = &x;
```

- **Evitar aritmética de punteros compleja:** La aritmética de punteros puede ser propensa a errores. Evítala si es posible o úsala con cuidado.

```
1 int arr[10];
2 int* ptr = arr;
3 ptr += 2; // Apunta al tercer elemento del array
```

- **No retornar desde una función, un puntero a una variable del stack:** Si retornas un puntero a una variable del stack, la variable se libera cuando la función termina y el puntero se convierte en un puntero colgante.

```
1  int* foo() {
2      int temp = 50;
3      return &temp;
4  }
5  void bar() {
6      int temp = 66;
7      return;
8  }
9
10 int main() {
11     int* r = foo();
12     cout << *r; // Imprime 50
13     bar();
14     cout << *r; // Imprime 60
15 }
```

6.4.3 Punteros en lenguajes manejados

En lenguajes manejados como Java, C# y Python, los punteros no son accesibles directamente. En su lugar, se utilizan referencias. Una referencia es similar a un puntero, pero más seguro y más fácil de usar. Una referencia no puede ser nula y no puede ser reasignada. Una referencia es simplemente un alias para un objeto.

En Java, por ejemplo, se utilizan referencias para acceder a objetos. Por ejemplo:

```
1  class MyClass {
2      int x;
3  }
4
5  public class Main {
6      public static void main(String[] args) {
7          MyClass obj = new MyClass();
8          obj.x = 10;
9
10         MyClass obj2 = obj; // Shallow-Copy
11         System.out.println(obj.x); // Imprime 10
12     }
13 }
```

La mayoría de los lenguajes manejados tienen recolección de basura, lo que significa que no es necesario liberar la memoria manualmente. El recolector de basura se encarga de liberar la memoria automáticamente cuando un objeto ya no es accesible. Utiliza un algoritmo de marcado y barrido para determinar qué objetos son accesibles y cuáles no. Los objetos inaccesibles se marcan para ser liberados. Para determinar si un objeto es accesible, el recolector de basura sigue las referencias a través de los objetos.

Otros lenguajes modernos como Rust, tienen un sistema de tipos que garantiza la seguridad de la memoria sin necesidad de un recolector de basura. Rust utiliza un sistema de tipos basado en el concepto de *ownership* y *borrowing* para garantizar la seguridad de la memoria. Por ejemplo:

```
1 fn main() {  
2     let mut x = 10;  
3     let y = &x;  
4     let z = &x;  
5     println!("{}", x); // Imprime 10  
6 }
```

El código anterior no compilará en Rust. Rust garantiza que no haya referencias múltiples a un objeto mutable. En este caso, `x` es mutable, pero `y` y `z` son referencias inmutables. Rust garantiza que no haya referencias múltiples a un objeto mutable para evitar condiciones de carrera y errores de memoria.

6.5 Referencias adicionales

- Modern Operating Systems, Andrew S. Tanenbaum, Herbert Bos, Pearson, 2014.
- <https://www.geeksforgeeks.org/memory-layout-of-c-program/>
- <https://www.geeksforgeeks.org/cpp-pointer-operators/>

7 Análisis de algoritmos

En este capítulo se aborda el tema de análisis de algoritmos. Dicho conocimiento es esencial para el diseño de algoritmos eficientes y para la toma de decisiones en el desarrollo de software. La industria ha hecho estandar las entrevistas enfocadas en algoritmos y estructuras de datos junto con análisis de complejidad.

7.1 Definición de Algoritmo

Es un procedimiento para cumplir una tarea. Es la idea detrás de un programa.

¿Programa vs Algoritmo?

Un programa es la implementación de un algoritmo en un lenguaje de programación.

Opera sobre un problema bien definido, toma un input y lo transforma en un output deseado. Un ejemplo cotidiano puede ser una receta de cocina:

- Input: Ingredientes
- Output: Comida
- Instrucciones: Algoritmo

Las características de un algoritmo son:

- No son ambiguos: cada paso tiene un solo significado
- Entrada bien definida: Debe ser clara y consistente
- Salida bien definida: Debe indicar que tipo de output genera
- Finitos: Terminan en un tiempo determinado.
- Factibles: Pueden ser ejecutados con los recursos tecnológicos disponibles

Un algoritmo puede ser representado de varias formas:

- Pseudo código
- Lenguaje natural
- Definición formal
- Diagramas de flujo

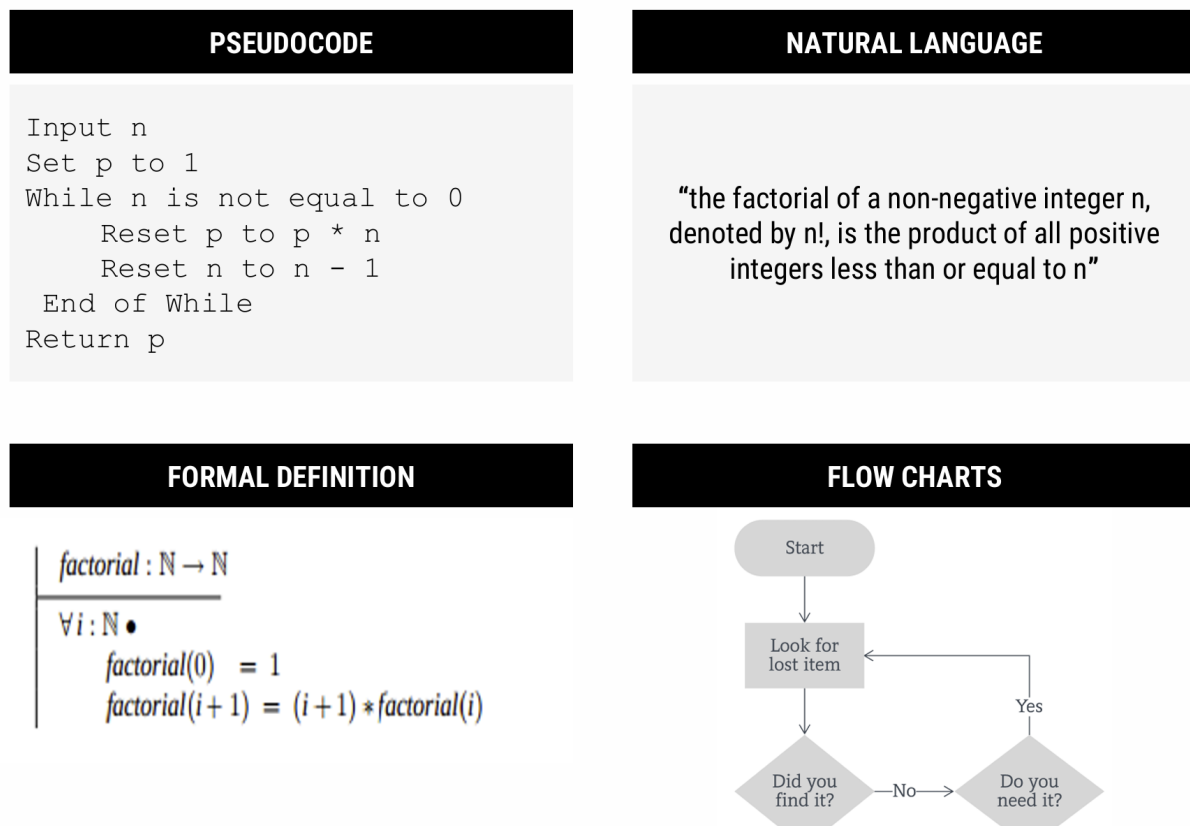


Figura 72: Representación de un algoritmo

7.2 Análisis de Algoritmos

En términos simples, analizar un algoritmo es el proceso para determinar la eficiencia de un algoritmo, medida en términos de tiempo y espacio. Esto no es una práctica de tiempos recientes, sino que se remonta a los inicios de la computación. Por ejemplo, Charles Babbage observó que:

"As soon as an Analytic Engine exists, it will necessarily guide the future course of the science. Whenever any result is sought by its aid, the question will arise—By what course of calculation can these results be arrived at by the machine in the **shortest** time?"

De manera similar y décadas después, Turing observó:

"It is convenient to have a measure of the amount of work involved in a computing process, even though it be a very crude one. We may count up the number of times that various elementary operations are applied in the whole process"

Ambos visionarios se referían a la necesidad de determinar la eficiencia de un algoritmo, con el fin de buscar la optimización del mismo o de comparar entre otros algoritmos de la misma “familia”.

La realidad es que hay muchos factores que pueden afectar la eficiencia de un algoritmo. Desde aspectos tecnológicos como el hardware, el sistema operativo, el lenguaje de programación, hasta aspectos más abstractos como la complejidad del algoritmo, la cantidad de datos, la pericia del programador, entre otros. Sea cual sea el factor, se desea analizar algoritmos para:

- Predecir comportamiento de un algoritmo y el programa que lo implementa.
- Comparar distintos algoritmos para el mismo propósito.
- Conociendo el comportamiento, podemos optimizar
- Clasificar el algoritmo según complejidad.

En las siguientes secciones se abordan dos formas de analizar algoritmos: *análisis empírico* y *análisis teórico*.

7.3 Análisis empírico de algoritmos

El término empírico se refiere a algo que se basa en la experiencia y la observación. En el análisis empírico, se evalúa el desempeño de un algoritmo ejecutándolo, conocido como *benchmarking*. El proceso entonces sería:

1. Marcar un tiempo de inicio
2. Ejecutar un programa con un input dado
3. Marcar el tiempo final
4. Calcular el tiempo transcurrido
5. Ejecutar la prueba varias veces, calcular mediana

Aunque pueda parecer contra-intuitivo, el análisis empírico es muy utilizado en la práctica profesional dado a que hay ambientes controlados donde se puede medir el desempeño de un algoritmo.

Profiling es análisis empírico. Esta es una técnica de análisis dinámico para encontrar cuellos de botella o secciones del código de un programa con mal rendimiento. Provee una visión completa: tiempo de ejecución, uso de memoria, uso de CPU, etc.

El problema clave de análisis empírico es que depende de la máquina en la que se ejecuta el algoritmo. Por lo tanto, no es posible generalizar los resultados obtenidos. No es lo mismo ejecutar un algoritmo en una máquina de hace 10 años que en una moderna.

7.4 Análisis teórico de algoritmos

Si el análisis empírico no provee una solución a prueba del tiempo y plataforma, ¿cómo se puede analizar un algoritmo? La respuesta es el análisis teórico.

El análisis teórico asume un modelo computacional en el que:

- Cada operación simple (+, -, *, /, =, ==, etc) toma tiempo constante. Estas son las operaciones elementales a las que se refería Turing.
- Los *loops* y subrutinas/funciones/métodos/procedimientos no son operaciones simples, sino que son la composición de muchas operaciones simples.
- Cada acceso a memoria toma tiempo constante.

Este modelo es una abstracción de la realidad, pero es perfecto para determinar el comportamiento de un algoritmo. No nos interesan los nano-segundos que toma una operación, sino el comportamiento general del algoritmo, **cual es la tasa de crecimiento de la cantidad de operaciones realizadas conforme el tamaño de la entrada.**

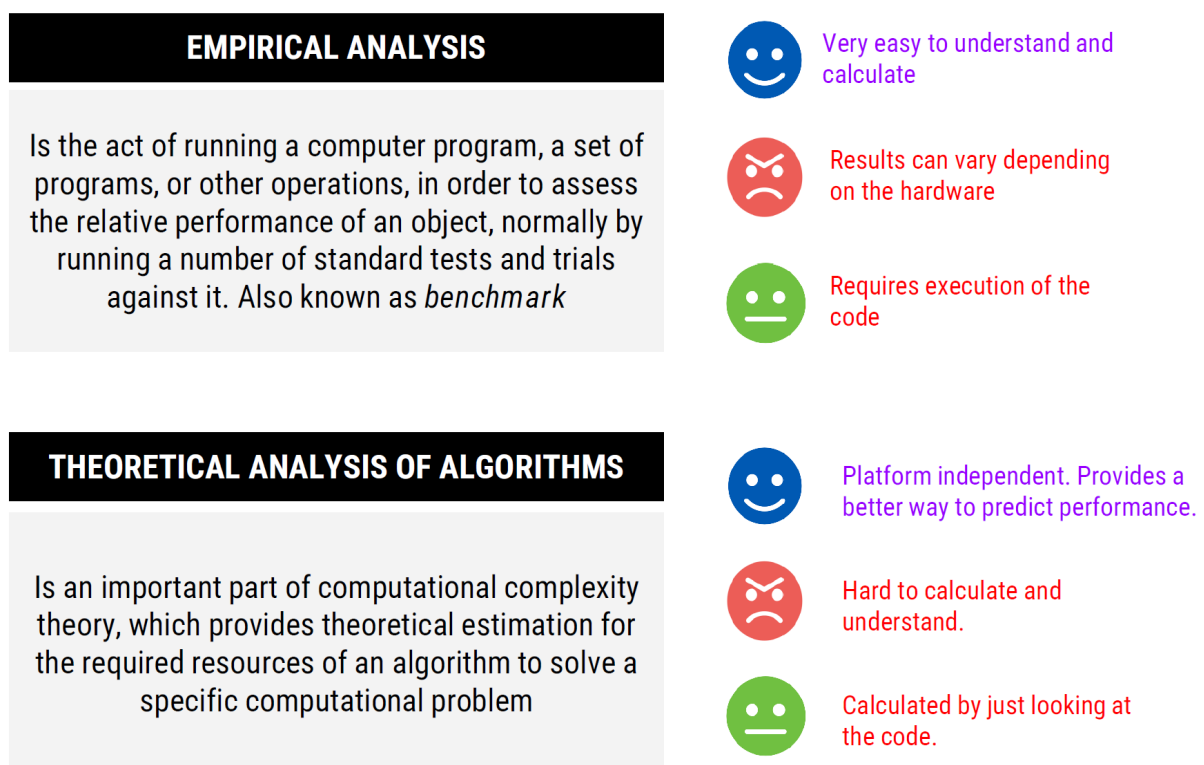


Figura 73: Comparación entre análisis empírico y teórico

Utilizando este modelo, podemos encontrar una función con base en el tamaño de la entrada. Por ejemplo, para el siguiente algoritmo

```

1  int max(int[] array) {
2      int max = array[0];
3      for (int i = 1; i < array.length; i++) {
4          if (array[i] > max) {
5              max = array[i];
6          }
7      }
8      return max;
9  }
```

la función que describe el tiempo que se requiere para encontrar el máximo de un arreglo de tamaño n es (en el peor caso) sería

```

1  int max = array[0]; -----> 3T
2  for (int i = 1; i < array.length; i++) { ---> T + 2nT
3      if (array[i] > max) { -----> 2T(n-1)
4          max = array[i]; -----> 2T(n-1)
5      }
6  }
7  return max; -----> T
8
9  f(n) = 6Tn + T
```

Encontrar la función a este nivel de detalle no es práctico. Imagínese el trabajo que implicaría una función como $f(n) = 12754n^2 + 4353n + 834\lg_2 n + 13546$. Más adelante veremos un enfoque que nos permite simplificar el trabajo.

7.4.1 Mejor, peor y caso promedio

Cuando se analiza un algoritmo, el **mejor caso** es el escenario en el que se realizan la menor cantidad de operaciones. Por ejemplo, en el algoritmo de búsqueda secuencial, el mejor caso es cuando el elemento buscado es el primer elemento del arreglo.

Al enfocarse en el **peor caso**, nos enfocamos en el escenario que causa que la mayor cantidad de operaciones se ejecute. Por ejemplo, en el algoritmo de búsqueda secuencial, el peor caso es cuando el elemento buscado está al final del arreglo.

En el **caso promedio**, tomamos todos los posibles inputs y calculamos el tiempo promedio que tomaría el algoritmo. Se suman todos los valores calculados y se divide entre el total de entradas. Por ejemplo, para el algoritmo de búsqueda secuencial, el caso promedio es cuando el elemento buscado está en cualquier posición del arreglo.

- Si el elemento está en la posición i se ejecutan i operaciones.

- Promedio de comparaciones es: $(1 + 2 + 3 + \dots + n) / n = n(n+1)/2$
- Promedio por elemento: $n(n+1)/2/n = (n+1)/2$

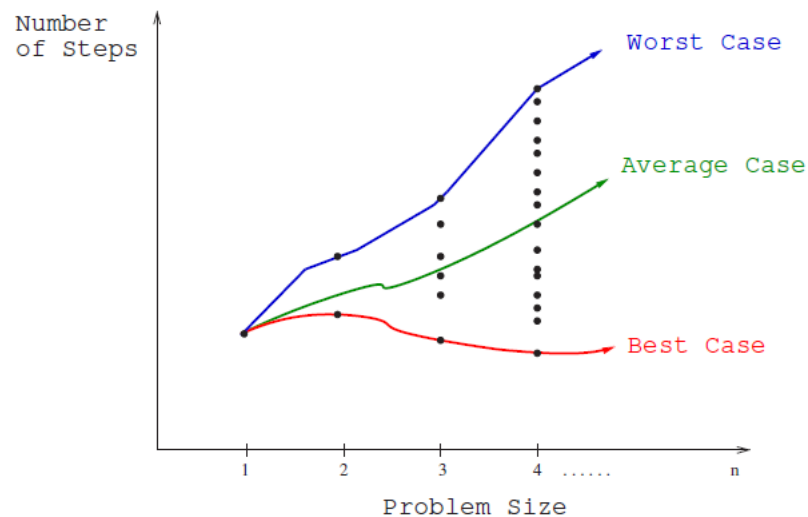


Figura 74: Comparativa de mejor, peor y caso promedio

El **peor caso** es el que resulta más útil para el análisis de algoritmos. Es el que nos da una idea de la eficiencia del algoritmo en el peor escenario posible. Conociendo lo peor que puede llegar, y siendo este aceptable, podemos asumir que el algoritmo es eficiente.

7.4.2 Análisis asintótico y notaciones comunes

Suponga que usted necesita enviar un archivo a un amigo en Guanacaste. ¿Qué es más rápido, enviarlo por correo/FTP o llevarlo personalmente? Asumiendo que ir a Guanacaste sin presas, tarda siempre 3 horas, podríamos tener el siguiente gráfico:

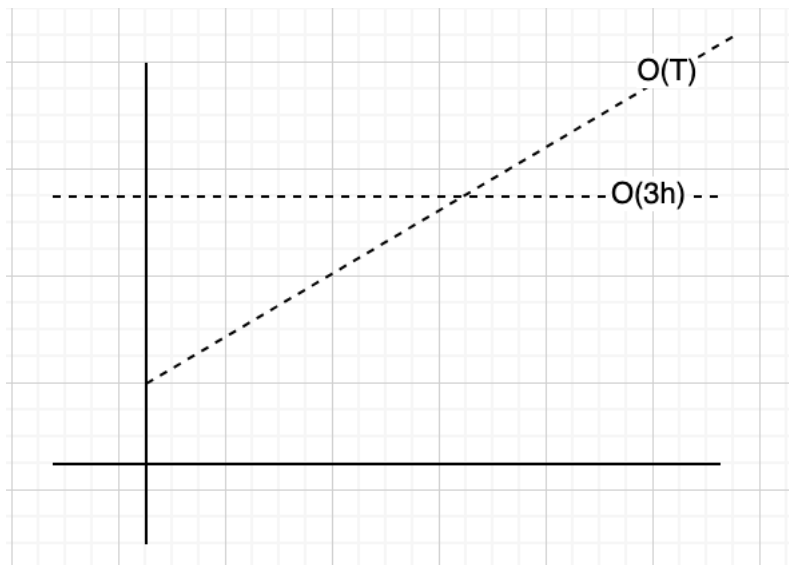


Figura 75: Ejemplo cotidiano sobre análisis asintótico

No importa que tan grande sea el archivo, llevarlo físicamente siempre tarda lo mismo. Por medio electrónico, el tiempo de transferencia depende del tamaño del archivo ($O(n)$) y en algún momento será mayor que las 3 horas que tarda llevarlo físicamente ($O(3)$).

Análisis asintótico busca encontrar la función que represente el crecimiento con respecto a n , sin entrar en detalles abrumadores de la función, es decir, podemos descartar los términos de menor relevancia.

Considere la función que calculamos tiempo atrás: $f(n) = 4T + 6nT$. Enfocándose en *análisis asintótico*, se descartan los términos de menor relevancia, es decir, los que no son significativos para el crecimiento de la función. De igual forma las constantes se eliminan. Por lo tanto, dicha función se puede expresar como

$$f(n) = O(n)$$

Lo que nos interesa en análisis asintótico, es la escalabilidad del algoritmo.

$$O(n^2 + n) \Rightarrow O(n^2)$$

$$O(n + \log(n)) \Rightarrow O(n)$$

$$O(5 \cdot 2^n + 100n^2) \Rightarrow O(2^n)$$

Por ejemplo, considere el siguiente algoritmo:

```
1 for (int i = 0; i < vector.length(); i++) {
2     foo(vector[i]); //Suponga que foo es tiempo constante
3 }
```

La gráfica de este algoritmo sería:

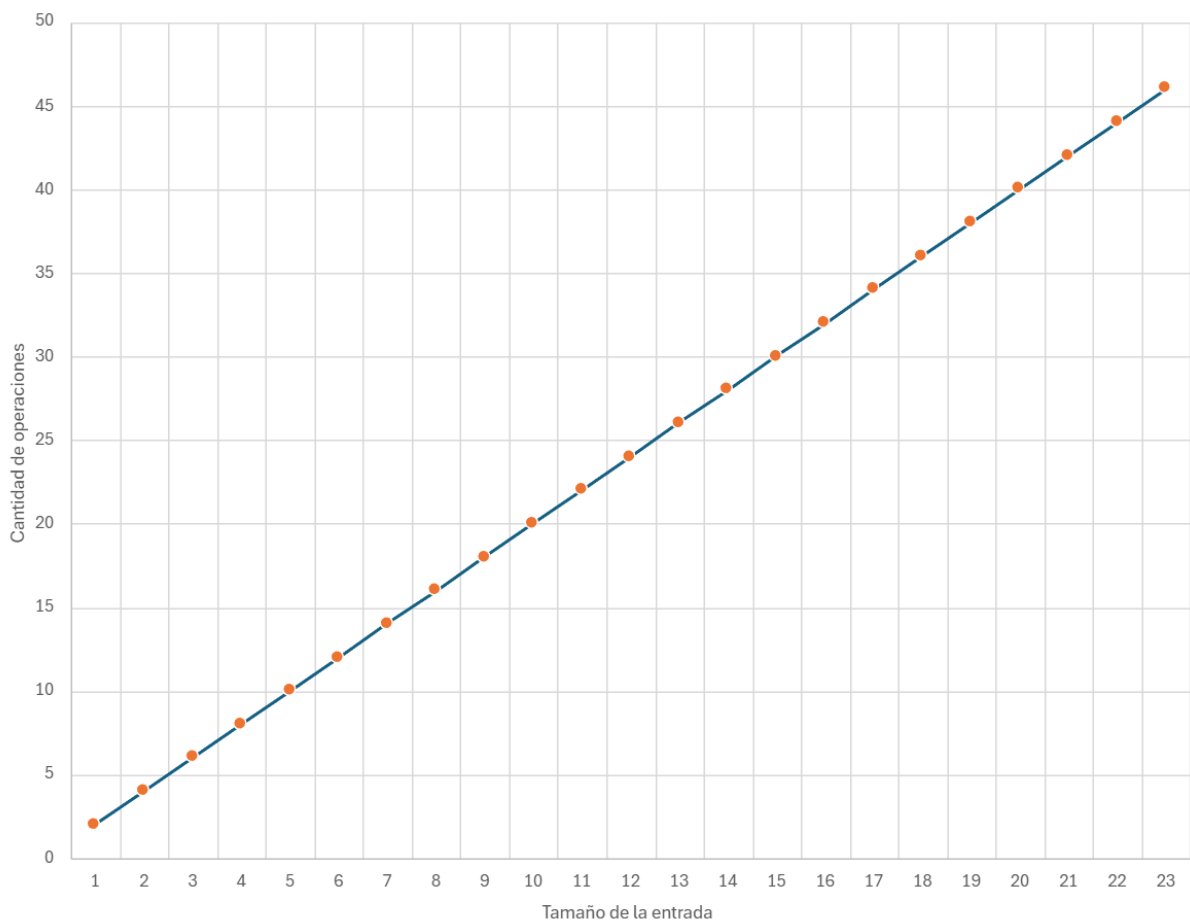


Figura 76: Gráfica de complejidad lineal

En caso que la constante cambie, se la función seguirá siendo lineal. Aunque la pendiente cambien, la tendencia del algoritmo seguirá siendo la misma.

7.4.3 Notación Big O, Big Omega y Big Theta

Son notaciones para describir la ejecución de un algoritmo en términos de su comportamiento asintótico.

7.4.3.1 Big O Describe el límite superior. Por ejemplo $O(n^2)$, $O(n)$, $O(2^n)$. El algoritmo es al menos tan rápido como este límite. No sobrepasa el límite dictado por Big O. Se define formalmente como

$f(n) = O(g(n))$, donde $c \cdot g(n)$ es un límite superior para $f(n)$. Es decir, existe una constante c tal que $f(n) \leq c \cdot g(n)$ para todo n mayor que un n dado.

$f(n)$ es la función que representa el algoritmo con precisión, por ejemplo, $f(n) = 3n^2 - 100n + 6$. $g(n)$ es el intento de clasificar nuestra función, por ejemplo, $g(n) = n^2$. La constante que multiplica a $g(n)$ la podemos escoger arbitrariamente, por ejemplo, $c = 3$. Si graficamos $g(n) = 3n^2$ y $f(n)$, veremos que $3n^2$ es siempre mayor que $f(n)$, concluyendo que $f(n) = O(n^2)$.

7.4.3.2 Big Omega Describe el límite inferior. Es decir, el algoritmo tendrá un comportamiento al menos tan lento como este límite. Se define formalmente como $f(n) = \Omega(g(n))$, donde $c \cdot g(n)$ es un límite inferior para $f(n)$. Es decir, existe una constante c tal que $f(n) \geq c \cdot g(n)$ para todo n mayor que un n dado.

Por ejemplo, dada la función $f(n) = 3n^2 - 100n + 6 = \Omega(n^2)$, porque para $c = 2$, $2n^2 < f(n)$ cuando $n > 100$.

7.4.3.3 Big Theta Describe el comportamiento exacto del algoritmo. Es decir, el algoritmo se comporta exactamente como esta función. Es el límite ajustado, *Big O* y *Big Omega*. Se define formalmente como $f(n) = \Theta(g(n))$, donde $c_1 \cdot g(n)$ es un límite inferior y $c_2 \cdot g(n)$ es un límite superior para $f(n)$. Es decir, existen constantes c_1 y c_2 tal que $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ para todo n mayor que un n dado.

Por ejemplo, dada la función $f(n) = 3n^2 - 100n + 6 = \Theta(n^2)$, porque $O(n^2)$ y $\Omega(n^2)$ aplican..

El mejor, peor y caso promedio, se puede describir con cualquiera de las notaciones de complejidad asintótica. Es incorrecto que *Big O* solo se refiere al peor caso, *Big Omega* es el mejor y *Big Theta* es el promedio.

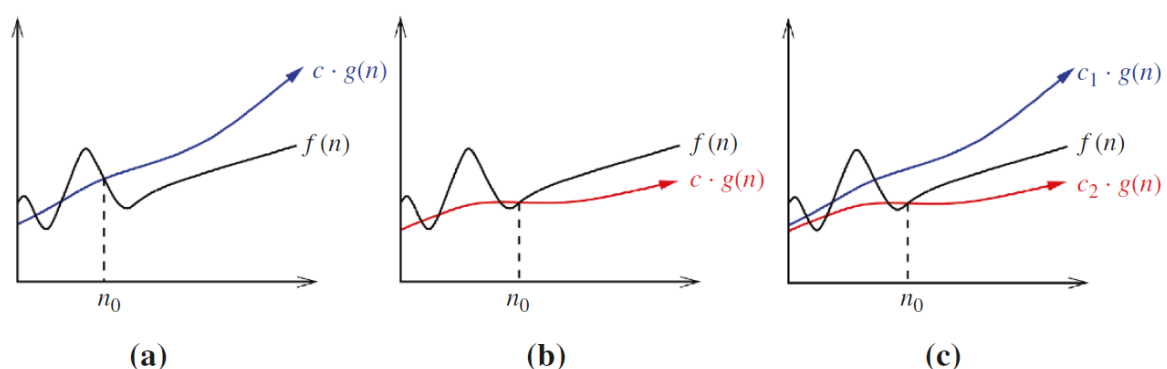


Figura 77: Visualización de Big O, Big Omega y Big Theta

7.4.3.4 Sobre la complejidad espacial Aunque lo visto hasta el momento se enfoca en complejidad en tiempo, la cantidad de memoria que el algoritmo requiere se puede describir con Big-O, Big-Omega y Big-Theta también.

```
1 int sum(int n) {
2     if (n <= 0) {
3         return 0;
4     }
5     return n + sum(n-1);
6 }
```

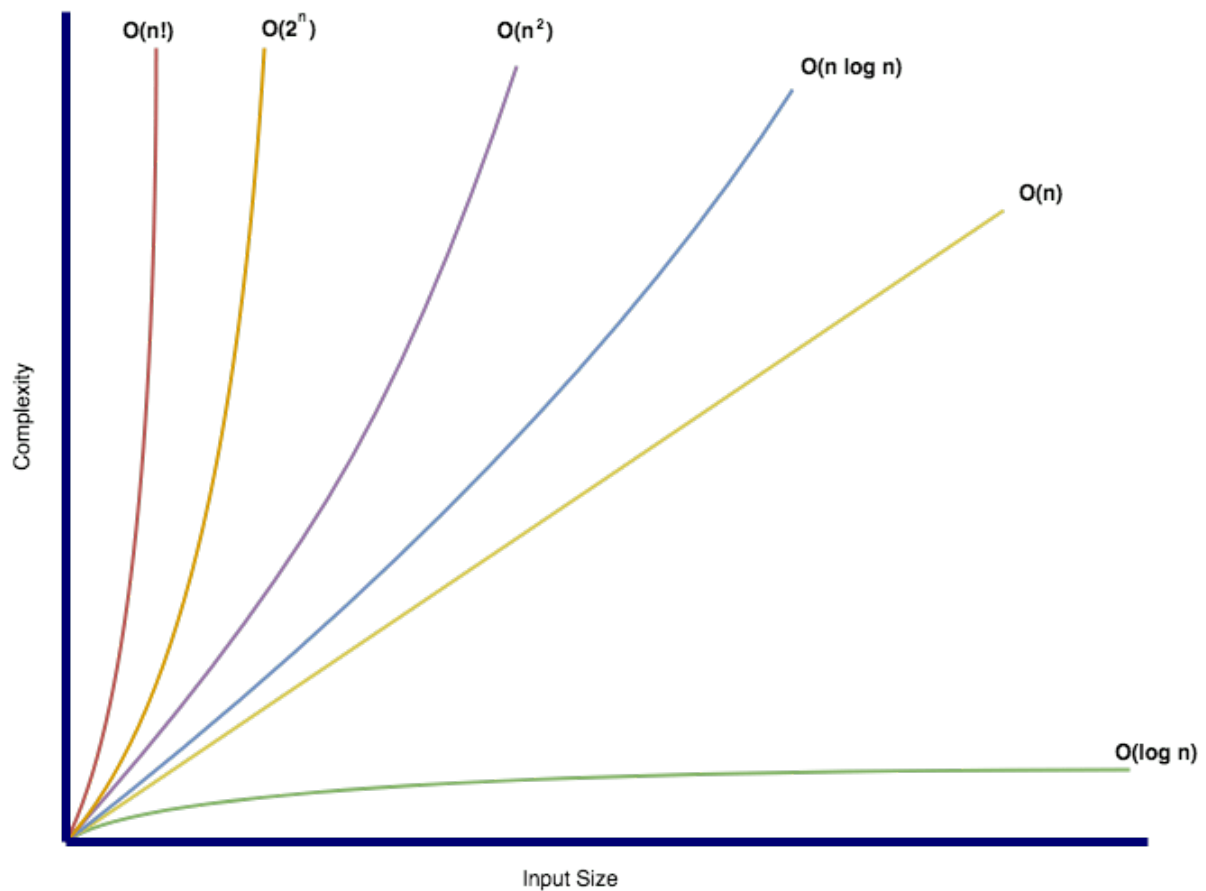
La complejidad espacial de este algoritmo es $O(n)$ dado que se requiere almacenar n llamadas recursivas en la pila de llamadas.

¿Cuál es la complejidad espacial de este algoritmo?

```
1 int foo(int n) {
2     int sum = 0;
3     for (int i = 0; i < n; i++) {
4         sum += bar(i);
5     }
6     return sum;
7 }
8
9 int bar(int a, int b) {
10     return a + b;
11 }
```

No hay llamadas anidadas $\Rightarrow O(1)$

7.4.3.5 Big-O conocidas El siguiente gráfico muestra las complejidades comunes con las que se clasifican muchos de los algoritmos conocidos:

**Figura 78:** Gráfico de Big-O conocidas

7.5 Determinar la complejidad de un algoritmo

Para determinar la complejidad de un algoritmo, dependerá del tipo de instrucciones que se utilicen en el código.

7.5.1 Secuencia de instrucciones

Para código de la forma:

```
1 statement 1;  
2 statement 2;  
3 ...  
4 statement n;
```

El total se calcula sumando la complejidad de cada instrucción:

$\text{complejidad}(\text{statement } 1) + \text{complejidad}(\text{statement } 2) + \dots + \text{complejidad}(\text{statement } n)$

7.5.2 If-Then-Else

Para código de la forma:

```
1  if (condition) {  
2      statement 1;  
3  } else {  
4      statement 2;  
5  }
```

La complejidad es $\max(\text{complejidad}(\text{statement } 1), \text{complejidad}(\text{statement } 2))$

7.5.3 Loops

Para código de la forma:

```
1  for (int i = 0; i < n; i++) {  
2      statement;  
3  }
```

La complejidad es $n * \text{complejidad}(\text{statement})$. Asumiendo que la complejidad de `statement` es $O(1)$, la complejidad del loop es $O(n)$.

Ahora considere el siguiente código con loops secuenciales:

```
1  for (int i = 0; i < n; i++) {  
2      statement 1;  
3  }  
4  for (int j = 0; j < m; j++) {  
5      statement 2;  
6  }
```

La complejidad es $O(n + m)$.

7.5.4 Anidamiento

Para código de la forma:

```
1  for (int i = 0; i < n; i++) {  
2      for (int j = 0; j < m; j++) {  
3          statement;  
4      }
```

```
5 }
```

La complejidad es $n * m * \text{complejidad}(\text{statement})$. Si ambos loops son de tamaño n , la complejidad es $O(n^2)$.

Conforme se anidan más loops, la complejidad crece exponencialmente. Por ejemplo, si se anidan 3 loops, la complejidad es $O(n^3)$.

7.5.5 Complejidad logarítmica

Para código de la forma:

```
1 int i = n;
2 while (i > 0) {
3     statement;
4     i = i / 2;
5 }
```

La complejidad es $O(\log(n))$. En cada iteración, i se divide por 2. ¿Cómo se llega a esta conclusión?

- En la primera iteración, i es n
- En la segunda iteración, i es $n/2$
- En la tercera iteración, i es $n/4$

En general, i es $n/2^k$ en la k -ésima iteración. La complejidad es $O(\log_2(n))$ porque k es el número de veces que se puede dividir n por 2 hasta llegar a 1.

7.5.6 Recursión

Para la recursión no hay una regla general, dado que depende de lo que haga el código. Por ejemplo, para el siguiente código:

```
1 int foo(int n) {
2     if (n <= 0) {
3         return 0;
4     }
5     return n + foo(n-1);
6 }
```

La complejidad es $O(n)$. La función se llama n veces, decrementando n en cada llamada.

Para el siguiente código:

```
1 int foo(int n) {
2     if (n <= 0) {
```

```

3     return 0;
4   }
5   return n + foo(n-1) + foo(n-1);
6 }

```

La complejidad es $O(2^n)$. En cada llamada, se hacen dos llamadas recursivas. ¿Cómo se llega a esta conclusión?

Gráficamente, se puede ver que la cantidad de llamadas se duplica en cada nivel de la recursión, como se muestra en la siguiente figura:

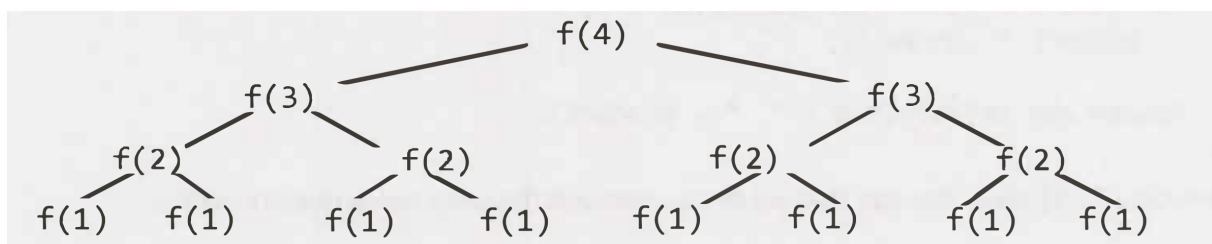


Figura 79: Visualización de recursión

Tabulando los datos anteriores:

Nivel	# de Nodos	
0	1	2^0
1	2	2^1
2	4	2^2
3	8	2^3
4	16	2^4

Por lo tanto, hay $2^{(n+1)} - 1$ nodos y la complejidad es $O(2^n)$. En muchos casos, la complejidad recursiva se puede generalizar como $O(\text{ramas}^{\text{profundidad}})$, donde *branches* es el número de llamadas recursivas y *depth* es la profundidad de la recursión.

7.5.7 Más ejemplos

7.5.7.1 Ejemplo #1 Para el siguiente código:

```

1 void foo(int[] array) {
2   int sum = 0;

```

```
3     int product = 1;
4     for (inti= 0; i < array.length; i++) {
5         sum += array[i];
6     }
7     for (int i= 0; i < array.length; i++) {
8         product*= array[i];
9     }
10    System.out.println(sum + ", " + product)
11 }
```

La complejidad sería $O(n)$. Iterar dos veces el arreglo no importa puesto que sería $O(2n) = O(n)$.

7.5.7.2 Ejemplo #2 Para el siguiente código:

```
1 void printUnorderedPairs(int[] array) {
2     for (int i= 0; i < array.length; i++) {
3         for (int j = i + 1; j < array.length; j++) {
4             System.out.println(array[i] + "," + array[j]);
5         }
6     }
7 }
```

Se realizan $(N-1) + (N-2) + \dots + 1 = N(N-1)/2$ operaciones. La complejidad es $O(n^2)$.

7.5.7.3 Ejemplo #3 El siguiente código:

```
1 void printUnorderedPairs(int[] arrayA, int[] arrayB) {
2     for (inti= 0; i < arrayA.length; i++) {
3         for (int j = 0; j < arrayB.length; j++) {
4             if (arrayA[i] < arrayB[j]) {
5                 System.out.println(arrayA[i] + "," + arrayB[j]);
6             }
7         }
8     }
9 }
```

Tiene una complejidad de $O(ab)$, donde a es el tamaño de `arrayA` y b es el tamaño de `arrayB`.

7.5.7.4 Ejemplo #4 El siguiente código:

```
1 void printUnorderedPairs(int[] arrayA, int[] arrayB) {
2     for (int i= 0; i < arrayA.length; i++) {
3         for (int j = 0; j < arrayB.length; j++) {
4             for (int k = 0; k < 1000000; k++) {
5                 System.out.println(arrayA[i] + "," + arrayB[j]);
6             }
7         }
8     }
9 }
```

```

8     }
9 }

```

Tiene una complejidad de $O(ab)$, donde a es el tamaño de *arrayA* y b es el tamaño de *arrayB*. El loop interno no afecta la complejidad dado que es $O(100000)$.

7.5.7.5 Ejemplo #4

```

1 void reverse(int[] array) {
2     for (int i= 0; i < array.length/ 2; i++) {
3         int other= array.length - i - 1;
4         int temp= array[i];
5         array[i] = array[other];
6         array[other] = temp;
7     }
8 }

```

En este caso, se recorre solo la mitad del arreglo. La complejidad es $O(n/2) = O(n)$.

7.5.7.6 Ejemplo #5 Para sumar todos los nodos de un BST:

```

1 int sum(Node node) {
2     if (node == null) {
3         return 0;
4     }
5     return sum(node.left) + node.value + sum(node.right);
6 }

```

Dado que se recorren todos los nodos del árbol, la complejidad es $O(n)$.

7.5.7.7 Ejemplo #6

```

1 boolean isPrime(int n) {
2     for (int x = 2; x <= sqrt(n); x++) {
3         if (n % x == 0) {
4             return false;
5         }
6     }
7     return true;
8 }

```

La complejidad es $O(\sqrt{n})$.

7.5.7.8 Ejemplo #7

```

1 int fib(int n) {
2     if (n <= 0) return 0;
3     else if (n == 1) return 1;
4     return fib(n - 1) + fib(n - 2);
5 }

```


La complejidad es $O(2^n)$.

7.5.7.9 Ejemplo #8

```
1 int factorial(int n) {  
2     if (n < 0) {  
3         return -1;  
4     } else if (n == 0) {  
5         return 1;  
6     } else {  
7         return n * factorial(n - 1);  
8     }  
9 }
```

La complejidad es $O(n)$. Se recorren n llamadas recursivas.

7.6 Revisitando los algoritmos y estructuras de datos

El peor caso de algunas estructuras de datos comunes se resume en la siguiente tabla:

Data structure	Access	Search	Insertion	Deletion
Array	$O(1)$	$O(N)$	$O(N)$	$O(N)$
Stack	$O(N)$	$O(N)$	$O(1)$	$O(1)$
Queue	$O(N)$	$O(N)$	$O(1)$	$O(1)$
Singly Linked list	$O(N)$	$O(N)$	$O(N)$	$O(N)$
Doubly Linked List	$O(N)$	$O(N)$	$O(1)$	$O(1)$
Hash Table	$O(N)$	$O(N)$	$O(N)$	$O(N)$
Binary Search Tree	$O(N)$	$O(N)$	$O(N)$	$O(N)$
AVL Tree	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$
Binary Tree	$O(N)$	$O(N)$	$O(N)$	$O(N)$
Red Black Tree	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$

Figura 80: Resumen de complejidad Big O

7.7 Referencias

- Skiena S. 2020. The Algorithm Design Manual. Springer.

- <https://www.geeksforgeeks.org/what-is-algorithm-and-why-analysis-of-it-is-important/>
- Laakmann Gayle L. 2020. Cracking the Coding Interview. 6th Edition. CareerCup.

8 Diseño de Algoritmos

En este capítulo se exploran algunos paradigmas de diseño de algoritmos utilizados en muchos de los algoritmos conocidos y utilizados en la actualidad. Analizar estos paradigmas, permitirá utilizar técnicas similares para nuevos problemas de ingeniería de software.

8.1 Divide y conquista

Divide y conquista es uno de los paradigmas introducidos en muchos de los cursos de programación introductorios. En términos generales, se divide el problema recursivamente en sub-problemas más pequeños, se resuelven de forma recursiva y se combinan las soluciones para obtener la solución final. Cuando la combinación toma menos tiempo que la resolución de los sub-problemas, se obtiene una mejora en la eficiencia.

La **fase de división** incluye:

- Divide el problema en sub-problemas más pequeños
- Los sub-problemas deben ser de la misma forma que el problema original, pero más pequeños
- Se divide hasta llegar al caso base

La **fase de conquista** incluye:

- Resolver cada sub-problema individualmente
- Si el sub-problema es lo suficientemente pequeño, se resuelve de forma directa
- El objetivo es resolver los sub-problemas independientemente

La **fase de combinación** incluye:

- Combinar las soluciones de los sub-problemas para obtener la solución del problema original
- Una vez que los sub-problemas se resuelven, se combinan para obtener la solución final
- El objetivo es obtener la solución del problema original

8.1.1 Ejemplos de algoritmos de divide y conquista

8.1.1.1 Merge Sort Merge Sort es un algoritmo de ordenamiento que utiliza el paradigma de divide y conquista. La idea es dividir el arreglo en dos mitades, ordenar cada mitad y luego combinar las dos mitades ordenadas.

Visualmente, el algoritmo se ve de la siguiente manera:

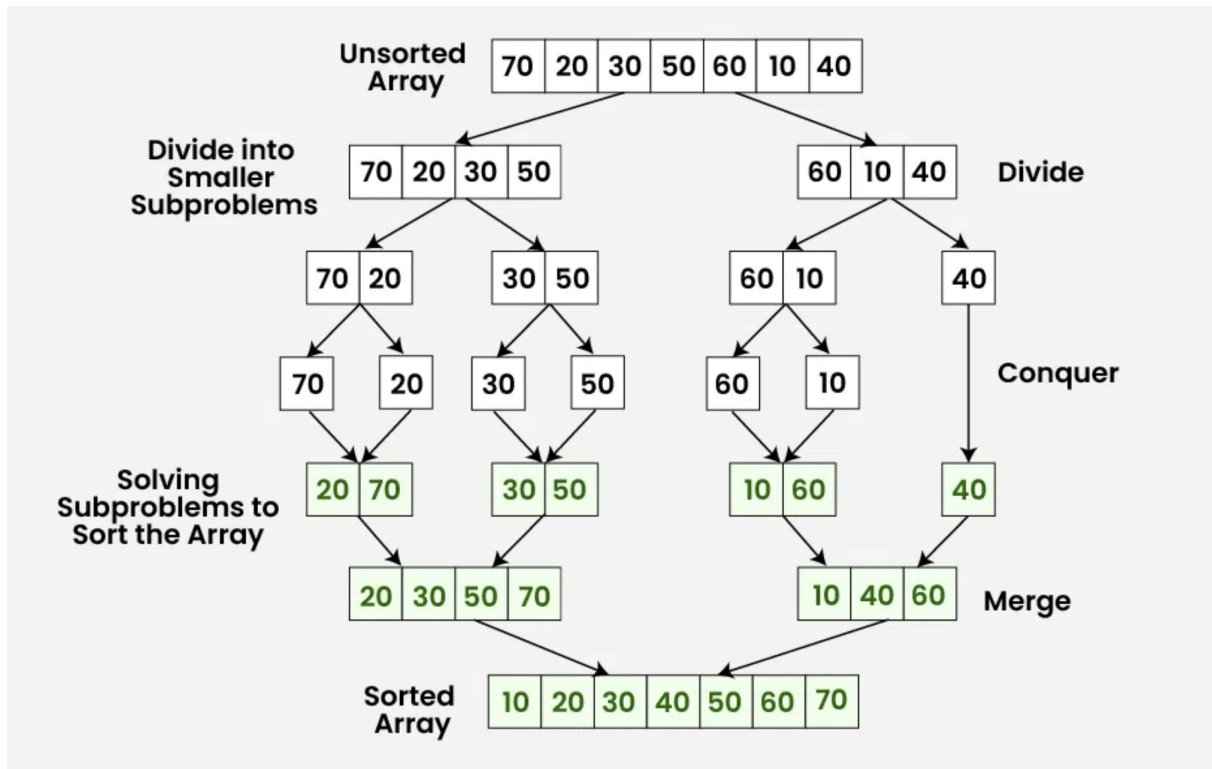


Figura 81: Visualización de MergeSort

En este algoritmo, en la **fase de división**, se generan $\log_2(n)$ niveles. En la **fase de merge**, combinar cada nivel toma $O(n)$ tiempo. Por lo tanto, la complejidad de tiempo de Merge Sort es $O(n \log n)$.

8.1.1.2 Búsqueda binaria Búsqueda binario, tal y como lo hemos visto en cursos anteriores, es un algoritmo de búsqueda que utiliza el paradigma de divide y conquista. La idea es dividir el arreglo en dos mitades, comparar el elemento con el valor medio y decidir en qué mitad continuar la búsqueda.

```

1  int binary_search(item_type s[], item_type key, int low, int high) {
2      int middle;
3      if (low > high) {
4          return (-1);
5      }
6      middle = (low + high) / 2;
7      if (s[middle] == key) {
8          return(middle);
9      }
10     if (s[middle] > key) {
11         return(binary_search(s, key, low, middle-1));
12     } else {

```

```
13         return(binary_search(s, key, middle +1, high));
14     }
15 }
```

La complejidad de tiempo de la búsqueda binaria es $O(\log n)$. En cada paso, el tamaño del problema se reduce a la mitad y no hay necesidad de mezclar los resultados.

8.1.1.3 Encontrar el máximo de un arreglo Para encontrar el máximo de un arreglo, la solución trivial sería recorrer el arreglo y comparar cada elemento con el máximo actual. La complejidad de tiempo de esta solución es $O(n)$:

```
1 int findMax(int[] a, int n)
2 {
3     int max = Integer.MIN_VALUE;
4     for (int i = 0; i < n; i++)
5     {
6         if (a[i] > max)
7         {
8             max = a[i];
9         }
10    }
11    return max;
12 }
```

Utilizando divide y conquista, se puede resolver de la siguiente manera:

```
1 static int findMax(int[] a, int lo, int hi)
2 {
3     if (lo > hi)
4     {
5         return Integer.MIN_VALUE;
6     }
7     if (lo == hi)
8     {
9         return a[lo];
10    }
11    int mid = (lo + hi) / 2;
12    int leftMax = findMax(a, lo, mid);
13    int rightMax = findMax(a, mid + 1, hi);
14    return Math.max(leftMax, rightMax);
15 }
```

La complejidad de tiempo de este algoritmo es $O(n)$. El análisis refleja que aunque sea $O(n)$, realiza menos comparaciones que la solución trivial.

Investigar: ¿Por qué la complejidad de tiempo de este algoritmo es $O(n)$? ¿Por qué no es $O(\log n)$?

8.1.1.4 Paralelismo en divide y conquista Computación paralela es una técnica que permite realizar múltiples tareas simultáneamente aprovechando múltiples núcleos de procesamiento. Cada partición del problema se puede asignar a un núcleo de procesamiento, lo que permite reducir el tiempo de ejecución.

Investigar: ¿Cómo se puede implementar paralelismo en el algoritmo de búsqueda binaria y en *merge sort*?

8.2 Programación dinámica (DP)

Aunque tiene el término *programación* en su nombre, no se refiere a escritura de código fuente. Acuñado por Richard Bellman en los años 50, *programar* se refiere a *planificar*, es decir, planificar óptimamente procesos de múltiples etapas. - Comparte similitudes con la técnica de *divide y vencerás*. - DP divide problemas en sub-problemas y *memoiza* las soluciones de los sub-problemas para resolverlos una *sola* vez. - En DP, los sub-problemas se traslapan, es decir, el resultado de uno puede ayudar a resolver otro.

Memoización vs Memorización Memoización se refiere a una técnica para optimizar recordando. Memorizar se refiere a poner algo en la memoria.

Por ejemplo, para calcular *fibonacci(4)* utilizando un enfoque tradicional:

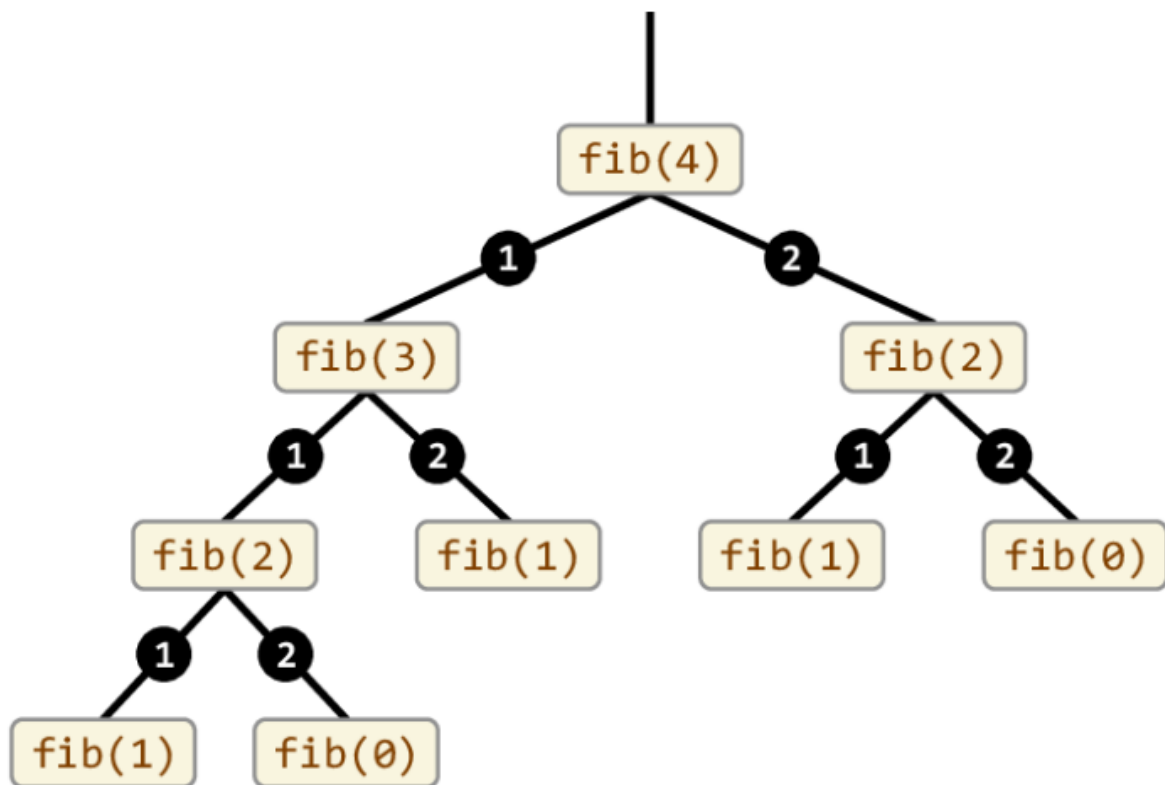


Figura 82: Árbol de recursión de Fibonacci

Se puede notar un claro traslape de sub-problemas, lo que implica un desperdicio de recursos computacionales. Utilizando un enfoque de DP, el problema se puede resolver de la siguiente manera:

```
1 int fib(n) {
2     mem[n + 2]; // ----> En caso que N sea 0 o 1
3     mem[0] = 0;
4     mem[1] = 1;
5
6     for (i = 2; i < n + 1; i++) {
7         mem[i] = mem[i - 1] + mem[i - 2];
8     }
9     return mem[n];
10 }
```

El código anterior utilizaría internamente, un arreglo que se vería de la siguiente manera:

```
1 mem[0] = 0
2 mem[1] = 1
3 mem[2] = 1
4 mem[3] = 2
5 mem[4] = 3
```


En el enfoque divide y conquista, el algoritmo de Fibonacci tiene una complejidad de tiempo de $O(2^n)$. En cambio, con DP, la complejidad de tiempo se reduce a $O(n)$.

El siguiente diagrama ilustra el enfoque DP vs Divide y Vencerás de forma general:

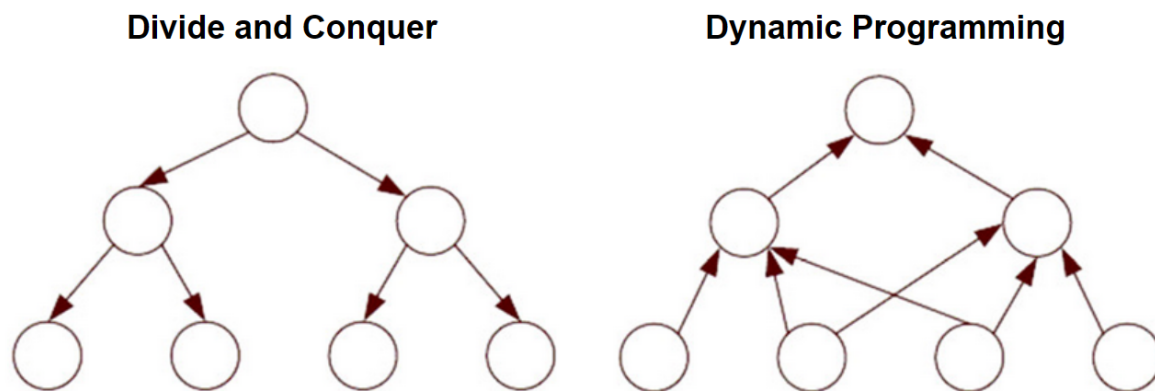


Figura 83: Divide y conquista vs programación dinámica

8.2.1 Ejemplo de programación dinámica: Longest Common Subsequence (LCS)

Cadena más larga común entre dos strings, no necesariamente contigua. Por ejemplo,

```
1 S1 = "BCDAACD"
2 S2 = "ACDBAC"
```

Tiene las cadenas comunes: BC, CDAC, DAC, ...

Implementándolo usando el enfoque tradicional:"

```
1 // Returns length of LCS for X[0..m-1], Y[0..n-1]
2 int lcs(String X, String Y, int m, int n)
3 {
4     if (m == 0 || n == 0)
5         return 0;
6     if (X.charAt(m - 1) == Y.charAt(n - 1))
7         return 1 + lcs(X, Y, m - 1, n - 1);
8     else
9         return max(lcs(X, Y, m, n - 1), lcs(X, Y, m - 1, n));
10 }
```

Para las cadenas BCDA y ACDB, se generaría un árbol de recursión de la siguiente manera:

```
1 (BCDA, ACDB)
2 / \
3 MAX[(BCD, ACDB) (BCDA, ACD)]
4 / \ / \
```

5 MAX[(BC, ACDB) (BCD, ACD) (BCD, ACD) (BCDA, AC)]

Claramente vemos como se resolvería varias veces el mismo sub-problema, siendo $O(2^{nm})$. Utilizando DP, se puede resolver de la siguiente manera:

Se crea una matriz de tamaño $(m+1) \times (n+1)$ y se llena de la siguiente manera:

```

1 static int lcs(String X, String Y, int m, int n)
2 {
3     int[,] L = new int[m + 1, n + 1];
4
5     for (int i = 0; i <= m; i++) {
6         for (int j = 0; j <= n; j++) {
7             if (i == 0 || j == 0)
8                 L[i, j] = 0;
9             else if (X[i - 1] == Y[j - 1])
10                 L[i, j] = L[i - 1, j - 1] + 1;
11             else
12                 L[i, j] = max(L[i - 1, j], L[i, j - 1]);
13         }
14     }
15     return L[m, n];
16 }

```

	A	C	D	B
0	0	0	0	0
B	0	0	0	0
C	0	0	1	1
D	0	0	1	2
A	0	1	1	2

Esto resulta en complejidad temporal $O(nm)$

8.3 Backtracking

- Popularizado por Henry Lehmer, matemático estadounidense.
- Es una forma metódica de probar distintas secuencias de decisiones hasta encontrar una que funcione
- Se puede conceptualizar como un árbol de decisiones
 - Cada nodo del árbol solo puede ver sus hijos directos

- Si un nodo conduce a error, se regresa al anterior y se prueba con otro hijo

La estructura general en código se puede resumir como:

```
1 backtrack(x) {
2     if (x == solucion) {
3         return true;
4     }
5     for (nodo in x.nodos) {
6         if (nodo == solution) {
7             return true;
8         } else {
9             return backtrack(nodo);
10        }
11    }
12 }
13 }
```

8.3.1 Ejemplo: el problema de las N-reinas

Dado un tablero de ajedrez de $N \times N$, colocar N reinas de tal forma que no se ataquen entre sí. Una reina puede atacar a otra si están en la misma fila, columna o diagonal.

```
1 bool solve(board[][] col) {
2     if (col == N) {
3         return true;
4     }
5     for (int i = 0; i < N; i++) {
6         if (isSafe(board, i, col)) {
7             board[i][col] = 1;
8             if (solve(board, col + 1)) {
9                 return true;
10            }
11            board[i][col] = 0;
12        }
13    }
14    return false;
15 }
```

8.4 Algoritmos ávidos

Es un paradigma de diseño de algoritmos que construye la solución parte por parte, siempre escogiendo la siguiente pieza que ofrezca el beneficio obvio e inmediato.

Algunas de sus características son:

- Son simples y sencillos de implementar

- Eficientes en términos de tiempo de ejecución, retornando el resultado en poco tiempo
- No reconsideran opciones previas

8.4.1 Ejemplo: Número mínimo de monedas

Suponga que tiene un conjunto infinito de monedas de 1, 2, 5, y 10 colones. ¿Cuál es el número mínimo de monedas que se necesitan para dar un cambio de 39 colones?

Utilizando un enfoque ávido, el proceso sería el siguiente:

- Seleccionar la denominación más grande que sea menor o igual al cambio restante. En este caso, sería 10 colones.
- Restar el valor seleccionado del cambio restante. Agregar la moneda seleccionada al resultado.
- Repetir los pasos anteriores hasta que el cambio restante sea 0.

No necesariamente la solución será la óptima en todos los casos, pero en muchos casos, la solución ávida es suficiente. Por ejemplo, si el cambio es 20 y las monedas disponibles son 1, 10 y 18, la solución ávida daría 18, 1 y 1, cuando la solución óptima sería dos monedas de 10.

8.5 Algoritmos Probabilísticos

Son algoritmos que utilizan aleatoriedad para tener una mejora en desempeño sacrificando la confiabilidad de los resultados obtenidos:

- No se produce ningún resultado
- Se produce un resultado incorrecto
- Se produce una respuesta aproximada

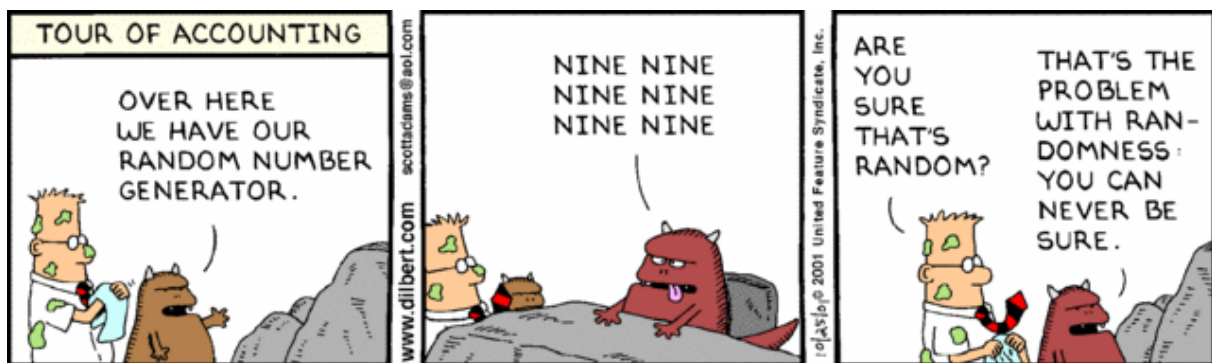
Ejecuciones distintas pueden producir respuestas distintas.

8.5.1 Clasificación

- **Monte Carlo:** Algoritmos que **siempre** retornan un resultado, pero puede no ser correcto. Se intenta minimizar la probabilidad de error. Múltiples ejecuciones reducen dicha probabilidad.
- **Las Vegas:** Algoritmos que **siempre** retornan un resultado correcto, pero pueden producir ningún resultado. Múltiples ejecuciones reducen la probabilidad de no obtener un resultado.

8.5.2 Aleatoriedad

- Provisto por un generador de números random. Estos generadores son pseudo-aleatorios, ya que generan una secuencia de números que parecen ser aleatorios, pero son deterministas. El único valor random que existe en nuestra realidad es la decadencia radioactiva.
- Los generadores de pseudo-random generan números en una secuencia dentro de un rango y requieren un elemento inicial llamado semilla (seed). Cada número en la secuencia se genera a partir del anterior.
- Hay posibilidad de ciclos dado que un número puede repetirse en la secuencia.



8.5.2.1 Ejemplo de pseudo-random: Método de cuadrado medio Eleve el número inicial (seed) al cuadrado y tome los dígitos del medio como el nuevo número. Por ejemplo, si el seed es 1234, el cuadrado es 1522756 y el número medio es 2275.

Una secuencia sería:

- 1234 (semilla)
- 2275 (1234^2)
- 5180 (2275^2)
- 6884 (5180^2)

y así sucesivamente. Dependiendo de la semilla, la secuencia puede ser cíclica muy rápido.

8.5.3 Algoritmos de Monte Carlo

Considere el siguiente requerimiento: “Dado un array de N elementos, determinar si hay un elemento que sea mayoritario, es decir que aparezca más de $N/2$ veces”

La solución trivial sería:

1. Recorra el array y cuente cuántas veces aparece cada elemento.

2. Si encuentra un elemento que aparece más de $N/2$ veces, retorne verdadero.

El problema es que la complejidad de este algoritmo es $O(N^2)$ y no es eficiente.

Utilizando un algoritmo de Monte Carlo, tendríamos:

```
1 boolean tieneElementoMayoritario(array, lenght) {
2     i = random(0, n - 1);
3     x = array[i];
4     k = 0;
5     for (j = 0; j < n; j++) {
6         if (array[j] == x) {
7             k++;
8         }
9     }
10    return k > n / 2;
11 }
```

Como se puede notar, podría devolver **false** aún cuando no haya un elemento mayoritario. La probabilidad de error es de $1/2$.

Ejecutar varias veces el algoritmo reduce la probabilidad de error (si el psuedo-random es bueno). Después de k ejecuciones, la probabilidad de error es de $1/2^k$.

8.5.4 Algoritmos de Las Vegas

Características:

- No garantizan un resultado y no tiene un uppber-bound en tiempo de ejecución. Siempre se obtiene un resultado correcto, pero no siempre se obtiene un resultado.
- Utilizados para explorar un espacio de soluciones de las cuales algunas son las correctas. Usan random para moverse en dicho espacio de soluciones. Si hay muchas soluciones correctas, la probabilidad de encontrar una es alta en poco tiempo.

Considere el problema de las n -reinas. La solución clásica es mediante backtracking. Sin embargo, un algoritmo de Las Vegas podría ser:

1. Colocar las reinas en posiciones aleatorias, una en cada columna
2. Verificar si hay colisiones
3. Si no hay colisiones, retornar la solución
4. Si hay colisiones, repetir el proceso

8.5.5 Estructuras de datos probabilísticas

- Proveen respuestas aproximadas a consultas sobre grandes sets de datos.

- Sacrifican precisión para fomentar eficiencia temporal.
- Utilizan *hashing* y *randomness* para reducir la complejidad espacial y temporal.

Una diferencia clave de las estructuras probabilísticas con respecto a los algoritmos probabilísticos, es que estos últimos no necesariamente se comportan de forma impredecible para el usuario. Es decir, pueden devolver resultados correctos. Las estructuras de datos probabilísticas, no dan respuestas definitivas.

8.5.5.1 Filtros de Bloom Considere el siguiente escenario:

La página de registro de usuarios de un sitio web, permite al usuario especificar un *username*. No se permiten *username* duplicados. ¿Cómo se puede verificar si un *username* ya existe?

Algunas posibles soluciones serían:

- Aplicar búsqueda secuencial, pero el problema es que la complejidad es $O(N)$ y si tenemos millones de usuarios, la búsqueda sería muy lenta.
- Búsqueda binaria, que es mucho mejor, pero requeriría tener ordenados los usuarios y cargados en memoria.

Un filtro de Bloom permite determinar si un elemento pertenece a un set o no. Al ser probabilístico, puede dar falsos positivos (sí está), pero no falsos negativos.

- Utiliza un array de bits de tamaño fijo para representar todo el set de datos
- Agregar un elemento nunca falla, pero los falsos positivos aumentan conforme el set se llena
- Nunca genera falsos negativos
- No se pueden eliminar elementos del set

Implementación

Se utiliza un arreglo de bits de largo m .

Se necesitan k funciones de hash. Para agregar un elemento, se calculan las k funciones de hash y se setean los bits correspondientes a 1. Por ejemplo, si se define que se van a usar 3 funciones de hash, y se va a insertar la palabra "TEC" en el set:

$$h_1(\text{TEC}) \% m = 1 \quad h_2(\text{TEC}) \% m = 3 \quad h_3(\text{TEC}) \% m = 5$$

Ahora agregamos la palabra "QUIZ":

$$h_1(\text{QUIZ}) \% m = 3 \quad h_2(\text{QUIZ}) \% m = 5 \quad h_3(\text{QUIZ}) \% m = 4$$

Como se puede notar, en este caso, TEC y QUIZ tuvieron algunos resultados similares. El set se comienza a llenar, activando bits que no necesariamente corresponden a elementos diferentes.

Supongamos que se busca la palabra “PERRO”, la cual no se ha insertado aún, y que las funciones de hash generan los siguientes valores:

$$h1(\text{PERRO}) \% m = 1 \quad h2(\text{PERRO}) \% m = 4 \quad h3(\text{PERRO}) \% m = 5$$

Estos índices ya están en 1, por lo que el filtro de Bloom dirá que el elemento está en el set, cuando en realidad no lo está. Esto es un falso positivo.

Depende del tipo de aplicación, puede ser que esto sea aceptable. Por ejemplo, en el caso de la verificación de *username*, si el filtro de Bloom dice que el *username* ya existe, se puede hacer una verificación adicional para confirmar. Pero si el filtro dice que no existe, entonces no se necesita hacer nada más y se ahorra tiempo considerable.

Probabilidad de un falso positivo: $P(1 - [1 - 1/m]^{kn})^k$

8.5.5.1.1 Complejidad:

- Tiempo de inserción: $O(k)$
- Tiempo de búsqueda: $O(k)$
- Espacio requerido: $O(m)$

8.5.5.1.2 Selección de la función de hash

- Deben ser funciones rápidas
- Usar hash criptográfico proveerá una mayor estabilidad pero tiene un hit de performance muy importante

8.5.5.1.3 Aplicaciones conocidas de filtros de bloom

- Medium.com lo utiliza para identificar los post ya vistos por el usuario
- Cloudflare lo utiliza para identificar IPs maliciosas
- Google Chrome lo utiliza para identificar URLs maliciosas
- Apache Cassandra lo utiliza para buscar registros inexistentes en disco

8.6 Algoritmos Genéticos

Se fundamentan en el trabajo de John Holland en 1962, quien luego publica en 1975 su libro “Adaptation in Natural and Artificial Systems”. Los algoritmos genéticos son una técnica de optimización y búsqueda basada en la teoría de la evolución de Darwin.

En 1980 se aplican en problemas de optimización y en 1989 en problemas de aprendizaje automático.

Se pueden definir como una técnica de búsqueda para problemas de optimización y aprendizaje automático que simula el proceso de selección natural.

Se consideran como algoritmos heurísticos, ya que no garantizan la obtención de la solución óptima, pero sí una solución aceptable en un tiempo razonable.

Heurística significa “encontrar” en griego. Se refiere a la búsqueda de soluciones a problemas de forma rápida y eficiente, pero no necesariamente óptima. Es el pasado de *eureka*.

Utilizan técnicas inspiradas en la biología evolutiva, como la selección natural, la reproducción y la mutación.

8.6.1 Definiciones esenciales

- **Individuo:** Representa una solución al problema. Puede ser una cadena de bits, un vector de números, una estructura de datos, etc.
- **Población:** Conjunto de individuos/posibles soluciones.
- **Fitness:** Función que evalúa la calidad de una solución/individuos
- **Características:** Representan las propiedades de un individuo. Pueden ser los genes de un individuo.
- **Genoma:** Conjunto de características de un individuo.

Dependiendo del problema se pueden usar múltiples cromosomas para representar un individuo.

8.6.2 Representación de un individuo

El objetivo es optimizar el espacio de búsqueda escogiendo una representación adecuada para el problema. Usualmente se recomienda el uso de bit-vectors, donde cada entrada indica si cierta característica está presente o no.

8.6.3 Funcionamiento general

Inician con una población de individuos aleatorios. En cada generación, se evalúa el fitness de cada individuo, se seleccionan los mejores y se reproducen para generar una nueva generación.

La nueva población reemplaza a la anterior y se repite el proceso hasta que se cumple un criterio de parada.

¿Cuándo se detiene el algoritmo? Puede ser cuando se alcanza un número máximo de generaciones, cuando se alcanza un fitness mínimo, cuando se alcanza un fitness máximo, etc.

8.6.4 ¿Cómo seleccionar los padres?

Después de aplicar la función de fitness, se obtiene un grupo de posibles padres:

- Secuencia: 1 - 2, 3 - 4...
- Random

8.6.5 Introducir variabilidad

Después de seleccionar los padres, se aplica recombinación y mutaciones.

8.6.6 Recombinación

Se mezclan los genes de los padres para generar nuevos individuos. Suponiendo que los siguientes bit-vectors representan los padres:

1		0	1	2	3	4	5
2	P1:	0	0	0	0	0	0
3	P2:	1	1	1	1	1	1
4				^			
5							

Cross-over point

Se generan los siguientes hijos:

1		0	1	2	3	4	5
2	H1:	0	0	0	1	1	1
3	H2:	1	1	1	0	0	0

8.6.7 Mutación

Con base en alguna probabilidad determinada, se hace flip a algunos bits aleatorios de cada bit-vector de los hijos generados.

8.7 Referencias

- <https://www.geeksforgeeks.org/introduction-to-divide-and-conquer-algorithm/>

-<https://www.geeksforgeeks.org/introduction-to-greedy-algorithm-data-structures-and-algorithm-tutorials/>

9 Algoritmos de búsqueda

Este tema abarca los algoritmos de búsqueda en estructuras de datos como *arrays* y expande para incluir búsquedas en planos (aplicables a juegos por ejemplo). No incluye motores de búsqueda textuales.

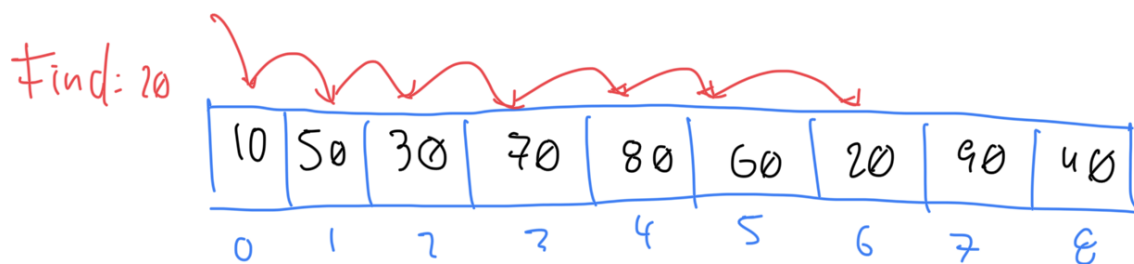
9.1 Búsqueda en arrays

9.1.1 Búsqueda secuencial

Búsqueda secuencial es la solución trivial para buscar en una colección de elementos. Cuando la colección está desordenada, es la única opción. Revisa/procesa cada elemento hasta encontrar el elemento deseado.

- En el peor caso, cada elemento de la colección es comparado contra la llave de búsqueda
- Si hay un *match*, la búsqueda termina y se retorna el índice del elemento
- Si no hay *match*, se retorna -1 (u otro índice negativo)

Por ejemplo, dado el siguiente array, para buscar el elemento 20, se seguiría el siguiente proceso:



9.1.1.1 Implementación

```
1 public class SequentialSearch {  
2     public static int search(int[] arr, int x) {  
3         for (int i = 0; i < arr.length; i++) {  
4             if (arr[i] == x) {  
5                 return i;  
6             }  
7         }  
8         return -1;  
9     }  
10 }
```

9.1.1.2 Consideraciones importantes

- Tiene una complejidad temporal de $O(n)$ para el peor caso y $O(1)$ para el mejor caso.
- Es ineficiente para colecciones grandes
- Si el array no está ordenado o se quiere evitar ordenar, es la única opción

9.1.2 Búsqueda binaria

Búsqueda binaria es un algoritmo de búsqueda que encuentra la posición de un valor en un arreglo **ordenado**. A diferencia de la búsqueda lineal, que recorre el arreglo desde el primer elemento hasta el último, la búsqueda binaria divide el arreglo en dos mitades y compara el valor buscado con el elemento en el medio.

- Si el valor buscado es *menor que el elemento en el medio*, la búsqueda continúa en la mitad izquierda del arreglo.
- Si el valor buscado es *mayor que el elemento en el medio*, la búsqueda continúa en la mitad derecha del arreglo.

Este proceso se repite hasta que el valor buscado sea encontrado o hasta que el subarreglo de búsqueda sea vacío.

9.1.2.1 Ejecución de búsqueda binaria

Dado el siguiente arreglo:

1	Indices	0	1	2	3	4	5	6	7	8	
2	Elementos	1	2	3	4	5	6	7	8	9	

Para buscar el número 9:

- Comenzamos comparando el número 9 con el elemento en el medio del arreglo, que es 5.
- Como 9 es mayor que 5, la búsqueda continúa en la mitad derecha del arreglo.
- Ahora comparamos el número 9 con el elemento en el medio de la mitad derecha del arreglo, que es 8.
- Como 9 es mayor que 8, la búsqueda continúa en la mitad derecha de la mitad derecha del arreglo.
- Ahora comparamos el número 9 con el elemento en el medio de la mitad derecha de la mitad derecha del arreglo, que es 9.
- Como 9 es igual a 9, hemos encontrado el número que buscábamos.

Visualmente, se puede representar de la siguiente manera:

1	Indices	0	1	2	3	4	5	6	7	8	
2	Elementos	1	2	3	4	5	6	7	8	9	
3		-----									
4	5 < 9					^					
5							-----				
6	7 < 9						^				
7								-----			
8	8 < 9							^			
9									--		
10	9 == 9								^		

9.1.2.2 Implementación en Java

```

1 public static int binarySearch(int[] arr, int target) {
2     int left = 0;
3     int right = arr.length - 1;
4
5     while (left <= right) {
6         int mid = left + (right - left) / 2;
7
8         if (arr[mid] == target) {
9             return mid;
10        }
11
12        if (arr[mid] < target) {
13            left = mid + 1;
14        } else {
15            right = mid - 1;
16        }
17    }
18
19    return -1;
20 }

```

9.1.3 Búsqueda por interpolación

- Es una mejora sobre **búsqueda binaria**, específicamente si los valores en el array están distribuidos uniformemente. La búsqueda por interpolación calcula la posición de la mitad del array basado en el valor del elemento buscado y los valores en los extremos del array.
- Búsqueda binaria siempre compara contra el elemento central del array, mientras que búsqueda por interpolación considera la llave de búsqueda antes de decidir contra cual elemento del array comparar.
- El código es igual al de búsqueda binaria, con la diferencia de que la posición del elemento medio se calcula de manera diferente:

```
1 mid = low + ((high - low) / (arr[high] - arr[low])) * (x - arr[low])
```

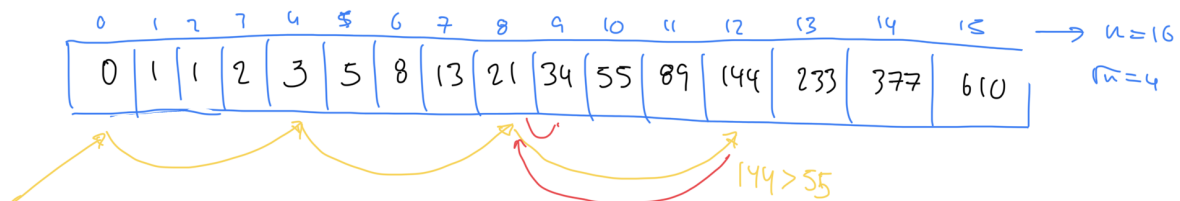
Low y high se refieren a los índices del array, y arr[low] y arr[high] a los valores en esos índices.

La mejora en rendimiento con respecto a búsqueda binaria se da en el caso promedio (que depende de la distribución uniforme de los elementos en el array), con una complejidad de tiempo de $O(\log \log n)$.

9.1.4 Búsqueda por salto (Jump Search)

Aplicable para arrays ordenados, comparandos menos elementos que búsqueda lineal saltándose n elementos a la vez. Determinar el tamaño del bloque de saltos es crucial para el rendimiento del algoritmo.

Normalmente se utiliza $\sqrt{n}$ como tamaño de bloque, donde n es el tamaño del array



Complejidad temporal: $O(\sqrt{n})$

9.2 Pathfinding

Pathfinding se refiere a búsqueda de caminos entre dos puntos en un plano, especialmente útil para video juegos y simulaciones. Incluye una amplia variedad de algoritmos y no hay una solución única para todos los casos. Por ejemplo, la elección del algoritmo depende de:

- ¿Es el destino estacionario o móvil?
- ¿Hay obstáculos en el mapa?
- ¿Hay diferente tipos de terreno en el mapa?

9.2.1 Pathfinding básico

Se refiere a encontrar el camino más corto entre dos puntos en un plano sin obstáculos. Por ejemplo, cómo se mueve a un personaje de un punto A a un punto B en un mapa de videojuego.

```
1 if(positionX > destinationX)
2     positionX--;
3 else if(positionX < destinationX)
4     positionX++;
5 if(positionY > destinationY)
6     positionY--;
7 else if(positionY < destinationY)
8     positionY++;
```

Esta es una solución simple, pero no es óptima. No considera obstáculos, terreno, o la distancia real entre los puntos. Además de esto, el movimiento resultante no es “natural”:

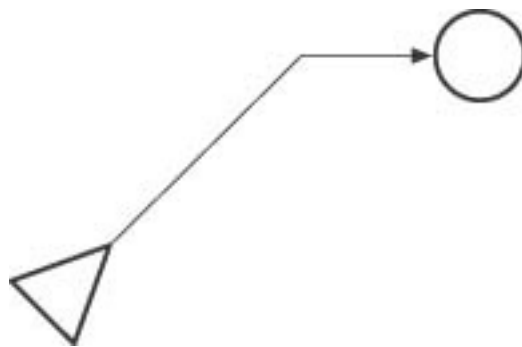


Figura 84: Movimiento poco natural en pathfinding

Utilizando algoritmos de línea de visión, se puede mejorar el movimiento del personaje:

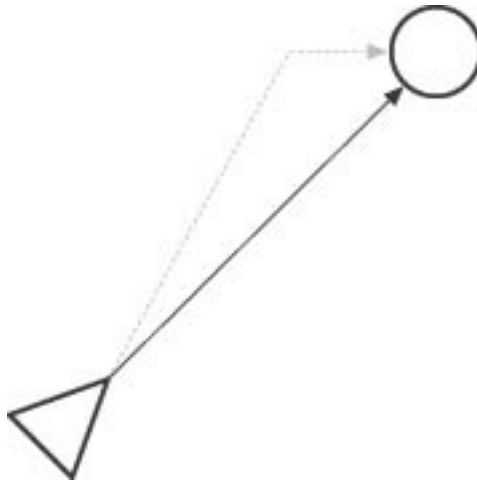


Figura 85: Movimiento esperado en pathfinding

Cualquiera de estos métodos produce resultados precisos. Por eso, se deben utilizar cuando sea posible. *Pathfinding* no es un problema que se resuelve con un solo algoritmo, sino con una combinación de

algoritmos. Al encontrarse obstáculos, estos enfoques no son suficientes.

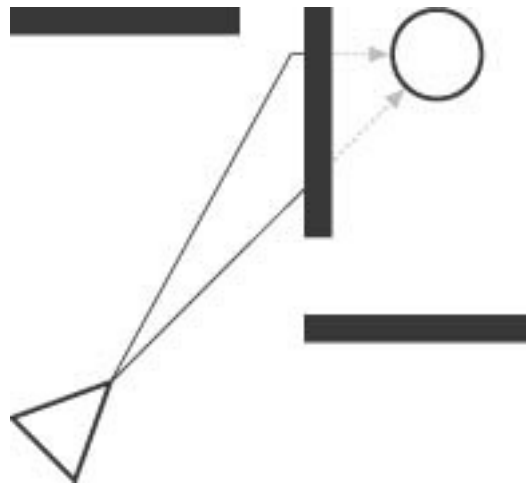


Figura 86: Pathfinding y obstáculos

9.2.2 Movimiento aleatorio ante obstáculos

Aunque parezca poco eficiente, el movimiento aleatorio es una solución simple y efectiva para evitar obstáculos, especialmente en ambientes con relativamente pocos obstáculos.

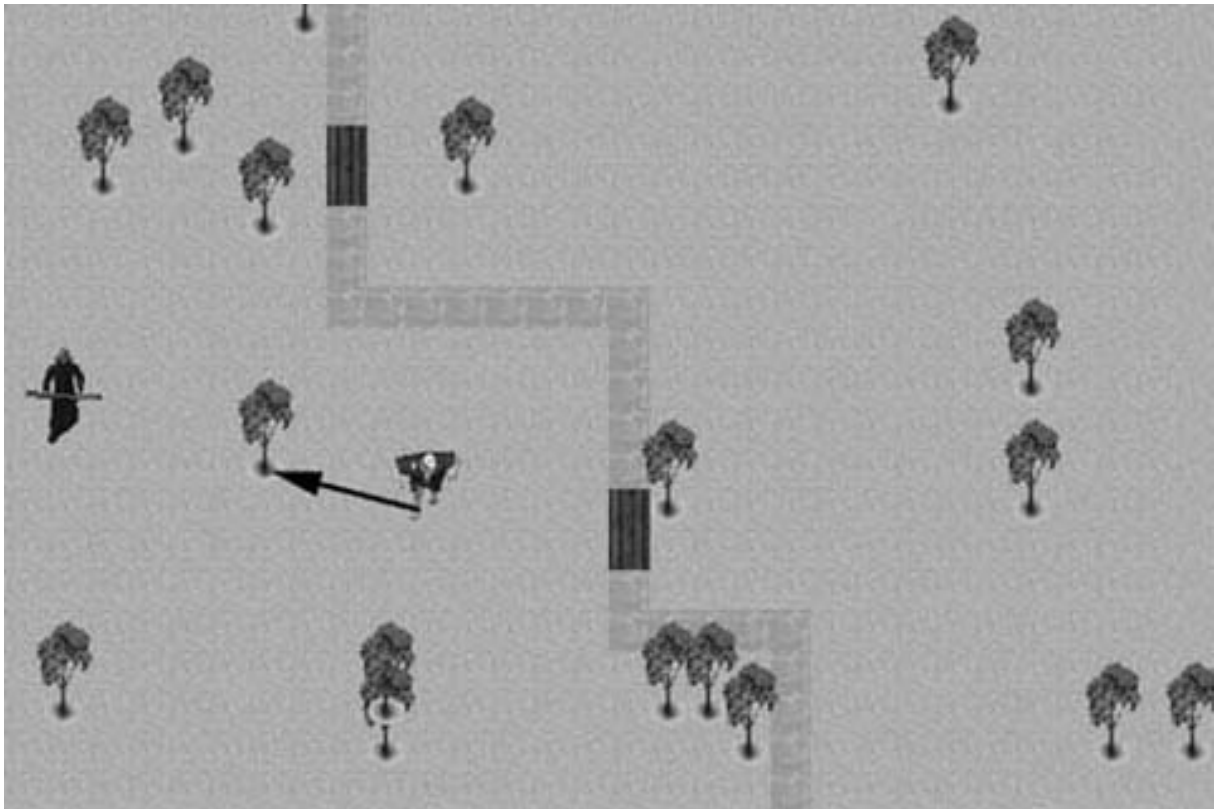


Figura 87: Movimiento aleatorio

Como se aprecia en la figura anterior, un movimiento aleatorio para evitar al enemigo puede ser suficiente y evita caer en un gasto computacional innecesario.

```
1 if Player In Line of Sight
2 {
3     Follow Straight Path to Player
4 }
5 else
6 {
7     Move in Random Direction
8 }
```

9.2.3 Rodear obstáculos

En lugar de moverse en línea recta hacia el destino, se puede rodear los obstáculos. Es relativamente simple y útil para evitar obstáculos grandes en el plano, tales como montañas o masas de agua. El proceso sería:

1. El personaje intenta un *pathfinding* básico hasta el objetivo.

2. Si encuentra un obstáculo, se cambia a modo de rodeo.
3. Sigue el borde del obstáculo hasta que pueda volver a intentar el *pathfinding básico* con éxito.

El problema del rodeo es que puede ser difícil de determinar cuando salir de dicho modo. Una forma puede ser determinar antes de entrar al modo de rodeo, cual es la ruta donde se puede salir de este.

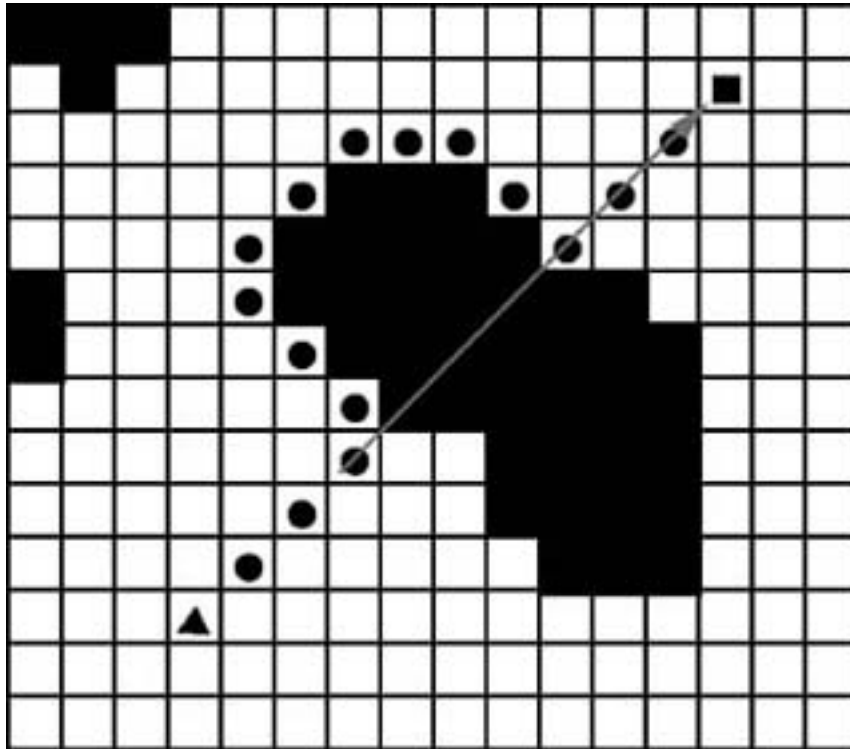


Figura 88: Rodeo de obstáculos

Para hacer el movimiento más natural, se puede intentar línea de visión en cada paso. Si el personaje puede ver el objetivo, se mueve en línea recta hacia él. Si no, se sigue rodeando el obstáculo.

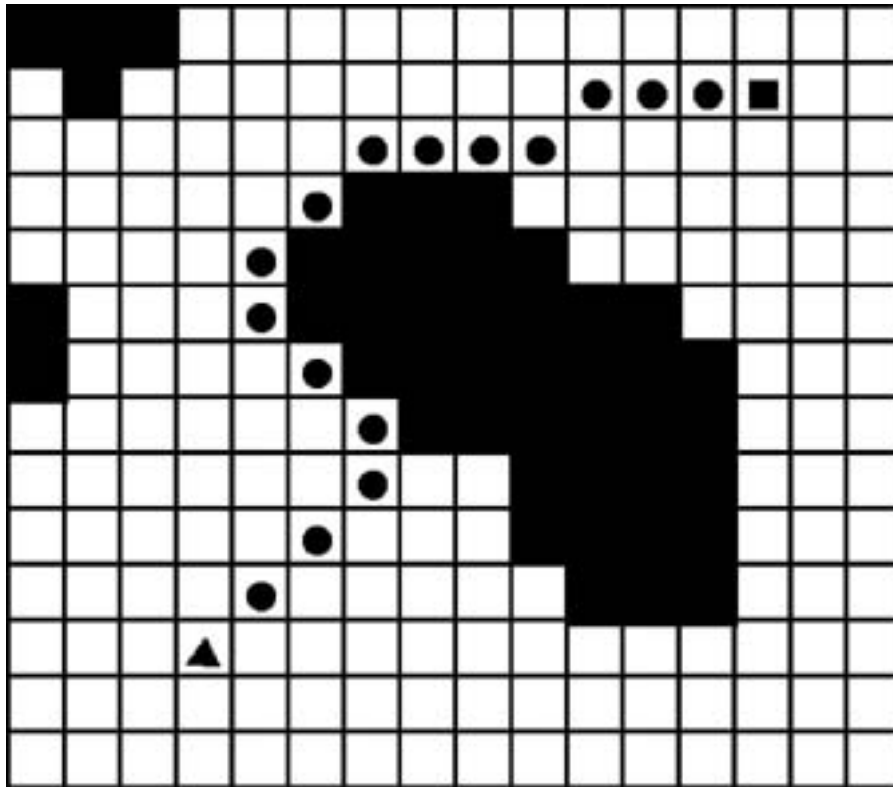


Figura 89: Rodeo de obstáculo y línea visión en cada paso

9.2.4 Pathfinding basado en grafos

En este enfoque, el mapa se representa como un grafo, donde los nodos son las posiciones en el mapa y las aristas son las conexiones entre las posiciones. Cada arista tiene un costo asociado, que puede ser la distancia entre los nodos o el tiempo que toma recorrerla.

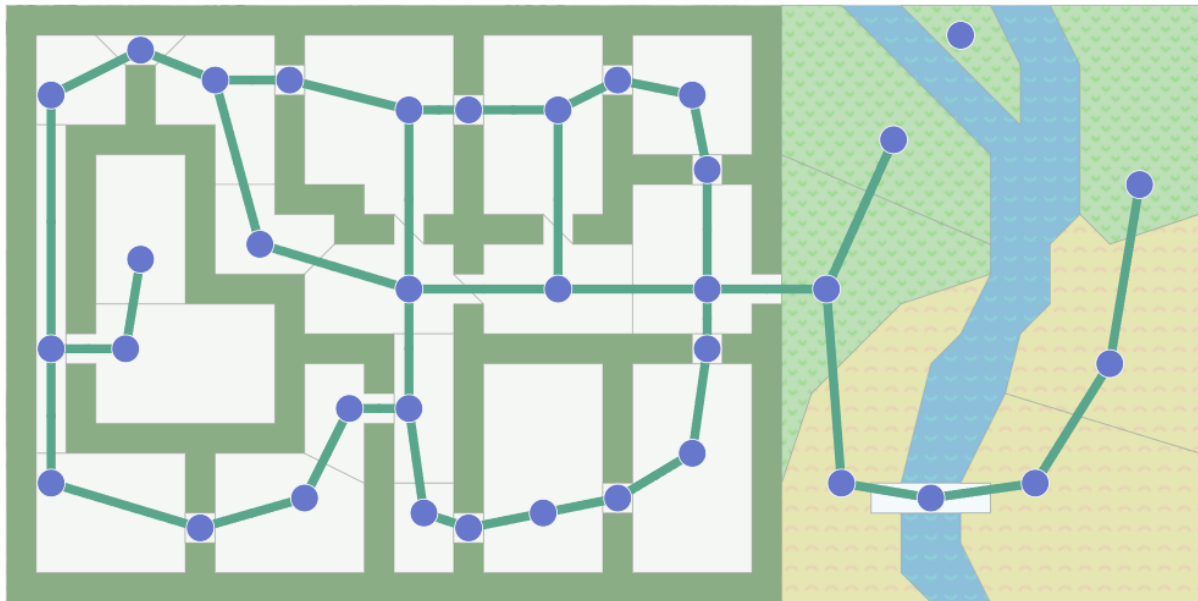


Figura 90: El mapa como un grafo

Si el mapa se modela como una rejilla, se puede ver como un tipo de grafo especial, como se muestra en la siguiente imagen:

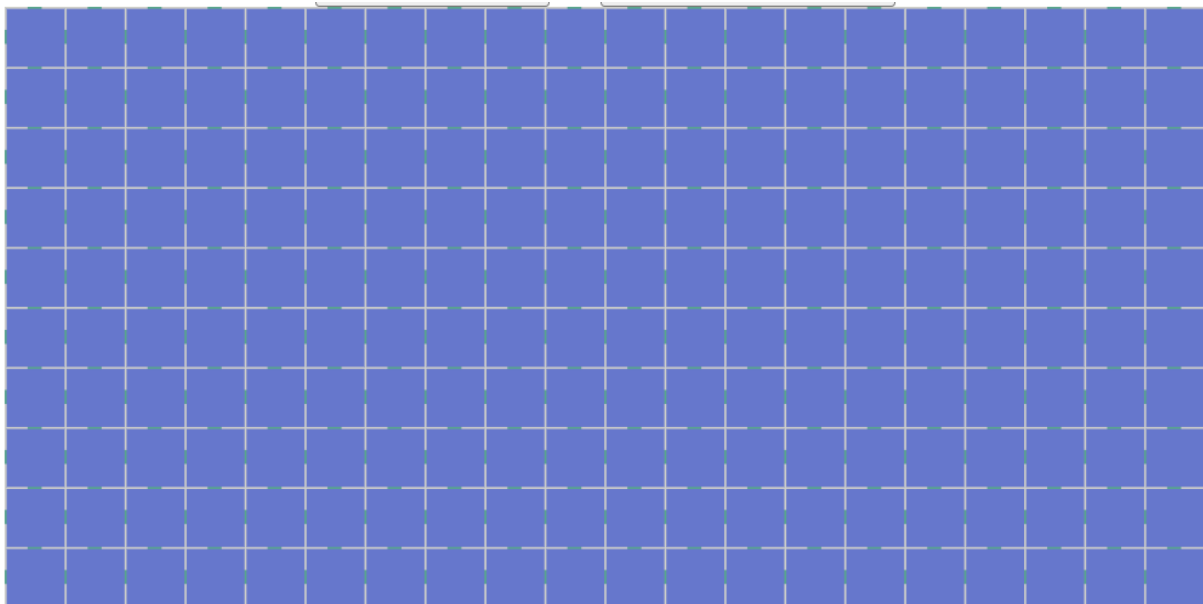


Figura 91: El mapa como un grid

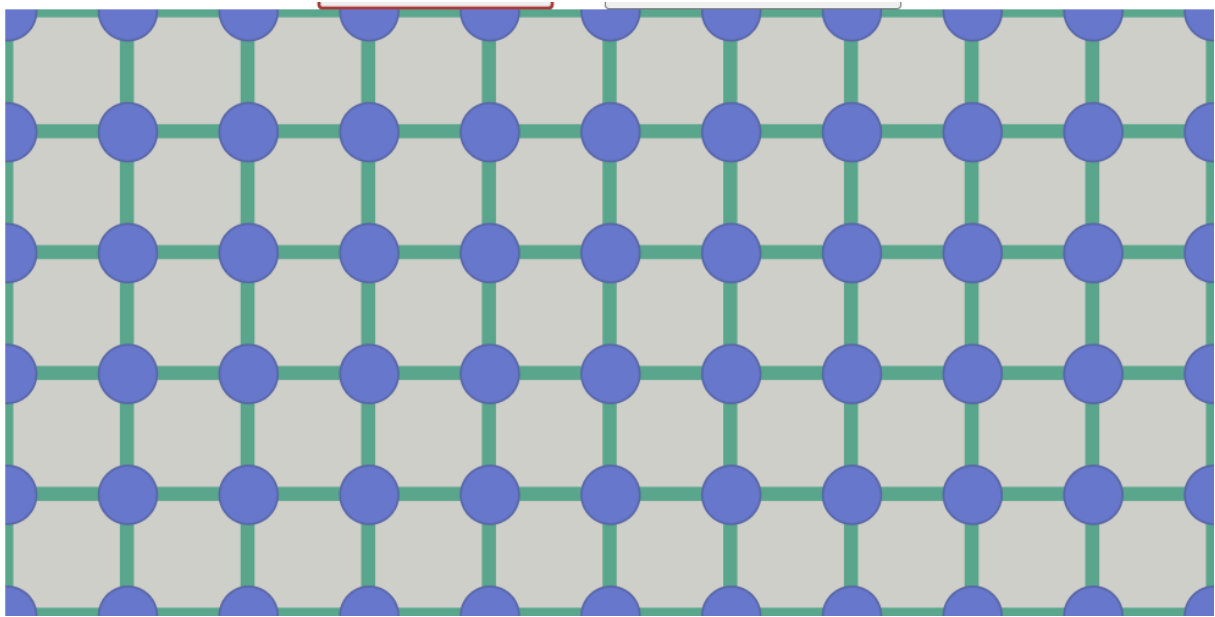


Figura 92: El grid es un grafo

9.2.4.1 Breath-first search (BFS) El algoritmo BFS (búsqueda en amplitud) es un algoritmo de búsqueda de grafos que comienza en un nodo raíz y explora todos los nodos vecinos a la raíz antes de avanzar a los nodos vecinos de estos. Visualmente, se puede ver como una expansión en todas las direcciones:

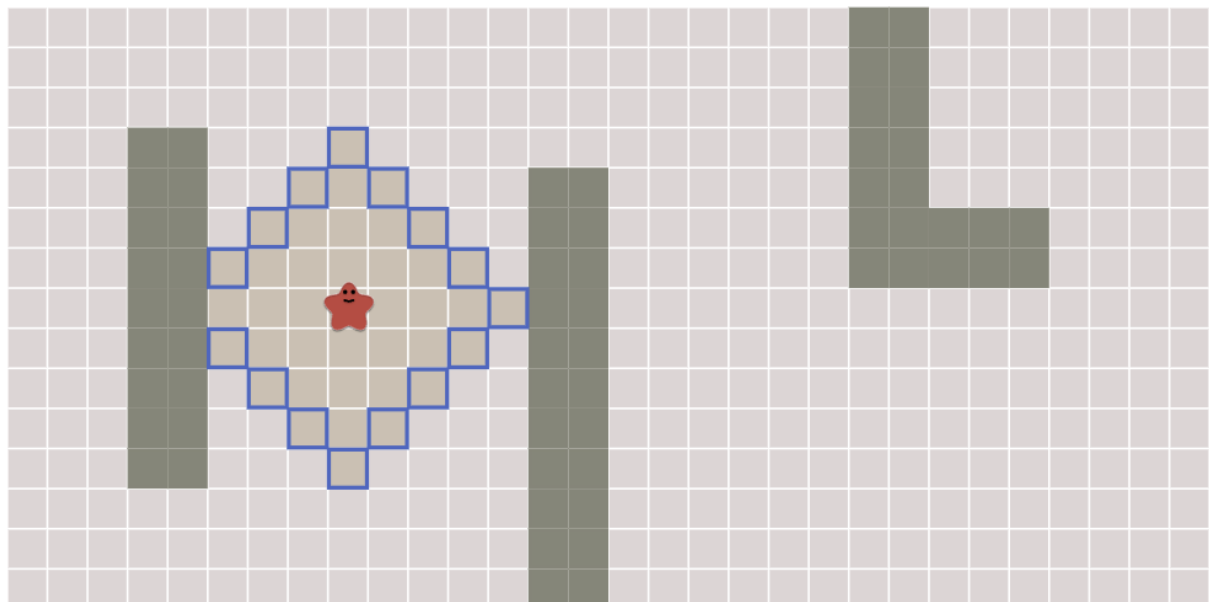


Figura 93: BFS

El algoritmo es tal y como se vio en el curso de Estructuras de Datos 1. A manera de recordatorio, se presenta el pseudocódigo:

```
1 frontier = Queue()
2 frontier.put(start )
3 came_from = dict()
4 came_from[start] = None
5
6 while not frontier.empty():
7     current = frontier.get()
8
9     if current == goal:
10        break
11
12    for next in graph.neighbors(current):
13        if next not in came_from:
14            frontier.put(next)
15            came_from[next] = current
```

9.2.4.2 Dijkstra BFS considera el movimiento hacia cualquier nodo como igual. Sin embargo, en muchos escenarios, distintas posiciones en el mapa tienen distintos costos o pesos. Por ejemplo, en algunos juegos de estrategia, distintos tipos de terreno (bosque, agua, desierto, entre otros) tienen diferentes efectos en la velocidad de los personajes. En estos casos, el algoritmo de *Dijkstra* es más adecuado.

Dijkstra lleva el rastro del costo acumulado y utiliza una cola de prioridad para escoger el siguiente paso. El código es similar al siguiente pseudocódigo:

```
1 frontier = PriorityQueue()
2 frontier.put(start, 0)
3 came_from = {}
4 cost_so_far = {}
5 came_from[start] = None
6 cost_so_far[start] = 0
7 while not frontier.empty() {
8     current = frontier.get()
9     if current == goal {
10        break
11    }
12    for next in graph.neighbors(current) {
13        new_cost = cost_so_far[current] + graph.cost(current, next)
14        if next not in cost_so_far or new_cost < cost_so_far[next] {
15            cost_so_far[next] = new_cost
16            priority = new_cost
17            frontier.put(next, priority)
18            came_from[next] = current
19        }
20    }
```

21 }

Visualmente, se puede entender la diferencia entre BFS y Dijkstra en esta animación. A continuación se incluyen una captura de pantalla de la animación:

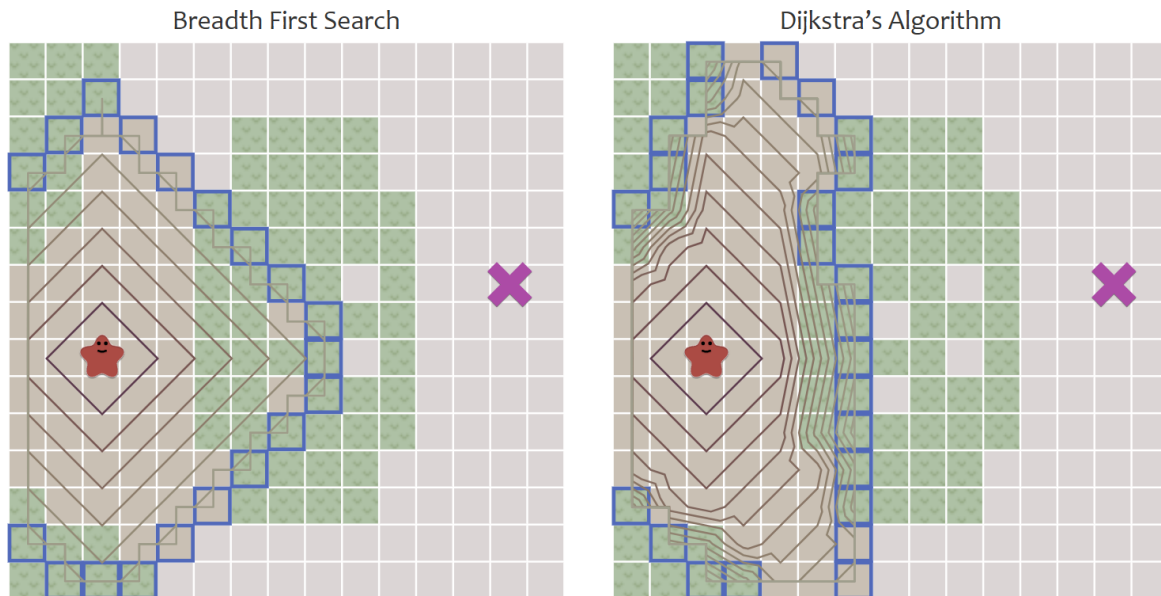


Figura 94: Dijkstra

9.2.4.3 A* Considera el costo real (similar a Dijkstra) y una heurística que estima el costo restante. La heurística es una función que estima el costo de llegar al destino desde un nodo dado. La heurística debe ser admisible, es decir, nunca sobreestimar el costo real.

Se utiliza con ciertas mejoras, en game engines modernos, como Unity.

```

1 frontier = PriorityQueue()
2 frontier.put(start, 0)
3 came_from = {}
4 cost_so_far = {}
5 came_from[start] = None
6 cost_so_far[start] = 0
7 while not frontier.empty():
8     current = frontier.get()
9     if current == goal:
10         break
11
12     for next in graph.neighbors(current):
13         new_cost = cost_so_far[current] + graph.cost(current, next)
14         if next not in cost_so_far or new_cost < cost_so_far[next]:
15             cost_so_far[next] = new_cost

```



```
16         priority = new_cost + heuristic(goal, next)
17         frontier.put(next, priority)
18         came_from[next] = current
19     }
20 }
21 }
```

Para calcular la heurística, se puede utilizar la distancia Manhattan o la distancia Euclidiana. La distancia Manhattan es la suma de las diferencias en las coordenadas x y y, mientras que la distancia Euclidiana es la distancia en línea recta.

```
1 // Manhattan distance
2 function heuristic(a, b) {
3     return abs(a.x - b.x) + abs(a.y - b.y)
4 }
```

Dependiendo del escenario, la heurística puede ser precalculada y almacenada en una tabla de búsqueda.

A* y Dijkstra son similares, pero A* es más rápido porque la heurística guía la búsqueda hacia el destino. La visualización aquí permite compararlas.

Aunque A* y Dijkstra encuentran el camino, la cantidad de comparaciones realizadas por A* es mucho menor. En el peor caso, A* es igual a Dijkstra, pero en el mejor caso, A* es mucho más rápido.

9.2.5 Referencias

- <https://iq.opengenus.org/time-complexity-of-interpolation-search/>
- <https://www.geeksforgeeks.org/jump-search/>
- Pathfinding
- Pathfinding on Grids
- Bourq, David M. (2004). AI for Game Developers. O'Reilly Media. ISBN 978-0-596-00555-2.

10 Estructuras de almacenamiento externo

La jerarquía de memoria, se puede representar con la siguiente pirámide:

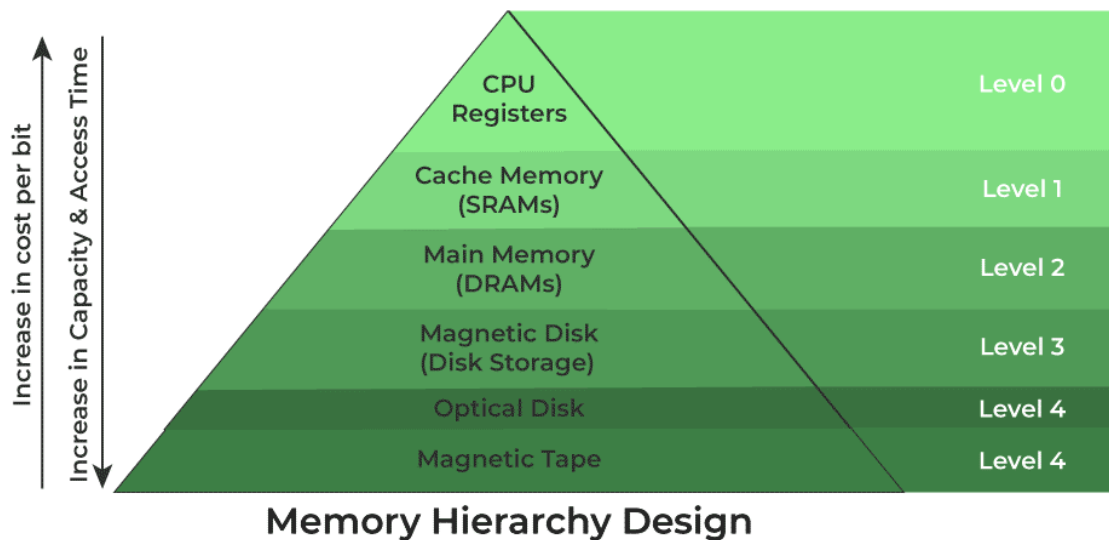


Figura 95: Jerarquía de memoria

Conforme se “sube” en la jerarquía, la cantidad de memoria disminuye pero el costo y velocidad de la misma aumentan. Conforme se desciende, el costo y velocidad disminuyen pero la cantidad de memoria aumenta. Es por eso, que el cache, es una memoria muy rápida pero de poca capacidad, mientras que el disco duro es una memoria lenta pero de gran capacidad.

10.1 Conceptos esenciales

- **Dato:** Sucesión de símbolos representados con números o letras. No contienen ninguna información en sí mismo, sino que representan un *hecho* con algún significado dado por una unidad y su magnitud. Requieren organización y procesamiento para extraer información. Por ejemplo: \$10, 20 años, 3 kg, 10 metros, 22 km.
- **Información:** Es la interpretación significativa de los datos
- **Sistemas de almacenamiento:** Sistemas para almacenar datos en formato binario sin importar la información que se pueda extraer de dichos datos. Solo ven bloques crudos de bits.

10.2 Discos Duros (Hard-disk drives)

No deben confundirse con discos de estado sólido (SSD).

Son dispositivos de almacenameinto persistente que mantiene la información almacenada incluso cuando no están siendo alimentados con electricidad.

- Los datos se almacenan mediante magnetización de partículas dentro del material magnético de los discos.
- El disco es capaz de leer los datos al detectar los patrones de magnetización creados al escribir los datos.

10.2.1 Componentes

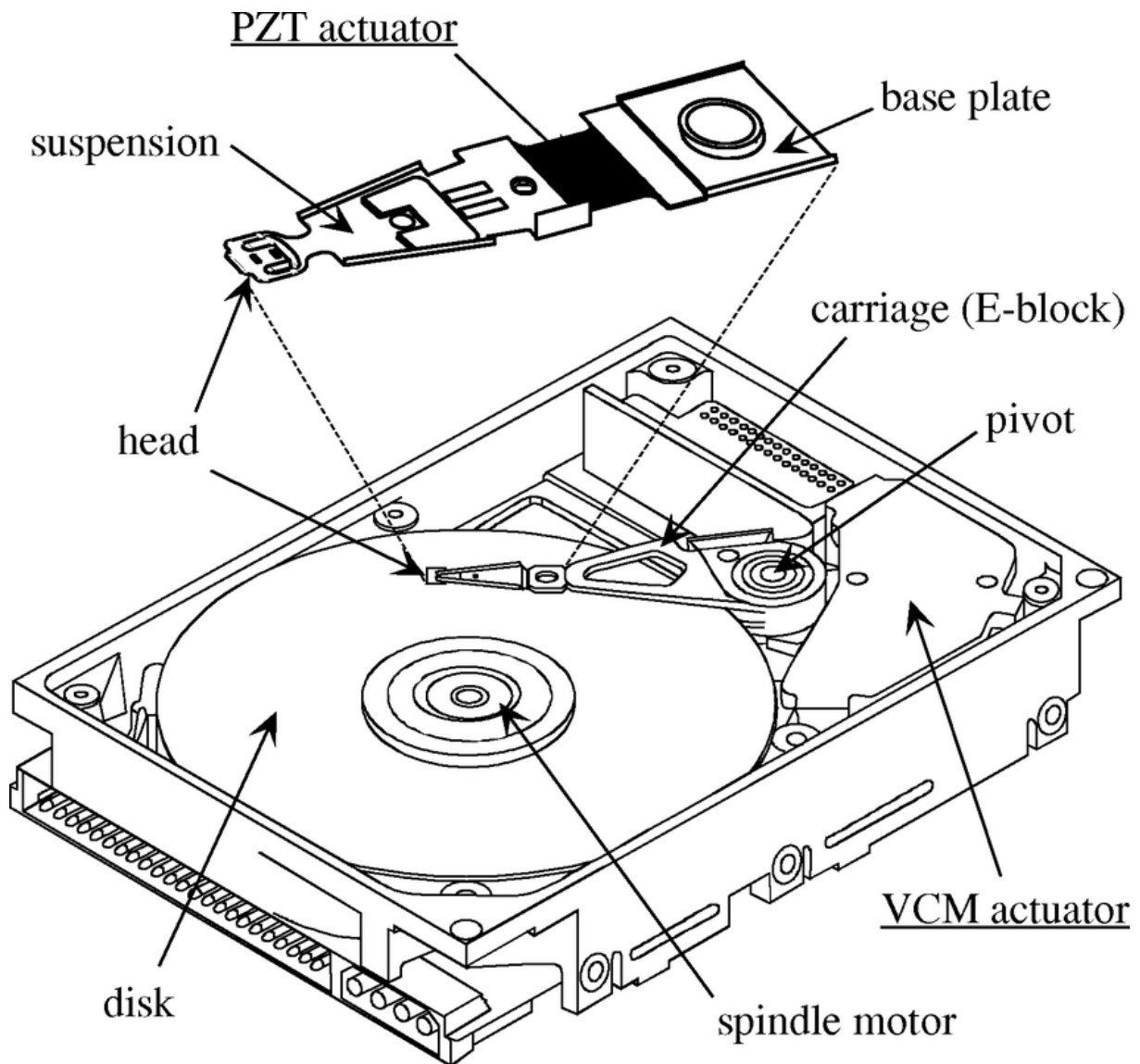
- *Platos*: Discos circulares de material magnético en ambas caras
- *Eje*: sostiene uno o más platos conectados a un motor que gira los platos a un RPM constante.
- *Brazo actuador*: Mueve las cabezas de lectura/escritura a lo largo de los platos.
- *Cabezas*: Dispositivos electromagnéticos que leen y escriben datos en los platos.
- *Hard disk assembly*: Combinación de los platos, eje, brazo actuador y cabezas.

10.2.2 Funcionamiento

La velocidad del giro influencia directamente a la velocidad de I/O. A mayor velocidad, mayor consumo energético y mayor costo.

- Velocidades para uso general (consumidor):
 - 5400 RPM
 - 7200 RPM
- Velocidades para uso profesional (servidores):
 - 10000 RPM
 - 15000 RPM

Los datos se escriben en círculos concéntricos llamados *pistas* y se dividen en *sectores*. Un sector es la unidad mínima de almacenamiento en un disco duro y generalmente tiene un tamaño de 512 bytes.



10.2.3 Tiempos de acceso

El tiempo de búsqueda (*seek time*) es el tiempo que tarda el brazo actuador en moverse a la pista deseada (8 - 10ms en discos de 7200 RPM). El tiempo de latencia es el tiempo que tarda el sector deseado en pasar por debajo de la cabeza de lectura/escritura (4 - 5ms en discos de 7200 RPM).

El tiempo de acceso total es la suma del tiempo de búsqueda y el tiempo de latencia.

10.2.4 Tiempo de transferencia

Los discos duros tienen un tiempo de transferencia que es el tiempo que tarda en leer o escribir un bloque de datos. Este tiempo depende de la velocidad de rotación del disco y de la densidad de los datos. Por ejemplo, un disco de 7200 RPM tiene un tiempo de transferencia de 0.5ms.

El *external data rate* es la cantidad de datos que se pueden transferir por segundo. Por ejemplo, un disco de 7200 RPM con un external data rate de 300MB/s puede transferir 300MB de datos por segundo.

10.2.5 Interfaces

El HDD se conecta a otros componentes de la computadora a través de una interfaz. Las interfaces más comunes son:

- PATA (Parallel Advanced Technology Attachment)
 - Interfaz de 40 pines que se conecta a la placa madre.
 - Velocidad de transferencia de 133MB/s.
 - No es hot-swappable.
 - No se usa en la actualidad.
 - Solo para discos internos
- SATA (Serial Advanced Technology Attachment)
 - Evolución de PATA.
 - Estandar en computación moderna.
 - Velocidad de transferencia de 600MB/s.
 - Bajo consumo energético.
 - Hot-swappable.
- SCSI (Small Computer System Interface)
 - Interfaz de alta velocidad.
 - Se usa en servidores y estaciones de trabajo.
 - Hot-swappable.
- SAS (Serial Attached SCSI)
 - Evolución de SCSI.
 - Velocidad de transferencia de 12GB/s.
 - Hot-swappable.
 - Se usa en servidores y estaciones de trabajo.

10.3 Discos de Estado Sólido (SSD)

No son discos duros, sino dispositivos de almacenamiento de estado sólido que utilizan memoria flash para almacenar datos. No tienen partes móviles, lo que los hace más rápidos y menos propensos a fallas que los discos duros.

- Utiliza memoria flash para almacenar datos.
- Menor acceso aleatorio que los discos duros.
- No tiene latencia de búsqueda.
- Writes limitados (menor tiempo de vida útil).
- El precio es más alto que los discos duros.

10.3.1 Componentes

- **Controlador:** Procesador que ejecuta el firmware y es responsable de la gestión de la memoria, la corrección de errores y la interfaz con el sistema operativo.
- **Memoria NAND:** Memoria flash que almacena los datos. Se divide en celdas que pueden ser de tipo SLC, MLC, TLC o QLC.
- **DRAM:** Memoria volátil que se utiliza para almacenar datos temporales y mejorar el rendimiento.
- **Firmware:** Software que controla el funcionamiento del SSD.
- **Conector:** Interfaz que conecta el SSD a la placa madre. Los conectores más comunes son SATA y PCIe (utilizando el protocolo NVMe).
- **Form factor:** Tamaño y forma del SSD. Los form factors más comunes son 2.5", M.2 y U.2. También existen SSDs en formato de tarjeta de expansión PCIe.

10.3.2 Desempeño

La transferencia de datos de un SSD es mucho más rápida que la de un disco duro. Los SSDs modernos pueden alcanzar velocidades de lectura de hasta 550 MB/s y velocidades de escritura de hasta 520 MB/s. Un SSD PCIe NVMe puede alcanzar velocidades de lectura de hasta 3500 MB/s y velocidades de escritura de hasta 3300 MB/s.

10.4 Particionamiento de discos

Un solo disco físico puede ser dividido en varias particiones, cada una de las cuales se comporta como un disco independiente (lógico). Las particiones se pueden formatear con diferentes sistemas de archivos y se pueden montar en diferentes puntos de montaje.

En enlace entre lo físico y lo lógico se hace a través de la *metadada* almacenada en la tabla de particiones. La tabla de particiones es un registro que contiene información sobre las particiones del disco, como su tamaño, ubicación y tipo de sistema de archivos. Esta tabla se encuentra en el primer sector del disco (MBR) y es leída por el sistema operativo al arrancar.

Una partición puede existir sin inicializarse.

10.4.1 Volumen

Es una partición inicializada con un *sistema de archivos*. Es la interfaz lógica que utiliza el sistema operativo para acceder a los datos almacenados en el disco.

10.5 Sistemas de archivos

El sistema de archivos puede entender como la abstracción que provee el Sistema Operativo para acceder a los datos almacenados en el disco (interfaz lógica entre usuario y la capa física de almacenamiento). Determina la forma en que los archivos son nombrados, organizados y almacenados en el disco. Almacena metadata/atributos de los archivos. Por ejemplo el nombre, tamaño, fecha de creación, etc.

Hay distintos sistemas de archivos conocidos y usualmente se asocian con uno u otro Sistema Operativo. Por ejemplo, Windows utiliza *NTFS*, Linux utiliza *ext3* o *ext4*, MacOS utiliza *HFS+*, y así por el estilo.

El sistema de archivos requiere espacio en disco para almacenar la metadata. Por ejemplo, si se tiene un archivo de 1 KB, el sistema de archivos necesita espacio adicional para almacenar la metadata del archivo (nombre, tamaño, fecha de creación, etc.). Por tal motivo, posterior a formatear un disco, el espacio disponible es menor al espacio total del disco.

10.5.1 Propósito

Algunas de las funcionalidades principales del sistema de archivo incluyen:

- Nombrar y organizar archivos.
- Proveer un API para el acceso a los archivos: Creación, lectura, escritura, eliminación, etc.

- Almacenar un índice de archivos y directorios.
- Gestionar permisos y restricciones de acceso para mantener la integridad y seguridad de los datos.
- Funciones avanzadas como compresión, cifrado, cuotas de disco, etc.

10.6 Archivos

Se puede definir como el mecanismo de abstracción para hacer transparente al usuario, los detalles complejos del disco. Dicha abstracción provee un “canal de comunicación” para poder ejecutar operaciones como:

- Leer bytes
- Escribir bytes
- Desplazarse a una posición específica en el disco

Un archivo se puede procesar de varias formas:

- **Acceso secuencial:** byte por byte desde el principio hasta el final según el orden en que fueron escritos.
- **Acceso aleatorio:** acceso directo a cualquier parte del archivo (conocido como acceso directo) independientemente del orden de escritura.

Acceso secuencial es más rápido que el acceso aleatorio, pero el acceso aleatorio es más flexible.

- **Acceso indexado:** acceso a través de un índice que contiene la ubicación de los datos. Depende de la estructura y propósito del archivo. Por ejemplo, si es un archivo que contiene registros de clientes, un índice puede mapear el identificador de cada cliente, a la posición de su registro en el archivo. Precursor de las bases de datos.

10.6.1 Estructura de un archivo

Las estructuras más comunes de archivos son:

- Secuencia de bytes
- Registros de tamaño fijo
- Árboles de registros

10.6.1.1 Secuencia de bytes En este caso, se trata de archivos planos que contienen una secuencia de bytes. No tienen estructura interna y se pueden leer o escribir byte por byte. El sistema operativo

no tiene conocimiento de la estructura interna del archivo, por lo que no puede realizar operaciones avanzadas como búsqueda o indexación.

Los archivos se reducen a ser “cubetas” de bytes, maximizando la flexibilidad. Windows/Linux/macOS utilizan este tipo de archivos.

10.6.1.2 Registros de tamaño fijo En este caso, el archivo se organiza en registros de tamaño fijo. Cada registro contiene una estructura de datos con campos de tamaño fijo. Los registros se pueden leer o escribir de forma secuencial o aleatoria. Este tipo de archivos permite realizar operaciones de búsqueda y ordenamiento.

Al crear un archivo de registros de tamaño fijo, se debe especificar la estructura de cada registro. Por ejemplo, si se tiene un archivo de registros de empleados, cada registro puede contener los campos: nombre, apellido, edad, salario, etc.

Muy utilizados en mainframes y bases de datos.

10.6.1.3 Árboles de registros Similar al caso anterior, pero en lugar de almacenar los registros de forma secuencial, se almacenan en una estructura de árbol. Cada nodo del árbol contiene un registro y apunta a otros nodos. Este tipo de archivos permite realizar operaciones de búsqueda y ordenamiento de forma eficiente.

10.6.2 Tipos de archivos

- **Archivos regulares:** Contienen datos de usuario. Pueden ser de texto o binarios.
- **Directorios:** Archivos que contienen una lista de archivos y directorios. Son partes esenciales del sistema de archivos.
- **Character special files:** Representan dispositivos de caracteres, como terminales y puertos serie.
- **Block special files:** Representan dispositivos de bloques, como discos duros y unidades flash.
- **Sockets:** Archivos que permiten la comunicación entre procesos.
- **Pipes:** Archivos que permiten la comunicación entre procesos.
- **Symbolic links:** Archivos que apuntan a otros archivos.
- **Binary files:** Archivos que contienen datos binarios, como imágenes, videos, etc.

10.7 Cache

- El cache se basa en el principio de localidad, que establece que los programas tienden a acceder a un conjunto reducido de direcciones de memoria en un corto periodo de tiempo.

- El concepto de caching puede ser aplicado a cualquier capa de computación, como CPU, disco, red, etc
- Se puede aplicar en distintas capas de la arquitectura de un sistema de información.
- Busca mantener datos usados frecuentemente en una ubicación óptima: cercano al usuario, cercano a la aplicación, en memoria más rápida, etc.

10.7.1 Funcionamiento general

1. Se solicita un dato
2. Se busca en el cache
3. Si se encuentra, se devuelve al usuario (cache hit)
4. Si no se encuentra, se busca en la fuente original (cache miss) y se almacena en el cache para futuras solicitudes.

10.7.2 Operaciones en el cache

Dado que la capacidad de un caché es limitada, usarlo de forma eficiente es crucial. Las operaciones más comunes son:

- **Hit:** Cuando la información solicitada se encuentra en el caché.
- **Miss:** Cuando la información solicitada no se encuentra en el caché.

Cada vez que se produce un *miss*, se debe decidir qué información se debe eliminar del caché para hacer espacio para la nueva información. Existen diferentes políticas de reemplazo, como *Least Recently Used* (LRU), *First In First Out* (FIFO), *Least Frequently Used* (LFU), *Most Recently Used* (MRU), etc. Veamos algunas de estas

10.7.2.1 Least Recently Used (LRU) La política LRU reemplaza la entrada que no ha sido utilizada por más tiempo. Es la política más común y se basa en la idea de que si un elemento no ha sido utilizado recientemente, es menos probable que se utilice en el futuro.

10.7.2.2 First In First Out (FIFO) La política FIFO reemplaza la entrada que ha estado en el caché por más tiempo. Es la política más simple y se basa en la idea de que los elementos que han estado en el caché por más tiempo son menos probables de ser utilizados en el futuro.

10.7.2.3 Least Frequently Used (LFU) La política LFU reemplaza la entrada que ha sido utilizada menos veces. Se basa en la idea de que los elementos que han sido utilizados menos veces son menos probables de ser utilizados en el futuro.

10.7.2.4 Most Recently Used (MRU) La política MRU reemplaza la entrada que ha sido utilizada más recientemente. Se basa en la idea de que los elementos que han sido utilizados más recientemente son más probables de ser utilizados en el futuro.

10.7.3 Tipos de cache a nivel general

10.7.3.1 En memoria Se almacena en la memoria RAM. Se utiliza para almacenar datos que se acceden con frecuencia a nivel del proceso.

10.7.3.2 Caché de disco Se almacena en el disco duro. Se utiliza para almacenar datos que se acceden con frecuencia a nivel del disco. Por ejemplo, los datos de un archivo que se accede con frecuencia.

10.7.3.3 Caché distribuido Se almacena en varios servidores. Se utiliza para almacenar datos que se acceden con frecuencia a nivel de la red. Por ejemplo, los datos de un sitio web que se accede con frecuencia.

10.7.4 Tipos de cache a nivel específicos

10.7.4.1 Application server cache Es una especialización del cache en memoria. Se almacena en la memoria RAM del servidor de aplicaciones. Se utiliza para almacenar datos que se acceden con frecuencia a nivel de la aplicación.

10.7.4.2 Caché global Se almacena en varios servidores. Se utiliza para almacenar datos que se acceden con frecuencia a nivel global. Por ejemplo, los datos de un sitio web que se accede con frecuencia en todo el mundo.

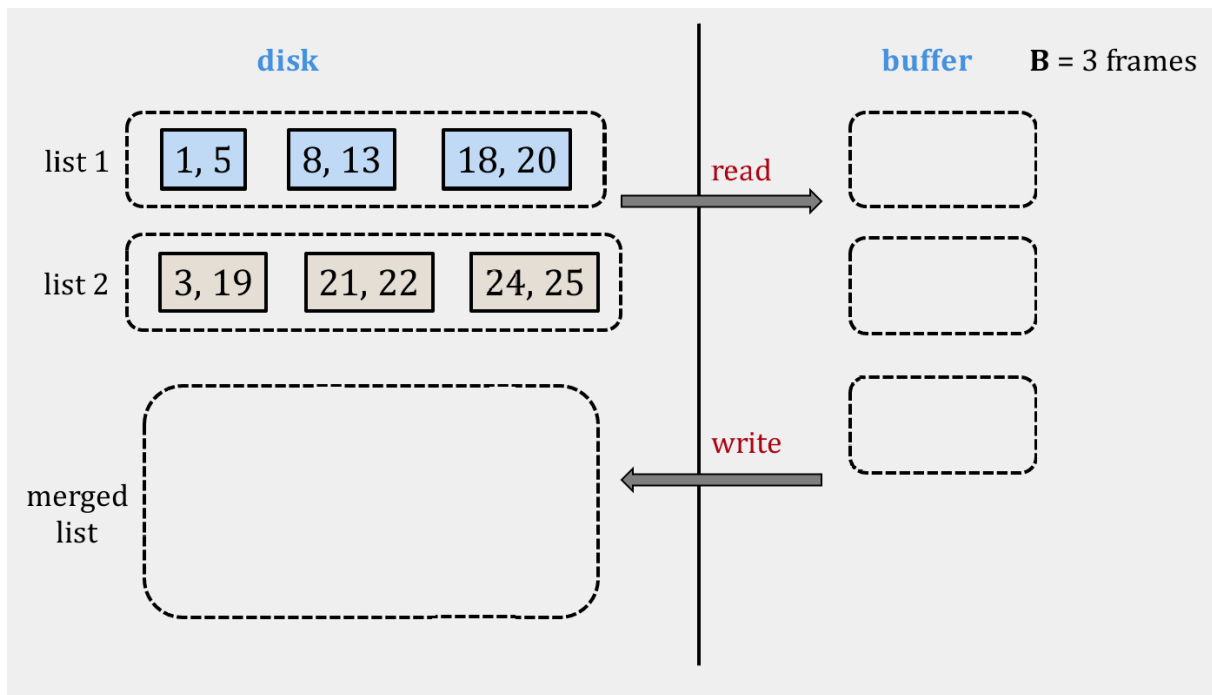
10.8 Algoritmos de ordenamiento externo

Los algoritmos de ordenamiento externo son algoritmos que se utilizan para ordenar grandes conjuntos de datos que no caben en la memoria principal. Usualmente utilizan un enfoque híbrido que combina la memoria principal y la memoria secundaria para ordenar los datos.

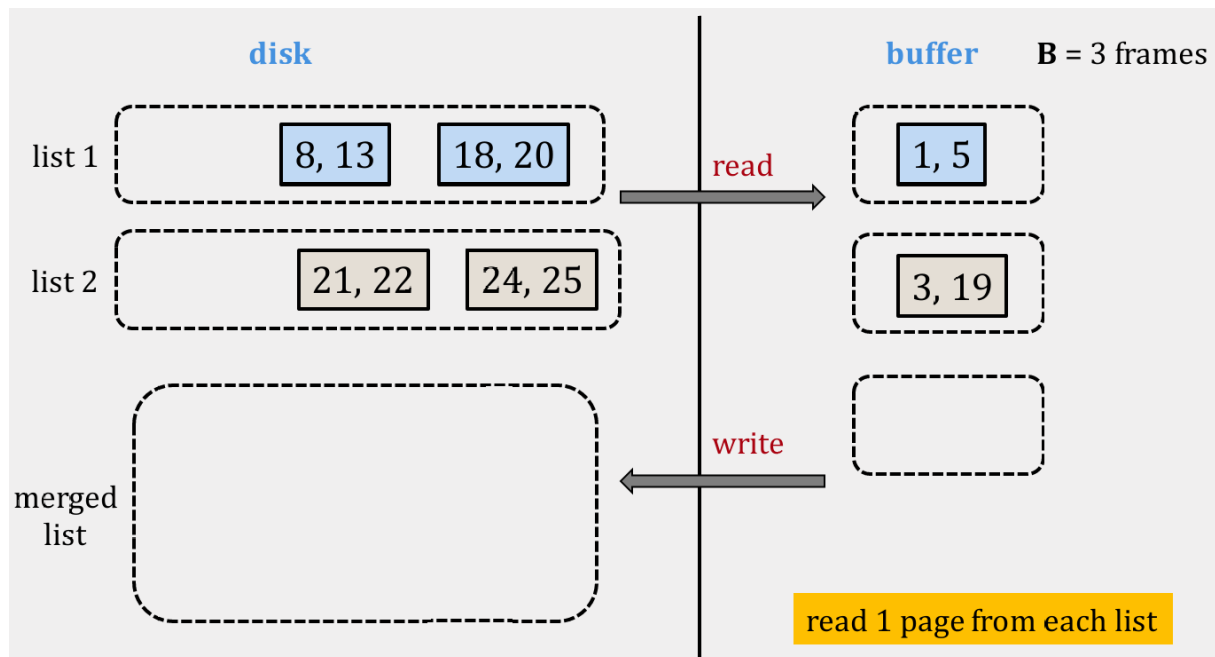
Algunos de estos algoritmos incluyen: - External merge - External merge-sort

10.8.1 External merge

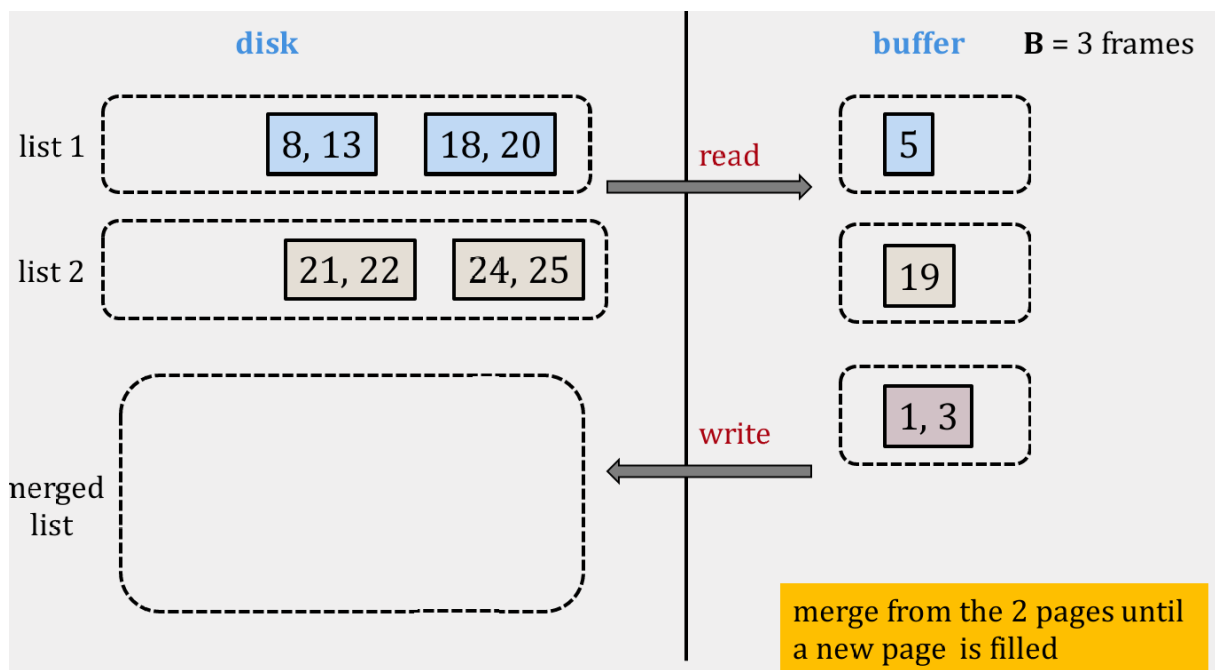
Suponga que se necesitan unir dos archivos **ordenados** que no caben en memoria. El algoritmo de *external merge* divide los archivos en bloques que caben en memoria, los ordena y los guarda en un archivo temporal. Luego, combina los bloques ordenados en un solo archivo ordenado. Si cada bloque es de tamaño n , en memoria se mantendrán tres frames de tamaño n .

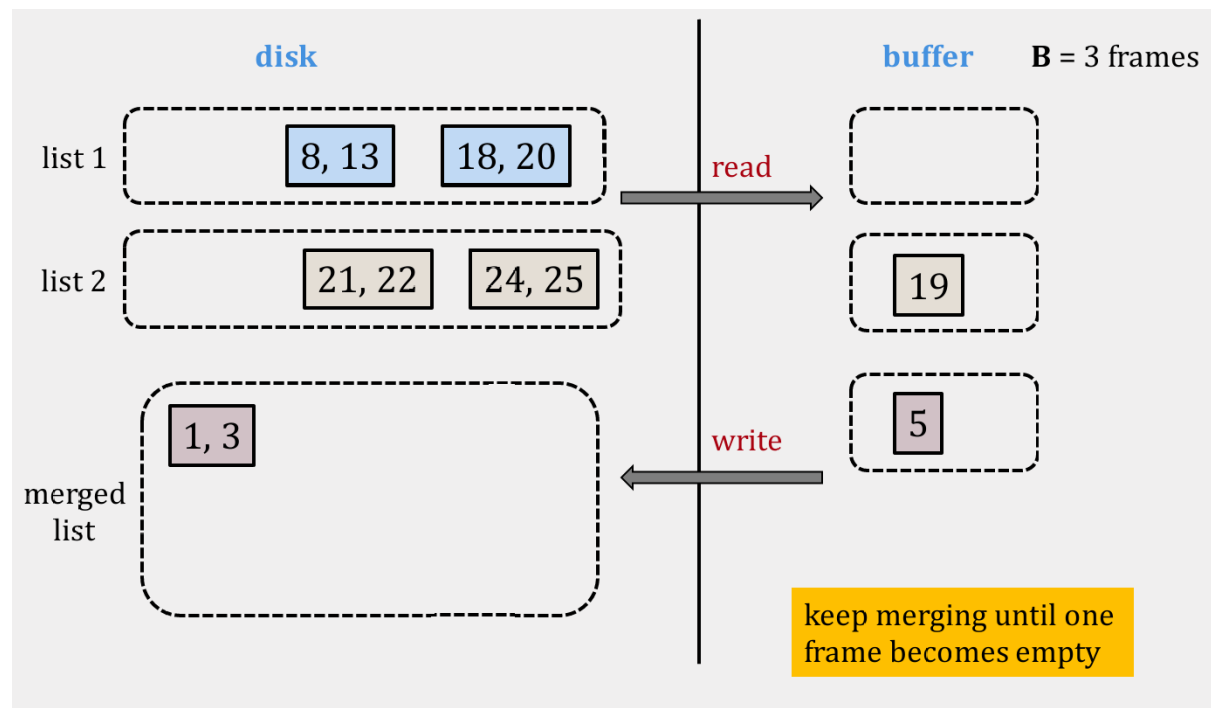
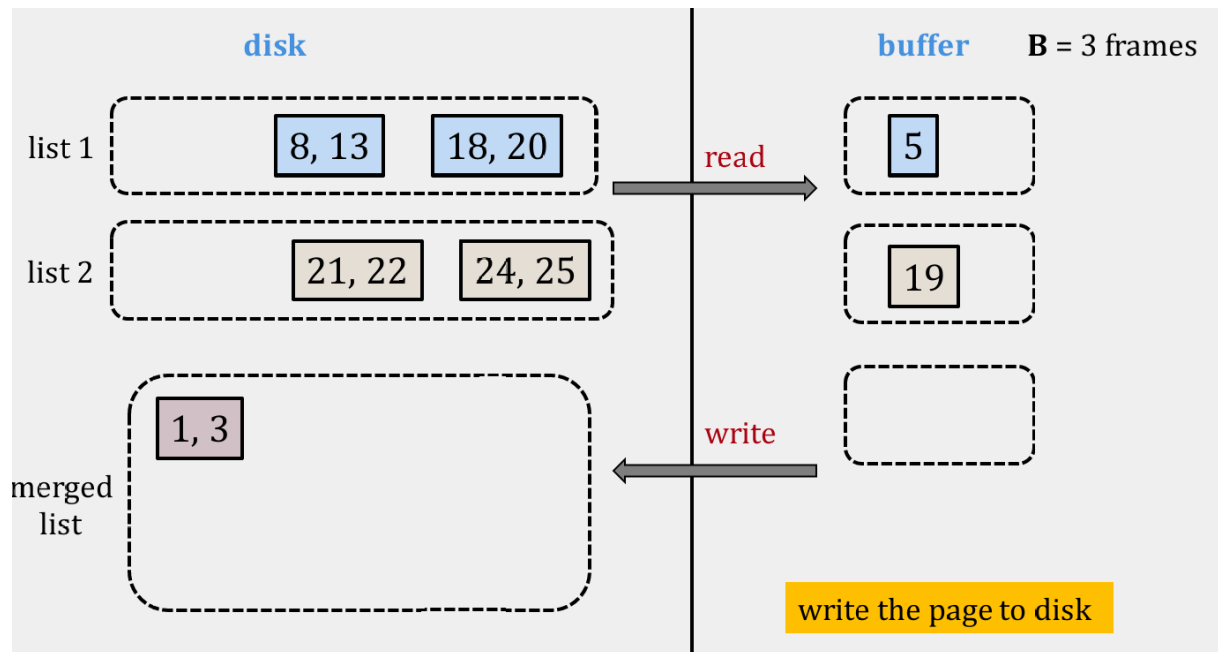


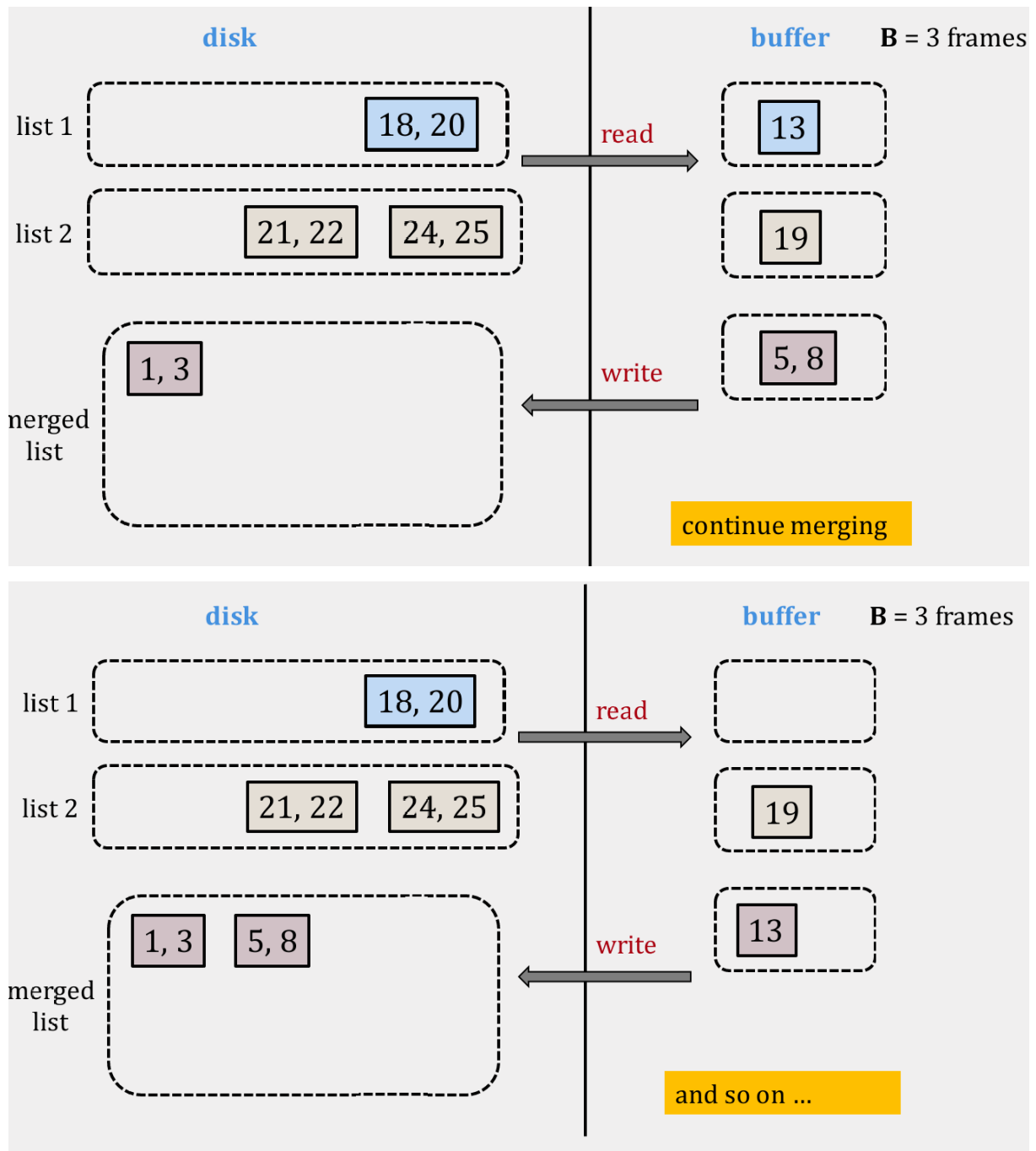
Se leen los primeros bloques de ambos archivos en memoria



Usando el frame temporal, se van comparando los elementos de ambos bloques y se ordena en dicho frame. Cuando el frame temporal se llena, se escribe en el archivo de salida.







10.8.2 External merge-sort

Para ordenar un archivo cuyos elementos están desordenados, podemos aplicar una forma del algoritmo tradicional de merge-sort. En merge-sort en memoria, el array se divide en sub-array en memoria. En el caso de *external merge-sort*, en vez de crear sub-array en memoria se crean *archivos temporales*

en disco de tamaño incremental que actúen como dichos sub-arrays. Cada archivo temporal se llama *run*.

El algoritmo irá creando runs de tamaño 1, tamaño 2 y así sucesivamente hasta que se logre ordenar el archivo completo. Un *run de tamaño 1* es el bloque mínimo de datos (*pages*) que se puede tener en memoria. La cantidad de elementos de dicho bloque dependerá de la naturaleza de los datos en el archivo.

En memoria únicamente se mantiene tres frames. En cada frame cabe un run de tamaño 1. Estos frames se ordenan aplicando el algoritmo de *external merge* como veremos más adelante.

El proceso es el siguiente para la *etapa de división*:

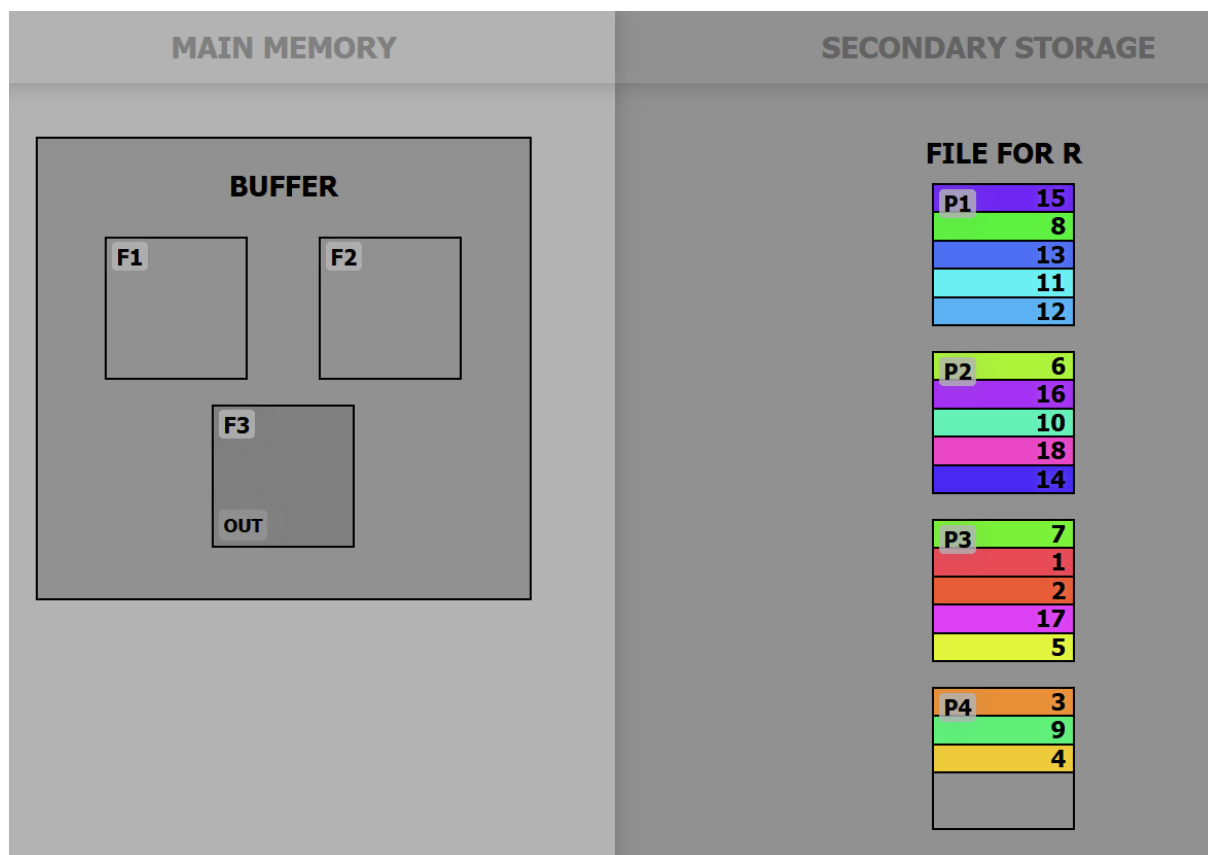
1. Se realiza una pasada inicial para leer página por página del archivo.
2. Se ordena cada página y se escribe en un archivo temporal (*run* de tamaño 1). Se generan n archivos temporales, donde n es la cantidad de páginas del archivo original

Para la *etapa de mezcla*, se sigue el siguiente proceso:

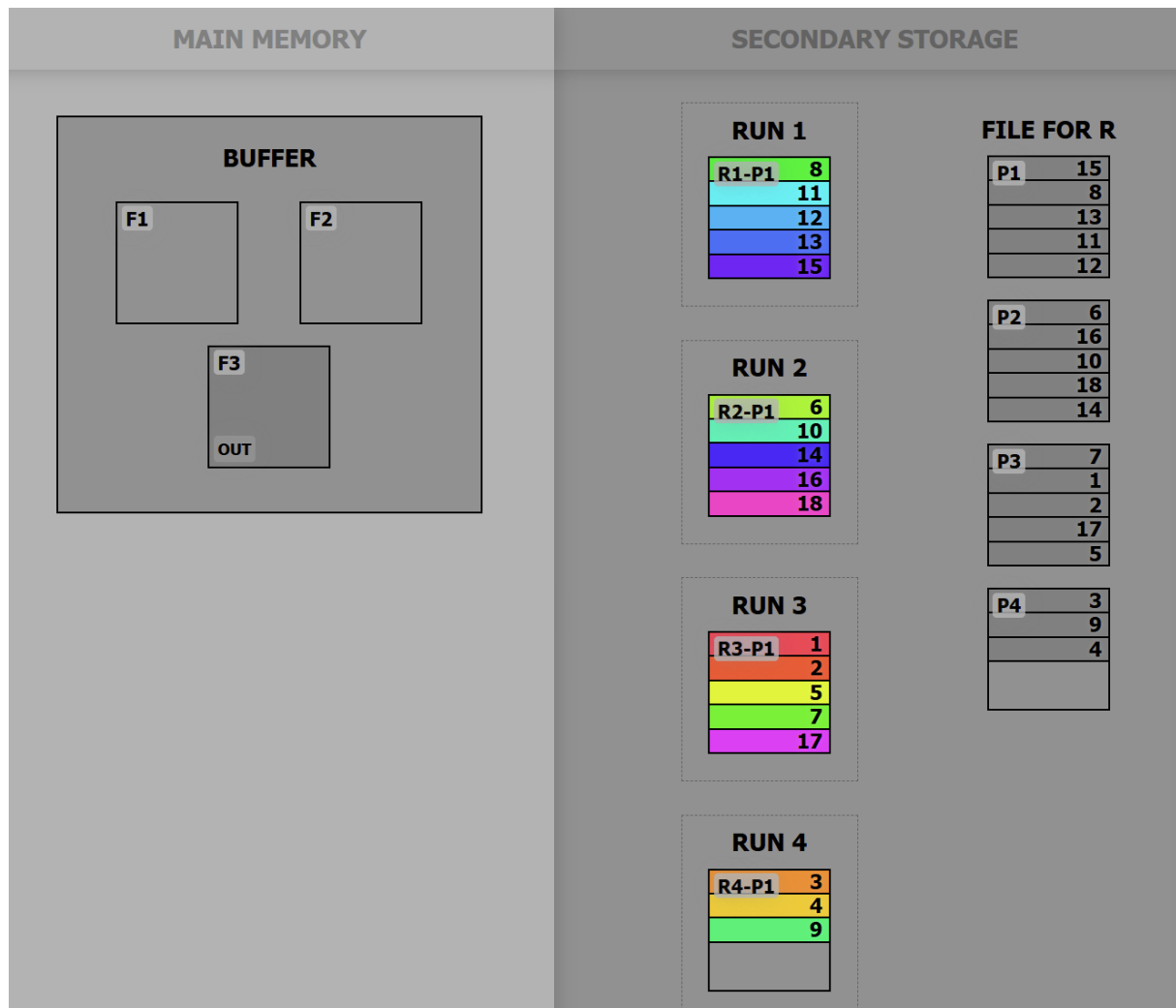
1. Por cada página de cada run i , se carga una página de cada uno en memoria y se aplica ordenan en un nuevo frame. Dicho frame se escribe en un nuevo run de tamaño $i+1$.
2. Se repite el proceso por cada set de *runs*

La visualización del algoritmo en este enlace es sumamente útil. A continuación se adjunto algunas capturas de la misma.

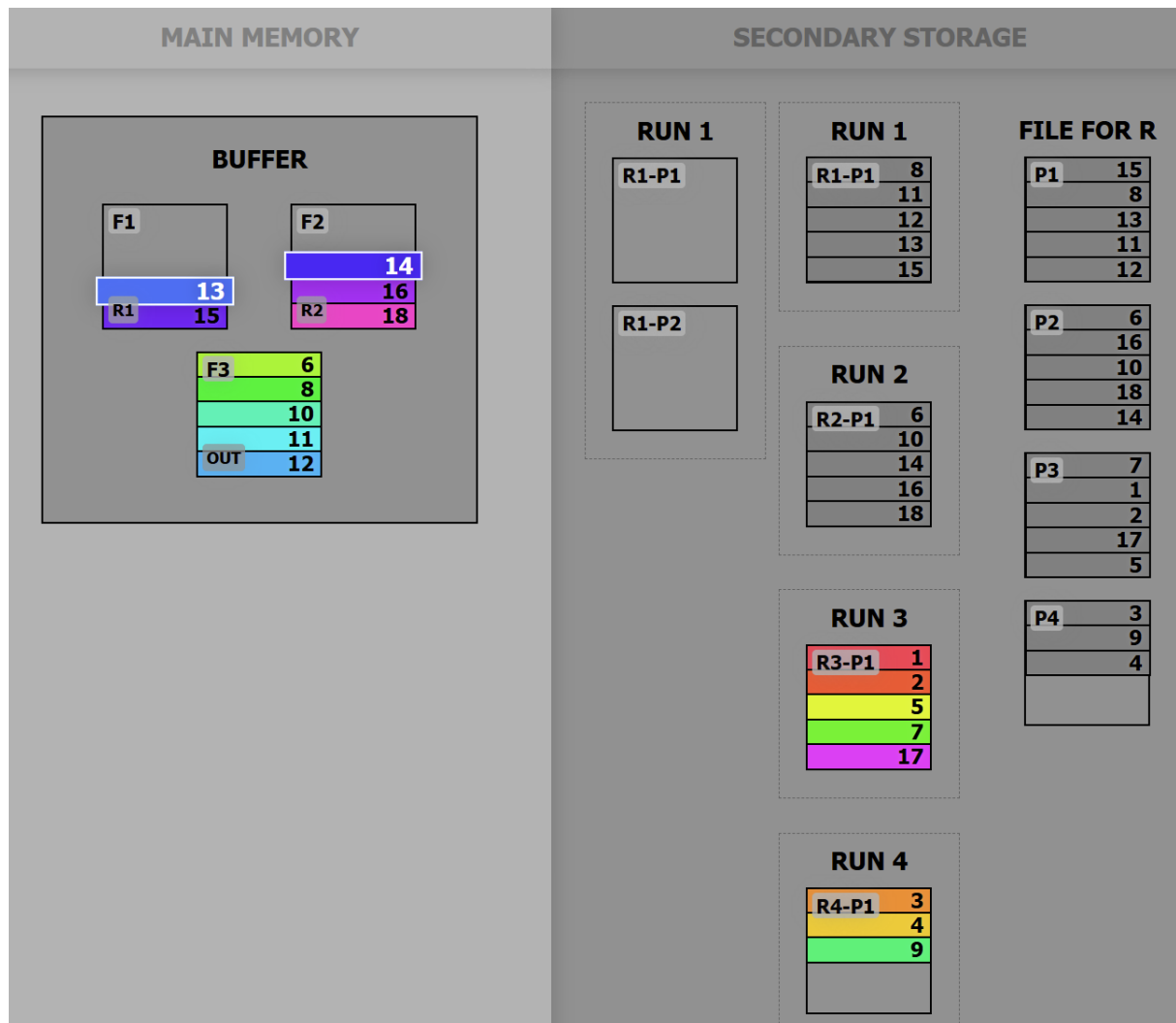
Por ejemplo, el estado inicial es el siguiente:



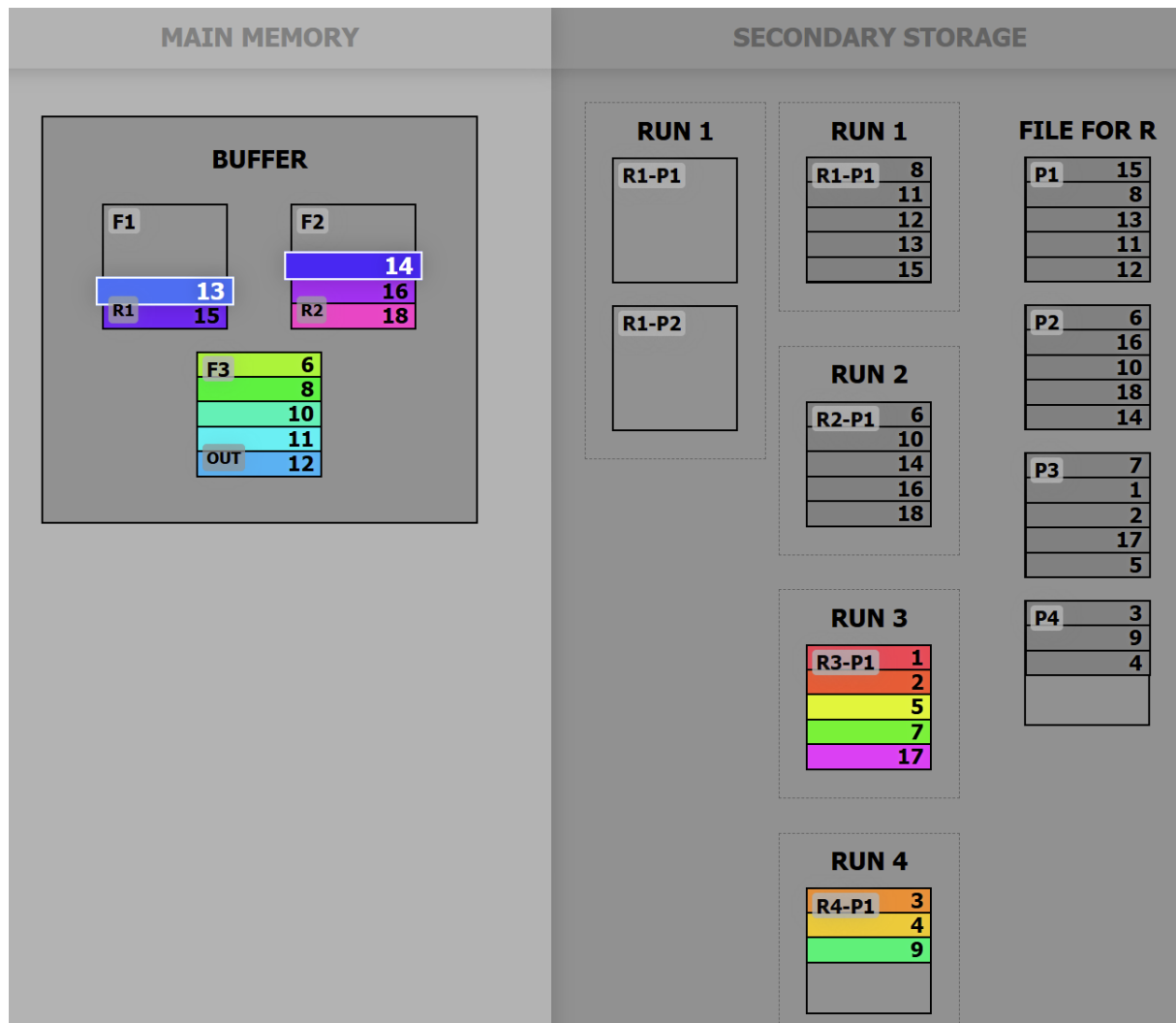
Posterior a la etapa de división, tenemos 4 runs de tamaño 1:



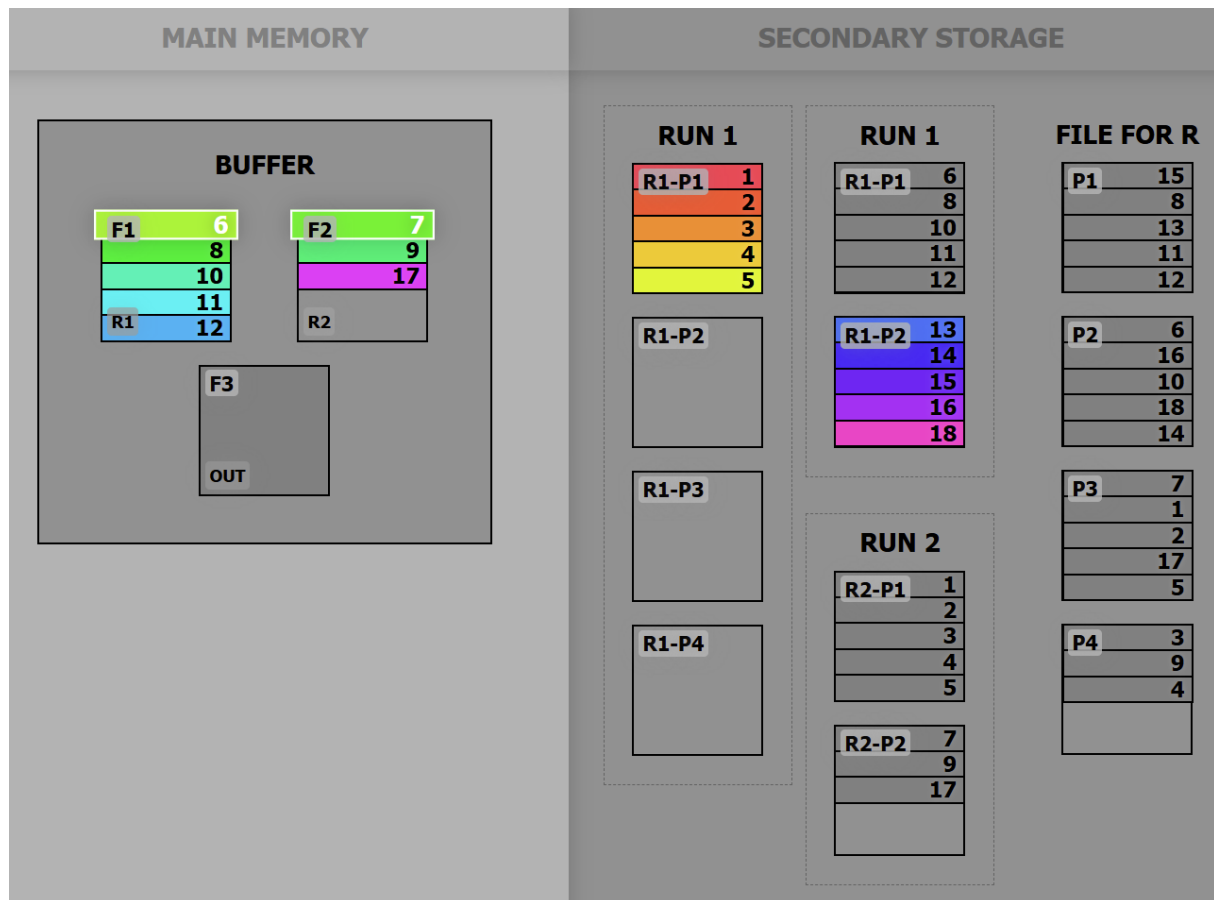
En la etapa de mezcla, observe como se las primeras páginas (y únicas en este caso) de los primeros dos runs de tamaño 1:



Cada vez que el buffer se llena, se escribe en un nuevo run de tamaño 2:



Se repite el proceso con los runs de tamaño dos. Y así sucesivamente hasta que se logre ordenar el archivo completo.



Se pueden hacer mejoras sobre el algoritmo de *external merge-sort* como el uso de *multiway merge* para reducir el número de pasadas sobre el archivo.

10.9 Referencias

- <https://www.geeksforgeeks.org/external-sorting/>
- Koutris. P. CS 564 [Spring 2018]. <https://pages.cs.wisc.edu/~paris/cs564-s18/lectures/lecture-16.pdf>
- <https://valeriodiste.github.io/ExternalMergeSortVisualizer/External%20Merge%20Sort%20Visualizer/index.html>

11 Bases de Datos

En este capítulo, se introducen conceptos fundamentales de bases de datos relacionales y NoSQL. Dicho conocimiento proveerá una conclusión al tema de almacenamiento externo y funcionará como base para los cursos posteriores centrados en bases de datos.

11.1 Definiciones fundamentales

Un **dato** es una representación simbólica (numérica, alfabética, algorítmica, etc.) de un atributo o variable cuantitativa o cualitativa. Los datos describen hechos, entidades o relaciones entre ellos. Por ejemplo, un dato puede ser la edad de una persona, el nombre de un producto o la fecha de un evento.

Una **base de datos** es una colección de datos organizada para un propósito específico. Las bases de datos puede ser almacenadas en medios no electrónicos, como la base de datos de una biblioteca, donde cada tarjeta de papel, tiene datos de un libro específicos. Para efectos de este capítulos, consideramos las bases de datos dentro del contexto de Tecnologías de la Información, donde los datos son almacenados y gestionados electrónicamente.

Las bases de datos tiene un *dominio* o universo de discurso específico, que describe el tipo de datos que almacenan. **No hay bases de datos genéricas.**

Un sistema administrador de bases de datos (DBMS) es un software que permite a los usuarios crear, leer, actualizar y eliminar datos en una base de datos. Ejemplos de DMBS comunes son MySQL, PostgreSQL, Oracle, SQL Server, SQLite, DB2, MongoDB, entre otras.

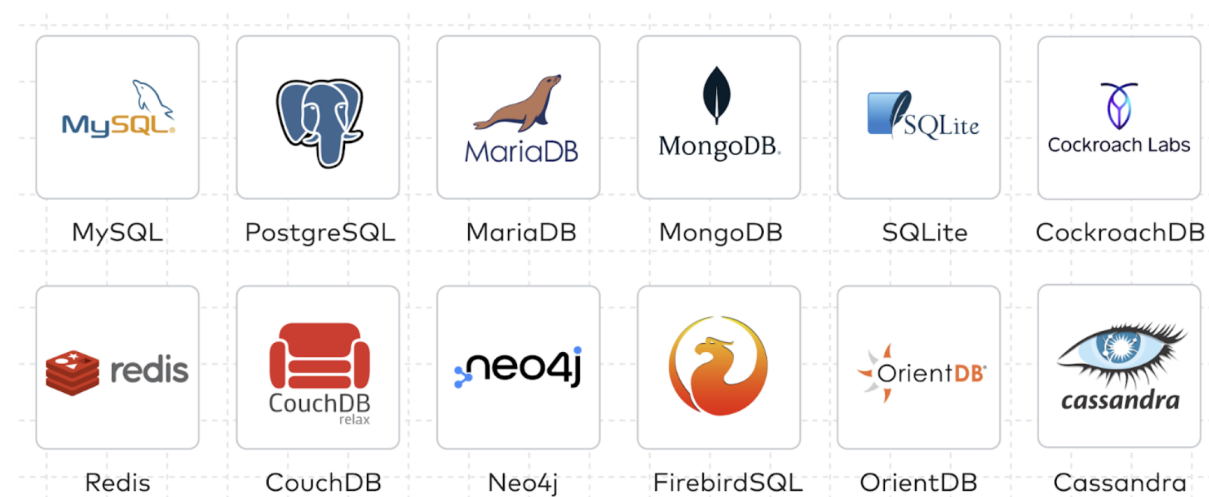


Figura 96: DBMS populares

11.2 Enfoques en el manejo de la información

11.2.1 Enfoque orientado a archivos

Antes de la era de los DBMS, los datos generados por los sistemas de información se almacenaban en archivos simples. El programa o el sistema que los accediera, era responsable de interpretar el formato de cada registro y asegurar la consistencia del archivo completo. Usualmente, un programa accedía a un único archivo con información diseñada para este. Sin embargo, conforme la complejidad de los sistemas crece, surgen problemas como:

- Formatos inconsistentes
- Redundancia (duplicación) de datos en archivos separados
- Diseño de datos ineficiente
- Inseguridad en el manejo de los datos
- Cambios en los archivos requerían cambios en los programas (acoplamiento alto)
- Fragilidad en la integridad de los datos, un error en un programa podía corromper los datos

Bajo el enfoque de archivos, los datos se almacenaban secuencialmente en los archivos, en registros compuestos de campos por ejemplo:

Salesperson Number	Salesperson Name	City	State	Office Number	Commission Percentage	Year of Hire
119	Taylor	New York	NY	1211	15	2003
137	Baker	Detroit	MI	1284	10	1995
186	Adams	Dallas	TX	1253	15	2001
204	Dickens	Dallas	TX	1209	10	1998
255	Lincoln	Atlanta	GA	1268	20	2003
361	Carlyle	Detroit	MI	1227	20	2001
420	Green	Tucson	AZ	1263	10	1993

Figura 97: Estructura de un archivo

En los DBMS modernos, los datos estructurados, se almacenan *lógicamente* con una estructura igual, es decir, registros y campos.

11.2.2 Enfoque orientado a bases de datos

Tal y como se mencionó en secciones anteriores, el DBMS es un software que permite a los usuarios crear, leer, actualizar y eliminar datos en una base de datos. Bajo este enfoque, se crea una capa de indirección entre los programas y los datos, delegando la responsabilidad de la gestión de los datos al DBMS. Los programas acceden a los datos a través de consultas y comandos, sin necesidad de conocer la estructura interna de la base de datos.

Las ventajas de este enfoque son:

- **Independencia de los datos:** Los programas no necesitan conocer la estructura interna de la base de datos. Los DBMS son auto-descriptivos, es decir, pueden describir su estructura interna.

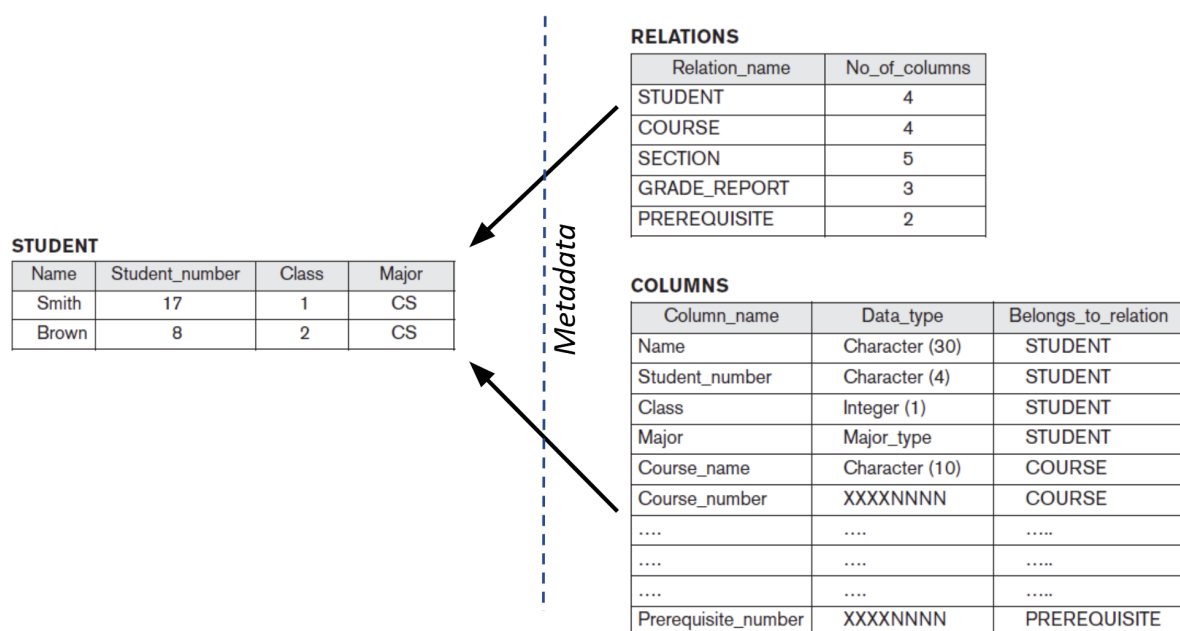


Figura 98: Metadata en los DBMS

- **Integridad de los datos:** Los DBMS pueden aplicar reglas de integridad para garantizar la precisión y consistencia de los datos.
- **Seguridad:** Los DBMS pueden controlar el acceso a los datos y protegerlos contra accesos no autorizados.
- **Concurrencia:** Los DBMS pueden gestionar el acceso simultáneo a los datos por múltiples usuarios.
- **Escalabilidad:** Los DBMS pueden manejar grandes volúmenes de datos y crecer con las necesidades de la organización.

- **Facilidad para compartir datos:** Los DBMS permiten compartir datos entre múltiples usuarios y aplicaciones, sin necesidad de duplicarlos.

Algunos de los componentes de un DBMS incluyen:

- **Datos:** La información almacenada en la base de datos.
- **Motor de base de datos:** Interpreta y ejecuta las consultas y comandos enviados por los usuarios.
- **Lenguaje de consulta:** Permite a los usuarios interactuar con la base de datos mediante consultas y comandos.
- **Gestor de transacciones:** Controla las operaciones de inserción, actualización y eliminación de datos para garantizar la consistencia y la integridad.
- **Gestor de almacenamiento:** Administra el almacenamiento físico de los datos en el disco.
- **Gestor de seguridad:** Controla el acceso a los datos y protege la base de datos contra accesos no autorizados.

11.3 Tipos de bases de datos

De forma análoga a los lenguajes de programación en los que se adopta un paradigma que determina sus características, las bases de datos pueden adoptar un enfoque:

1. **Relacional:** Las bases de datos relacionales almacenan datos en tablas, donde cada fila representa un registro y cada columna un campo. Las relaciones entre las tablas se establecen mediante claves primarias y claves foráneas. Se fundamentan en el modelo relacional propuesto por Edgar Codd en 1970.
2. **Orientadas a objetos:** los datos se modelan como objetos, con atributos y métodos. Las bases de datos orientadas a objetos permiten almacenar objetos complejos y sus relaciones de manera eficiente.
3. **NoSQL:** Las bases de datos NoSQL (Not Only SQL) son una alternativa a las bases de datos relacionales, diseñadas para manejar grandes volúmenes de datos no estructurados o semi-estructurados de manera flexible y escalable. NoSQL se refiere a una amplia gama de tecnologías de bases de datos que no utilizan el modelo relacional tradicional basado en tablas.

Algunos DBMS pueden soportar más de un enfoque, por ejemplo, Oracle Database soporta bases de datos relacionales y objetos. Por eso, decimos que la base de datos sigue un enfoque específico en vez de referirse al DBMS como tal.

Los tipos de bases de datos listadas en esta sección excluyen tipos utilizados en el pasado. Podrá aprender más sobre esto en el curso de bases de datos.

11.3.1 Bases de datos relacionales

Se fundamentan en el modelo relacional propuesto por Edgar Codd en 1970. En este modelo, los datos se almacenan en tablas, donde cada fila representa un registro y cada columna un campo. Cada fila tiene un identificador único llamado clave primaria, que permite identificar de manera única cada registro en la tabla. Considere la siguiente tabla llamada **ESTUDIANTE**:

ID	NOMBRE	EDAD	CARRERA
1	Juan	20	Ingeniería en Sistemas
2	María	22	Ingeniería Civil
3	Pedro	21	Ingeniería Industrial

La columna ID es la clave primaria de la tabla ESTUDIANTE. No pueden haber dos o más registros con el mismo ID.

Las relaciones entre las tablas se establecen mediante claves foráneas, que son campos en una tabla que hacen referencia a la clave primaria de otra tabla. Por ejemplo, considere la tabla **CURSO**:

ID	NOMBRE	ESTUDIANTE_ID
1	Matemáticas	1
2	Física	2
3	Química	3

La columna **ESTUDIANTE_ID** es una clave foránea que hace referencia a la clave primaria de la tabla **ESTUDIANTE**. Esta relación permite asociar un curso con un estudiante. No hace falta duplicar la información del estudiante en la tabla **CURSO** (tener todos los campos de **ESTUDIANTE** de nuevo en la tabla **CURSO**).

11.3.1.1 Terminología clave Algunos de los conceptos clave en las bases de datos relacionales son:

- *Relación*: Una tabla que almacena datos relacionados.
- *Atributo*: Una columna en una tabla que representa un campo de datos.
- *Esquema de la relación*: La estructura de una tabla, que incluye los atributos y las restricciones. Puede verse como una clase en Orientación a Objetos que define la estructura pero no es un objeto en sí.
- *Tuplas*: Una fila o registro en una tabla que representa una instancia en particular de dicha relación.
- *Clave primaria*: Un atributo o conjunto de atributos que identifica de manera única cada tupla en una tabla.
- *Clave foránea*: Un atributo en una tabla que hace referencia a la clave primaria de otra tabla.
- *Clave candidata*: Un atributo o conjunto de atributos que pueden ser claves primarias.

11.3.1.2 Integridad referencial La integridad referencial es una restricción que garantiza que las relaciones entre las tablas sean válidas. En una relación entre dos tablas, la clave foránea en la tabla secundaria debe hacer referencia a una clave primaria existente en la tabla principal. Por ejemplo, en la tabla `CURSO`, el campo `ESTUDIANTE_ID` debe hacer referencia a un `ID` existente en la tabla `ESTUDIANTE`.

Esto permite que las relaciones se “normalicen”, es decir, que se evite la redundancia de datos y se garantice la consistencia de los datos.

11.3.1.3 Transaccionalidad ACID ACID es un acrónimo que describe las propiedades de las transacciones en una base de datos relacional. Las transacciones son operaciones que modifican los datos en una base de datos y deben cumplir con las siguientes propiedades:

- *Atomic*: Una transacción es atómica si se ejecuta completamente o no se ejecuta en absoluto. Si una parte de la transacción falla, se deshace la transacción completa.
- *Consistent*: Una transacción es consistente si lleva la base de datos de un estado consistente a otro estado consistente. La base de datos debe cumplir con todas las restricciones de integridad antes y después de la transacción.
- *Isolated*: Una transacción es aislada si su ejecución es independiente de otras transacciones. Las transacciones concurrentes no deben interferir entre sí.
- *Durable*: Una transacción es duradera si los cambios realizados por la transacción persisten en la base de datos incluso después de un fallo del sistema.

11.3.1.4 El lenguaje SQL (Structured Query Language) Es un lenguaje estandarizado para interactuar con bases de datos relacionales. SQL permite realizar diversas operaciones para gestionar datos de manera eficiente y precisa. Las operaciones SQL se pueden clasificar en dos tipos:

- **DDL (Data Definition Language):** Utilizado para definir y modificar estructuras de bases de datos ([CREATE](#), [ALTER](#), [DROP](#)).
- **DML (Data Manipulation Language):** Utilizado para manipular datos dentro de objetos de la base de datos ([SELECT](#), [INSERT](#), [UPDATE](#), [DELETE](#)).

A continuación, se presentan algunas operaciones SQL de ejemplo sobre las tablas [ESTUDIANTE](#) y [CURSO](#).

1. **Creación de tablas:** Para crear tablas, la instrucción SQL [CREATE TABLE](#) se utiliza y tiene la siguiente estructura:

```
1 CREATE TABLE nombre_tabla (  
2     columna1 tipo_dato1,  
3     columna2 tipo_dato2,  
4     ...  
5 );
```

Nótese que los tipos de datos pueden variar dependiendo del DBMS utilizado y son diferentes a los tipos tradicionales en la programación. Un resumen de la jerarquía de tipos de datos en SQL se puede visualizar en este enlace.

Para crear la tabla [ESTUDIANTE](#), se puede utilizar una instrucción SQL similar a la siguiente:

```
1 CREATE TABLE ESTUDIANTE (  
2     ID INT PRIMARY KEY,  
3     NOMBRE VARCHAR(50),  
4     EDAD INT,  
5     CARRERA VARCHAR(50)  
6 );
```

De igual forma, para crear la tabla [CURSO](#):

```
1 CREATE TABLE CURSO (  
2     ID INT PRIMARY KEY,  
3     NOMBRE VARCHAR(50),  
4     ESTUDIANTE_ID INT,  
5     FOREIGN KEY (ESTUDIANTE_ID) REFERENCES ESTUDIANTE(ID)  
6 );
```

En este ejemplo, durante la creación de la tabla [CURSO](#), se establece una clave foránea ([FOREIGN KEY](#)) que hace referencia a la clave primaria de la tabla [ESTUDIANTE](#). No es necesario definir la integridad durante la creación de la tabla, puesto que se puede usar la instrucción [ALTER TABLE](#) para agregar restricciones de integridad después de la creación de la tabla.

2. **Insertión de datos:** Para insertar datos en una tabla, se utiliza la instrucción SQL [INSERT INTO](#) con la siguiente estructura:

```
1 INSERT INTO nombre_tabla (columna1, columna2, ...)
2 VALUES (valor1, valor2, ...);
```

Por ejemplo, para insertar un nuevo estudiante en la tabla **ESTUDIANTE**:

```
1 INSERT INTO ESTUDIANTE (ID, NOMBRE, EDAD, CARRERA)
2 VALUES (1, 'Juan', 20, 'Ingeniería en Sistemas');
```

De igual forma, para insertar un nuevo curso en la tabla **CURSO**:

```
1 INSERT INTO CURSO (ID, NOMBRE, ESTUDIANTE_ID)
2 VALUES (1, 'Matemáticas', 1);
```

3. **Actualización de datos:** Para actualizar registros existentes en una tabla, se utiliza la instrucción SQL **UPDATE** con la siguiente estructura:

```
1 UPDATE nombre_tabla
2 SET columna1 = nuevo_valor
3 WHERE condición;
```

Ignorar la cláusula **WHERE** en una instrucción **UPDATE** puede resultar en la actualización de todos los registros en la tabla. Es importante recalcar que los nuevos valores serán revisados contra las restricciones de integridad de la tabla.

Por ejemplo, para actualizar la edad de un estudiante en la tabla **ESTUDIANTE**:

```
1 UPDATE ESTUDIANTE
2 SET EDAD = 21
3 WHERE ID = 1;
```

4. **Eliminación de datos:** Para eliminar registros de una tabla, se utiliza la instrucción SQL **DELETE FROM** con la siguiente estructura:

```
1 DELETE FROM nombre_tabla
2 WHERE condición;
```

Al igual que con la instrucción **UPDATE**, ignorar la cláusula **WHERE** en una instrucción **DELETE** puede resultar en la eliminación de todos los registros en la tabla.

Por ejemplo, para eliminar un curso de la tabla **CURSO**:

```
1 DELETE FROM CURSO
2 WHERE ID = 1;
```

Cuando se eliminan datos de una tabla “padre”, los registros relacionados en las tablas “hijas” deben ser eliminados o actualizados para mantener la integridad referencial. Esto se puede lograr mediante la

definición de restricciones de integridad en la base de datos. Esto se conoce como “acción en cascada” y es una característica común en los DBMS.

5. **Consultar los datos:** La operación más común en SQL es la consulta SELECT, que se utiliza para recuperar datos de una o más tablas. Dicha operación tiene la siguiente estructura:

```
1 SELECT columna1, columna2
2 FROM tabla
3 WHERE condición;
```

Por ejemplo, para consultar los estudiantes de la tabla **ESTUDIANTE**:

```
1 SELECT *
2 FROM ESTUDIANTE;
```

Para consultar los cursos de la tabla **CURSO**:

```
1 SELECT *
2 FROM CURSO;
```

Para consultar los cursos de un estudiante específico:

```
1 SELECT *
2 FROM CURSO
3 WHERE ESTUDIANTE_ID = 1;
```

Para unir la información de las tablas **ESTUDIANTE** y **CURSO**:

```
1 SELECT E.NOMBRE, C.NOMBRE
2 FROM ESTUDIANTE E
3 JOIN CURSO C ON E.ID = C.ESTUDIANTE_ID;
```

La sentencia **JOIN** y sus variantes, puede entenderse visualmente mediante el siguiente diagrama:

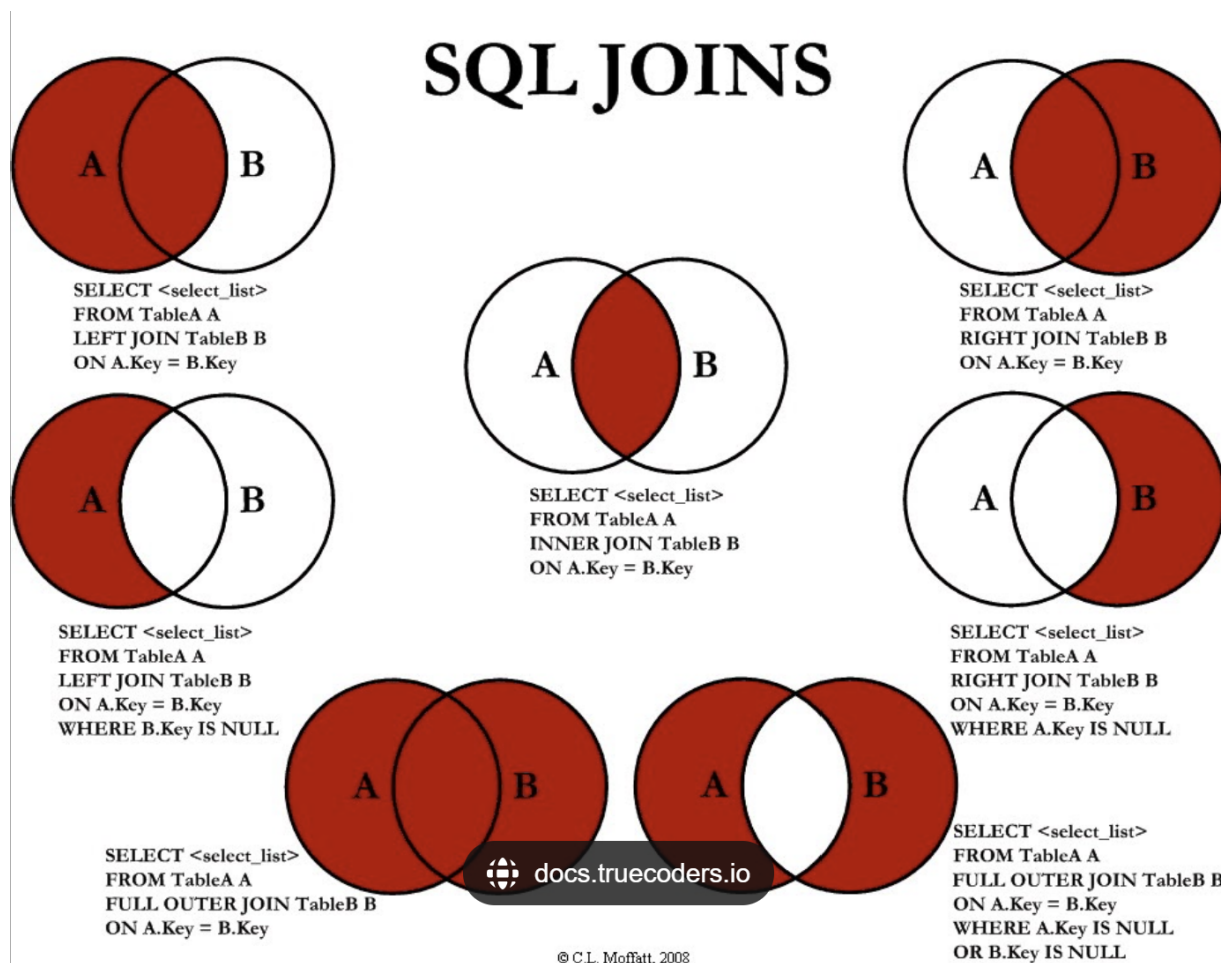


Figura 99: Tipos de JOIN

11.3.2 Bases de Datos NoSQL

NoSQL (Not Only SQL) es un término utilizado para describir bases de datos que no utilizan el modelo relacional tradicional basado en tablas. Estas bases de datos están diseñadas para manejar grandes volúmenes de datos no estructurados o semi-estructurados de manera flexible y escalable.

El término *NoSQL* fue acuñado por Carl Strozzi en 1998 cuando lanzó su base de datos “NoSQL”, que era un sistema de bases de datos que no usaba SQL. Su base de datos estaba más centrada en ser ligera y simple para aplicaciones específicas, en lugar de seguir los principios y características de las bases de datos relacionales.

Sin embargo, este nombre fue interpretado de manera errónea. El No en NoSQL inicialmente hacía referencia a “No solo SQL”, lo que indicaba que no era una base de datos exclusivamente relacional, sino que existía una alternativa. En otras palabras, NoSQL significa “No solo SQL”, lo que sugiere que las

bases de datos NoSQL pueden usar otros lenguajes de consulta o no requieren de SQL en absoluto.

11.3.3 Características de NoSQL

- **Estructura Flexible:** Permite almacenar datos con estructuras flexibles sin necesidad de un esquema fijo.
- **Escalabilidad Horizontal:** Capacidad para manejar grandes volúmenes de datos distribuyendo la carga en múltiples servidores o nodos.
- **Modelos de Datos Diversos:** Soporta varios modelos de datos como documentos, grafos, columnas y clave-valor, optimizados para diferentes tipos de aplicaciones y cargas de trabajo.

11.3.4 Tipos de Bases de Datos NoSQL

1. **Bases de Datos de Documentos** Son bases de datos que almacenan datos en documentos JSON o BSON (una representación binaria de JSON). Cada documento es una entidad independiente que contiene datos y metadatos. Los documentos se pueden agrupar en colecciones, que son similares a las tablas en una base de datos relacional. Ejemplos de bases de datos de documentos incluyen MongoDB, Couchbase y CouchDB.
2. **Bases de Datos de Grafos** Son bases de datos que modelan datos como nodos y relaciones entre ellos. Son útiles para representar relaciones complejas entre entidades. Ejemplos de bases de datos de grafos incluyen Neo4j, Amazon Neptune y ArangoDB.
3. **Bases de Datos de Columnas** Son bases de datos que almacenan datos en columnas en lugar de filas. Son eficientes para consultas analíticas y agregaciones. Ejemplos de bases de datos de columnas incluyen Apache Cassandra, HBase y Google Bigtable.
4. **Bases de Datos Clave-Valor** Las bases de datos clave-valor almacenan datos en pares clave-valor, donde cada clave es única y se asocia con un valor. Son eficientes para operaciones de lectura y escritura rápidas. Ejemplos de bases de datos clave-valor incluyen Redis, Amazon DynamoDB y Riak.

11.3.5 Casos de Uso de NoSQL

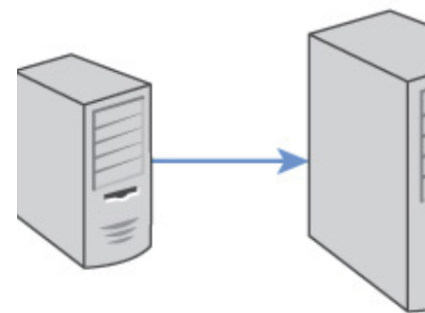
- **Aplicaciones Web Escalables:** Ideal para aplicaciones web que requieren escalabilidad horizontal y manejo eficiente de grandes volúmenes de datos.
- **Análisis de Datos en Tiempo Real:** Utilizado en bases de datos como Apache Kafka o Elasticsearch para análisis de datos en tiempo real y búsqueda de texto completo.

11.3.6 Consideraciones y Limitaciones

- **Consistencia:** Algunas bases de datos NoSQL pueden sacrificar consistencia eventualmente consistente.
- **Herramientas y Ecosistema:** Aunque cada vez más robusto, el ecosistema y las herramientas de NoSQL pueden ser menos maduras en comparación con las bases de datos relacionales establecidas.

11.3.7 Ventajas de las bases de datos NoSQL

- **Sintaxis más flexible:** A diferencia de SQL, que tiene una sintaxis rígida, las bases de datos NoSQL permiten una mayor libertad en la estructura de los datos.



- **Escalabilidad horizontal:** Pueden crecer fácilmente agregando más servidores.
- **Rendimiento:** Operan principalmente en memoria, lo que reduce los tiempos de lectura y escritura.

Scale Up

11.4 Referencias

Gillenson. M. (2012). Fundamentals of Database Management Systems. John Wiley & Sons. Sullivan M. (2019). NoSQL for Mere Mortals. Addison-Wesley.